



Beyond the Veil:

A Guide to Privacy, Security, and
Anonymity in the Digital Era

Mohammad Kamal

Contents

1	An Introduction to Cybersecurity for a 1.5 Hour Talk	21
1.1	The Motivation	21
1.2	Prerequisites	21
1.2.1	Data	22
1.2.2	Instruction/Command Execution	22
1.2.3	Operating System (OS)	23
1.2.4	The Internet and Server	23
1.2.5	Security Vulnerability	23
1.2.6	Encryption	24
1.3	Security	24
1.3.1	Definition	24
1.3.2	Hardware Security	25
1.3.3	Software Security	30
1.3.4	Network Security	40
1.3.5	Conclusion	46
1.4	Privacy	47
1.4.1	What Does It Mean by Privacy?	47
2	Quick Introduction to Hardware	51
2.1	What is Hardware?	51
2.2	Why We Should Care About Hardware?	51
2.3	Hardware Components	51
2.3.1	Hard Disk Drive (HDD) & Solid State Drive (SSD)	52
2.3.2	Central Processing Unit (CPU)	52
2.3.3	Random Access Memory (RAM)	53
2.3.4	Register	54
2.3.5	Cache	54
2.3.6	The Bigger Picture	56
2.3.7	Graphical Processing Unit (GPU)	58

2.4	Do Software Engineers Need to Know About Hardware?	58
2.4.1	Branch Prediction & Speculative Execution	58
3	Difference between Privacy, Anonymity, and Security	63
3.1	Security	63
3.1.1	Definition	63
3.1.2	Prevention, Detection and Response	63
3.1.3	No System is 100% Safe	66
3.1.4	The Weakest Link	68
3.1.5	The Importance of Transparency: Public Algorithms in Security Assurance	69
3.1.6	Open Source and Its Security	74
3.1.7	Don't Rely on Trial and Error When Creating Services and Troubleshooting Computer Errors!	80
3.1.8	The Software Industry Needs More Secure Products, Not More Security Products-CISA	81
3.1.9	Downgrade Attacks	85
3.1.10	Social Engineering	86
3.1.11	Be Cautious of Hacking and Security Groups	87
3.1.12	Avoid Getting Involved in Governmental Hacking Games	87
3.2	Privacy	89
3.2.1	Why Privacy Matters?	89
3.2.2	Story of the Government	94
3.2.3	Freedom of Information Act (FOIA)	94
3.2.4	Who Watches the Watchers?	95
3.2.5	It's Not All About Security Agencies	95
3.2.6	Analysis on Data & Data is Power	96
3.2.7	The Double-Edged Sword of Monitoring and Collecting Data	99
3.2.8	The Chain Reaction of Backdoors: How one Backdoor can Lead to Many	100
3.2.9	The Paradox of Data Overload: Valuable Insights Lost in the Noise	101
3.2.10	Not Only Yourself, But You Harm Others	101
3.2.11	The Solution	102
3.2.12	If You Don't Pay the Product, You Are the Product! . . .	102
3.2.13	Reading List	103
3.3	Anonymity	103
3.3.1	Zero Trust Model	104
3.4	End of the Chapter Questions	104

3.5	End of the Chapter Solutions	105
-----	--	-----

4 Cryptography 111

4.1	Why you should know about cryptography	111
4.2	Reading List	111
4.3	What problem am I solving?	112
4.4	Cryptographic Hash Function	113
4.4.1	Birthday Problem and Hash Collision	115
4.4.2	Applications	116
4.4.3	Standardization of Hash Functions	118
4.4.4	SHA-3	118
4.5	Password Hashing Algorithms	119
4.5.1	Traditional Brute-Force	120
4.5.2	Hash Table Attack	120
4.5.3	Rainbow Table Attack	120
4.5.4	Salting	121
4.5.5	Different Password Hashing Algorithms	122
4.6	Applications of Hash Functions	123
4.6.1	Bitcoin (SHA2-256)	123
4.6.2	Cloud-base Applications	126
4.6.3	Reading List	126
4.7	Encoding	127
4.8	Encryption	127
4.8.1	Kerckhoff's Principle	128
4.8.2	Steganography	129
4.8.3	Number Station	131
4.8.4	What if using a pad several times?	132
4.9	Symmetric Encryption	133
4.9.1	Application of Symmetric Encryption	134
4.9.2	One-time Pad	134
4.9.3	Analysis of the Basic Attack (Advanced)	135
4.10	Symmetric Encryption Operating Modes	138
4.10.1	ECB (Electronic Codebook)	139
4.10.2	CBC (Cipher Block Chaining)	140
4.10.3	PCBC (Propagating Cipher Block Chaining)	142
4.10.4	CFB (Cipher Feedback)	143
4.11	ASCON	146
4.12	Asymmetric Encryption	146
4.12.1	Meddler-in-the-Middle Attack (MitM Attack)	147

Contents

4.12.2	Digital Signature	149
4.12.3	RSA	150
4.12.4	Low-entropy Message Attack	151
4.12.5	Is RSA bulletproof?	152
4.13	Randomness	152
4.13.1	Pseudo-random number generators (PRNGs)	154
4.13.2	True random number generators (TRNGs)	155
4.14	An example of an E2EE system	156
4.15	Post-Quantum Cryptography	157
4.15.1	Debunking RSA “Breaks”: The Reality Behind the Head- lines	159
4.16	MAC and HMAC	159
4.16.1	Message Authentication Code (MAC)	159
4.16.2	Designated Verifier Signature	160
4.17	Homomorphic Encryption	161
4.18	Shamir’s Secret Sharing	161
4.18.1	Secret Sharing	162
4.18.2	Naive Approach	162
4.18.3	Lagrange Interpolation	162
4.18.4	Shamir’s Secret Sharing	164
4.19	Zero-Knowledge Proofs	167
4.20	Bit-commitment	168
4.21	Combining Multiple Block Ciphers	169
4.22	What to know?	169
4.23	End of Chapter Questions	172
4.24	End of the Chapter Solutions	174
5	Hardware Security	179
5.1	Physical Access	179
5.1.1	Cold Boot Attack	182
5.2	What to do?	183
5.3	Hardware Attacks	183
5.3.1	A simple Backdoor with Mux	184
5.3.2	Rowhammer Attack	184
5.3.3	Fault Attack	184
5.3.4	Side-channel Attacks	184
5.4	Career Opportunities	187
5.5	End of Chapter Questions	189
5.6	End of the Chapter Solutions	189

6	Operating System (OS) and Software Security	193
6.1	What is an Operating System?	193
6.2	OS security	193
6.3	What OS to use?	194
6.4	Malware: The Most Common Threat	195
6.4.1	Virus	196
6.4.2	Ransomware	197
6.4.3	Trojan	201
6.4.4	Keystroke Logger (Keylogger)	202
6.4.5	Adware	203
6.4.6	Spyware	203
6.4.7	Botnet	204
6.4.8	Denial-of-service attack (DoS) & Distributed Denial-of-service attack (DDoS)	208
6.4.9	Backdoor	211
6.4.10	Fileless malware	211
6.4.11	Coinminer	211
6.5	What to Do?	213
6.6	The Dark Side of Connectivity: Security Risks in the Internet of Things	215
6.7	Isolation from the Actual System	217
6.7.1	Sandbox	217
6.7.2	Virtual Machine	220
6.7.3	Isolated Environments: The Key to Resilient Software Design	221
6.7.4	See Also	221
6.8	Anti-virus	221
6.8.1	Signature-based detection	222
6.8.2	Behavioral-based detection	223
6.8.3	Anti-virus bypass methods	224
6.9	Code Security	224
6.9.1	Input Validation	224
6.9.2	Sign and Unsigned Comparison	226
6.9.3	Reach the End of Function Without Return	226
6.9.4	Floating-Point Comparison	227
6.9.5	Timing Attack	228
6.9.6	Careless programmers/Faster is not always better	231
6.9.7	Race Condition	232
6.9.8	An attempt to backdoor the kernel	236
6.9.9	Out-of-Bound Access	237

6.9.10	Memory-Safe Languages	239
6.9.11	Telegram Emoji Suggestion	245
6.10	End of the Chapter Questions	246
6.11	End of the Chapter Solutions	247
7	Network	253
7.1	Look-alike Domain	253
7.1.1	How to determine the domain of a website?	253
7.2	DNS	262
7.3	Virtual Private Network (VPN)	266
7.3.1	Why should I be careful when choosing a VPN?	266
7.3.2	Why should we even use a VPN?	267
7.3.3	What Data Can They Collect	269
7.3.4	When choosing a VPN (or generally an app), what should you pay attention to?	270
7.4	Vulnerabilities Related to Network and Connection	276
7.4.1	Leak of Traffic During System Shutdown and Reboot	276
7.4.2	Heartbleed	277
7.4.3	Crawling	278
7.5	End of the Chapter Questions	278
7.6	End of the Chapter Solutions	279
8	Fingerprint	283
8.1	What is Fingerprinting?	283
8.2	Narrowing Down: How Characteristics Lead to Identification	284
8.3	Does This Fingerprinting Have Any Positive Uses?	285
8.3.1	Detecting Suspicious Behavior	285
8.4	Behavioral Fingerprinting	286
8.4.1	Authorship Identification	286
8.4.2	Coding Style	286
8.4.3	How You Type	286
8.4.4	How You Use Your Phone	286
8.5	Device Fingerprinting	287
8.6	Browser Fingerprinting	288
8.6.1	User-Agent (HTTP Header)	289
8.6.2	Accept Header	289
8.6.3	Extensions and add-ons	289
8.7	Solutions	290
8.7.1	Blending In	290
8.7.2	Randomizing	291

8.7.3	Browser Isolation	291
8.7.4	Reading List	292
9	Web Browsing	295
9.1	Web Browser	295
9.1.1	Chromium-based or Non-Chromium based?	296
9.1.2	Extensions and addons	297
9.1.3	Search Engines	297
9.1.4	Reading List	298
9.2	Deepweb and Darkweb	298
9.2.1	What is the Deep Web?	298
9.3	The Onion Router (Tor)	299
9.3.1	Tor comes in	299
9.3.2	Timing Attacks	301
9.3.3	Dangerous behaviors lead to de-anonymizing	303
9.3.4	Reading List	309
9.4	Metadata	309
9.4.1	What to do?	311
9.4.2	Reading List	312
9.4.3	Silk Road	312
9.4.4	Reading List	315
9.5	Behavior on Social Media	315
9.5.1	Registration	315
10	Security in Action	319
10.1	Log Analysis	319
10.2	Bug Bounty	320
10.3	End of the Chapter Questions	324
10.4	End of the Chapter Solutions	324
11	Online Scams	327
11.1	Ponzi Schemes	327
11.1.1	How They Deceive You	327
11.1.2	If We Got Scammed, What Should We Do?	336
11.2	A Common Misconception	338
11.3	Why Do Nigerian Scammers Say They are From Nigeria?	338
11.4	Illegal Activities (Mainly Illegal Gambling)	339
11.4.1	Why Illegal Gambling Sites Shouldn't Be Compared to Legal Betting Sites	339
11.5	End of the Chapter Questions	340

Contents

11.6 End of the Chapter Solutions	340
---	-----

About

Who am I?

I am currently a university student at Shiraz University, one of the top five universities in Iran, and I am proud to hold the rank of fourth among my peers in my cohort.

My journey into serious writing and editing began when I took on the role of student editor for Mobtakeran Publisher. This experience ignited my passion for the written word and led to several publications, including a book titled "Introduction to Python," which is available in Persian on my GitHub profile.

In addition to my writing activities, I am teaching a course called "Online Security and Privacy Workshop" in our department at the university. This is my third time delivering this course, and it has inspired me to compile the related content into a comprehensive book.

[Github](#) | [Telegram Channel](#) | [Youtube](#)

What is this book/course about? Why read this first?

There are numerous books on security, but they often fail to cover the hacker's mindset with sufficient attention to fundamental concepts, a gap I aim to fill. My book is designed as a foundational text that explores the essentials of privacy, security, and anonymity. It is intended to be the first resource for newcomers to the field. Many existing books and resources skip the basics, which makes it difficult for beginners to truly grasp the material. This book seeks to bridge that gap by providing a comprehensive introduction.

Bridging the Gap

I have observed that many security books often skip fundamental concepts, making it difficult for newcomers to fully understand the material. This realization inspired me to write an introductory book that thoroughly covers the

essentials of privacy, security, and a touch of anonymity for those just starting their journey in this field. When I began learning about security, I relied on blogs, but I struggled to find high-quality resources. My aim is to bridge that gap.

Fostering Critical Thinking

With my writing experience, I aim to present the fundamental concepts in a way that fosters critical thinking and helps readers develop a hacker's mindset. I do this by introducing ideas that are relevant across various domains, rather than confining the discussion to a single area. Additionally, I utilize a question-and-answer approach in my teaching, which I find to be an effective method for enhancing understanding and fostering intellectual growth.

Profound Understanding

How can someone read a security concept in a computer and understand it profoundly without having a hacker's mind? People often think they understand a concept just by reading it; however, they are just memorizing it. If something new happens, they are less likely to create a connection between a past event and the new information.

- **But we can achieve this hackery mind by learning more and more.**
- **OK! But think about a book which teaches you this from the beginning? Isn't it going to be really helpful?**

Unlike other courses that focus on a single technology or tool, this course/-book builds your knowledge from the ground up, starting with hardware. This layered approach ensures you understand the core principles, which can then be applied to any system, regardless of its specific applications or protocols. This is the key to developing a truly flexible and adaptable hacker mindset.

I begin with hardware, teaching you how it works and then providing an overview of where things can go wrong and how. We will then examine some primitive solutions (perhaps common wrong answers), then some better solutions, and finally how to solve them correctly (this is less covered in this book, as these concepts are often described in hundred-page documents, which I will cite for your further reading). Afterwards, I will go layer by layer to help you develop this hacker's mindset in different contexts.

For this purpose, we do not deal with standard functions or developer documentation. We want to delve deeper into the fundamentals of how to think like a hacker. I have gathered knowledge through reading appropriate books, numerous articles, and credible documents (all referenced in my book through citations) to present this comprehensive text.

What I bring to the table?

- Separating personal life from academic life ensures that any previous emotions or biases do not affect my lecturing performance.
- I possess deep knowledge of security, privacy, and anonymity concepts, gained through extensive reading of books, articles, and credible documents.
- I have teaching experience, notably having conducted the "Online Security and Privacy Workshop" course for three consecutive years at Shiraz University.
- I am the first to arrive in class, not the last to enter and set up the necessary tools, including the projector.
- You can ask me questions directly through Telegram.

Expectations and Grading

What I expect from you?

- Attend in classes and elaborate on topics.
- Studies show that after just 24 hours, your memory loses a considerable amount of what you've learned, unless you review it. With each review, you maintain the learning and it is transferred into longer-term memory. Therefore, I require you to review the topics after one day, then after 3 days, then after a week, and then after a month.

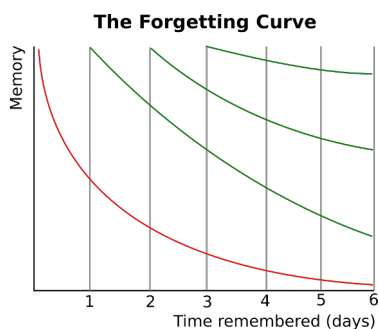


Figure 1: Forgetting Curve. Src: Icez, Wikipedia, public domain

The grading consists of 3 parts (total of 20):

1. **Class Activity (6 points):** Attendance + participation in class discussions. This course is designed for a 10-week, 1 hour 40-minute period. As a result, I require you to attend at least 9 sessions. Beyond that, your grade will be decreased by 0.75 points for each session. (If less sessions are held, the required attendance will be adjusted accordingly.)
2. **Exam (9 points):** Solely based on class lectures.
3. **Research/Project (5 points):**
 - A research topic is given to you (often an overview of privacy features and challenges in apps or protocols based on what you have learned).
 - Creating slides based on the course notes.
 - Knowledge improvements based on the reference book.
 - Students are welcome to propose self-chosen projects that align with the course objectives, showcasing creativity and relevance in exploring innovative solutions to contemporary challenges in online privacy and security.

Note 1: All projects are required to be accepted by me before starting.

Note 2: The project can be done in groups of up to 3 students.

Note 3: The project must be submitted by the end of the 9th week.

Note 4: Three types of assistance are provided for grading: one is **often** adding a specific number of points to your exam, the second is awarding additional points to those who perform better in class activities, and the third is calculating your grading based on the below formula:

$$Max((C6, E9, P5), (C6, E6, P8))$$

Why This Book Doesn't Dive Deep into Practical Perspectives

Security is a unique topic that differs significantly from others. For instance, if you want to learn about concept X in Python, there are countless resources available that are sufficient for your implementation and generally work well. However, the crucial question is: are they based on secure-by-design principles? The answer is often no.

The challenge in security lies not only in knowing how to do things correctly but also in understanding how systems can fail. This dual requirement makes the subject particularly complex. Let me quote a paragraph from the book *Cryptography Engineering*[1]:

“ As a colleague once told Bruce: “The world is full of bad security systems designed by people who have read Applied Cryptography.” That makes this a very dangerous book. Some people will read this book, and then turn around and design a cryptographic algorithm or protocol. When they’re finished, they’ll have something that looks good to them, and maybe even works, but will it be secure? Maybe they’ll get 70% right. If they’re very lucky, they may get 90% right. But there is no prize for being almost right in cryptography. A security system is only as strong as its weakest link; to be secure, everything must be right. And that is something you simply can’t learn from reading books.

— Bruce Schneier

Security, privacy, and anonymity are complex topics.

“ Anyone, from the most clueless amateur to the best cryptographer, can create an algorithm that he himself can’t break.

— Bruce Schneier

We often design systems in a way that gives us an illusion of security, but the truth is that no one can anticipate all potential problems. Practical implementations typically require extensive reading, including multiple books and papers, as well as years of experience. This is evident from the fact that NIST recommendations are developed by teams of experts rather than a single individual.

I do not claim to be an expert in practical applications, nor do many who assert such expertise. Therefore, I will not attempt to explain how to design specific systems. Most of these topics require hundreds of pages of reading and in-depth knowledge, which is beyond the scope of this book.

It is essential to verify anything you read. To support this, I strive to provide the reasoning behind my statements and cite relevant papers and books. I welcome corrections for any errors you may find or suggestions for improvement. In contrast to many other documents, I pay close attention to detail. For instance, you may encounter someone making recommendations based on NIST or BSI documents, but you might not realize that these documents often contain important assertions at the beginning. For example, BSI in one of its documents states¹:

¹Section 1.1. Security Objectives and Selection Criteria, Page 18, BSI TR-02102-1: “Cryptographic Mechanisms: Recommendations and Key Lengths” Version: 2024-1: <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.html>

“ The realisation of concrete implementations is not considered here.

— *BSI*

I've seen a lot of people on Twitter, Instagram, and other platforms who claim to be experts in security and privacy, but they often overlook this crucial detail. They may even say, "BSI says to use AES!" without considering the implications of the statement or assertions made in the document. This is a common issue in the field of security, where people often make claims without fully understanding the context or the limitations of the information they are presenting.

I *try* to avoid such oversights and provide you with the complete picture. If you find any errors, please let me know. I am always open to feedback and suggestions for improvement.

License

This book is licensed under a Creative Commons Attribution-NonCommercial 4.0 International License. You are free to share and adapt the content for non-commercial purposes, provided you give appropriate credit to the author. For more information, visit <https://www.creativecommons.org/licenses/by-nc/4.0/>.

Contents

Donation

Why do you pay for ice cream? Because there are costs associated with producing the milk, as well as expenses for the people involved in the production process.

In the digital world, applications and content are also created by individuals who invest their time and effort. Therefore, using these resources naturally incurs a cost. **Just because something is not physical does not mean it has no associated expenses.**

Free services often rely on various revenue models:

- The provider values privacy and receives funding from organizations. In some countries, when you engage in nonprofit activities, you can request funding by stating that you are helping people for free and need financial support. Thus, you may receive a portion of public revenue or taxes.
- Advertising.
- Donations (which we discussed at the beginning of the book).
- Collecting and selling data to other companies. This is why we say, "If you don't pay for the product, you are the product!"

For example, you might say, "Hey, you who are providing free education are dedicating your time and effort to improve society. I appreciate your work and would like to contribute a certain amount to support you."

Alternatively, you might say, "Firefox, you are creating a free browser that respects our privacy and doesn't exploit our data like other browsers do. I see your work as beneficial to the community, so I want to donate a certain amount to you." Essentially, financial support from individuals is what we call donations. (Donation is not limited to money; it can also include resources like servers, bandwidth, and so on.)

Many free services worldwide operate on this donation model. For instance, there are nonprofit organizations where volunteers pick up trash, plant trees, and so on. Their living expenses are also covered by donations and financial support from the public.

When you want to use Microsoft Office, such as Word, PowerPoint, or Excel, you have to pay for the software. The price for Microsoft Office 2021 for home use is over \$100, which not everyone can afford! This is where an alternative comes in. Some individuals voluntarily create free and open-source software, so if I need to open a file, I don't necessarily have to buy Microsoft Office; I can open that file for free. Additionally, these alternatives often strive to respect our privacy more! One such option is LibreOffice. A group of concerned individuals came together to create a program that allows us to open our files without cost.

Where does their income come from? Donations. They might say, "Did you enjoy using my program? Did you appreciate the good work I did? If you want, feel free to support us with whatever amount you can!"

Below are some examples of donation pages from well-known free and open-source projects:

Donations are among the most noble forms of support because they make software accessible to everyone, allowing individuals to contribute financially as they are able. We need to learn to spend money on our digital needs just as we do in the physical world. After all, content creators and developers have their own lives and expenses. Just as you receive a salary, they also need to earn a living. If we don't support them, they may have to sell their programs, which could limit access for many people. They might even compromise their values and start collecting and selling our data.

NOTE

If you enjoy my writing, you can support me with your donation.^a Donation really helps me continue my work and write more content. Thank You :)

^a<https://github.com/Mohammad-Kamal-mk/Books?tab=readme-ov-file#donate>

1.0 An Introduction to Cybersecurity for a 1.5 Hour Talk

This chapter is designed to provide a brief introduction to cybersecurity and privacy concepts, suitable for a 1.5-hour talk. It covers the fundamental ideas and principles of cybersecurity, privacy, and anonymity, with a focus on understanding the importance of these topics in today's digital world.

Read this chapter to gain a basic understanding of cybersecurity concepts, including the motivation behind cybersecurity, and key terms in the field. This chapter is specially designed to make you motivated.

1.1 The Motivation

Why should we care about cybersecurity?

- **Data is valuable:** Data is the new oil. It is the most valuable asset in the digital world. Companies and individuals collect and store vast amounts of data, including personal information, financial records, and intellectual property. We later find out that even people's destinies can be changed by knowing how to interact with them (which comes from data collection).
- **Cybercrime is on the rise:** Cybercrime is a growing threat, with cybercriminals using increasingly sophisticated techniques to exploit vulnerabilities in systems and networks. The cost of cybercrime is estimated to be in the billions of dollars.

1.2 Prerequisites

The prerequisites are described here in a non-technical way. The goal is to provide a basic understanding of the concepts. If you are familiar with these terms, you can skip this section.

1.2.1 Data

In general, data refers to information—essentially, everything we know. For example, weather information is considered weather data. The details of students' grades and scores are data related to their academic performance. In fact, anything can be data! Even the words I'm using right now are a form of data.

Everything on a computer is stored as data, just in different forms. For example, consider an image. How does a computer recognize that it's an image? Well, behind that image, there is a set of information stored—specifically, the data for each pixel (which are the tiny units you can see when you look at a monitor from a few centimeters away). The computer uses this information to determine the color of each pixel, thus rendering the image.

For instance, take a look at the image below:

Address	Hex	Decoded Text
00000000	D7 A1 34 D7 4B AF 15 93 88 D2 B6 07 D4 7C 5D 49	4 K I
0000000C	AA 95 41 30 49 92 D4 8C 3D 88 53 DF 55 9D F6 6C	. A 0 I . . . + S U . l
00000018	C6 4F D4 BD 00 49 1A 06 56 06 FE 02 7C A8 D0 0B	0 . I . V . . .
00000024	69 C2 78 22 93 E0 5D 0D 88 F6 D8 89 8D D8 2A 02	1 x " . .] m . . . x
00000030	61 23 88 23 88 1E 64 0B 57 30 7F 11 1B 13 F7 8F	a # . . d W 0 . . .
0000003C	35 DA 0C 0E 0A F5 11 59 15 17 13 59 F2 80 62 25	5 Y . . Y . . b s
00000048	22 80 79 2C F5 6F CE 07 6D 4F A2 6C F7 C4 36 AE	" y . . o . m 0 l . . 6
00000054	61 27 E2 F1 B1 74 18 07 50 A6 F1 C0 EF 88 5D 6B	a ' . . t . P . . .] k
00000060	BE FE 36 C0 45 74 DF 21 0C 05 6D 0C FC 9D E8 21	6 E t . l m . . . i
0000006C	89 78 1B D7 31 0E F8 31 F8 FE 36 AE 41 92 2A D5	x 1 . 1 . 6 A + .
00000078	AB A9 D8 92 24 A9 3A 30 00 BF A2 03 36 43 9E 23	\$: : . . 6 C #

Figure 1.1: Binary data behind a picture

- If it's binary, why isn't it just zeros and ones?
→ You're right! This is another way of representing numbers, called hexadecimal. But that's not our focus here. Just know that the computer sees these numbers and understands how to display the image accordingly.

1.2.2 Instruction/Command Execution

In general, we give instructions to the computer, telling it to do this now, or that there, and so on. When the computer carries out our commands, we say that the instructions have been executed.

Everything on a computer operates through the execution of commands. For example:

1. Get a password.
2. Check if the password is correct.
3. If it is correct, allow access.
4. Otherwise, display an error message.

All of these are commands. This is exactly how programs work; it's all about executing instructions. So, when we say a program has run or commands have been executed, that's what we mean.

1.2.3 Operating System (OS)

Think of a big restaurant. You have many robots that are chefs. If two robots want to access the refrigerator at the same time, they will crash. So you need a manager to manage them—deciding when to give access to the refrigerator to one and when to give it to another. This is the job of the operating system.

You have hardware. Programs need to access the hardware. But if two programs want to access the hardware at the same time, what should we do? When should we give the CPU to one and when to another? This is the job of the operating system.

1.2.4 The Internet and Server

How does sending a letter by mail work? You write a letter, put it in an envelope, and send it to the post office. The post office then delivers the letter to the recipient. The Internet is similar to this process, but instead of letters, we send packets of data. These packets contain information, just like letters do. The Internet is a network of computers that communicate with each other, allowing us to send and receive data (including emails, files, and web pages) over long distances.

There are two main ways of sending data:

- peer-to-peer (P2P): Everyone can send data to anyone else. Do we really use the Internet this way? No, we use a client-server architecture.
- client-server: A computer is designed to give services to other computers. For example, when you want to access a website, your computer (the client) sends a request to the server hosting that website. The server processes the request and sends back the requested data (like a web page) to your computer.

Have you noticed the URL at the top of your browser? For instance, it might say: `https://en.wikipedia.org/wiki/Ariobarzanes_of_Persis` This means that, for example, Firefox is being instructed to go to the server of `en.wikipedia.org` then request the “Ariobarzanes_of_Persis” page from the “wiki” folder. The server will then respond with the requested data.

1.2.5 Security Vulnerability

Suppose you have fastened your bike with a lock and chain. If someone wants to steal your bike, they must break the lock or chain. Where does the chain break? At the weakest link (where it is vulnerable to attack). Similarly, in the digital world, security is only as strong as its weakest link. No matter how robust your security measures are, if there is a weak link, attackers will exploit it. This is exactly what hackers do: they find the weakest link and exploit it to gain unauthorized access to your data. To ensure security, you must secure all

1.3 Security

the links, including hardware (e.g., electronic devices), operating systems (e.g., Windows, Linux, etc.), networks (e.g., Wi-Fi, LAN, etc.), applications (e.g., browsers, email clients, etc.), and users (e.g., you and me). This is the job of security experts.

Like a broken window that thieves use to enter your house, a security vulnerability (or a security bug) is a point that hackers exploit to gain access to your system. This is why it is recommended to update the software you use. Programmers regularly search for vulnerabilities and fix them through updates.

Have you ever noticed that whenever you update WhatsApp, the app doesn't change much? That's because WhatsApp mostly just fixes vulnerabilities rather than adding many new features. In contrast, Telegram frequently introduces new features.

1.2.6 Encryption

Imagine I want to write a letter to someone. Well, it needs to be sent by mail. How can I be sure that the postal worker won't open it to read it? There's no guarantee that someone won't read it along the way!

This is where the concept of encryption comes in. It means that I can write a message in a coded form that only I and the recipient can understand. For example, what do you think the following text means?

uoy era woh

If you read it backwards, you'll see that it says:

how are you

This is a simple example of encryption. The message is changed in a way that only the recipient (who knows how to decode it) can understand. This is the basis of encryption. It's like a lock. You have the key, and only you can open it. If someone else tries to open it, they won't be able to.

1.3 Security

1.3.1 Definition

In simple terms, security is like a lock on a door. It prevents unauthorized access to your data—meaning anyone you don't want to have access. Security ensures that your data is not tampered with or stolen. It protects your information from unauthorized access, disclosure, or modification. For example, you certainly wouldn't want your bank account data to be accessed or altered by someone else. Even if that person couldn't access the actual data, they could still damage it, making it unusable.

Security is mainly defined by three principles, known as the CIA Triad:

1. **Confidentiality:** It ensures that the data is only accessible to authorized users.
2. **Integrity:** It ensures that the data is not tampered with.
3. **Availability:** It ensures that the data is available when needed.

1.3.2 Hardware Security

Side-channel Attacks

Let's start with a real-world example.

Have you seen in movies when someone tries to open a safe by placing a stethoscope on it? They listen for the sounds of the tumblers inside the safe to determine when the correct combination is found. This is similar to how a door lock works.

A door lock consists of several pins attached to springs. When the key is inserted, the pins move up and down so that their cut sections align with the rotating part of the lock. When the key is turned (because the pins are positioned correctly), the lock opens.

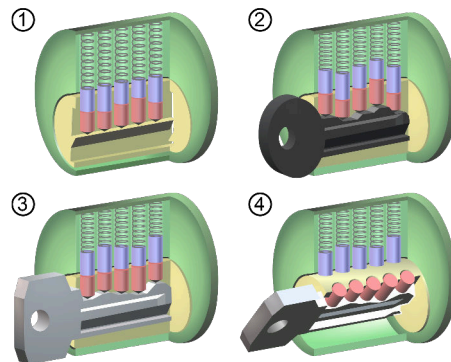


Figure 1.2: How a lock works

When they place the stethoscope on the safe, they are essentially listening to see which pins move up or down and where they need to be positioned. When a pin is correctly aligned, it makes a tiny "click" sound, indicating that they've found the right position. They listen to each pin one by one, adjusting them until all are in the correct position.

Now, imagine the safe's combination is four digits. For each digit, they test numbers from 0 to 9 to see which one produces that specific sound. Once they hear the sound for the first digit, they move on to find the next three.

Instead of randomly guessing the combination, which would require 10,000 attempts (from 0000 to 9999), they can use the stethoscope to listen for the

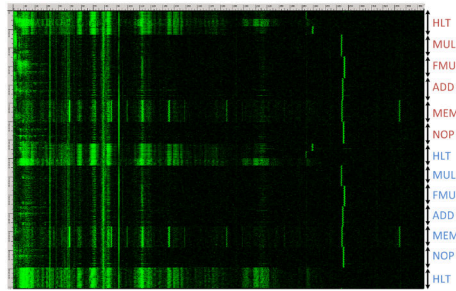


Figure 1.3: “Acoustic measurement frequency spectrogram of a recording of different CPU operations. The horizontal axis is frequency (0-310 kHz), the vertical axis is time (3.7 sec), and intensity is proportional to the instantaneous energy in that frequency band.” Image and description from [2]

clicks. They use their knowledge of how the safe works and listen for the sounds to extract information from the system.

Now, let’s translate this into the computer world:

The most fundamental aspect of a computer is its hardware. Everything runs on hardware. If the hardware is not secure, it’s like having a poorly built foundation for your house; eventually, it will collapse, and your data could be compromised due to hardware vulnerabilities.

Do you agree that hardware consists of electronic components? Do you also agree that electronic components emit noise and sound when they operate as electricity flows through them?

I can use special devices to capture these noises and sounds and identify patterns. For example, I might say that if the sound is like this, it means the CPU is doing this operation. If the sound is like that, it means the CPU is doing that operation. By analyzing the sounds, I can determine what the CPU is doing and what data it’s processing. For instance, I could potentially discover someone’s password as they type it in! [2]

Or, for example, by analyzing the amount of energy consumed by the CPU and the differences in energy usage for various tasks and inputs we provide to the system, I could gain insights into what’s happening inside!

These attacks are called **side-channel attacks**. Forms of side-channel attacks include:

- **Timing attacks:** These attacks exploit the time it takes for a system to respond to different inputs. By measuring the time it takes for a system to perform operations, an attacker can infer information about the data being processed.
- **Power analysis attacks:** These attacks analyze the power consumption of a device during cryptographic operations. By observing variations in

power usage, attackers can extract secret keys or sensitive information.

- **Electromagnetic attacks:** These attacks capture electromagnetic emissions from devices to gather information about their internal operations. This can include data being processed or cryptographic keys.
- **Acoustic attacks:** These attacks use sound waves emitted by devices to extract information. For example, the sounds produced by a keyboard can be analyzed to determine what keys were pressed.
- **Optical attacks:** These attacks involve capturing light emissions from devices, such as LEDs or screens, to gather information about their state or the data being displayed.

What does a security expert do? Their role is to prevent such attacks. For example, they might use special materials to shield the device from electromagnetic emissions or design the hardware in a way that minimizes power variations during operations to make it harder for attackers to extract information.

You may have heard of the *Trusted Platform Module (TPM)*, a chip required for Windows 11. This chip is designed to prevent certain physical attacks. Secret keys are stored in this chip, and it is designed to be tamper-resistant.

Physical Access

The most important thing to remember is that physical access equals being hacked. If I have physical access to your computer, I can do anything with it.

I can install malware using a USB flash drive or download it from the Internet. I can change your sensitive settings.

For this reason, high-security servers are often kept in locked rooms. Only authorized personnel can enter.

A security professional might evaluate the server room's physical safeguards by testing lock-picking techniques or employing social-engineering methods to convince staff to grant access.

Another problem that may occur is about deleting files. You delete a sensitive file and think it's gone. But it is not! Only the file's pointer in the operating system (OS) is deleted. This means that the OS no longer knows if a file is there and where it is. But if we scan the disk with special software, we can find the file and recover it. It's not that hard. Files have specific structures. For example, opening a file with a text editor reveals its structure. Software can search for these structures and recover them.

Wow! How can we prevent it? Have you ever tried to cover up something you've erased by scribbling over it with a pencil, writing random things to hide what was originally there? It's similar in the world of computers. When you understand how hardware writes data to a hard drive, you can create algorithms that repeatedly write and erase data in the areas where files have been deleted, making it difficult to recover the original file!



Figure 1.4: Physical security

Yes, there are researchers who have developed these algorithms and created software that implements them. Perhaps one day you will be one of them or be one who give recommendations for secure deletion of files.

Career Opportunities

For example, you design the hardware in such a way that information extraction is not possible.

- What is needed for this part?

- Understanding hardware! As long as you don't know how computer hardware works, how the components interact, and how information is exchanged, how can you understand its security? So, the first step is to gain knowledge and understanding. (You need to comprehend how the CPU operates, the methods of data transfer between the CPU and other components, protocols for data transfer and how to bypass the currently implemented security measures and where potential data leaks might occur.)

There are no shortcuts to becoming a cybersecurity expert. Anyone who tells you to just learn this or that to become a hacker is misleading you! Security means having an understanding of:

1. How a system works.
2. How to make it malfunction or finding a way to extract information from a functioning system.
3. Fixing the problem once you identify it.

"Illegally accessing Wi-Fi or social media accounts is unethical and offers no future. Right now, you might be young and seeking excitement, thinking, "Wow, I did this cool thing!" But it's not impressive! These activities won't provide you with a sustainable livelihood. You need to find a real job—one that will support you when you're forty years old. At that point, no one will be covering your expenses; you'll have to take care of yourself! Hacking Instagram or Wi-Fi is trivial and illegal, with no future. Instead, focus on gaining the knowledge required to become a security engineer—an ethical and well-respected profession.

So, videos that claim to teach you how to hack Instagram or have titles like "Become a Hacker" or "Let me teach you how to hack a website" are likely just offering a few exciting tidbits that won't lead to a real career. Instead, take a structured approach and read the right books. Not every book on security is good, especially those in Persian that may have poor translations or are just poorly copied versions of foreign texts without proper explanations.

Yes, the path to becoming a security engineer is long, and you won't get there in two or three months. You always need to stay updated. If you don't keep up, those who do will take your place. You don't need artificial intelligence to replace you; it will be people with more knowledge who will take your spot.

1.3.3 Software Security

Input Validation

Consider the following Python code:

```
1 num = int(input())  
2
```

Listing 1.1: Getting an integer from the user

Explanation: The Python interpreter processes the innermost parentheses first. It takes user input, which is initially a string, and then attempts to convert it to an integer using the `int` function.

Now, suppose a user enters a word instead of an integer. What happens?

The program will throw an error such as:

```
1 ValueError: invalid literal for int() with base 10: 'hello'  
2
```

Listing 1.2: Error when entering a word instead of an integer

In other words, this results in an error, causing the software to crash.

The basic idea behind fuzz testing is to provide invalid or unexpected input data to the system to identify crashes or potential security vulnerabilities (for example, unauthorized access).

Consider the following scenarios:

- If the program asks for a number, what happens if I enter:
 - A negative number?
 - Zero?
 - A very large positive number?
 - A very small (or overly negative) number?
- If the program expects an age between 1 and 100, what happens if I input a value like 200? (The idea is to enter data that falls outside of the acceptable or expected range.)
- If the program requires a selection between options 1 to 5, what happens if I choose an invalid option like -1?
- If the program expects a website URL (for example, `https://example.com`), what happens if I mistakenly provide `https:/example.com` (using one slash instead of two)?
- If it requires a file input, what happens if I supply an incorrect or oddly formatted file (for instance, with a strange file extension)?

1.3 Security

The core idea is to operate against the main flow of the program. In security testing, not only should you understand how the system is supposed to work, but you should also explore ways of interacting with it that the programmer did not anticipate. This can reveal vulnerabilities where the system might crack, allow unauthorized access, or even lead to data breaches.

● But my program is large and complex. I can't test all possible inputs!

→ That's true! You can't test every possible input. But you can use your programming skills to write a program that generates random inputs. This way, you can automate the process of testing various inputs against the system. This highlights the importance of coding skills in security testing. The process of giving random inputs is called **fuzzing**.

If you are familiar with languages like C that have `int` and `long int`, consider the following case:

```
1 int res;  
2 scanf("%d", &num1);  
3 scanf("%d", &num2);  
4 res = num1 * num2;  
5
```

Listing 1.3: C code that multiplies two integers

If the product of `num1` and `num2` becomes too large to fit within the range of an `int`, an overflow may occur. It might even underflow, resulting in a very small or very negative number that does not fit within the range of `int`.

Solution?

- Never trust user-provided data! Always validate critical inputs.
- Always check user data for validity before storing it. For example, if a number is expected but the user enters something like `14212a1643`, it is clearly invalid!

Timing Attack

Do you remember what the `==` operator does in Python (or in other languages like Java)? It checked whether the two items provided were exactly equal. For example:

```
1 'abcd' == 'bbbb'  
2
```

Listing 1.4: Python code that checks if two strings are equal

Python starts comparing the two strings character by character, beginning with the first character. It checks whether the first character of the first string is equal to the first character of the second string. If they are not equal, it immediately concludes that the two strings are not equal—even if the rest of the characters might match. There is no need to continue checking, as the

1.3 Security

mismatch of one character is enough to declare that the strings are different; this avoids unnecessarily slowing down the program.

Now, consider the case where the first characters of two strings are the same:

```
1 'abcd' == 'aaaa'
2
```

In this case, Python will check the first character, find it equal, and then move on to the second character. It will continue checking until it finds a mismatch or reaches the end of the string. This means that if the two strings are equal, Python will take longer to check them than if they are not equal.

This type of vulnerability is known as a *timing attack*. An attacker can use this information to determine whether two strings are equal or not based on how long it takes for the program to respond. For example, if the attacker provides a string and receives a response quickly, they know that the first character is not equal. But if it takes longer, they know that the first character is equal and can continue checking the next characters.

To prevent timing attacks, use constant-time comparison functions that execute in the same amount of time, regardless of whether the strings are equal.

Now, suppose the password on a website is the string '123456789'. If I want to guess the password, how might I approach it? I might try the following sequences in order:

'000000000', '111111111', '222222222', ...

Why would this work? If you look at the code, you notice that Python compares the strings character by character. It stops as soon as it finds a mismatch. Now, suppose the test takes 1 millisecond for '000000000', 2 milliseconds for '111111111', and 1 millisecond for '222222222', and so on. The longer duration for '111111111' suggests that the first character is correct and that Python took additional time to check the second character as well. In effect, you would have determined the first character of the password.

Thus, with about 10 tests (for each possible digit) you could determine the first character. In total, for each character in the password you only need about 10 tests.

For a 10-character password, that would be approximately $10 * 10 = 100$ tests instead of testing all 10^{10} possibilities. This is a significant reduction in the number of tests required to guess the password.

These types of attacks are called **timing attacks**. They exploit the time it takes for a system to respond to different inputs. By measuring the time it takes for a system to perform operations, an attacker can infer information about the data being processed.

● Does adding random delays help?

→ No! It doesn't fully mitigate the problem. An attacker can still measure the time and determine the average time it takes for the system to respond. They can then use this information to make educated guesses about the data being

processed.

To prevent timing attacks, use constant-time comparison functions that execute in the same amount of time, regardless of whether the strings are equal. This way, an attacker cannot gain any information about the data being processed based on the time it takes for the system to respond. A simple way to compare is:

```
1 # Assuming the size of both strings is the same
2 def compare_strings(s1, s2):
3     result = 0
4     for i in range(len(s1)):
5         result |= ord(s1[i]) ^ ord(s2[i])
6
7     return result == 0
8
```

Listing 1.5: Python code that compares two strings in constant time

This function compares two strings in constant time, regardless of whether they are equal or not. It uses the XOR operator to compare each character and returns true only if all characters are equal.

A better way to do this is to use *C* and check for compiled assembly code to know the code is actually constant time or not.

To fully mitigate the problem of timing attacks, hardware should be designed to prevent timing attacks. For example, the CPU could be designed to take the same amount of time to perform operations regardless of the data being processed. Here also you need to have a good understanding of hardware and how it works. This is why security experts need to have a deep understanding of both software and hardware.

Timing attacks are not limited to input validation. Let's consider a more complex example. You use WhatsApp, right? When you send a message, it shows a single tick mark, indicating that the message has been sent. When the recipient receives the message, it shows two tick marks. But how does the tick marks appear? When you send a message and the recipient receives it, a special packet is sent to you indicating that the message has been received.

A hacker mindset would be to think, "Hmm, I wonder if I can gather information about the recipient's location based on the time it takes for the message to be delivered." See *Timing Attacks on WhatsApp, Signal, and Threema can Reveal User Location*[3] for more information. ¹

Careless programmers/Faster is not always better

Have you ever heard of the phrase "faster is not always better"? This is especially true in programming. Sometimes, programmers try to optimize their code to make it run faster, but this can lead to security vulnerabilities.

¹Or you may just read the news: <https://cyberinsider.com/timing-attacks-on-whatsapp-signal-threema-reveal-user-location/>

1.3 Security

When you crop an image, many image-editing software programs do not remove the pixels. Instead, they use flags to indicate which pixels are part of the image and which are not. For example (**IEND** is a flag that indicates the end of the image; Computer understand it as the end of the image):

- Original image: `sdklfjsadfjfdslkweruoiIEND`
- Cropped image: `sdklfjsadfjfIENDdslkweruoi`

In fact, the cropped image is not really cropped; it's just a flag that indicates which pixels are part of the image and which are not. This optimization is often implemented to save time and improve performance

But a hacker might think, "Hmm, I see the size of the cropped image is not that much smaller than the original image. I wonder if there would be the original image in the cropped image." So they open the cropped image as binary, remove the flag, and voila! They have the original image.

• Wow! Does this applies to all image-editing softwares?

→ No! Some have this problem.

Similar is true for drawing. For example, if you have a picture of a person and you want to hide their face with drawing on it, the software might not actually remove the pixels; it just added a layer of drawing on top of the original image. This means that if someone knows how to remove the overlay, they can reveal the original image. Similar thing is also true for blurring.

This shows that programmers need to be careful when optimizing their code. They should not sacrifice security for speed. The code should be secure, even if it takes a little longer to run. Security is more important than speed. If the code is not secure, it can lead to serious vulnerabilities that can be exploited by hackers.

Race Condition

Let your CPU has only one core and you have multiple programs running on it. Does the CPU run all of them at the same time? No! It runs one program at a time. But how do we feel like they are running at the same time? It's because the CPU switches between them very quickly. It gives microseconds to each program, and we feel like they are running at the same time. This process is called **context switching**.

This switching is very quick and is done by the operating system (OS). The OS is like a manager that decides which program gets to use the CPU at any given time. It ensures that all programs get a fair share of the CPU's time and that they don't interfere with each other. Now consider the following instructions:

Listing 1.6: Instructions taken for applying a discount coupon code

```
Get the code from user
Check validity
```

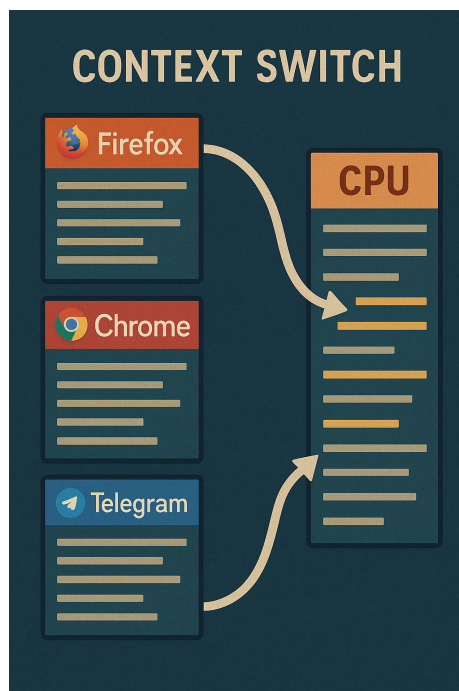


Figure 1.5: Context Switching

1.3 Security

```
Apply code
Validity = False
```

This code is used to check if a discount coupon code is valid. The first line gets the code from the user. The second line checks if the code is valid. The third line applies the code. The fourth line sets the validity to false. Now, suppose two users are trying to apply their codes at the same time. The OS switches between them, and the following happens:

Listing 1.7: How the OS may switch between two processes

```
Get the code from user 1
Get the code from user 2
Check validity of code 1
Check validity of code 2
Apply code 1
Apply code 2
Validity = False
Validity = False
```

or

```
1 if coupon_validation == True:
2     enable_discount() # (I)
3     coupon_validation = False # (II)
4
```

Listing 1.8: Python code that applies a discount coupon code

If two user try to apply their codes at the same time, the switching of the OS can cause a problem. As you can see, OS does a little bit of work on each program before switching to the next one. However, it may cause both code to be applied. Because two process can enter the block before the validity is set to false. This issue is an example of a *race condition*, a type of concurrency vulnerability.

The same issue can occur if two requests to transfer money are sent simultaneously. Consider the following code:

```
1 my_balance = 100 # Get the balance from the bank API (I)
2 transfer_amount = 100
3 his_balance = 0
4 if my_balance >= transfer_amount:
5     his_balance = his_balance + transfer_amount
6     my_balance = my_balance - transfer_amount # (II)
7
```

Listing 1.9: Python code that transfers money

If two requests are sent at the same time, the OS will switch between them. The first request will check the balance and find that it is sufficient. Then, the second request will also check the balance and find that it is still sufficient. Both requests will then proceed to transfer the money, resulting in a double

1.3 Security

transfer. By this, the hacker can transfer more money (200) than they actually have (100).

Have you spotted other potential problems? Yes! Input validation is not done. The code does not check if the transfer amount is valid (for example, if it is a negative number). This could lead to other issues, such as transferring a negative amount (resulting in a positive balance for the sender)!

Race condition may also occur in other scenarios. For example, two account can be created with the same email address.

An attempt to backdoor the kernel

Consider the following code:

```
1 #define __WCLONE 1
2 #define __WALL 2
3 #define EINVAL 1
4 int options = 2;
5 int retval;
6 int currUid;
7
8 int fun()
9 {
10     if (((options == __WCLONE) || (options == __WALL)) && (
11         currUid = 0))
12         retval = -EINVAL;
13 }
```

Listing 1.10: Checking for root access?

This code was an attempt to be added to the Linux kernel.² Can you spot the problem? The code attempts to check if the user is root (user ID 0), but it mistakenly uses a single equal sign (=) for assignment instead of a double equal sign (==) for comparison. This means that instead of checking if the user is root, it assigns the value 0 to `currUid`. As a result, the process that has called this function gets root access!

This is a classic example of a *backdoor*: a hidden vulnerability intentionally or unintentionally introduced, known only to its creator. If this code had been merged into the kernel, it would have introduced a critical security vulnerability, allowing the creator or others aware of the issue to gain root access by exploiting it.

Various tools, such as static analysis tools, are available to detect such issues in code. For example, `cppcheck` is a static analysis tool that can be used to find bugs in C/C++ code. It can detect issues like this one and help prevent security vulnerabilities.

²<https://lwn.net/Articles/57135/>

Out-of-Bound Access

Example 1 Let's return to the code for checking if two strings are equal. What problem do you think the following C code might have (aside from the timing issue we discussed)?

```
1 #include <string.h> // For strlen
2
3 int isPassValid(char pass[], char inp_pass[])
4 {
5     // Assuming pass is a null-terminated string.
6     // strlen(pass) gives the length of the correct password.
7     for (int i = 0; i < strlen(pass); i++)
8     {
9         // If inp_pass is shorter than pass, this can read out of
10        // bounds.
11        if (pass[i] != inp_pass[i])
12            return 0; // Not equal
13    }
14    // This part is only reached if all characters up to strlen(pass)
15    // matched.
16    // It doesn't check if inp_pass might be longer than pass.
17    // A more robust check would also ensure inp_pass[strlen(pass)]
18    // == '\0'
19    // or strlen(pass) == strlen(inp_pass).
20    return 1; // Equal up to length of pass
21 }
```

→ Yes! An “out-of-bounds read.” If the user-provided `inp_pass` is shorter than the actual `pass` (e.g., `inp_pass` has 2 characters and `pass` has 10), the loop continues as long as `i < strlen(pass)`. When `i` exceeds the length of `inp_pass` (e.g., `i` becomes 2), `inp_pass[i]` attempts to read memory outside the allocated space for `inp_pass`. This means you are reading memory locations that do not belong to `inp_pass`, which can lead to crashes, incorrect behavior, or information disclosure.

Example 2 → To avoid accidentally accessing an incorrect index when working with arrays (or lists in Python), pay close attention to your `while` or `if` conditions. For example, consider this Python code:

```
1 size = 5
2 arr = [1, 2, 3, 4, 5]
3 # This loop intends to print elements in reverse
4 while size >= 0:
5     print(arr[size-1]) # Accesses arr[4] down to arr[-1]
6     size -= 1
7
```

Listing 1.11: Python code that prints elements in reverse

This code has a problem. Let's trace its execution:

1.3 Security

- When `size` is 5, it prints `arr[4]` (which is 5). `size` becomes 4.
- When `size` is 4, it prints `arr[3]` (which is 4). `size` becomes 3.
- When `size` is 3, it prints `arr[2]` (which is 3). `size` becomes 2.
- When `size` is 2, it prints `arr[1]` (which is 2). `size` becomes 1.
- When `size` is 1, it prints `arr[0]` (which is 1). `size` becomes 0.
- When `size` is 0, it prints `arr[-1]`. In Python, `arr[-1]` is a valid way to access the last element of the list (which is 5). `size` becomes -1.

The loop then terminates. The output sequence is 5, 4, 3, 2, 1, 5. So, the value 5 is printed twice. While Python handles negative indexing by wrapping around, in languages like C or C++, accessing `arr[-1]` would be an out-of-bounds memory access, likely causing a program crash or undefined behavior. The condition `while size >= 0` combined with `arr[size-1]` is tricky. If the intent was to iterate through valid positive indices from `size-1` down to 0, the condition should be `while size > 0`. Always be careful to define your loop conditions and array/list indexing correctly to ensure you only access memory within the allocated bounds.

Example 3 Now, what problem do you think the following C code has? (The logic is: it takes an integer `length` as input from the user, then reads `length` characters from the input and stores them in the `str` array.)

(P.S.: For this example, assume `scanf` and `getchar` are functioning correctly as input mechanisms, and disregard other potential security concerns related to them.)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int length;
7     scanf("%d", &length);
8     getchar(); // To consume the newline character left by scanf
9
10    // Allocate memory for 'length' characters
11    char *str = (char *)malloc(sizeof(char) * length);
12    if (str == NULL) {
13        // Handle memory allocation failure
14        return 1;
15    }
16
17    int i;
18    for (i = 0; i < length; i++)
19    {
20        str[i] = getchar(); // Read characters
21        if (str[i] == EOF) { // Handle premature end of input
22            break;
```

1.3 Security

```
23     }
24 }
25 // Attempt to null-terminate the string
26 str[i] = '\0';
27
28 // printf("String: %s\n", str); // Using the string
29 free(str);
30
31 return 0;
32 }
33
```

Listing 1.12: Reading a string from user input

→ The problem here is an “out-of-bounds write.” The code allocates memory for `length` characters using `malloc(sizeof(char) * length)`. The `for` loop then reads `length` characters into `str[0]` through `str[length-1]`. After the loop completes (assuming no EOF is encountered prematurely), the value of `i` will be `length`. The line `str[i] = '\0';` attempts to write the null terminator at `str[length]`. However, the allocated memory block only has valid indices from 0 to `length-1`. Writing to `str[length]` means writing one byte beyond the allocated buffer. This can corrupt adjacent memory, lead to unpredictable program behavior, or cause a crash. To correctly store `length` characters plus a null terminator, you need to allocate `length + 1` bytes: `char *str = (char *)malloc(sizeof(char) * (length + 1));`.

1.3.4 Network Security

DDoS

One type of attack that can be launched against a server is a DoS/DDoS attack, which stands for Denial of Service/Distributed Denial of Service. As the name suggests, it aims to deprive users of services.

Imagine you are an employee assisting clients. When someone approaches, you greet them and ask how you can help. Now, if someone wants to disrupt your performance, they could send a thousand people to surround you, and each one would just say hello and shake your hand before leaving. This would create such a crowd that you wouldn't be able to attend to your actual clients.

In a DDoS attack, a similar scenario occurs. When a computer communicates with HTTPS websites, it first sends a request to the server, essentially saying, “Hello, are you there?” The server responds, “Yes, I am here.” This initial exchange is known as a handshake, much like a greeting.

Now, imagine if millions of “hello” requests are sent to the server. The server has to respond to each one, saying, “Yes, I am here.” This flood of requests overwhelms the server, leaving it too busy to handle legitimate tasks. As a result, the website becomes extremely slow and may even become inaccessible. This means that legitimate customers cannot access the service, and you might also

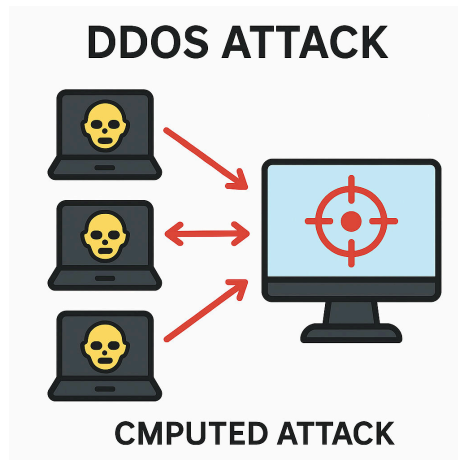


Figure 1.6: DDoS Attack. The attacker might wait for a response from the server.

struggle to maintain a connection with your server! This is why websites often slow down during sales or promotions. Because many people (here legitimate users) are trying to access the site at the same time.

A security engineer can implement several strategies to mitigate DDoS attacks:

- **Traffic Scrubbing Centers:** These are centralized data-cleaning stations where incoming traffic is analyzed. Malicious traffic (like that from a DDoS attack) is identified and separated from legitimate traffic, which is then forwarded to the server. ³
- **Rate Limiting:** Configure servers or network devices to restrict the number of requests a single IP address can make within a specific time frame. This prevents any single source from overwhelming the server with requests. (This is a way to prevent DoS attacks.)
- **Firewalls and Web Application Firewalls (WAFs):** These can be configured to block known malicious IP addresses or to filter out common attack patterns.

Preventing DDoS attacks often involves a combination of these techniques and requires ongoing vigilance and adaptation as attack methods evolve.

³How? The way a real user interacts with the server is different from how a botnet (a network of compromised computers) interacts with it. For example, a real user might take time to read the page, while a botnet might send requests in rapid succession.

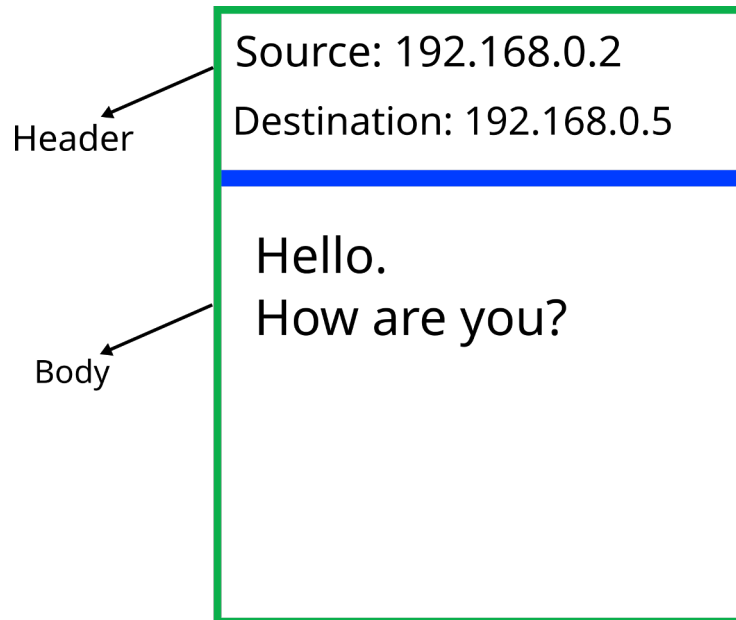


Figure 1.7: A packet

HTTP vs. HTTPS

The internet is a space where everyone is connected. We also know that if I want to send a message to someone in the physical world, I need to go to the post office, write a letter, and specify the sender's and recipient's addresses. In the world of the internet, it's quite similar. Everyone has an address. If I want to connect to Google, I find Google's address and send a message to that address! This address is called an IP address, or Internet Protocol Address.⁴

Each packet has a header and a body. The header includes the sender's and recipient's addresses, the protocol used, connection parameters, and the packet size. (Remember this! We will come back to it later.) The body contains the actual data being sent. The header is like the envelope of a letter, while the body is like the content of the letter itself.

But the problem is that the body (the content) is not encrypted. Thus, anyone who can see the packet (like the postman –the ISP) can read the content. This is a big problem because it means that anyone can see what you're sending and receiving over the internet. This is where encryption comes in. We encrypt the body of the packet so that only the sender and recipient can read it:

⁴That's why it's said that it's better for no one to know your IP address, because it's like knowing your home address!

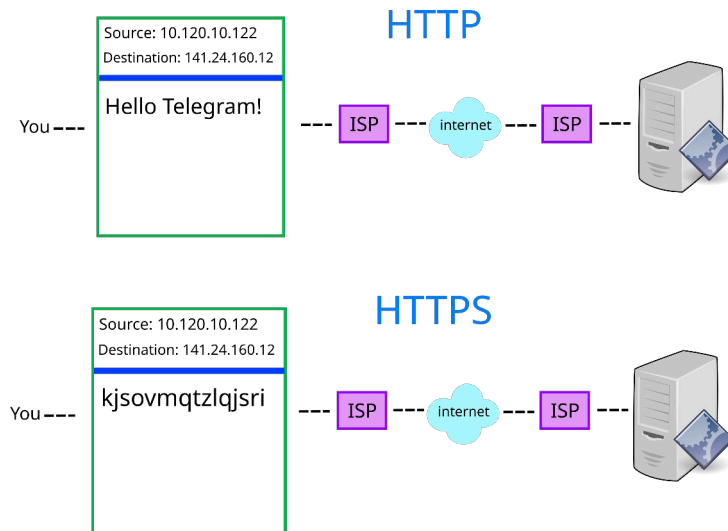


Figure 1.8: HTTP vs. HTTPS

VPN Kill Switch

First let's look at how VPN works: Imagine that your VPN disconnects due to some issue. Normally, this would mean that your data is transmitted outside the secure VPN tunnel, potentially exposing your IP address. To address this, there's a feature called a "kill switch." This feature disconnects your internet if the VPN connection drops, preventing data from being transmitted outside the secure tunnel. It will only allow data transfer once the VPN is reconnected.

However, consider the scenario where I turn off the device. If the program isn't well-designed, the kill switch process might be terminated before the system shuts down. This creates a window of time between the VPN disconnection and the termination of the kill switch process, during which data could be transmitted outside the VPN while the internet is still active.

Alternatively, if you have the "auto-connect" and "launch app on start-up" options enabled, the VPN should start automatically when the system boots up, preventing any data from leaking outside the VPN. However, due to various factors, there might be a brief moment before the VPN establishes a connection, during which data could be transmitted outside the VPN, revealing our real IP address.

In essence, in the realm of security, we focus on identifying vulnerable points and edge cases. We ask ourselves, "What happens if we encounter a sensitive situation at that critical moment?"

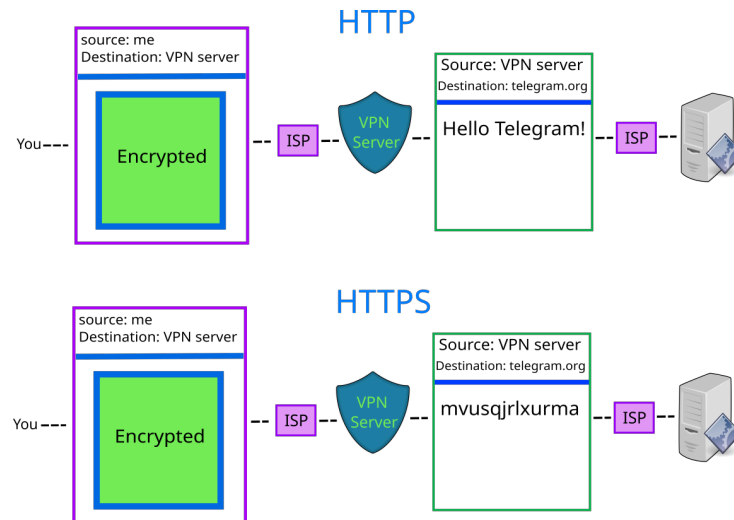


Figure 1.9: VPN vs non-VPN

Heartbleed

Opening HTTPS connection takes time. So, to speed up the process, the server and client keep a connection open for a while. This way, they don't have to establish a new connection every time they want to send data.

The process of keeping the connection open is called **heartbeat** (does the heart of the server beat? Is it alive?). But the problem arises when the server doesn't check the length of the data being sent. For example, let I send a message saying "bird" and I say that the length of the message is 500 bytes. All data is stored in the RAM (including the bird). The server will read the first 500 bytes of the RAM from the location where the bird is stored. But the problem is that the server doesn't check if the length of the data is correct. So, instead of reading only 4 bytes (the length of the word "bird"), it reads 500 bytes. This means that it will read the data that is stored in the RAM after the bird. This data could be anything, including sensitive information like passwords, credit card numbers, and so on.

A security expert would check protocols and find out where things might go wrong. In this field, you need to read and understand how the protocols work.

Log Analysis

A log is a record of events that occur in a system or network. For example, see the following log entry:

Listing 1.13: Log entry example

```
2025-10-01 12:34:56 [INFO] User 'admin' logged in from IP address
```

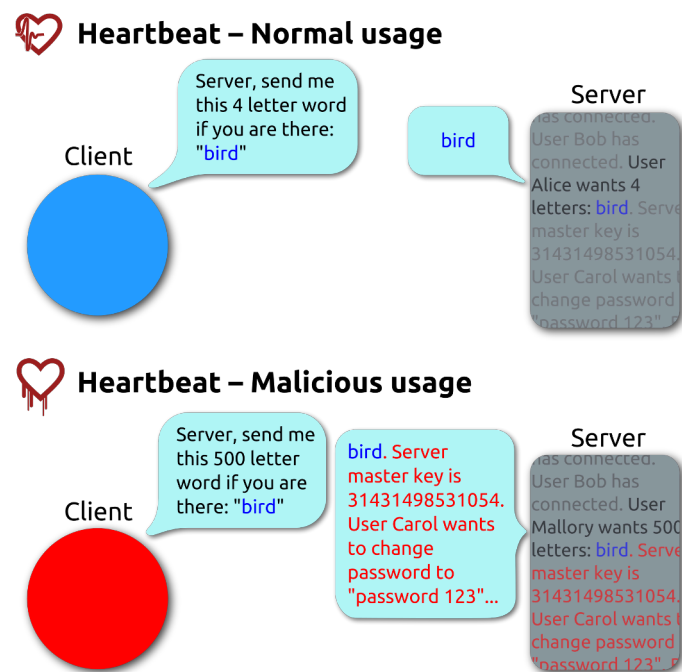



Figure 1.10: FenixFeather, Simplified Heartbleed explanation, CC BY-SA 3.0

1.3 Security

```
2025-10-01 12:35:00 [ERROR] Failed login attempt from IP address
2025-10-01 12:35:05 [INFO] User 'admin' logged Out
2025-10-01 12:35:10 [INFO] User 'admin' accessed file 'sensitive_data.txt'
```

Logs show which files were accessed, when they were accessed, and by whom. This information can be used to identify unauthorized access to sensitive files or data.

Logs can also be used to investigate security incidents after they occur. By analyzing the logs, security teams can determine what happened, how the incident occurred, and what steps need to be taken to prevent it from happening again.

Furthermore, if lots of failed login attempts are detected from a single IP address, it may indicate that someone is trying to attack the system.

A security expert would check write a program to analyze the logs and find out if there are any unusual patterns.

To analyze logs, you need to be able to code and use terminal commands. In many servers, only terminal commands are available. You need to be able to write scripts to automate the process of analyzing logs.

- What are terminal commands?

- When you use a computer, you usually use the graphical user interface (GUI) to interact with the system. However, there is another way to interact with the system using text-based commands. This is called the command line or terminal. In the terminal, you can type commands to perform various tasks, such as creating files, moving files, and running programs.

1.3.5 Conclusion

In summary, security is about:

- Understanding how the system works and how it communicates with other systems.
- What can go wrong.
- How to prevent it from happening.
- If it happens, how to detect it. (Detect a hacker is in the system.)
- How to respond to it. (Isolate the system to prevent further damage, Fix the system and learn from past mistakes.)

Potential jobs in this field include:

- Hardware security engineer
 - Check physical access

1.4 Privacy

- Design secure hardware
- While connecting systems to each other, what security measures related to hardware should be taken?
- software security engineer
 - Code reviewer
 - Writing parts of the code related to security (e.g., encryption, login systems)
- Network security engineer
 - Check how the network is configured
 - Check how the network communicates with other networks
 - Prevent Network Attacks

In your journey, you may also consider becoming a bug bounty hunter. This is a person who finds bugs in software and reports them to the company. The company pays the bug bounty hunter for finding the bug. Bug bounty hunting is an excellent way to learn about security while earning money.

Some argue that bug bounty hunting is not a full-time job and is better suited as a side activity alongside regular employment. Additionally, it is crucial to understand the legal aspects of bug bounty hunting. Some companies have strict rules about what you can and cannot do when testing their software. You need to make sure you understand these rules before you start testing.

I must emphasize that security is a complex subject. Unlike many other areas of computing, results in security often take time to materialize and you need to put in significantly more effort. However, when you do achieve results, you will see your own growth and realize how much more knowledge you have compared to others.

Finally I should say: while the journey in cybersecurity is challenging, the rewards of protecting systems and making the digital world safer are unparalleled.

1.4 Privacy

1.4.1 What Does It Mean by Privacy?

Privacy is the right to control access to your personal information. It means that you have control over who can access your data and how it is used.

- Do you like someone to look at your phone in a public place?
- Do you like someone to read your emails without your permission?
- Do you like to be monitored by someone without your knowledge?

1.4 Privacy

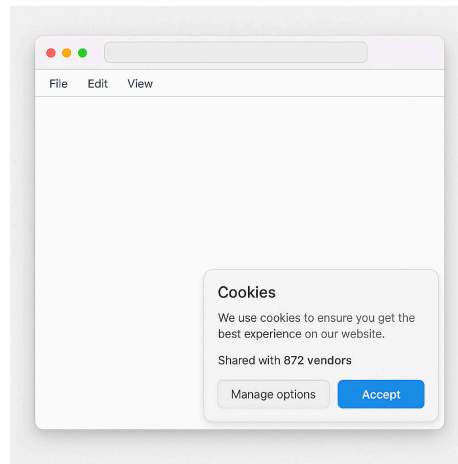


Figure 1.11: Cookie Banner (Example)

- Would you like to sit in the middle of a restaurant (which is the center of attention) or at a corner where less people are looking at you?
- Do you like all of your data being monitored by hundreds of companies? If you don't, why you blindly click on "Accept" button in cookie consent pop-ups?

I don't care. I'm not a criminal. I'm not doing anything wrong. I'm not that important. I am not a celebrity. I am not a politician. Why should I care about privacy? Data collection doesn't affect me.

Large-scale data collection has the potential to influence your future. For example, they know how to give you ads in order to convince you to vote for a specific person. For instance, they know you like cats. They can fabricate fake news targeting individuals who oppose certain topics, such as cats. They can show you that news and make you hate that person. They may subtly promote certain individuals by embedding favorable narratives within published content, often without your awareness. See "Scottish independence debate targeted by Iran-based fake social media accounts".⁵

Even small amounts of data, when analyzed in context, can reveal insights you might never have anticipated. For example, there is a famous story that says those who buy milk in large bottles likely have ancestors who experienced famine. They do this to ensure they always have a supply on hand, driven by their subconscious. Or see *Epigenetic signatures of intergenerational exposure to violence in three generations of Syrian refugees*[4]. Or *Pizza Meter* story⁶

⁵<https://www.heraldsotland.com/politics/19867365.scottish-independence-debate-targeted-iran-based-fake-social-media-accounts/>

⁶A sudden increase in pizza orders at Domino's by U.S. government offices, prior to major events like the invasion of Iraq into Kuwait, etc. See more at: <https://tradesmith.com>.

1.4 Privacy

“ Arguing that you don’t care about the right to privacy because you have nothing to hide is no different than saying you don’t care about free speech because you have nothing to say.

— *Edward Snowden*

The important note is that privacy is not just about keeping your data private. It’s also about having control over your data. You should be able to decide who can access your data and how it is used. It is your right!

In privacy field, we create solutions to protect people’s data. We create tools to help people keep their data private. We fight against data collection and surveillance.

1.4 Privacy

2.0 Quick Introduction to Hardware

This chapter is an introduction to hardware. It is not a comprehensive guide to hardware, but it will give you an overview of the hardware components and how they work together to you be ready for the next chapters.

2.1 What is Hardware?

Any software you run on your computer is executed by the hardware. Hardware is the physical components of a computer system. Simply, anything you can hit with a hammer is hardware. It includes the CPU, memory, storage, and input/output¹ devices (like keyboard, mouse, monitor, etc.). It is the foundation of a computer system. Without hardware, there is no computer system.

2.2 Why We Should Care About Hardware?

Almost any computer science student should have a basic understanding of hardware. Without having a basic understanding of hardware, you won't be able to understand how the software works. You won't be able to write efficient code! Yes efficient codes are not just about algorithms and data structures. They are also about how the hardware works and how you can optimize it for that specific hardware.

2.3 Hardware Components

To make it easier for you to understand, I like to compare computer hardware to a kitchen.

¹A device is called an input device if it receives data from the user and sends it to the computer. A device is called an output device if it receives data from the computer and sends it to the user.



Figure 2.1: HDD vs SSD

2.3.1 Hard Disk Drive (HDD) & Solid State Drive (SSD)

A kitchen needs a storage area to keep its supplies. Similarly, in the world of computers, you can't just have a program like Firefox floating in thin air; it needs to be stored somewhere. This is where components like HDDs and SSDs come in. They are responsible for this storage. Crucially, these components must retain their data even when the power is turned off. They are designed for persistent storage, ensuring that when you turn your computer on again, Firefox (and your other programs and files) will still be there. That's why we use non-volatile memory for this purpose.

HDD has a mechanical part that spins a disk to read and write data. This technology causes it to be slower than SSDs, which use flash memory to store data. SSDs have no moving parts, making them faster and more reliable (HDD hardware can fail due to mechanical issues. If you drop your laptop, the HDD can be damaged, and you may lose your data). SSDs are also more energy-efficient, which is important for laptops and mobile devices. However, they are more expensive than HDDs and the lifespan of SSDs are limited by the number of read and write cycles they can endure. This means that SSDs can wear out over time, especially if they are used heavily. However, modern SSDs have improved significantly in terms of durability and lifespan.

Typical size of these storage devices is in the range of hundreds of gigabytes (GB) to terabytes (TB).

2.3.2 Central Processing Unit (CPU)

CPU is like the chef in a kitchen. It is the brain of the computer. It does the tasks that are given to it by the software. For instance, if a program needs to calculate 2×2 , it is the task of the CPU to do that calculation.

Have you ever heard of CPUs with 8 or 16 cores? It's like having multiple chefs in the kitchen. Each chef can work on a different task at the same time. This means that the computer can do more tasks at the same time, which makes it faster. In computer, this means more programs can be open (to be

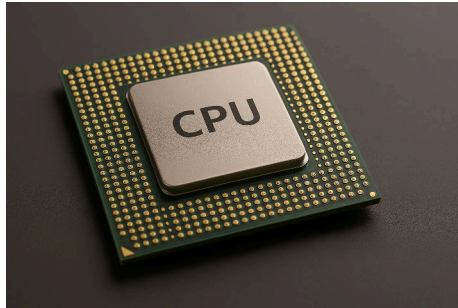


Figure 2.2: CPU

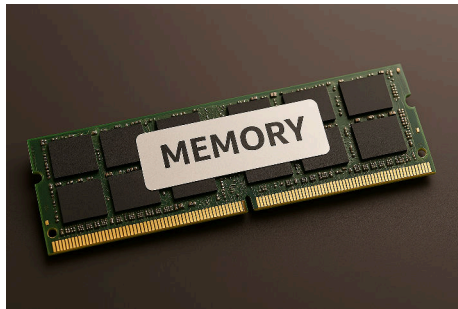


Figure 2.3: RAM

more precise, run) at the same time. For example, you can run a web browser, a text editor, and a game at the same time without any issues.

2.3.3 Random Access Memory (RAM)

Think about a chef again. When a chef wants to cook, do they work directly with all the meat and vegetables in the storage room, or do they bring the ingredients they need temporarily into the kitchen and place them on a shelf? It's the same in the computer world. When the computer turns on and we click on programs, the parts we need to work with are brought into a component called RAM (Random Access Memory), which temporarily holds our data. When we close a program or shut down the system, any changed information is saved back to the hard drive to be permanent.

You might wonder why we don't just use the hard drive. The reason is that RAM, due to its different structure, is much, much faster than a hard drive. RAM also has features that make working with it preferable to working directly with the hard drive for active tasks. The downside is that RAM is volatile memory, meaning it loses its data when the power is turned off. So, we use RAM to store data that we are currently working with, and we use hard drives

2.3 Hardware Components

to store data that we want to keep permanently.

Each program consist of two part: the code and the data. The code is the instructions that the CPU executes. For example, the code for adding two numbers is:

```
1 result = var1 + var2
2
```

Listing 2.1: Python code that adds two numbers

And the data is the information that the program uses. (`var1` and `var2` in the above code.) The data is stored in RAM. When you run a program, the code is loaded into RAM, and the data is also loaded into RAM. The CPU then executes the code and uses the data to perform its tasks.

Typical size of RAM is in the range of 4 to 128 gigabytes (GB). The more RAM you have, the more programs you can run at the same time without slowing down your computer.

2.3.4 Register

Does a chef work directly with all the ingredients on the storage shelves, or do they bring what they need to a table to chop and prepare it? It's similar in the computer world. The small part of a program that needs to be executed (like a line of code) is brought to this "table" (the registers) to be worked on. Registers are very small memory units (around 64 bits in size) where data is temporarily held for processing by the CPU.

Take the following line of code as an example:

```
1 result = var1 + var2
2
```

Listing 2.2: Python code that adds two numbers

When the CPU executes this line, it brings the values of `var1` and `var2` into registers, performs the addition, and then stores the result back in a register. After that, the result is stored in where the `result` variable is located in RAM.

2.3.5 Cache

Our chef gets tired of going to the storage shelf every time they need something. So, we've placed a small box next to their work table where we put items they use frequently or are likely to use soon. This way, the chef first checks this box, and if the item isn't there, then they go to the RAM (the main shelf). We call this box "Cache."

Cache is a very small and very fast memory located close to or within the CPU. It stores frequently accessed data and instructions from RAM. When the CPU needs data, it first checks the cache. If the data is found in the cache (a "cache hit"), it can be retrieved much faster than if it had to be fetched

2.3 Hardware Components



Figure 2.4: A tired chef



Figure 2.5: Happy chef with cache

2.3 Hardware Components

from RAM (a “cache miss”). This significantly speeds up processing because accessing cache is much faster than accessing RAM.

There are typically multiple levels of cache (L1, L2, L3, L4), with L1 being the smallest and fastest, and closest to the CPU cores. L2 is large, but slower than L1, and L3 is even larger and slower, often shared among multiple cores. CPU first checks L1 cache, then L2, and finally L3 if the data is not found in the previous caches. If the data is not found in any of the caches, it is fetched from RAM.

Typical sizes:

- L1 Cache: Tens of kilobytes (KB) per core.
- L2 Cache: Hundreds of KB to a few megabytes (MB) per core.
- L3 Cache: Several MB to tens of MB, shared.

Now you might ask: why not just use cache instead of RAM? The answer is that cache is very expensive to build, and it is very small compared to RAM. And the other reason is that if cache becomes too large, it will become slower.

• I have heard that cache is significantly faster just because it is close to the CPU. Is that right?

→ No. It is mainly because of the structure and size of the cache.

2.3.6 The Bigger Picture

Programs are stored in HDD or SSD. When you turn your computer on, the operating system is loaded into RAM. (Since the operating system is a program, it is also stored in HDD or SSD.) When you run a program, the code and data of that program are loaded into RAM.

Each core of the CPU has its own registers and cache. The CPU fetches instructions from RAM, understands what needs to be done, and what variables and data it needs to work with. It then brings the necessary data into its registers, performs the calculations, and stores the values first in the registers and then in RAM.

For instance, consider the following code:

```
1 result = var1 + var2
2
```

Listing 2.3: Python code that adds two numbers

First CPU fetches the instruction and understands that it needs to add two numbers. Then it fetches the values of `var1` and `var2` from RAM. It brings these values into its registers, performs the addition, and stores the result back in a register. Finally, it writes the result back to RAM in the location where `result` is stored.

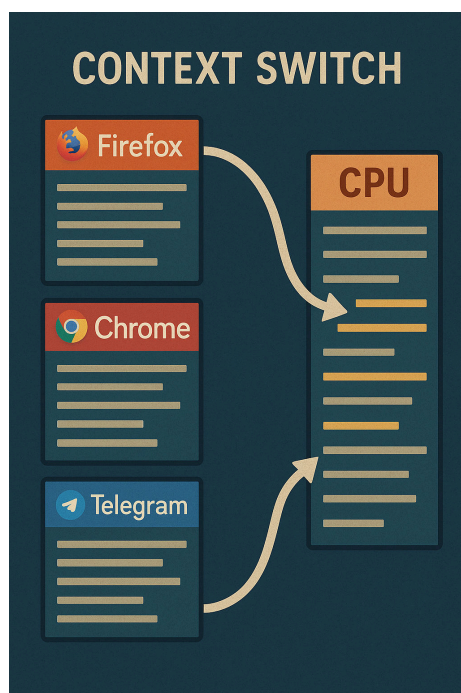


Figure 2.6: Context Switching

For both code and data, the CPU uses hierarchy of memory. The hierarchy is designed to optimize speed and efficiency. The CPU first checks the cache, then the registers, then the RAM, and finally the HDD or SSD if the data is not found in the previous levels. This hierarchy is designed to optimize speed and efficiency.

Each CPU core² has its own registers and cache and can only run one program at a time. But how are we able to run more programs than the number of CPU cores? The answer is that operating system (OS) is designed to do something called **time-sharing**. This means that the OS divides the CPU time among the running programs. Each program gets a small amount of time to run, and then the OS switches to another program. This gives the illusion that multiple programs are running at the same time, even though only one program is running on each core at any given moment.

This switching is done very quickly, so you don't notice it. For example, when you open a web browser and a text editor at the same time, the OS switches between them, doing a little bit of work on each program before switching to the next one. This is called **context switching**.

²Let's forget hyper-threading

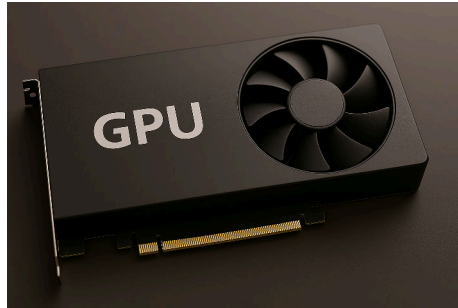


Figure 2.7: GPU

2.3.7 Graphical Processing Unit (GPU)

A GPU is like a sous-chef in a kitchen. It is responsible for handling the graphics and visual effects of a program. For example, when you play a game, the GPU is responsible for rendering the graphics and visual effects of the game. It does this by performing complex calculations and rendering images on the screen.

2.4 Do Software Engineers Need to Know About Hardware?

Yes, software engineers need to have a basic understanding of hardware. This is because software and hardware work together to make a computer system. If you don't understand how the hardware works, you won't be able to write efficient code. You won't be able to optimize your code for the hardware. You won't be able to write code that takes advantage of the hardware features.

2.4.1 Branch Prediction & Speculative Execution

When software engineers write code, they often focus on algorithmic efficiency (like choosing the right sorting algorithm or data structure). However, the way modern hardware, specifically the CPU, executes code can also have a significant impact on performance, sometimes in non-obvious ways. One such area involves how CPUs handle conditional branches (like `if` statements).

Consider a simple Python program designed to count how many ages in a list are greater than 18:

```
1 count = 0
2 # Let's assume 'ages_list' is a list of 100 ages
3 for age in ages_list:
4     if age > 18:
5         count = count + 1
```

Listing 2.4: Python code to count ages above 18

Now, suppose we run this code with two different lists of ages, both containing the same numbers but in a different order:

1. **Sorted List:** 1, 3, 4, 6, 8, 10, 20, 21, 24, 25, 29, 30, ... (and so on...)
2. **Unsorted List:** 21, 4, 29, 3, 30, 8, 10, 21, 1, 6, 25, 20, ... (same numbers, random order)

At first glance, one might think the execution time would be the same for both lists since the number of operations appears identical. However, the version processing the **sorted list** can often be faster. Why? The answer lies in CPU features called **branch prediction** and **speculative execution**.

Imagine a doctor's receptionist. If a particular patient has asked for their medical file every time they've visited for the past five appointments, the receptionist might get smart. The next time they see that patient approaching, they might proactively fetch the file, even before the patient explicitly asks. If the patient indeed wants the file, time is saved. If they want something else, the receptionist simply puts the file back, having lost only a little effort.

Modern CPUs do something similar. When they encounter a conditional branch (the `if age > 18:` line), they have to decide which path of code to execute next. Waiting for the condition to be fully evaluated can cause delays (a "pipeline stall"), as fetching data from RAM to cache and then to registers takes time. To avoid this, the CPU employs a **branch predictor**. This is a piece of hardware that tries to guess which way the branch will go (e.g., will `age > 18` be true or false?) based on the recent history of that branch.

If the branch predictor guesses, say, that the `if` condition will be true, the CPU can then **speculatively execute** the instructions inside the `if` block.

- If the guess was correct (the condition indeed turns out to be true), the CPU has saved time because the results of those instructions are already available.
- If the guess was wrong (the condition is false), the CPU discards the results of the speculatively executed instructions and executes the correct path (e.g., the `else` block, or simply skips the `if` block if there's no `else`). This incurs a penalty, but the hope is that correct predictions are frequent enough to provide an overall speedup.

How does this relate to sorted vs. unsorted data?

- **With the sorted list (e.g., 1, 3, ..., 10, 20, 21, ...):** The outcome of the `if age > 18` condition will be very predictable for long stretches. For the initial smaller numbers, it will consistently be false. Once the ages

2.4 Do Software Engineers Need to Know About Hardware?

cross 18, it will consistently be true. This highly regular pattern makes it easy for the branch predictor to learn and make correct guesses most of the time.

- **With the unsorted list (e.g., 21, 4, 29, 3, ...):** The outcome of `if age > 18` will flip unpredictably (true, false, true, false, true, true, false...). This makes it much harder for the branch predictor to guess correctly, leading to more mispredictions and thus more time wasted discarding speculative work and restarting on the correct path.

Therefore, even though the number of logical operations is the same, the sorted list allows the CPU's branch predictor to work more effectively, leading to faster execution due to successful speculative execution. This demonstrates that understanding low-level hardware behavior can be crucial for writing truly high-performance software.

In addition to this, if you don't know the hardware, you will probably write unsafe codes. see subsection 6.9.5.

- But this is the job of security engineer to find these problems and I should not be concerned.

→ While I get your point, I see many real-world projects that are designed by an individual or a small group of people where no formal structure, like those seen in big companies, is present, and security is often a missing part. Therefore, each programmer should have a basic understanding of security issues.



0110101
0100101



011001010
001000101



8

00101010
01010101



2.4 Do Software Engineers Need to Know About Hardware?

3.0 Difference between Privacy, Anonymity, and Security

These three terms are often used interchangeably, but they have different meanings.

3.1 Security

3.1.1 Definition

In simple terms, security is like a lock on a door. It prevents unauthorized access to your data—meaning anyone you don’t want to have access. Security ensures that your data is not tampered with or stolen. It protects your information from unauthorized access, disclosure, or modification. For example, you certainly wouldn’t want your bank account data to be accessed or altered by someone else. Even if that person couldn’t access the actual data, they could still damage it, making it unusable.

Mainly security is defined by three principles (CIA Triad):

1. **Confidentiality:** It ensures that the data is only accessible to authorized users. (For example, your bank account data should only be accessible to you and the bank.)
2. **Integrity:** It ensures that the data is not tampered with. (Even if someone can’t access your bank account data, they shouldn’t be able to change it. For example, if someone changes your bank account balance from \$1000 to \$100, that’s a breach of integrity. This will be more clear when we discuss encryption.)
3. **Availability:** It ensures that the data is available when needed.

3.1.2 Prevention, Detection and Response

Security consists of three steps: prevention, detection, and response.



Figure 3.1: Prevention, Detection and Response

Prevention is the first step in security. It involves implementing measures to prevent attacks or make them impossible or difficult to execute. These measures can include using strong passwords, enabling two-factor authentication, avoiding the download of untrusted apps, and refraining from using unsafe VPNs, as they may be used to collect your data.

An important note is that you always need to *overestimate* attacks. (It's not limited to just writing programs; it also includes protecting your data.) There are certainly attacks you're not aware of. New attacks are discovered daily. Additionally, computational power is increasing rapidly. What was secure yesterday might not be secure today. For instance, in 1977, *Ron Rivest* (one of the inventors of the RSA) said[5] for factoring a 125-digit number, we need 40 quadrillion years of running power.¹ Yet in 1994, researchers factored a 129-digit number![6] (Because of development of significantly better factoring algorithms.)

We read from *Cryptography Engineering*[7]:

A security engineer has to take a malicious wind into account. What if the wind blows up and down instead of just from the side, and what if it changes directions at the right frequency for the bridge to resonate? Bridge engineers will dismiss this kind of talk out of hand: "Don't be silly, the wind doesn't blow that way." That certainly makes the bridge engineers' jobs much easier. Cryptographers don't have that luxury. Security systems are attacked by clever and malicious attackers. We have to consider all types of attack. ...Since the attack may occur long after we design the system, she has

¹With best known algorithms was available at that time.

3.1 Security

the advantage of five or ten years' more research, and can use technology of the future that is not available to us.
...we also assume that the attacker might have any number of other goals, ...

Many only thinks of certain related type of attacks. Any type of attack should be considered.

Not only that, there are some groups and agencies that collect data now and attack later when they have enough computational power or when a new vulnerability is discovered which is known as *Store Now, Decrypt Later*.²

This is also affecting our privacy. See section 3.2 for more information.

Among the parameters recommended by reputable organizations (and not necessarily any higher existing parameter) for the protocols you utilize to safeguard against potential future attacks.

An approach is to have multiple layers of defense (Defense in Depth). In this way, if one layer is compromised, the attacker still needs to pass through the other layers. One example is to combine multiple block ciphers with different keys. See section 4.21 for more information.

Detection is the second step in security. It involves monitoring systems and networks for signs of attacks or breaches. This can include using intrusion detection systems, log analysis³, and network monitoring tools.

You can use tamper-evident seals to detect if someone has tampered with your device. For example, if you have a laptop and you want to make sure no one has opened it, you can put a tamper-evident label on it. If someone opens the laptop, the label will be damaged, and you will know that someone has tampered with your device.

Some individuals position their office chairs in a specific way. If they notice that the chair has been moved, they immediately recognize that someone has entered their office.

Response is the third step in security, involving actions taken in reaction to an attack or breach. Some people don't have a checklist for what to do in case of an attack. When attack happens, you don't have time for thinking to what you should do. You should have a complete plan in place for what to do in case of an attack. This can include isolating the affected systems, notifying users, and searching for traces of the attackers.

Disaster recovery is an essential component of the response process. It involves having a plan in place to recover from an attack or breach. This can include restoring the system from backups. For instance, having a complete disk backup allows you to restore the system to a point in time—such as ten

²For instance, *Tempora* is a British surveillance program that taps into fiber-optic cables to collect data and store it for later analysis.

³Logs are files that record events that occur in a system or network. They can be used to track user activity, system performance, and security events.

3.1 Security



Figure 3.2: Permanent tamper evident numbered label showing the label applied to the surface, the label voiding and the permanent void message left on the surface of the container. Image and caption from Wikipedia: TamperTechTeam, Permanent tamper evident numbered label, CC BY-SA 4.0

days ago—when it was free from viruses.

Finally, we should learn from our mistakes. After an attack, we should analyze what went wrong and how we can prevent it from happening again.

A robust security strategy should include **all** three steps.

3.1.3 No System is 100% Safe

Systems are created by humans, and humans are not infallible. No one knows all the security issues, and no one can write code without flaws, which is why any system **might** be broken at some point.

Furthermore, systems can have vulnerabilities that are unknown to the system administrators or developers but are known to attackers (these are often called “zero-day”⁴ vulnerabilities). Attackers are always one step ahead.

Additionally, any file can be dangerous. Why? Consider a PDF file for example. The PDF reader software is designed to open and display PDF files. However, if there is a vulnerability in the PDF reader software, a maliciously crafted PDF file could exploit that vulnerability when opened. This could lead to various harmful actions, such as executing arbitrary code.

Even if your server is behind a firewall, it can still be hacked. Firewalls are not foolproof. They can be bypassed, and they can be misconfigured. A firewall is like a door lock; it can be picked or broken.

⁴Which means that the developers had zero days to fix the vulnerability before it was exploited

3.1 Security

- even computers that have no external ports and no internet connection?!

→ Yes! The co-inventor of the RSA algorithm⁵ himself demonstrated that we can use computer noise⁶ to steal private data of RSA operations![2] He hacked a particular implementation of the algorithm (Math was created by him. Not implementation). This illustrates the beauty of falsifiability in science, which will be discussed later.

- So, does that mean there's no security at all? Should we just give up and not do anything about security?

→ Well, that's exactly one of the common mistakes people make. They think, "Since everything is hackable, why bother?" That's a misconception! When we say everything is hackable, it means there's always a chance that someone with enough: • **Knowledge** • **Resources** • **Capabilities** • **Money** • **Time** can come along and hack it.

Almost all door locks can be opened. There's always someone who knows a specific method to unlock them. So, do you not lock your door at home? Why do you lock it then? Because you know that if it's left open, it's easy for someone to come in. But if I lock it, it protects me against a group of thieves. At the very least, a thief might think, "Why would I go to a place with a locked door? I'll go somewhere that's easier to break into, so I don't waste my time."

1. Some individuals lack technical knowledge and are simply nosy about your life, wanting to snoop on your phone. You can likely prevent these people from accessing your device with a good screen lock. (If your password is a pattern you draw on the screen, it can be easily recognized after just one glance. Even after you set your phone aside, if you hold the black screen towards the light, the shape you drew remains visible due to the oils from your fingers, making it detectable again.)

2. There are individuals with some technical knowledge, limited money, and time, who know it's easy to boot up a live operating system and steal your data. For example, some people steal laptops and want to extract the information from them to sell later. With a good BIOS password (and by keeping your system and BIOS updated) and full-disk encryption, you can likely prevent them from accessing your data. If they want to bypass these protections, they will need time and money, which they may not have, or they might lack the knowledge to do so. (Didn't understand what these mean? No problem! We're just outlining things for now. We'll go into detail later!) (If you didn't understand what these mean, don't worry! We're just outlining things for now. We'll go into detail later!)

So far, with simple security measures, we've prevented the intrusion of two groups. These are the two groups that threaten us the most, and most daily threats come from them.

Don't you want to take some simple steps to prevent many everyday attacks?

⁵The algorithm behind HTTPS and many other protocols that keep you safe

⁶Any electronic device creates some noise. With enough analysis of the noise behavior, we can understand what is going on inside that electronic device, such as stealing passwords.



Figure 3.3: The weakest link

With just a few straightforward actions, you can stop them. You will definitely take these steps! This is the most basic and effective approach, and it prevents many attacks. It's like a lock; there's someone in the world who can open it, but you still lock the door! Those who say that no matter what you do, you'll get hacked, so don't bother, are giving completely wrong advice. Be careful not to fall for such statements!

3. Individuals, groups, or organizations with extensive knowledge, resources, capabilities, and significant funding; Organizations like the FBI fall into this category.

In this case, you can reduce the chances of them accessing your information, but you can't bring that risk down to zero! These groups are familiar with rare and hidden vulnerabilities, and they have the means to exploit them.

If your goal is to prevent access from this third group, you need to implement more robust security measures. Pay attention to the details and dig deeper!

Security is about risk management and making attacks prohibitively expensive or difficult for the attacker to make a hack infeasible in practice. If your data worth \$100k, I probably won't spend \$100 million to access it.

3.1.4 The Weakest Link

Suppose you have fastened your bike with a lock and chain. If someone wants to steal your bike, they must break the lock or chain. Where does the chain break? At the weakest link (where it is vulnerable to attack). Similarly, in the digital world, security is only as strong as its weakest link. No matter how robust your security measures are, if there is a weak link, an attacker will exploit it. This is exactly what hackers do: they find the weakest link and exploit it to gain unauthorized access to your data. To ensure security, you must secure all

3.1 Security

the links, including hardware (e.g., electronic devices), operating systems (e.g., Windows, Linux, etc.), networks (e.g., Wi-Fi, LAN, etc.), applications (e.g., browsers, email clients, etc.), and users (e.g., you and me).

Like a broken window that thieves use to enter your house, a security vulnerability (or a security bug) is a point that hackers exploit to gain access to your system. That's why we recommend that you update the software you use. Programmers regularly search for vulnerabilities and fix them through updates.

Have you ever noticed that whenever you update WhatsApp, the app doesn't change much? That's because WhatsApp mostly just fixes vulnerabilities rather than adding many new features. In contrast, Telegram frequently introduces new features.

Many times, bad actors gather information about what software and version you are using and then search for vulnerabilities in that software. If they find a vulnerability, they can exploit it to hack into your system.

Additionally, some use package managers to install software. Package managers are tools that help you install software on your system. However, some package managers don't provide updates in a short time. It's important to support package managers with our donations to help them provide updates faster.

In this book, I will provide you with the mindset of a hacker so that you can think like one and secure your data. I will start with the basics and mention some real-world examples to help you understand the concepts better. I will also explain how famous privacy tools work and how you can use them to secure your data.

3.1.5 The Importance of Transparency: Public Algorithms in Security Assurance

In security, we either identify a provably secure algorithm or method (protocols that have been proven to be secure)⁷, or we rely on a method that many experts have not found any vulnerabilities in and believe to be secure.

Falsifiability

Let's first discuss falsifiability, a concept introduced by Karl Popper. Especially in the context of the COVID-19 pandemic, there have been many odd claims. For instance, some people say, "Take this medicine; it's totally natural and will get rid of the virus." When you ask if they have tested it, they respond, "Yes, I gave it to some people, and they got better." They use these examples as evidence and proof. However, Popper argues that just because something seems to have worked doesn't mean it's true. It might be due to other factors, such

⁷For example, they demonstrate that an algorithm or protocol is secure if certain underlying mathematical problems (like factoring large numbers) are hard, or if certain idealized models (like the random oracle model) accurately represent reality. Proofs don't guarantee absolute security, and proofs themselves can contain errors.

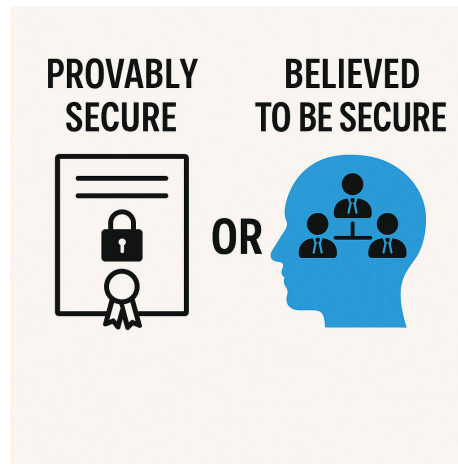


Figure 3.4: Provable or Believed to be Secure (AI)

as the person's immune system, a healthy diet, trust in the process, and a lack of worry⁸.

So, Popper asserts that a scientific theory must be testable and potentially refutable by evidence. A non-falsifiable claim isn't necessarily false, but it falls outside the realm of science.

The beauty of science lies in its foundation of falsifiability. Scientists make hypotheses and then attempt to disprove them. If they can't disprove a hypothesis, it becomes stronger (but not an absolute truth). This is how science progresses.

- But science is unreliable; they change their minds all the time.

- Yes, things change in science. However, science provides the best available solution at the moment. Science doesn't claim, "This is the truth; either accept it or die." Instead, science encourages questioning, testing, and attempts to disprove ideas. This is the beauty of science.

Science does not claim that examples serve as proof. This is because human nature tends to seek explanations for the phenomena it observes. Just as in ancient times, people believed that certain actions would anger the sea god (because they had once seen a flood occur after performing that action), countless superstitions and deities were created from this belief. Numerous gods were invented to explain the phenomena that humans observed.

The difference between scientific experiments and the claims of a seller is that, in drug development, researchers typically conduct controlled trials. For example, they might gather a large group of participants and divide them ran-

⁸Many people believe that if they get sick, they will die. This belief can make them more anxious and stressed, which can weaken their immune system. On the other hand, if someone believes that they will get better, they are more likely to recover.



Figure 3.5: Falsifiability

3.1 Security

domly into two groups. Both groups are placed in similar environments with comparable diets and activities. One group receives the actual drug, while the other receives a placebo (an inactive substance). If a significantly higher percentage of the group receiving the real drug shows improvement compared to the placebo group, this provides strong evidence that the drug is effective within the tested population. However, even positive results require further testing and scrutiny. Science builds evidence through rigorous experimentation and remains open to revision based on new findings; it rarely claims absolute, final proof. The strength of science lies in this iterative process and its willingness to adapt based on evidence.

Moreover, science has advanced significantly. Today, there are newer and more controlled methods than there were thirty years ago. Laboratory conditions are much more refined now, leading to more precise results.

Returning to the example of medicine, while it is true that many drugs are derived from natural substances, researchers examine which components are beneficial and which may be harmful. They isolate the beneficial compounds and formulate them into drugs, ensuring precise dosages, such as 5 milligrams of a specific ingredient. This is not the same as someone claiming to have created a remedy and suggesting that you take a spoonful every day without any scientific basis or testing to verify its effectiveness. Some medications are effective only up to a certain dosage; if taken in excess, they can even have adverse effects. For example, a drug may be beneficial at 5 milligrams, but beyond that, it may not only lose its effectiveness but also harm the body.

Falsifiability in Security

In security, the principle of seeking systems that can be rigorously tested and potentially falsified applies directly. We primarily trust algorithms and protocols that have been publicly available and scrutinized by many experts over several years without known vulnerabilities being found (these often become established standards). Regular, independent testing is also crucial. Be wary when companies claim their products are secure based on internal testing alone, especially if they don't provide test results or demonstrate ongoing testing. Whenever you see such claims, ask about the regularity and recency of their testing.

In security, the same principles apply. We primarily have to trust algorithms that have been tested over several years by many experts, who have not found any vulnerabilities in them (often, these are established standards). Regular testing is also crucial. Sometimes, companies claim they have tested their products and that they are secure, but they do not provide the results of these tests and do not conduct regular testing. Whenever you see a company claiming that they test their product, ask them whether they test regularly and when the last test was conducted.

Security does not come from obscurity; it comes from transparency. Additionally, the implementation of the algorithm is also important. We have seen many instances where companies used self-developed algorithms or their own

3.1 Security

implementations, which turned out to be vulnerable [8]:

The design and implementation details should be well audited and reviewed by independent researchers and should not rely on the difficulty of reverse engineering proprietary systems

“ Anyone, from the most clueless amateur to the best cryptographer, can create an algorithm that he himself can’t break.

— Bruce Schneier

Even if you are a good cryptographer/security expert, it doesn’t mean that your algorithm is OK. It should be tested. For instance *Vincent Rijmen* and *Bart Preneel* about *McGuffin* (a cipher created by *Bruce Schneier*) say[9]:

Modifying a scheme with only existing attacks in mind however is not a good design principle.

Bruce Schneier himself says⁹:

“ Key schedules are very hard, and I didn’t understand them very well in 1993 when I designed Blowfish. I’m much prouder of the key schedule in Twofish and Threefish.

— Bruce Schneier

When big companies like *Samsung* and *Qualcomm*, as well as prominent figures in cryptography like *Bruce Schneier*, can go wrong, how can you trust a proprietary algorithm?

Conclusion: Use well-audited algorithms. Don’t change them, even if you think a modification would improve them. Simply don’t! For this reason, we sometimes prefer older algorithms (as long as they are still secure, tested, and updated) over newer ones because they have been tested for a longer time. Untested algorithms are dangerous.

Reading List

- Snake Oil¹⁰ (Highly recommended)
- An example of snake oil¹¹ (Highly recommended)

⁹https://www.schneier.com/blog/archives/2009/09/the_doghouse_cr.html

¹⁰<https://www.schneier.com/crypto-gram/archives/1999/0215.html#snakeoil>

¹¹<https://www.schneier.com/crypto-gram/archives/2003/0215.html#4>

3.1 Security

- Snake Oil Warning Signs: Encryption Software to Avoid¹²
- Desktop Google Finds Holes¹³ (Cache of encrypted files, must be inaccessible)

3.1.6 Open Source and Its Security

Imagine you've bought a car designed by the manufacturer in such a way that you can only drive it. There's no way to see the engine, the radiator, or any of the wiring—everything is enclosed in a metal box that you can't open. The only thing you can do is get in, drive around, and come back! You have no idea if quality parts were used or not!

On the other hand, there's another company that provides complete transparency. They don't hide the engine in a box! They say, 'Look, everything is clear. You can see everything!'

Behind every program, there's a set of code—a series of texts and instructions that tell the computer what to do. This code is like the engine of a car. If you can't see the code (in closed-source programs), you can't know what the program is doing. You can't know if it's secure or not. You can't know if it's doing something harmful or not. You can't know if it's collecting your data or not.

An important note is that this doesn't mean that closed-source programs cannot be reviewed. If you have an executable binary file, you can disassemble it to an assembly file to be able to analyze it. Ghidra¹⁴ goes further by giving you a C file! Additionally, there are many tools to analyze what the program does.

However, reverse engineering is complex, time-consuming, may have legal restrictions (EULA), and doesn't guarantee a full understanding equivalent to having the original source code.

Now, let's go back to that transparent car we were discussing. Yes, you can see the car's parts and check if they're good or not, but there's a problem! If something goes wrong with the car, you have to wait for the company to come and fix it! You can't fix it yourself! If you try to fix it, the company can sue you! They might say, 'Sure, I let you see the parts, but you don't have the right to change them! You have to come to me to get them replaced!'

Moreover, they make you sign a lengthy contract before selling you the car, stating that you can only use it in a specific city. If you drive it on banned roads, they'll sue you. They impose a lot of conditions on you.

Now, you might be thinking, "What do you mean?! I bought the car! I want to be able to repair it myself! What do you mean if I try to fix it, you'll sue me?! What do you mean I can't drive on certain roads?! I bought it!"

¹²<https://web.archive.org/web/20030207174457/https://www.interhack.net/people/cmcurtin/snake-oil-faq.html>

¹³https://www.schneier.com/blog/archives/2004/11/desktop_google.html

¹⁴<https://github.com/NationalSecurityAgency/ghidra>

3.1 Security

So, what can we do? Here's where the concept of Free Software comes in. Some people say this situation isn't right! We should allow everyone to modify their own software. What do you mean by imposing rules that prevent changing parts? They proposed that any software should have:

Freedom 0: The freedom to use the software for any purpose you want! This means you have the right to use the software for whatever reason you like! To clarify, regular software has specific rules set by the creator. For example, programs may state that you can only use them for personal and non-commercial purposes. If you want to use it for commercial purposes (like in your company), you have to pay to obtain the right to use it! Because you're using my program in a company that generates income, and my program has presumably helped your business, you should give me a portion of that money. For instance, \$20 a year.

Freedom 1: The freedom to study the program, understand how it works, and change it so it does what you want. What does it mean to have the freedom to study a program to understand how it works? It means you can see the code! You can see the engine! Free Software states that the program's code must be accessible! To understand how a program works, I need access to its code so I can read it and see how it operates. Unlike open-source software, Free Software asserts that just having the source code isn't enough! I must also have the right to change it. Why?

I might want to modify a program to add a new feature. For example, a messaging app has voice calling but lacks video calling. I can change the code myself to add video calling! I can add a feature that I want! I can fix a bug that annoys me! I can remove a feature that I don't like! I can do whatever I want with the program! I have the freedom to change it!

Imagine there's a security vulnerability in a program. If you don't have the right to change and fix the program, you have to wait for the developer to address it whenever they feel like it, and they might not even care to fix it, leaving you at risk! That's really bad!

- But I don't have coding skills to fix it!

→ Yes, you might not, but others do! Others can fix it and provide you with the corrected version. This is exactly what we call Freedom 3, which we'll discuss in a moment.

Freedom 2: The freedom to redistribute copies so you can help others. In the licenses of most software, it's specified that no one should copy the program files and share them with others! It's not very clear, is it? Let me give you an example:

You can't take the program file, put it on a USB drive, and give it to your friend so they can install it too! You can't upload a program file to your website for others to download, even if that program is free! You can't send a program file to your friend on Telegram! But Free Software says, "What's the point of that? Let people copy the program files and share them with their friends! Let's not impose so many restrictions!"

3.1 Security

Freedom 3: The freedom to distribute copies of your modified versions to others. This is the point we discussed earlier. If there's a problem and the company doesn't fix it, others have the right to fix it and share it with the public!

For instance, Telegram has a lot of features. There are also many unofficial versions of Telegram with various functionalities. In fact, Telegram is free software, which means anyone can work on improving it. That's why Telegram has so many features. For example, if someone notices that this messaging app has voice calling but lacks video calling, they can modify the code to add video calling! This way, an unofficial version of Telegram is created. Another person might say, "Oh, it has video calling, but it doesn't support group calls," so they add group calling. Yet another person might think, "It has this feature, but it's missing that one," and they add it too.

In essence, people create and release new versions. They can also share their code with the original developer, who might incorporate it into the main Telegram, or they can choose not to and create a new, unofficial version to publish.

In contrast, we have WhatsApp, which doesn't operate this way at all. It has very limited features, and no one can improve WhatsApp on their own! This highlights the difference between free software and closed software that imposes countless restrictions on you.

In fact, the existence of the source code is essential for all freedoms 1 to 3.

Now, you should understand the difference between Free Software and Freeware. Freeware simply refers to a program that is free of charge. However, Free Software means software that is truly free, encompassing the freedoms to use, study, modify, and distribute the software.

“ To understand the concept, you should think of “free” as in “free speech,” not as in “free beer.” We sometimes call it “libre software,” borrowing the French or Spanish word for “free” as in freedom, to show we do not mean the software is gratis.^a

^a<https://www.gnu.org/philosophy/free-sw.html>, Creative Commons Attribution-NoDerivatives 4.0 International License

— GNU Project

The above quote really explains the concept well.

- If everyone can modify the program, doesn't that put security at risk?
→ while everyone can modify and distribute the program, you have the option to use the original version from the original developer if you're not confident in the security of modified versions.

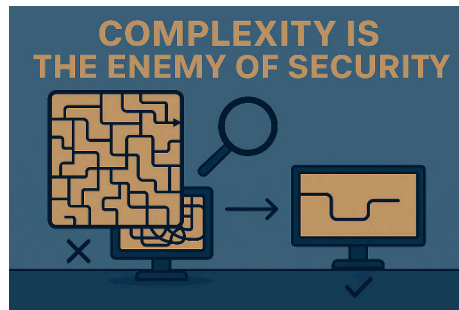


Figure 3.6: Complexity in security

Attack Surface & Complexity

Entering a house with 10 doors is easier than entering a house with only one door. The more doors you have, the more chances you have to be attacked. Programs are based on code. The more code you have, the greater the chances that a vulnerability exists.

While more features can be beneficial, they also increase the attack surface. Many security-focused apps reduce the attack surface by removing unnecessary features. For example, a browser that doesn't have a camera feature is more secure than a browser that does. The camera feature might have a vulnerability that can be exploited by an attacker. This principle can be extended to any app (including messaging apps). For instance, I personally don't recommend Telegram for someone who wants to be anonymous because it has many, many features.

Is editing a book with 1000 pages easier than editing a book with 100 pages? The more lines of code your app has, the more complexity you have in your code, the greater the chances that vulnerabilities arise and don't get detected. We mostly choose programs and protocols that have a smaller and easier to understand code. For example, *WireGuard* is a VPN protocol that has only several thousand lines of code, while *OpenVPN* has tens of thousands of lines of code. This means that *WireGuard* is easier to audit and review than *OpenVPN*.

As another example, enabling editing allows potentially malicious macros within the document to run if macro execution is also permitted by security settings or user confirmation. This is why you should not tick all the options you see which may or may not increase attack surface.

Many people install numerous dependencies and IDE extensions in their projects and don't remove the ones they don't use. Keep a list of dependencies and remove the ones you don't need. This reduces the attack surface.

In fact, some bad actors during job interviews ask you to do a live project and provide you with a project that has malicious dependencies to hack into your system. Additionally, some give you a project with legitimate but outdated dependencies. They want to know whether the project works on your computer

(if you are using an old dependency) and then hack into your system based on vulnerabilities that exist in that old dependency.

Remove dead code (code that is no longer used). It can be a security risk.

Open-source Does NOT Mean Free of Bugs

First of all, how can you be sure that the code provided is the same as the code of the executable file? Some provide a way to check it¹⁵. But I bet many of you have never checked it. I understand that checking it mostly requires technical knowledge and dependencies that you might not have.

Moreover, there was a problem in *OpenSSL* (a library that is used to secure the connection between your browser and the server and is used by millions of people), and nobody discovered it for years. This is an example that even big projects can have bugs.

- Why?

→ Well, nothing is 100% perfect! Sometimes, issues slip through the cracks of the people reviewing the code! No one has read all of the code of a program like *OpenSSL*! It's a huge project!

Additionally, many free software programs rely on libraries (pieces of code written by others that we can use instead of coding everything from scratch, like code to determine if a number is prime or not).

- What does that have to do with it?

→ I want you to see the connection. Think about it: okay, the product itself might not have any issues, but there could be problems in its supply chain... Many times, problems arise from the supply chain. This means that the developer of the program might be using a service, and that service uses libraries that are no longer being updated! The library developer may have abandoned it, leaving it without updates; when that happens, our program is using outdated code that may have security vulnerabilities or bugs that won't be fixed!

Actually, *supply chain attacks* are a serious issue. For example, the *SolarWinds* attack was a supply chain attack. The attackers didn't attack the target directly. Instead, they attacked a company that provided services to the target. The attackers compromised the software update mechanism of the company, and when the company sent an update to the target, the update contained malware. The target installed the update, and the attackers gained access to the target's network. This is a supply chain attack. The attackers didn't attack the target directly; they attacked a company that provided services to the target. This is a very serious issue.

A bad actor can gain trust in the community by contributing to open-source projects. They can contribute to the project for a long time, and when they gain the community's trust, they can insert malicious code (e.g., a backdoor) into the project. This is why it's important to review the code of the libraries

¹⁵e.g. Telegram: <https://core.telegram.org/reproducible-builds>

3.1 Security

you use. You can't just trust them blindly. You have to review them and make sure they're secure.

Let's assume the libraries are being updated. The developer of our program might not care about using the updated libraries. They might still be using older versions that have security issues!

- Why?
 - Many open-source projects are written for fun and to learn.¹⁶
 - A large number of developers are unaware of security issues and may not realize how important they can be! So when they're not aware of their significance, they don't prioritize updating the libraries!
 - They might simply lack the motivation to update the libraries. They might think, "Why should I update the library? It's working fine!"
 - The new versions of the libraries might come with new features and changes that our developer doesn't know how to work with, which is why they don't update!
 - Updating that library might cause the existing code to break, requiring several changes in the code. And the developer might not want to do that!

What to do?

→ Use libraries that are actively maintained and updated. Libraries that are made by real experts in the field, not just written for a fun project.

Typically, large companies use free libraries but don't provide any benefits to the creators. Essentially, these companies are profiting from someone else's work without giving any share of their earnings to the original developers. It would be better if the law required them to give a portion of their profits to the creators of those libraries, ensuring that the developers are financially supported and can dedicate more time and motivation to building their projects!

One issue that arises in free software is that developers may become discouraged by the fact that so many different entities are using their code without any compensation. After all, they need an income to sustain their lives! Eventually, they may get tired and abandon their projects. However, if they receive some income, they will be more motivated, spend more time on their work, and produce higher-quality code instead of just throwing together something mediocre!

Moreover, companies that use this code should make an effort to review it and not just be passive consumers! Unfortunately, many companies simply use the code without examining it. If every company were to review the code they use, the situation would change significantly. This way, many experts would be monitoring and reviewing the code, and the security of open-source software

¹⁶While companies have a security team to review the code, many open-source projects don't have a security team.

3.1 Security

relies on the fact that many people check the code. Otherwise, if everyone is just a user and doesn't check the code, it doesn't enhance security!

It might be funny, but we have open-source malware. They update it after anti-viruses detect it. They update it to bypass the anti-viruses.

Reading List

- Choosing Secure Open Source Package¹⁷¹⁸ (From Intel Open Source. Highly recommended!)
- The Cathedral and the Bazaar¹⁹

The section 'VERY OLD CODE' explains why we should avoid using very old and unmaintained code. I'd like to add a point: for example, if I wrote a library back in 2000, there were certain types of attacks that either didn't exist at that time or were so impractical that they were unlikely to occur. As a result, I didn't implement any defenses against them. However, now those attacks have become viable and could easily exploit vulnerabilities in the code.

It doesn't matter if it was written by the best cryptographer in the world or if it used the best encryption methods available at the time. What's important is that the world has changed. What was once theoretical or impractical has now been discovered to be feasible!²⁰

3.1.7 Don't Rely on Trial and Error When Creating Services and Troubleshooting Computer Errors!

Imagine you are trying to set up a program. During the installation, you encounter an error. You go online to search for a solution, and a website or video suggests that you run a specific command to fix the issue. Without fully understanding the implications, you blindly follow that advice. However, instead of resolving the problem, you encounter another error that states you need to run the program with admin access (the highest level of permission). Once again, you proceed without realizing that this could be potentially harmful.

In each instance, you are trying solutions that may either be safe or malicious. It's like a house where, at every step, you are opening different doors and windows that could later be exploited.

Instead of relying on blind trial and error, it's better to seek out reputable sources and official documentation. This approach will help you resolve issues effectively and safely while also preventing potential risks.

¹⁷<https://01.org/blogs/terrioda/2017/choosing-secure-open-source-packages-part-1>

¹⁸<https://01.org/blogs/terrioda/2017/choosing-secure-open-source-packages-part-2>

¹⁹https://en.wikipedia.org/wiki/The_Cathedral_and_the_Bazaar

²⁰For example, *differential cryptanalysis* was previously unknown to the academic world while it was known to the NSA: https://www.schneier.com/blog/archives/2013/09/the_nsas_crypto_1.html

3.1 Security

If you're a developer, you should write clear and detailed documentation for your software. This documentation should explain how to use the software, how to troubleshoot common issues, and how to fix errors. By providing this information, you can help users avoid potential risks and ensure they have a positive experience with your software.

Warning

Never do something you're not aware about its consequences. Unfortunately, many people blindly follow instructions found online, assuming that just because a method works or has received positive comments from others^a, doesn't mean it is safe.

^aSuch comments may be fake or even posted by users whose devices are infected with malware

3.1.8 The Software Industry Needs More Secure Products, Not More Security Products—CISA

Imagine you are building a house. You can either construct it using durable materials, strong pipes, and proper insulation from the start, or you can use cheaper materials and later add safety and security features like alarms, insulation, or repair broken pipes. Which approach is more effective?

A secure system is one that is designed to be secure from the beginning. This means that security is not an afterthought or an add-on, but rather an integral part of the design process. If you're designing an app without considering security from the start, you **must** think again!

Never design a system that requires user action for security. Most people still use the default browser, and many don't even know there are other browsers out there! Do you really want to rely on the security of the system being in their hands? No! Never do it! Security and privacy must be by default. As CISA in *Shifting the Balance of Cybersecurity Risk: Principles and Approaches for Secure by Design Software*²¹ suggests:

“ The complexity of security configuration should not be a customer problem. Organizational IT staff are frequently overloaded with security and operational responsibilities, thus resulting in limited time to understand and implement the security implications and mitigations required for a robust cybersecurity posture.

— CISA

Additionally, you must not trust user input. Always check whatever the user

²¹<https://www.cisa.gov/resources-tools/resources/secure-by-design>

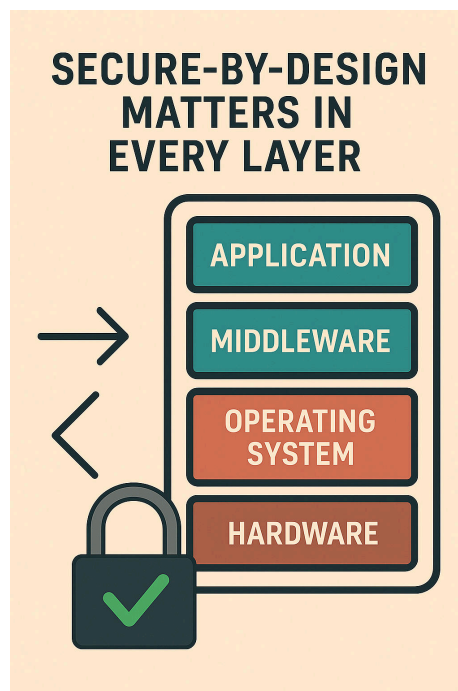


Figure 3.7: Secure by Design

3.1 Security

does on the server side²². Many security problems occur because of trusting user inputs.

For instance, some poorly designed password managers (an app that stores passwords securely) don't encrypt any fields other than passwords. This is a huge risk. Many people store backup codes for 2FA (Two-Factor Authentication) in the notes field of the password manager²³. Some also store security questions there. You should not have pre-assumptions about what users will do. You always need to think like this: "I have designed it like this. What if the user does something else?"

Secure-by-design principles also encourage minimizing the default attack surface. Providing options to disable features users don't need further supports this by allowing users to reduce their specific attack surface.

- I have hardening guide. I don't need secure-by-design practices!

→ CISA gives a good answer:

“ Hardening guides suffer from several common problems. Some hardening guides are hard to find and are not well supported. Others are complex to implement, occasionally requiring software development to write an extension module. Still, others assume the reader has extensive cybersecurity experience to understand the ways in which various settings change the attack surface. Practitioners who have an incomplete understanding of the ways in which attackers work may fail to properly implement hardening guide instructions, especially if the instructions do not make the trade offs clear. Further, not all hardening guides are written by engineers who are intimately familiar with attacker tactics and economics, causing them to create hardening guides that are ineffective even if faithfully implemented. Millions of customers are taking on the responsibility to harden multiple instances of software or systems, often in resource constrained environments. Relying on hardening guides simply does not scale.

— CISA

“ Similar to seat belt chimes in cars that continuously make noise when seat belts are not fastened, manufacturers should implement timely and repeated alerts when users or admins are in truly unsafe states.^a

^ae.g. not having 2FA

— CISA

²²Many client-side checks can be bypassed.

²³Which is wrong. Do you know why? Because if the password manager gets compromised, 2FA becomes lossless: https://keepassxc.org/docs/KeePassXC_UserGuide#_adding_totp_to_an_entry

Isolation: The New Key to Security

So far, we have discussed many concepts in security, such as *no system is 100% safe* and *using updated software*. However, in today's world, where AI and powerful tools are available to attackers, these concepts alone are not sufficient. We need to add another concept, which is a key principle in secure software development practices: *isolation*. The software we build should be designed in such a way that even if an attacker manages to compromise one part of the system, they cannot access other parts, ensuring that the rest of the system remains secure and functional.

“ **As automated and AI-driven attacks continue to expand, software security can no longer rely solely on patching vulnerabilities after the fact,^a**

^aSVP of Legal and Strategic Programs at the Linux Foundation, in <https://www.linuxfoundation.org/press/linux-foundation-launches-the-open-robust-compartmentalization-alliance-orca-to-advance-software-se>

— Mike Dolan

“ **The future of computing lies in trustworthy systems that can recover, adapt, and continue to serve even when parts fail.^a**

^aAssociate Professor in the Department of Computer Science and Engineering at the University of California, Merced, src: <https://www.linuxfoundation.org/press/linux-foundation-launches-the-open-robust-compartmentalization-alliance-orca-to-advance-software-se>

— Xiaoyi Lu

How can this be achieved? By limiting the interaction and communication between different parts of the system. For example, consider a banking application. The part of the application that handles user authentication (e.g., verifying usernames and passwords) should be isolated from the part that processes transactions. If an attacker compromises the authentication system, they should not be able to directly access or manipulate the transaction processing system. This isolation ensures that a breach in one area does not compromise the entire system.

To summarize: Not only should we build our components with overestimation in mind, but we should also isolate them from each other. This can be seen as a defense-in-depth strategy, where multiple layers of security are implemented to protect the system. Each layer provides a barrier that an attacker must overcome, making it more difficult for them to achieve their goals.

Join *ORCA (Open Robust Compartmentalization Alliance)*²⁴

²⁴<https://orca-lf.org/> to learn more about isolation and compartmentalization in software security.

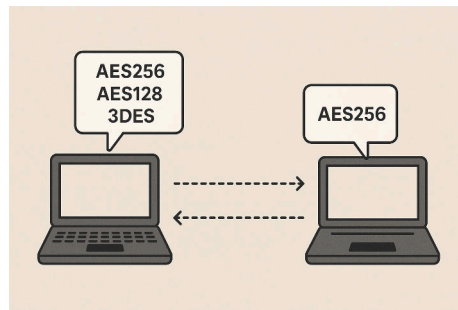


Figure 3.8: Version negotiation

3.1.9 Downgrade Attacks

Imagine you enter Iran; conversations typically start with greetings and handshakes. In Japan, for example, people begin conversations by nodding their heads. Essentially, the protocol (or agreement) for conversation in Iran is a greeting and handshake—it's a set of rules for interaction.

Computers are machines that require everything to be clearly defined for them. They need to know what to do in specific situations. For two computers to communicate with each other over a VPN connection, they must speak based on a shared discourse and agreement that allows them to understand each other. This method of communication is referred to as a *protocol*.

For example, the process of securing HTTP and converting it to HTTPS works like this: first, you send a message to the website saying, "Hello, I support version 1.2 and 1.3 of the security protocols with these specific algorithms." Then, the server responds with a greeting and says, "Okay, from what you mentioned, I will choose this option. From now on, let's communicate based on the agreement we've made!"

For that reason, in order for older systems to be able to communicate with newer systems, they must support the same protocol. For this reason, most systems support multiple versions of the same protocol. For example, a server might support both TLS 1.2 and TLS 1.3. When a client connects to the server, the server and the client negotiate the version of the protocol to use. Mostly, one party sends a message to the other party saying, "I support these versions of the protocol." The other party choose the highest version that both parties support. This is called *version negotiation*. For example, if one party supports TLS 1.2 and 1.3 and the other party only supports 1.2, 1.2 will be chosen.

However, this negotiation mostly happens when the connection is not yet secure (because the client and the server don't know each other yet). This means that an attacker can intercept the communication between the client and the server and modify the messages to make it appear as though the newer version of the protocol is not supported. The attacker can then force the client and the server to use an older, less secure version of the protocol that has known

3.1 Security

vulnerabilities that can be exploited. This is called a *downgrade attack*.

One approach is to drop support for older versions of the protocol. This way, the attacker cannot force the client and the server to use an older version of the protocol. However, this approach can cause compatibility issues with older systems that do not support the newer versions of the protocol. Another approach is to implement mechanisms that detect and prevent downgrade attacks.

As CISA | NSA | FBI | ACSC | CCCS | CERT NZ | NCSC-NZ | NCSC-UK | BSI | NCSC-NL NCSC-NO | NÚKIB | INCD | KISA | NISC-JP | JPCERT/CC | CSA | CSIRTAMERICAS suggest²⁵:

“ Prioritize security over backwards compatibility, empowering security teams to remove insecure features even if it means causing breaking changes.

— CISA

Some softwares drop the support for older versions (the software stops working) to protect users from downgrading attacks.

3.1.10 Social Engineering

Social engineering is the art of manipulating and controlling people. It is a psychological manipulation technique that exploits human error to gain access to systems, networks, or physical locations.

Have you ever noticed that some people, when they speak, seem to have you under their control, making you feel almost enchanted, doing whatever they want? You might agree with what they say without realizing that you are under their influence. This is because they have information about you and know how to behave.

Many high-impact attacks on companies are related to social engineering. For example, I can call the IT department and say, “I am from the head office. Why is our system always broken? Can’t you do your work correctly? We always have problems with our system. I am tired of this. I want you to fix it now! I am going to report you to the CEO!” The IT department might be scared and do whatever I want. This is a simple example of social engineering.

Alternatively, I can go to my SIM card provider in a Mercedes and wearing a suit and say, “My phone has been stolen. I am waiting for a very important call. Will you please forward my calls to this number?” Based on my appearance, they might think I am a legitimate person and do whatever I want. This is another example of social engineering.

Social engineering is a very powerful tool. It is much easier to exploit a person’s natural inclination to trust than to find ways to hack into a system.

²⁵<https://www.cisa.gov/resources-tools/resources/secure-by-design>

3.1 Security

Kevin Mitnick²⁶ is a prime example of a social engineer. He was able to gain access to computer systems simply by talking to people and convincing them to give him the information he needed.

We will later discuss why privacy matters and how the data you leak can be used to manipulate you.

3.1.11 Be Cautious of Hacking and Security Groups

I know many of you are eager to join hacking and security groups to learn more about hacking and security topics. However, it's crucial to be aware that a vast majority of hacking and security groups on platforms like Telegram do not provide you with valuable knowledge. A large portion of these groups focuses on cracking software and distributing pirated programs that have been cracked, along with sharing cracked accounts and engaging in illegal activities. Moreover, individuals with substantial knowledge often avoid these specific types of groups. They focus on more serious topics and discussions, such as security research, vulnerability analysis, and ethical hacking.

It's important to understand that many of the people in these groups may know more than you do, and among them could be cybercriminals, such as ransomware developers. Therefore, avoid downloading any files from these sources, including PDFs! Yes, even PDFs can be infected, and we will discuss this later. So, exercise caution. I don't want to scare you, but at the very least, you should be aware of malware and the different types of files.

Also, be very careful of individuals looking for hackers. Why? Because these people often say things like, "Come here and do this for us; we have your back," or "The boss is watching out for you." You might end up doing hacking work for them, but when things go wrong, you will be the one held responsible, and they will take no accountability. So, stay away from those seeking hackers. You might find yourself in trouble with international law enforcement or the FBI, facing travel bans and more.

3.1.12 Avoid Getting Involved in Governmental Hacking Games

Getting involved with governmental hacking groups (both within and outside your country) can be very dangerous. They may seem friendly at first, but their true intentions are often hidden. They might say things like, "We are here to help you," or "We want to work with you." However, their real goal is to use you for their own purposes. They may ask you to do things that are illegal or unethical, and if you refuse, they may threaten you or even harm you.

They may even claim that a high-ranking government official is backing them and will always have your back. They may even arrange a meeting with that person to build trust.

²⁶https://en.wikipedia.org/wiki/Kevin_Mitnick



Figure 3.9: involvement with governmental hacking groups

In some countries, students may not have the right to work or may have limited working hours, such as being allowed to work only 20 hours a week. However, under these circumstances, you may need money to cover your expenses. They might say, “We’ve got your back; we’ll give you work, but keep it quiet.” For example, they might ask you to write something for them, assuring you that they won’t disclose you as the author. They may even say, “We just need you to translate a text from Chinese to English,” but in the process, they will ask you to add sensitive information that is crucial to their agenda. Read about *Game of Pawns* from the FBI.²⁷

This path has no way back!

Once you are involved in such activities, you will be unable to escape. They will have evidence against you, and you will be forced to comply with their demands. If you’re targeting a country, you may ultimately end up in prisoner exchange negotiations. In politics, there are no friends; only interests matter. You may be used as a pawn in a game of chess, and when the game is over, you will be discarded like a piece of trash. There were many cases where prisoners were exchanged, and the person who was supposed to be exchanged was not even aware of it. This exchange even happens between those countries that happen to be enemies.

Even if your government don’t sell you, you can’t be sure that your government will last forever. What if your government is overthrown? What if a new government comes to power that doesn’t care about you?

- FBI Most Wanted Cyber Criminals²⁸ (Highly recommended to read. It reminds you that you can’t be hidden forever. You will be caught eventually.)

²⁷<https://www.fbi.gov/video-repository/newss-game-of-pawns/view>

²⁸<https://www.fbi.gov/wanted/cyber>

3.2 Privacy

In many cases, privacy is confused with security. Let me give you an example to illustrate the Difference.

Imagine you are trapped in a cage. There is food, water, and exercise equipment available. There are a hundred guards surrounding you, along with a thousand surveillance cameras, and a hundred people are monitoring everything. In this situation, you are safe and not in any danger, but you have no privacy! Whatever you do, the hundred guards are watching you! Do you like it?



Figure 3.10: Trapped in a cage

3.2.1 Why Privacy Matters?

This topic is highly controversial. When we talk about using strong passwords, avoiding unsafe VPNs, and not downloading untrusted apps, people often say, “Come on, take it easy! I have nothing to hide!”

This mindset comes from the fact that people don’t have a realistic understanding of the digital world.

- Let me ask you something. Do you like someone to watch your phone while you are using it on the bus?

- Of course not! Even if you are not doing anything wrong, you don’t want someone to watch your phone.

- Do you want someone to always be with you and watch everything you do?

- If you do, go and find a random person and ask them to watch you all the time for the rest of your life.

- But I don’t know him. The person could do something bad to me. Additionally, for the whole life is too much.

- Exactly! So tell me, do you really know the people who are watching you online? Do you know that anything you do online is permanent forever?



Figure 3.11: Do you want someone to always watch you?

We know the physical world, and we’ve been told to lock our cars because if we don’t, someone might steal them. We see the absence of the car and know that it’s stolen. But in the digital world, you don’t see the absence of your data. You don’t know that your data is stolen. You don’t know that your data is being used against you. You don’t know that your data is being sold to the highest bidder. You don’t know that your data is being used to manipulate you. You don’t know that your data is being used to make you do things you don’t want to do. You don’t know that your data is being used to ruin your life!

Just like kids who ask why we should lock the doors—there are many other houses around us. Why would someone steal our house? They are much richer than us. People also ask why we should secure our data—there are many other people around us. Why would someone steal our data? They are much richer than us.

Yes, they might be richer, but you are still valuable. Furthermore, thieves mostly target the easiest targets. If you don’t lock your door, you become the easiest target. If you don’t secure your data, you are the easiest target. We should be taught to secure our data just like we are taught to lock our doors.

I have read a really good ELI5 (Explain Like I’m 5) explanation of privacy on *Reddit*:

“ **Everybody poops, but we still close the bathroom door.^a**

^aMockingbird, Comment on ‘How do you counter the “I have nothing to hide?” argument?’, Reddit, August 22, 2015, <https://www.reddit.com/r/privacy/comments/3hynvp/comment/cubsirr>

— Mockingbird

Are we doing something wrong in the bathroom? No. We need space to do our private things. We need privacy.

“ Privacy isn’t about hiding bad things. It’s about protecting what makes us humans: our day-to-day behavior, our personality, our fears, our relationships, and our vulnerabilities. Everyone deserves privacy online.
^a

^a<https://twitter.com/torproject/status/1450488158416687107>

— Tor Project

“ Privacy is an inherent human right, and a requirement for maintaining the human condition with dignity and respect

— Bruce Schneier

Surveillance changes behavior

Think about being in a restaurant. Do you choose a table in the center of the restaurant or a table in the corner? Most people choose the table in the corner. Why? Because they want privacy. They don’t want everyone to watch them while they are eating. Is eating a bad thing?

Have you ever encountered a situation where you perform worse when you are being watched? For example, you might be playing a game and doing well. Then someone comes and watches you, and you start to do worse. Why? Because you are being watched. You are being judged. You are being evaluated.

People’s behavior changes when they know they are being watched. They become more cautious. They become more self-conscious. They become more anxious. They become more stressed. They become more paranoid. They become more fearful.

In the Digital World, Anything Might Be Permanent

Keep in mind that anything that has gone into the digital world might be permanent— including anything you’ve done on a website (they are stored in logs), anything you’ve sent to someone, anything you’ve searched for, and anything you’ve posted on social media. Even if you delete it, it might still be stored in backups, shared with third parties, kept in swap files²⁹, hibernate files, temporary files in RAM, etc., or AI may have learnt from it.

A simple search you make might manifest later when you become someone important. Are you okay with that?!

Have you ever read your past messages and thought, “What was I thinking?” The world is changing rapidly. For example, in very ancient times, slavery existed and was a common and normal thing. It was completely normal to have slaves. But now we say that slavery is wrong. Do we go back to the past and criticize those who had slaves? No! Because that was the norm of that time and the general understanding of society then. If someone didn’t treat their slave harshly, we actually commend them. It’s the same now, with the difference that society is changing quickly.

²⁹Of course, this is not used for everyone but rather for targeted attacks

3.2 Privacy

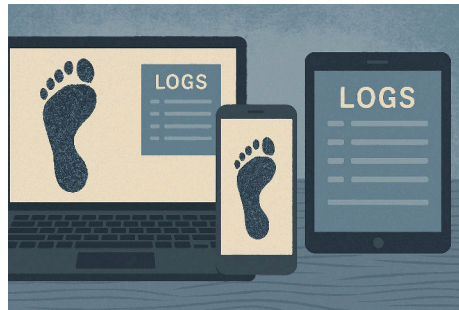


Figure 3.12: Anything that goes into the digital world might be permanent.

For instance, one of the tactics used to eliminate a competitor—whether in business, elections, or other areas—is to say, “You said this in the past! Shame on you!”

Nobody knows whether you might become an important person in the future. Nobody knows whether you might become a target in the future.

While that statement might have been acceptable at the time, the situation has changed now. It’s also possible that the person didn’t have a good understanding of the issue back then but has since learned and changed their mind. Yet, they are still judged based on their past. Did everyone believe in social equality from the very beginning? People grow up in different environments and have different experiences.

Unfortunately, society does not provide an opportunity for defense. The media can lead to someone being fired from their job without waiting for a response from the individual or proof in court. This is why privacy is important. People should have the right to change their minds, to learn, to grow, to make mistakes, and to be forgiven. People should have the right to be human.

Warning

Anything that goes into a digital device might be revealed to the public in the future. In a world where information is permanent, just one mistake can lead to irreversible consequences.

But what if you no longer use a service? Or want to sell your SIM card? Is there any way to delete your data from an app? Fortunately, yes! Some services provide you with the ability to delete your data. This is called the *Right to be Forgotten (RTBF)*. However, keep in mind that your data might still be stored in backups, shared with third parties, kept in swap files³⁰, hibernate files, temporary files in RAM, etc. Moreover, your data might still be present in information held by others. For example, your metadata (e.g. time and

³⁰Of course, this is not used for everyone but rather for targeted attacks

3.2 Privacy

duration of calls) might still be included in other users' data. Furthermore, it might even be used to train their AI. Additionally, not all "delete account" buttons truly delete your data; some may simply hide it. That being said, I personally recommend that you edit your data before deleting it. However, this is not a guarantee that your data in backups and logs will be replaced with the new data.

For example, I was arrested on charges of fraud. All the news outlets headline that this person has been caught as a fraudster. But then I go to court, and it turns out there was a mistake, and I am innocent.

So, where's the problem? The issue is that anyone who searches my name online will find that I am labeled as a fraudster. A thousand headlines calling me a fraudster pop up. Now, how do I fix this?

I could go to Google and say, "Look, here's the court ruling. I am innocent." Or I could approach the news agencies and ask them to reconsider, saying, "Look, I am innocent. Please take down your article."

This is where RTBF comes into play.³¹ It allows me to request the removal of that information from search engines and news outlets.

I have the right not to share everything

Have you heard that people say not to ask these questions:

- When do you want to get married?
- When do you want to have children?
- How much do you earn?
- Why are you getting leaner?
- Why are you getting fatter?

This is because these questions are related to privacy. People might not want to share everything about their lives. They might want to keep some things private. I have the right not to share my diary with you. It's not that you are necessarily a bad person; it's that I might want to keep some specific things private.

You don't need to have bad things to hide to want privacy. You need privacy because you are human!

“ Arguing that you don't care about the right to privacy because you have nothing to hide is no different than saying you don't care about free speech because you have nothing to say.

— Edward Snowden

³¹See also: <https://gdpr.eu/right-to-be-forgotten/>, <https://gdpr.eu/article-17-right-to-be-forgotten/>

Read-world Scenarios

Did you know that the NSA (National Security Agency) was collecting data on everyone? They were collecting data on everyone, even if you were not a criminal, and this was against the law.

Why I Mostly Talk About the NSA?

In this book, I mostly talk about the NSA because they are the most famous and have the most resources. They are the most powerful.³² There is real evidence that they have been collecting data on everyone. Furthermore, there are limited resources about other countries. But keep in mind that every country has an intelligence agency, and they are doing similar things or even worse things.

3.2.2 Story of the Government

There is a story that the government is actually my servant because we humans have historically tried to divide our tasks. For example, a person with high physical ability was assigned the responsibility of protecting others. We appointed someone to be responsible for cooking. We divided responsibilities. The modern government is the same. The police are actually my employees; I pay them to protect me.

This leads to the following law:

3.2.3 Freedom of Information Act (FOIA)

The Freedom of Information Act (FOIA) is a law that gives you the right to access information from the federal government. It is often described as the law that keeps citizens informed about their government. Under the FOIA, agencies must disclose any information requested unless it falls under one of nine exemptions that protect interests such as personal privacy, national security, and law enforcement.

Why? Because you are paying them! You have the right to know what they're doing with your money. This might seem strange in some countries because the government is seen as a higher authority. But in reality, the government is your servant. You are the boss.

A clear example of this is the request to the FBI regarding what data it can collect from encrypted messaging services through law enforcement actions.³³

³²They knew about some attacks before academics knew about them. For instance, they knew *differential cryptanalysis* twenty years before Adi Shamir found it.

³³<https://web.archive.org/web/20230108195447/https://propertyofthepeople.org/document-detail/?doc-id=21114562>

“ I must have the right to get accessed to the information of things that are done with my money. For example I might want to create a program that helps people to find nearest trash cans based on their location. I need to have the data of the trash cans.

— *Jadi Mirmirani*

FOIA Further helps us to disclose other real-world Scenarios.

3.2.4 Who Watches the Watchers?

Agencies are watching us. But who watches the agencies? Who watches them to ensure they are not doing bad things? To ensure they are not using their power against those they don't like? To ensure they are not using their power to manipulate people? To ensure they are not creating a dictatorship?

An agency might be good now, but is there any guarantee that it will be good in the future?

This is why police officers have body cameras.³⁴ Without body cameras, police officers can act as they wish. They can extort money from people. They can beat people. Subsequently, they can say that the person was resisting arrest. They can say that the person was attacking them, etc.

Reading List

- China's bank run victims planned to protest. Then their Covid health codes turned red³⁵

3.2.5 It's Not All About Security Agencies

Many companies do collect data on you. It's not just about security agencies like the NSA! Various companies can also collect our data and sell it. For instance:

- In the United States in 2017, internet service providers were allowed to sell users' browsing data for advertising purposes.³⁶
- Verizon has even been fined in the past for compromising individuals' privacy.³⁷
- AT&T Offers Data Privacy — for a Price³⁸

³⁴Read more: <https://nij.ojp.gov/topics/articles/body-worn-cameras-what-evidence-tells-us>

³⁵<https://www.cnn.com/2022/06/15/china/china-zhengzhou-bank-fraud-health-code-protest-intl-hnk>

³⁶Senate votes to let ISPs sell your Web browsing history to advertisers: <https://arstechnica.com/tech-policy/2017/03/senate-votes-to-let-isps-sell-your-web-browsing-history-to-advertisers/>

³⁷Verizon Injecting Perma-Cookies to Track Mobile Customers, Bypassing Privacy Controls: <https://www.eff.org/deeplinks/2014/11/verizon-x-uidh>

³⁸<https://www.wsj.com/articles/BL-DGB-40475>

3.2 Privacy

These are just a few examples of how companies can collect and sell our data. They might say, “We are not selling your data. We are just selling access to it.” But in reality, they are selling your data.

- So, why does this happen? Why do they want to keep tabs on us?

- It largely comes down to advertising. Advertisers often say, “For every 1,000 views, we’ll pay you, the company we’re advertising for, a dollar.” However, they want to ensure that those 1,000 views are genuine and not just inflated by bots that could easily generate fake traffic. To satisfy their clients, advertisers strive to create mechanisms that ensure these views are not only real but also targeted. For example, they might say, “We’ll show your white shoes to people who like white shoes.” This way, they’re not just showing the ad to everyone, but specifically to those who are more likely to pay attention to it and buy the product. As a result, instead of earning one dollar for every 1,000 views, they might earn five dollars!

In reality, this system benefits both the advertisers and the companies, but it comes at our expense. They need to collect data from us to display relevant ads, and this data isn’t just used for advertising purposes.

3.2.6 Analysis on Data & Data is Power

Why should we be careful about data? Even if the data is not sensitive, it might be used against you. Analysis makes it possible to know more about your past and predict or even change your future.

Inadvertent Data Collection: The Unseen Aspects of Your Online Presence

When we talk about collecting data, we don’t mention only the data that you have shared; we also mention the data that has been collected about you. This data might be collected unintentionally. It might be collected from other people you’re connected with. Or you might have accepted it through the tick you marked to accept the terms and conditions. Or you might have accepted the cookies.

Uncovering Hidden Patterns: The Surprising Insights of Contextual Data Analysis

Even small amounts of data, when analyzed in context, can reveal insights you might never have anticipated. For example, there is a famous story that says those who buy milk in large bottles likely have ancestors who experienced famine. They do this to ensure they always have a supply on hand, driven by their subconscious. Or see *Epigenetic signatures of intergenerational exposure*

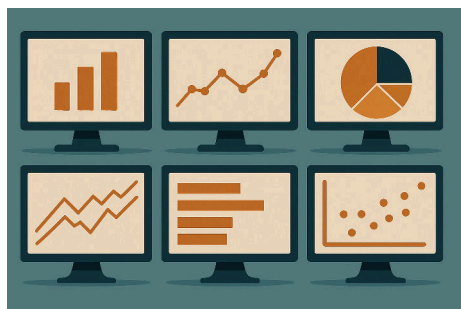


Figure 3.13: Analysis on Data and extracting meaningful patterns

to violence in three generations of Syrian refugees[4]. Or *Pizza Meter* story³⁹

Data-Driven Influence: How Data Analysis Shapes Your Digital Experience and Your Digital Life

When we talk about analyzing or monitoring data, we don't mean someone is sitting and watching you. We mean that the data is being processed by a computer and meaningful patterns extracted.

There is a huge list of tools that can search for you around the internet with just one click.⁴⁰ With a huge amount of data, computers are able to know more about you and possibly change your behavior.

Have you ever noticed that some people, when they speak, seem to have you under their control, and you feel almost enchanted, doing whatever they want? You might agree with what they say without realizing that you are under their control. This is because they have information about you and know how to behave.

Large-scale data collection and data analyzing allows the actor to know about you. What guarantees that they don't use it against you?!

Of course, this is not common in daily life. But in the digital world, this is made possible by analyzing your data.

With large-scale data collection, they can change your future. For example, they know how to give you ads in order to convince you to vote for a specific person. For instance, they know you like cats. They can fabricate fake news targeting individuals who oppose certain topics, such as cats. They can show you that news and make you hate that person. They may subtly promote certain individuals by embedding favorable narratives within published content, often without your awareness. See "Scottish independence debate targeted by

³⁹A sudden increase in pizza orders from Domino's by U.S. government offices, prior to major events like the invasion of Iraq into Kuwait, etc. As they likely had a meeting. See more at: <https://tradesmith.com/the-pizza-meter-a-seven-figure-retirement-and-you/>

⁴⁰<https://github.com/The-Usint-Toolbox/Email-Username-OSINT>

3.2 Privacy

Iran-based fake social media accounts".⁴¹⁴²

Have you noticed that when you search for something or watch videos related to a topic, you start to see ads about that subject? This happens because your data is being analyzed and used to show you ads that you might be interested in. This practice is called targeted advertising. While this is not inherently bad, there is no guarantee that the data won't be used against you.

Challenging Yet Essential: The Value of Upholding Small Privacy Rights

Protecting privacy can be challenging and comes with limitations, but failing to do so has even heavier consequences. When you don't defend your basic and small rights, you won't be able to stand up for larger rights later on! If you don't resist companies collecting data, it becomes normal for all companies to gather data. Each time, the next company will collect even more data for its own advancement, and you won't be able to question it. In the end, they will say, "Don't want to? Then don't use this app!" This means you won't be able to do anything about it later! Ultimately, all companies and agencies will say, "Well, if you don't want to, then don't use it!"

People often realize the value of something only when they lose it or find themselves in a difficult situation. Like people who have experienced war, they know the value of peace. You won't realize the value of privacy until you lose it.

So, defend your small rights, even if it doesn't seem to harm you personally. Because it harms others! Eventually, the world will become so bad that you will feel the impact too!

There is a really beautiful poem about this in Persian:

small stream can be blocked by a shovel. But when it becomes a
river, you can't even pass through it with an elephant.⁴³

So, don't let the small things go. Because they will become big things later on.

They Lie to You

Surely, no government or agency would tell you that they are collecting your data for a dictatorship. They always say they are collecting your data for your safety. They want to catch terrorists. They have to catch criminals. See also (How they manipulate): [10]⁴⁴

Once senator *Ron Wyden* asked *James Clipper*:

⁴¹[https://www.heraldscotland.com/politics/19867365.scottish-independence-debate-targeted-iran-based-fake-](https://www.heraldscotland.com/politics/19867365.scottish-independence-debate-targeted-iran-based-fake-iranian-pro-scottish-independence-accounts-go-silent-after-israel-attacks)

⁴²Iranian pro-Scottish independence accounts go silent after Is-
rael attacks: <https://www.scottishdailyexpress.co.uk/news/politics/iranian-pro-scottish-independence-accounts-35450209>

⁴³Saadi

⁴⁴<https://www.reuters.com/world/china/china-bank-protest-stopped-by-health-codes-turning-red-depositors->

3.2 Privacy

→ Does the NSA collect any type of data at all on millions or hundreds of millions of Americans?

● No sir.

→ It does not?

● Not wittingly. There are cases where they could inadvertently, perhaps, collect, but not wittingly.[11]

He later said: "I responded in what I thought was the most truthful, or least untruthful manner by saying no" This is the reason why Edward Snowden led the collection of data:

I would say sort of the breaking point is seeing the Director of National Intelligence, James Clapper, directly lie under oath to Congress.⁴⁵

President Obama said:

Nobody is listening to your telephone calls.⁴⁶

Let me respond to this with a quote by Stewart Baker (former NSA director):

Metadata absolutely tells you everything about somebody's life. If you have enough metadata you don't really need content

3.2.7 The Double-Edged Sword of Monitoring and Collecting Data

Having data is dangerous. How can we be sure that employees of the company don't use the data for their own benefit?

● We regularly check our employees.

→ How can we be sure you actually do this correctly?! You checked Edward Snowden! But what happened? He leaked the data about your data collection!

→ How can we be sure that a mistake won't happen and the information won't get leaked? We've seen many times that an employee from a security organization took confidential information home, and that's how the information got exposed!⁴⁷

→ Let's assume all the employees are good and won't make any mistakes (which is not possible). How can we be sure that nobody from outside would pressure them and threaten their families to get the data?

→ How can we be sure that the agency or company won't get hacked? At the very beginning, we said any system can be hacked! No matter how secure it is!

Nobody cares about you. They want money.

⁴⁵Snowden-Interview: Transcript: https://web.archive.org/web/20140131063748/https://www.ndr.de/ratgeber/netzwelt/snowden277_page-2.html

⁴⁶Obama: Nobody is listening to your calls <https://youtu.be/z0kHtzHJEZ0>

⁴⁷https://www.schneier.com/blog/archives/2017/10/yet_another_rus.html



Figure 3.14: A door hidden behind green bushes.

3.2.8 The Chain Reaction of Backdoors: How one Backdoor can Lead to Many

Imagine you have a house with a front door that is locked with a secure and strong lock. But you also have a door at the back of the house that is not locked. The door is hidden in the back, and nobody knows about it. But what if a thief finds it? They can easily enter your house.

Let me give you another example: You have a safe with a strong lock. But you also have a hidden key under the safe. Nobody knows about it. But what if a thief finds it? They can easily open the safe.

Definition 1. *Backdoor: A backdoor is a method, often secret and hidden, to bypass security mechanisms. It can be used to access a computer system, a mobile device, or a software program.*

One of the security agencies' ideas is that companies should implement a backdoor in their software to allow the agencies to access the data to catch criminals.

The problem here is that nobody can guarantee that the backdoor won't be found by bad people. If a backdoor is found by bad people, they can access the data and use it for their own benefit. This is why many people are against the idea of implementing a backdoor.

The same goes for security based on secrecy. If the security of a system is based on secrecy, as soon as the secret is revealed, the security is gone.

Also, if a country decides to force a company to implement a backdoor, other countries might do the same. They will say, "If you want to sell your software in our country, you have to implement a backdoor for us too. If you don't, we will ban your software for our citizens." This will lead to a situation where all

3.2 Privacy

companies have to implement a backdoor for all countries. This is a nightmare for privacy.⁴⁸

This is the exact idea of the conversation between Alex Stamos (CSO of Yahoo) and the director of the NSA [12].

Backdoors can't bring national security; they bring insecurity!

We have to have both privacy and security at the same time. *Benjamin Franklin* once said:

They who can give up essential Liberty to obtain a little temporary Safety, deserve neither Liberty nor Safety.

3.2.9 The Paradox of Data Overload: Valuable Insights Lost in the Noise

Useful information gets lost among all the other data. Organizations like the NSA collect so much information that valuable insights become buried within it. It's like searching for a needle in a haystack. They gather so much data that finding useful information, responding to it, and analyzing it becomes extremely difficult.⁴⁹

3.2.10 Not Only Yourself, But You Harm Others

By not respecting privacy, you also harm others. This way, individuals who need to use privacy-based tools like Tor for their work (such as journalists) become easily distinguishable, and that distinction is very harmful. It becomes clear that anyone using Tor must be hiding something, and they end up on a list that is always monitored.

For example, one of the advantages of Telegram is that you can communicate with others without revealing your phone number. However, when you use an unofficial client of Telegram, it might collect your contacts' numbers. This is detrimental to someone who wants to remain anonymous on Telegram, as others will now know the number associated with a particular account. Take a journalist who writes anonymously and goes against the opinions of powerful individuals; they would want to stay anonymous. Similarly, someone investigating cybercriminals must remain anonymous to avoid threats from those individuals!

● But is are no such person around me!

→ How do you know there aren't? Certainly, someone who wants to stay anonymous is not someone you would recognize. The person behind a certain

⁴⁸A little related -> Special Report: Amazon partnered with China propaganda arm <https://www.reuters.com/world/china/amazon-partnered-with-china-propaganda-arm-win-beijings-favor-document-shows-2021-12-17/>

⁴⁹1. NSA is so overwhelmed with data, it's no longer effective, says whistleblower (Former NSA official): <https://www.zdnet.com/article/nsa-whistleblower-overwhelmed-with-data-ineffective/> 2. The NSA's Call Record Program, a 9/11 Hijacker, and the Failure of Bulk Collection: <https://www.eff.org/deeplinks/2015/04/nsas-call-record-program-911-hijacker-and-failure-bulk-collection>

3.2 Privacy

account could very well be your next-door neighbor! That individual is likely not showing their true self in the real world either. If someone wants to remain anonymous, they definitely wouldn't want even their closest friends to find out! Because if they do, under certain circumstances, those friends might inadvertently reveal their identity or even be pressured to do so. Because they know your relationships. So, you definitely wouldn't know. Search "Ross Ulbricht"; does this face show he has launched one of the biggest drug websites on the dark web?

Or government can have excuse that why you don't want us to collect data? Are you a criminal?

3.2.11 The Solution

Privacy means that I have boundaries for myself! It's not about saying, "I won't give my data to Meta, but I will give it to Mozilla!" That's a mistake! The right approach is to not give my data at all! It's not about what's acceptable for one and wrong for the other; it's about maintaining my privacy across the board.

- So, what's the solution? What should we do?

- Well, to put it simply, there is no definitive solution! We can only offer some general suggestions.

- What do you mean? Please explain more!

- Privacy boundaries are different for everyone. I might not need to follow certain precautions, but you might need to based on your circumstances! For example, someone working in a specific section of the NSA has a job that requires confidentiality. If they don't maintain that secrecy, their life could be in danger! So, we really need to pay attention to our personal privacy boundaries and our threat model!

Definition 2. *Threat Model is a way of thinking about the potential risks you face and how you can protect yourself against them.*

Instead of watching all people, security agencies should invest in security tools which power individuals. Like what Germans do.⁵⁰

3.2.12 If You Don't Pay the Product, You Are the Product!

Why do you pay for ice cream? Because there are costs associated with producing the milk, as well as expenses for the people involved in the production process.

⁵⁰<https://www.evergabe-online.de/tenderdetails.html?0&id=397181&cookieCheck>

3.3 Anonymity

In the digital world, applications and content are also created by individuals who invest their time and effort. Therefore, using these resources naturally incurs a cost. Just because something is not physical does not mean it has no associated expenses. Free programs often have various revenue models:

- The provider values privacy and receives funding from organizations. In some countries, when you engage in a nonprofit activity, you can request funding by stating that you are helping people for free and need financial support. Thus, you can receive a portion of public revenue or taxes.
- Advertising
- Donations (We have talked about it in the beginning of the book)
- Collecting and selling data to other companies. This is where we say that if you don't pay the product, you are the product!

3.2.13 Reading List

- Citizenfour: A documentary about Edward Snowden and the NSA.

3.3 Anonymity

On the internet, everything you do is converted into messages, like sending a letter. Of course, in order to deliver that letter, you need to have an address. On the internet, you have an address too. This is called an IP address. This way, anything you do is traceable.

- Privacy: It is OK to others to know that you are doing something, but they shouldn't know what you are doing. (The thing is hidden) For example, in bathroom, you don't want others to know what you are doing. You just want them to know that you are in the bathroom.
- Anonymity: It is OK to others to know what is happening, but they shouldn't know who is doing it. (The person is hidden) For example, in bathroom, people see what's happening, but they don't know who is doing it. (The person is wearing a mask)

This, of course, requires a lot of work. You need to first learn about privacy and security and what actions you should take based on your own threat model. You should read all the topics in this book and take action. For now, to make it **harder** (not impossible) to trace you, you can use *Tor Browser*. It is a browser based on Firefox that routes your traffic through multiple servers. This way, the website you are visiting can't know who you are. It is slower than normal browsers because of the multiple servers, but it is worth it. It is important to not reveal anything that seems directly or indirectly related to your identity

3.4 End of the Chapter Questions

there. For example, don't use your real name, don't use your real email, don't use your real phone number, and don't talk about your interests, etc.

- Does Incognito mode/Private window help?

→ No! It only deletes browsing data from your computer after you've finished your browsing. Some settings for better privacy are also on. However, it doesn't prevent others (like your internet service provider or anyone who has access to your network) from seeing your browsing data.

3.3.1 Zero Trust Model

Always assume that the system is compromised. Don't trust anyone. There have been cases where a data breach or hack occurred and users' data was leaked. This is especially important for those working in sensitive areas, such as journalists, activists, and so on. For example, if you are using a service to publish your articles and the system gets compromised or attackers exploit a vulnerability, they can see your device information and probably your IP address. So always use Tor and related technologies to hide your identity.

Suppose you're an activist and want to publish an article about a sensitive topic through a service (e.g., a blog or someone's Telegram channel). The zero trust model tells us to assume the service is already compromised. Thus, attackers may also see who submitted the article to, for example, the admin of the Telegram channel. If you've used your real account, they can easily find out who wrote the article.

Always assume the other party you're communicating with may **NOT** do what you expect them to do. For example, you send a photo of your ID card to a service, expecting them to hide certain information before sharing it with others. The zero trust model tells us to assume they won't do what they're supposed to do. So, you should hide the information yourself before sending it. Additionally, always assume the data stored on the service is stored insecurely. Therefore, do NOT provide any sensitive information you think is not required.

3.4 End of the Chapter Questions

Problem 3.1 Some people suggest that with smart phones, protecting privacy is a joke. They suggest you should have a classic push button phone. In this way you cannot be tracked and hacked. What do you think about this idea?

Problem 3.2 When a critical vulnerability is discovered, users should update their software immediately to patch it. However, many users delay updating their devices. You are in charge of a company. One of your programmers suggests implementing a backdoor in your product so that, in essential cases, your company can access users' devices and update them. Is this a good recommendation? Why or why not?

3.5 End of the Chapter Solutions

Problem 3.3 Is open-source software inherently secure? Discuss potential problems that can arise in the open-source software supply chain.

Problem 3.4 When it comes to dual booting, we recommend having a separate password for each operating system. Why? Even if the password is very strong.

Problem 3.5 In situation where a critical vulnerability is found, people should update their software as soon as possible. But many people don't update their software soon. A programmer suggests that the your software should have a backdoor to we ourself can update the software. What do you think about this idea?

Problem 3.6 Implement a backdoor for a vending machine.

Problem 3.7 Create a threat mode for a journalist.

Problem 3.8 When a security algorithm is not "provably secure," what is the primary reason it might still be trusted? Is it because it has been kept secret so no one knows its weaknesses?

Problem 3.9 Which one is correct? In the field of security, genuine assurance and trust in algorithms primarily stem from:

- A. The algorithm's mathematical complexity and length of its keys.
- B. Keeping the algorithm's design and implementation details secret (obscurity).
- C. Endorsement by a single, highly reputable government agency.

Problem 3.10 Why should we avoid providing unnecessary information to services and always minimize the data we share with anyone (even friends)?

Problem 3.11 There are many Telegram channels and websites that claim to return a person's phone number, name, ID, or address given some information (for example, a phone number). What privacy issues can this cause?

3.5 End of the Chapter Solutions

Solution 3.1 No. Many push button phones use simple and old protocols which are not secure. For example, in Russia, some push button phones were sold with malware preinstalled.⁵¹

Solution 3.2 No! The problem with backdoors is that if they are discovered by someone else, others can also exploit them for their own purposes. This discovery can occur in several ways:

⁵¹<https://therecord.media/malware-found-preinstalled-in-classic-push-button-phones-sold-in-russia/>

3.5 End of the Chapter Solutions

- Other individuals might review the software and identify these vulnerabilities.
- An employee within the organization who has access to the code could discover the backdoor and sell this information for personal profit.
- Someone aware of the backdoor might be threatened or coerced into revealing the information.

Solution 3.3 No, open-source software is not inherently secure. The term “open-source” signifies that a program’s source code is publicly accessible, but this transparency alone does not guarantee its security. Several issues can arise in the open-source software supply chain, such as:

- Malicious actors can gain trust within a project by making consistent contributions. Once they become accepted members of the community, they might introduce malicious code or sabotage the project.
- Many open-source projects are initiated as hobbies rather than formal, serious undertakings. Consequently, they may lack structured protocols for security reviews, defined release processes, or rigorous security testing. This increases the likelihood of security vulnerabilities, particularly in less prominent projects.
- Due to the hobbyist nature of many projects, the use of outdated libraries is common. These outdated components can introduce known vulnerabilities into the software.
- Users often download and utilize software releases compiled by various, sometimes anonymous, individuals without verifying that the compiled binary perfectly matches the officially published source code. This discrepancy can lead to running unintended or malicious code.

Solution 3.4 The reason is that if you use the same password for both operating systems and one of them has a vulnerability that leads to password discovery, an attacker can access both systems. The weakest link is the most vulnerable operating system.

Solution 3.5 This is a bad idea. Because if a backdoor is found by bad people, they can access the data and use it for their own benefits. Or one of the programmers can sell it to bad people. Or one of the employees may be forced to give the backdoor to bad people.

Solution 3.6 if a person enters a specific code, the vending machine will give them a free drink.

Solution 3.7 empty!

Solution 3.8 No. The primary reason is that it has been used for a long time

and tested by many experts. It has been used in many real-world applications and has not been broken. "Security through obscurity" is not a good idea to rely on.

Solution 3.9 None of them. No single entity can guarantee the security of an algorithm. (C) The best way to ensure security is to have the algorithm publicly reviewed and tested by many experts in the field. (B) This is called "open security." To check the security of an algorithm, it should be simple and easy to understand. The more complex it is, the more likely it is to have hidden weaknesses. (A)

You may wonder that shouldn't one be correct? As the type of question suggest. I must say the wrong thing is wrong. Even if it is used by millions and are the default choice. A hacker mind should also think out of borders.

Solution 3.10 Just look at the many available services that map one piece of information to another (phone number to name, name to address, etc.). Some of this data is gathered from leaked databases (data exposed through insecure company storage) or from having your information saved in a friend's contact list. Friends may grant contact-access permission to any app they download, and some of those apps collect the information their friends have stored about them.

Adopt a *Zero Trust* mindset: don't assume anyone needs your full details. Share only the bare minimum.

Solution 3.11 Even if these services sometimes work, they create serious privacy risks. Operators can collect submitted data and link it with other records to build social graphs and deanonymize people. By sending a phone number (yours or someone else's) you reveal relationships and confirm identity ties that were previously private.

Such services also undermine privacy features (e.g., Telegram usernames that hide your number), because users voluntarily disclose numbers to the service.

Mitigations: do not submit other people's numbers without consent, avoid untrusted lookup services, and prefer privacy-preserving or official channels for any necessary lookups.

3.5 End of the Chapter Solutions



3.5 End of the Chapter Solutions

4.0 Cryptography

4.1 Why you should know about cryptography

Cryptography is one of the most important tools for securing data. Everyone who works in the field of computing should have a basic understanding of cryptography. Without cryptography, your data can be read by anyone. Without cryptography:

- Anyone can steal your banking information when you are doing online shopping
- Anyone can read your messages when you are chatting with your friends
- Someone can change your grades in the school system
- Someone can send a message to your friend using your name and profile.
- People can see what websites you are visiting
- You can't be sure the file you downloaded is the original one
- We wouldn't have cryptocurrencies.
- Secret military information can be stolen.

With cryptography, you can search for something in a search engine (e.g., Google, DuckDuckGo, etc.) without revealing who you are and without the search engine knowing what you've searched, yet still get the correct result! Yeah! This is possible. It is called *Privately Outsourcing Computation*

4.2 Reading List

I will try to explain the basics of cryptography mainly for non-technical people. It's important to note that cryptography is a complex field, and you should **NOT** take advice from me or anyone who is not a qualified professional. I recommend viewing this document as a good starting point, but please do not

4.3 What problem am I solving?

rely on the advice or information provided here. Instead, seek out more authoritative sources for guidance in this area. There are sources you may want to consider:

- **Books:**

- *Applied Cryptography: Protocols, Algorithms, and Source Code in C*, second edition, by Bruce Schneier (I recommend that anyone who wants to know more about security read this book. Although it is a little bit old, it is still a wonderful book.)
- *Serious Cryptography: A Practical Introduction to Modern Encryption*, second edition, by Jean-Philippe Aumasson
- *Cryptography Engineering: Design Principles and Practical Applications*, by Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno

- **Online Courses:** There are many online courses on cryptography. Some of them are:

- *Cryptography I* by Stanford School of Engineering¹
- *Cryptography II* by Stanford School of Engineering²
- *Cryptography* by Christof Paar on YouTube

- **Conferences:** There are many conferences on cryptography and security. Some of them are:

- *Crypto* by IACR
- *Eurocrypt* by IACR
- *RSA Conference* by RSA
- *Black Hat* by Black Hat

4.3 What problem am I solving?

There are many documents out there. Why another one? Most of them are either very technical, involving number theory basics, or they just use tools with no explanation about how they work or what not to do with them.

I want to explain wrong behaviors while using crypto technologies and suggest some better (but not ideal!) practices. In addition to that, I hope you become interested in the field and start to learn more about it, as well as write your programs and act in cyberspace more securely.

¹<https://online.stanford.edu/courses/soe-y0001-cryptography-i>

²<https://online.stanford.edu/courses/soe-y0002-cryptography-ii>

4.4 Cryptographic Hash Function

We will talk about **cryptographic** hash functions, not those hashes you may have heard of in hash tables

Imagine you have a website. People come and create accounts there. You have to store a username and password for each user. Something like this³:

Username	Password
Alice	1234
Bob	5678
Charlie	9012

This structure helps you authenticate users during login. You simply get the username and password from the user, then check whether they are valid. But is it safe and secure? **NO!** The problem is that if a person hacks into your servers, they can see all the passwords! So, what should you do?! Let me introduce you to hash functions:

I) A hash is a **one-way** function that gives a strange output for a given input. For example, if you hash 1234, you get

03ac674216f3e15c761ee1a5e255f067953623c8b388b4459e13f978d7c846f4

It is a strange text, isn't it? As it is one-way, it is impossible to extract 1234 from this text. This means there is no function that takes that strange text and returns 1234.

II) Hash is a **deterministic** function. It means if you run that algorithm in any system, you always get the same result.

III) Each text (or, more precisely, each binary) has its own hash (two different binaries have two different hashes):

hello → 2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

Kamal → d091eea64f6591f3e4f5f484543bfe196ce1c59f9cbb8b0fe1e7ee477a5fdb87

hi → 8f434346648f6b96df89dda901c5176b10a6d83961dd3c1ac88b59b2dc327aa4

IV) If you change one bit of the text, the hash will be completely different. For example, if you hash hello, you get:

2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c1fa7425e73043362938b9824

and if you hash Hello, you get:

185f8db32271fe25f561a6fc938b2e264306ec304eda518007d1764826381969

See? I only changed some bits of the text (see in ASCII), but the hash is completely different.

³Never ever use this method! Additionally never ever use these simple passwords.

4.4 Cryptographic Hash Function

V) Not only can I not find a function that takes a hash and returns the text, but I also can't guess what input will produce a specific hash. This means I can't determine if changing the 4th bit will change the output in a specific way (without running the algorithm).

In summary:

1. It is one-way. From output, you can't get the input.
2. It is deterministic. You always get the same output for a given input.
3. Each input has its own hash. No two different inputs have the same hash.⁴
4. Even if you change one bit, the hash will be completely different.⁵
5. It is fixed-length. No matter how long the input is, the output is always the same length. (e.g. 256 bits)⁶
6. An arbitrary output cannot be generated from a given input.⁷
7. It is a like a random mapping from all possible inputs to all possible outputs.[13]

The above list suggests that we can use a hash function for better password storage⁸. Instead of storing the password, we store the hash of the password:

Username	Hash
Alice	03ac674216f3e15c79e13f978
Bob	185f8db32271fe25f561a938b
Charlie	8f434346648f6b96df89dda90

When a user wants to log in, we get the password, hash it locally⁹, and compare it with the stored hash. If they are the same, we let the user in. This way, even if a person hacks into your servers, they can't see the passwords. They can see the hash, which, because of its one-way nature, prevents them from retrieving the password.

Let's create a hash algorithm. I associate each character with a number:

Character	Number
A	1
B	2
C	3
D	4
E	5

⁴Collision-resistant

⁵Avalanche effect

⁶Although we have variable length output hashes, but here we only consider fixed length one which are easier to understand.

⁷Pre-image resistant

⁸Still not secure!

⁹Because of the security of the user, it is important that passwords never leave the client device.

4.4 Cryptographic Hash Function

Now let's hash $ABD \rightarrow 1+2+4 = 7$ For better one-wayness, I can use the modulo operation. So, I can compute the hash of ABD as $(1+2+4) \bmod 5 = 2$ Now do you see? Someone doesn't know whether 2 means AA or B or AF , etc. This demonstrates that one-wayness is possible.¹⁰

But my hash function has several major problems:

- It only supports characters, not all binaries.
- Two different inputs can have the same output. This is called a collision.

4.4.1 Birthday Problem and Hash Collision

I have brought you this problem to explain why self-made hash functions are not secure and why it is important for cryptography-related algorithms to be public and analyzed by mathematicians and cryptographers.

Example 4.4.1. *In a room with 23 people, what is the probability that at least two people share the same birthday?*

The first answer that comes to mind is that the probability is something like $\frac{23}{365}$ or $1 - \frac{23}{365}$. But let's play with its mathematics. When it asks for the probability that at least two people share the same birthday, we can find the probability that no two people share the same birthday and subtract it from 1.

With two people, the probability that they don't share the same birthday is $\frac{365}{365} \times \frac{364}{365}$. It means that the first person can choose any day and leaves the second person with 364 choices. With three people, the probability that they don't share the same birthday is $\frac{365}{365} \times \frac{364}{365} \times \frac{363}{365}$. It means that the first person can choose any day, leaves the second person with 364 choices. After the second person chooses a day, the third person has 363 choices. With 23 people, the probability that they don't share the same birthday is $\frac{365}{365} \times \frac{364}{365} \times \dots \times \frac{343}{365} \approx 0.5$. So the probability that at least two people share the same birthday is $1 - 0.5 = 0.5$, or 50%. That is quite high, isn't it?

Conclusion: In general, if an element can take on N different values, then you can expect the first collision after choosing about $\sqrt{N}$ random elements (Because the probability is 50%)

Example 4.4.2. *You want to assign a unique and random number to each person in the world. If the population of the world is 8 billion, is a 64-bit integer enough for this task?*

The answer is no! With this, you will find a collision in roughly $\sqrt{2^{64}} = 2^{32} = 4,294,967,296$ people. This means that with 4,294,967,296 people, you will find two people with the same number.

• There is a question in my mind. You said that each binary has its own hash and no two hashes are the same. But I am skeptical about that. We can give

¹⁰Same is possible with XOR operation.

4.4 Cryptographic Hash Function

infinitely many inputs to a hash function. So, there should be a collision.

→ You are right. There is a collision. It is called *Pigeonhole Principle*. If you have N pigeonholes and M pigeons, and $M > N$, then there must be at least one pigeonhole with more than one pigeon. Nowadays we use 256-bit hash functions. 2^{256} is a very big number. For a collision, you would need about $\sqrt{2^{256}} = 2^{128} = 340,282,366,920,938,463,463,374,607,431,768,211,456$ attempts, which is an extremely large number. It is extremely difficult to find two different inputs with the same hash.

For example, *MD5* is a 128-bit hash function. This means that with about 2^{64} inputs, you will find two inputs with the same hash. It is not secure anymore.¹¹ *SHA-1* is a 160-bit hash function. It is not secure anymore.¹²

We use *SHA-2* and *SHA-3* which are recommended by NIST.

- Finding a collision for a hash is not practical. It requires a lot of memory to store values and to check each new value against the database.

→ No. There is a method called *Pollard's Rho*, which doesn't require much memory.

4.4.2 Applications

Do you remember I told you that with any change to the input, the output will be completely different? Think about this and find an application for this property.

Suppose you have received a file for an application from your friend. You don't trust them, and you want to know whether or not the file has been changed (and possibly contains viruses). You can hash the file and compare it with the hash provided by the developer. If they are the same, you can be sure that the file has not changed. If they are different, you can be sure that the file has been altered. Most developers provide the hash of the file on their website. For example, *Handbrake* (a video converter software)¹³ provides the hash of the file on their website:

¹¹<https://mail.python.org/pipermail/python-dev/2005-December/058850.html>

¹²<https://security.googleblog.com/2017/02/announcing-first-sha1-collision.html>

¹³You can also use Handbrake to reduce the size of a video by 80 percent without loss of quality! In the Dimensions menu, set the cropping option to None. In the Video section, test the RF number to see what works best for you. You can record a very short video and use it for testing, so it doesn't take too much time to reduce the file size. For example, record a 30-second video and test different numbers. Usually, 32 or 28 is a good value. You can test slightly higher or lower values. The higher the number, the more the file size will be reduced (but beyond a certain point, the quality will decrease). If the video quality is lower, you can increase this number. After that, click Start to begin.

4.4 Cryptographic Hash Function

Current Version: 1.5.1

**macOS**
For 10.13 and later
[Download \(Universal\)](#)

**Windows**
For 10 and later
[Download \(x64 64 bit\)](#)
[Download \(x64 64 bit Portable Zip\)](#)
For ARM Devices
[Download \(ARM 64\)](#)
[Download \(ARM 64 Portable Zip\)](#)

**Linux**
Flatpak
[Install via flathub.org](#)
or
[Download \(64bit\)](#)
[QuickSync Plugin Download \(64bit\)](#)

**Other**
[Command Line Version](#)
[Source Code](#)

Development Builds
[Hosted externally on Github](#)

Old Releases
[Release Archives](#)

Download Safety

Please take note that HandBrake.fr is the only official place where HandBrake can be downloaded from. There are many unofficial mirrors of HandBrake and while most of them offer legit versions of HandBrake, there are a few that don't.

- Read our guide to [Downloading and Installing HandBrake](#)
- Check the integrity of your download with [Checksums](#) (mirrored on our [Github Wiki](#))
- Check the authenticity of your download with [Open PGP](#) (mirrored on our [Github Wiki](#))

Validate your Download

Please see our guide to safely [downloading and installing HandBrake](#).

Version 1.5.1		
File	Size (MB)	SHA256
HandBrake-1.5.1-x86_64-Win_GUI.exe	19.49	01167a96a338cb394ee2e339545379cd156dfe4b1837af64764a08279f8af33e
HandBrakeCLI-1.5.1-win-aarch64.zip	14.64	dfcc82e756ddbc67ba3d71fcf67377d06f21aadf1c54a8413797e6398294d49d
HandBrakeCLI-1.5.1-win-x86_64.zip	17.45	496e9111341095305e46f331463281e93fabed926381d28b79a2bd3785c49954
HandBrakeCLI-1.5.1-x86_64.flatpak	9.8	61ec9503d8e656bc16d1d5e220497491eb9ae4dc484a51c4d79f365b164ba509
HandBrakeCLI-1.5.1.dmg	32.06	328ad1fbacb855b644b63899450c004cb18e5e819ad519549c4c4bc863a60f90
HandBrake-1.5.1-source.tar.bz2	15.39	3999fe06d5309c819799a73a968a8ec3840e7840c2b64af8f5cdb7fd8c9430f0
HandBrake-1.5.1-arm64-Win_GUI.zip	22.8	dc9e8395945778d6819bd0063ab21bd666dbc9abfd93130d4900652c80c36350
HandBrake-1.5.1-arm64-Win_GUI.exe	15.27	b5468cfb3e8d469e72a68a28f57624ad287e5b22c7407582c1aed4193ea70299
HandBrake-1.5.1-x86_64-Win_GUI.zip	27.44	69e499d88d6f77a5ce663c8f5ae3ff2e6210a908152a7c437d10bca7294d0be
HandBrake-1.5.1-x86_64.flatpak	23.26	23a459b3dd02c4ccd9c53c4c1585a452e7557d6011be276740afa6634a2ca66f
HandBrake-1.5.1.dmg	35.91	767cb16314e3869c42cff78db92bcad7a7f1aa861c70f97b1326fe3686c62b61f
Plugin.HandBrake.IntelMediaSDK-1.5.1-x86_64.flatpak	58.31	61768dce2776df0220d18477e2f5bda8e7512583bff6e994157e691f532efe46

Version 1.5.0		
File	Size (MB)	SHA256
HandBrake-1.5.0-source.tar.bz2	25.94	72d79e8e0c6759f5855407c9b4de4273eb5fb6cc363238bf8d9a992c4b2a3c1a
HandBrake-1.5.0-arm64-Win_GUI.exe	15.25	42a8520911a70d46fcd9b4358a79d69cf36f71044a64df483daf01ec66ad2c58
HandBrake-1.5.0-arm64-Win_GUI.zip	22.8	a3d67b951b20a378095f1c3cedc3e451d8ce7043fed37672a44789f3ead575c3

This is what happens when you download a package or software in GNU/Linux distributions. The package manager automatically checks the hash of the package and compares it with the hash provided by the developer. If they are the same, it installs the package. If they are different, it shows an error message.

What do you think are the properties of hash functions used in this application? Mainly, collision resistance. Additionally, it must be fast to compute.

4.4 Cryptographic Hash Function

Why? Because you don't want to feel like you are waiting a long time to install software. On servers, many hashes are computed every second, so it must be fast.

4.4.3 Standardization of Hash Functions

Why do we need to standardize hash functions? One of the most important principles in security is that algorithms should be public. This means that everyone can review and analyze the algorithm. Who is better suited to promote this than a standards organization? This is the best way to encourage public scrutiny of an algorithm.

NIST (National Institute of Standards and Technology) is one of the most important standards organizations. It organizes competitions for new algorithms. Over several years, researchers attempt to break each other's algorithms. The algorithm that withstands all attacks is selected as the new standard.

The *NSA* (National Security Agency) is another important organization. It is involved in the standardization process and has extensive experience in the field of security, which can help identify vulnerabilities in algorithms.

4.4.4 SHA-3

SHA-3 is the latest member of the Secure Hash Algorithm family. It was released by NIST and originated from a competition called the SHA-3 competition. The goal was to find a new secure hash algorithm. First, some workshops were held to discuss the requirements for the new algorithm. Then, NIST invited the world to submit their algorithms for the competition. After five years of competition, Keccak was selected as the winner. It is now known as SHA-3. Hooray! We have a new secure hash algorithm.

Now you see why we say that self-created hash functions (and any security protocols) are not necessarily secure: five years of research went into this!

Let's delve deeper into the SHA-3 project.¹⁴

“ As was the case for the AES competition, security is the most important factor when evaluating the candidate hash algorithms. However, there remains significant disagreement within the cryptographic community at large over what security definitions should be used to evaluate hash algorithms. While initially proposed for use in digital signatures, cryptographic hash algorithms are used in a wide variety of applications, including message authentication codes, pseudorandom number generators, key derivation, and one-way functions for obfuscating password files. All of these applications have different security

— NIST

¹⁴<https://csrc.nist.gov/Projects/hash-functions/sha-3-project>

4.5 Password Hashing Algorithms

“ requirements.

— NIST

It is obvious that a hash should be secure. A hash that does not have the properties we discussed is not secure.

“ FRN-Nov07 identified cost as the second-most important criterion when evaluating candidate hash algorithms. In this case, cost includes computational efficiency and memory requirements. Computational efficiency essentially refers to the speed of an algorithm. NIST expects SHA-3 to offer improved performance over the SHA-2 family of hash algorithms at a given security strength. Memory requirements refer both to code size and random-access memory (RAM) requirements for software implementations, as well as gate counts for hardware implementations.

— NIST

If something is to be used worldwide and by any device, it must be fast. Don't just consider your laptop; think about a smartwatch or a smart card. They have very low computational power. It must be fast for them too. Additionally, it must be memory efficient.

“ The SHA-3 competition has received many candidate algorithms with new and interesting designs, and with unique features that are not present in the SHA-2 family of hash algorithms. Candidate algorithms with greater flexibility may be given preference over other algorithms. This includes algorithms capable of running efficiently on a wide variety of platforms, as well as algorithms that use parallelism or instruction set extensions to achieve higher performance. In addition, simple and elegant designs are preferable, in order to encourage understanding, analysis, and design confidence.

— NIST

As we discussed regarding falsifiability, in order to analyze and implement different attacks on an algorithm, it must be simple and elegant. If it is complex, it is hard to understand and analyze. It is hard to find vulnerabilities and design attacks on it.

4.5 Password Hashing Algorithms

Until now, we have only talked about certain types of hash functions. Here, we want to learn more about password hashing algorithms.

4.5.1 Traditional Brute-Force

Imagine you want to open a safe, but you don't have the password. What can you do? You can try all possible passwords, like 1111, 1112, 1113, and so on. Eventually, you will find the password. This is called a brute-force attack. It is the most basic type of attack. It is very slow, but it is effective.

If the password database is stolen, the attacker can use this method to find the passwords. They can try all possible passwords, compute their hashes, and compare them with the database. How can we make this more difficult?

Password hashing algorithms are designed to be slow and to consume a lot of memory. This way, calculating many hashes for attacks becomes much harder (compared to SHA).

```
time printf "password" | argon2 somesalt -id -t 4 -m
    16 -p 1
```

You will find that it takes roughly a fraction of a second (compared to SHA, which takes a fraction of a microsecond or less). Therefore, this process takes much more time.

4.5.2 Hash Table Attack

What if there was a table that contained all the hashes of all common passwords? Like:

Password	Hash
Admin	c1c224b03cd9bc7b6a86d77f5d
Alex2000	85566b2fad150f8d8297b383c5
123456	8d969eef6ecad3c29a3a629280

The attacker can simply compare the hash with the table. This is called a *Hash Table Attack*. In fact, there are many such tables on the internet. People can download them and use them in their attacks.

4.5.3 Rainbow Table Attack

The problem with a hash table attack is that the table is very large. For 1 billion passwords (aside from the passwords themselves), we need $1,000,000,000 \times 256 = 256\text{GB}$. What should we do? Of course, we should store less data. But how can we do that?

Remember that we said if one bit of the hash changes, the hash will be completely different? A hash is like a random mapping. We can use a portion of the hash instead of the whole hash. For example, we can use the first 16 bits and the last 16 bits of the hash:

4.5 Password Hashing Algorithms

Password	Hash
Admin	c1c224b0 - 86d77f5d
Alex2000	85566b2f - 97b383c5
123456	8d969eef - 3a629280

So if the hash is 256 bits, we save 224 bits (87.5% smaller). This method is called a *Rainbow Table Attack*.

● Well, you're right that we first check the beginning and end of the hashes, and if they match, it's very likely that the hash belongs to the password, for example, "admin", for which we stored the first and last characters. However, it's possible that another password could also have similar first and last characters. For instance:

- Admin hash: **c1c224b03cd9bc7b6a86d77f5d**
- Another password hash: **c1c224b0abc8ab61c986d77f5d**

→ I agree that there could be another hash with the same first and last characters but different in the middle. However, the probability of that happening is very low for several reasons:

- The hash function is designed to be collision-resistant. This means that it is very, very hard to find two different inputs with the same hash.
- We are storing well-known passwords, so the likelihood of encountering a famous password is much higher than finding a random password that just happens to have the same hash.
- Let's assume we are extremely unlucky and your point is valid. That's fine! When I'm trying to breach accounts in the database, even if I manage to access just 1,000 out of a million accounts and sell their information, that's enough for me. I've made my money, and that's what matters! So, it doesn't really concern me if, with that very low probability, I can't access a specific account. It's not a big deal unless I specifically want to access a particular account. But again, based on points 1 and 2, the likelihood of that happening is very low!

4.5.4 Salting

The rainbow table attack arises from the fact that everyone knows the hash of "admin." So, if the database is compromised, they can access it. What can we do? We need to ensure that the hash of "admin" is different in each database. If we add a random string to the beginning of the password, the hash will change. For example, instead of calculating the hash of `admin`, we calculate the hash of `abcdeadmin`! People know the hash of `admin`, but they haven't calculated the

4.5 Password Hashing Algorithms

hash of `abcdeadmin!` This way, we can prevent rainbow table attacks and hash table attacks. For example, we can change the table to:

Username	Password	Salt
Alice	e5791349f63f04c733cb6ac4d	a1b59e0c811fd
Bob	1557ff6a427896d3b5c0e867c	b2ef229ba023b
Charlie	e4bef5d7964c9efea8a211332	eef6ecad3c29f

In this way, the attacker needs to calculate the hash of “admin” with each salt. And remember that password hashing algorithms are slow. It will take a lot of time. Since the salts are random, the attacker can’t use precomputed tables.

The security of this method depends on the randomness and length of the salt. If you are working with these algorithms, you should seek advice about the appropriate length and how to generate the salt from authoritative sources.

4.5.5 Different Password Hashing Algorithms

Many times password hashing algorithms are used to generate an encryption key. See NIST recommendation and its FAQ¹⁵.

Argon2

- Winner of the Password Hashing Competition¹⁶
- Used in KeepassXC¹⁷, Bitwarden¹⁸, Tuta¹⁹
- RFC 9106²⁰

Scrypt

- Used in Litecoin²¹
- RFC 7914²²

Bcrypt

- Used in Proton Pass²³
- Hashing in Action: Understanding bcrypt²⁴

¹⁵<https://pages.nist.gov/800-63-FAQ/> Especially “Q-B17”

¹⁶<https://www.password-hashing.net/>

¹⁷https://keepassxc.org/docs/KeePassXC_UserGuide#_database_settings

¹⁸<https://bitwarden.com/help/what-encryption-is-used/#argon2id>

¹⁹<https://tutanota.com/blog/best-encryption-with-kdf>

²⁰<https://datatracker.ietf.org/doc/html/rfc9106>

²¹<https://litecoin.org/>

²²<https://datatracker.ietf.org/doc/html/rfc7914>

²³<https://proton.me/blog/proton-pass-security-model>

²⁴<https://auth0.com/blog/hashing-in-action-understanding-bcrypt/>

4.6 Applications of Hash Functions

PBKDF2

- Used in Lastpass²⁵, Bitwarden²⁶, 1Password^{27,28}, Apple²⁹

NIST says³⁰:

In response to the public comments received, NIST proposes to revise SP 800-132,

- to approve an additional memory-hard password-based key derivation function and password hashing scheme, and
- to provide additional guidelines and clarifications on the use of PBKDF2.

NIST³¹ and RFC 8018³² say:

The iteration count shall be selected as large as possible, as long as the time required to generate the key using the entered password is acceptable for the users. Since the capability of user machines varies (e.g., from a smart card to high-end workstations or servers), reasonable iteration counts vary accordingly. For especially critical keys, or for very powerful systems or systems where user-perceived performance is not critical, an iteration count of 10,000,000 may be appropriate

Conclusion: Always overestimate the iteration count and security measures. It is better to have a slow login than a hacked account. Additionally use an algorithm that slows down attackers more than users.³³

4.6 Applications of Hash Functions

4.6.1 Bitcoin (SHA2-256)

Currency Evolution: How Trust and Value Shaped Modern Money

Here is a very simple and imprecise explanation of Bitcoin and blockchain (this explanation is not detailed and is just meant to provide a basic understanding!

²⁵<https://support.lastpass.com/s/document-item?bundleId=lastpass&topicId=LastPass%2Fabout-password-iterations.html>

²⁶<https://bitwarden.com/help/what-encryption-is-used/#pbkdf2>

²⁷<https://support.1password.com/1password-security/>

²⁸<https://1passwordstatic.com/files/security/1password-white-paper.pdf>

²⁹https://help.apple.com/pdf/security/en_US/apple-platform-security-guide.pdf

³⁰<https://csrc.nist.gov/news/2023/proposal-to-revise-nist-sp-800-132-pbkdf>

³¹<https://nvlpubs.nist.gov/nistpubs/Legacy/SP/nistspecialpublication800-132.pdf>

³²<https://datatracker.ietf.org/doc/html/rfc8018#section-4.2>

³³Read More at: <https://blog.1password.com/bcrypt-is-great-but-is-password-cracking-infeasible/>

4.6 Applications of Hash Functions

The focus here is not on technical accuracy, but simply to understand that hashes have a purpose):

First, we need to talk about the philosophy of money. In the past, people engaged in barter, meaning they exchanged goods directly. For example, one person might say, "I'll give you two apples if you give me two oranges." However, this system had its problems. For instance, if you wanted to travel to another city, what would you do? You couldn't carry apples and oranges with you. So, people started going to trusted individuals who were honest and reliable. For example, someone would give ten apples to a trusted person, and that person would provide a note stating that this individual had deposited a certain value with them. When the person arrived at their destination, they could receive goods or whatever they needed equivalent to that value, and the trusted person would later return the apples. Gradually, this practice became common and laid the foundation for modern money.

In fact, modern money works in a similar way. The cash you hold doesn't have intrinsic value; rather, it represents that you have a certain amount of money, say five dollars, stored in the central bank. This is why people talk about the backing of money. The backing of a country's currency consists of the gold reserves that correspond to the banknotes it prints in its central bank³⁴. However, it's not necessarily just gold; other things can also provide value to money. Gold is just a prime example.

This is why the question "Why don't they just print more money so everyone can be rich?" is dismissed. The reason is that money itself has no intrinsic value; its worth comes from the backing behind it. That's why it's said not to print money arbitrarily, as it would decrease the value of the currency.

For instance, imagine we have one million dollars in cash and one million dollars' worth of gold. If we print another million dollars in cash, we would then have two million dollars in cash for the same one million dollars' worth of gold. This would halve the value of our cash because now, for every two million dollars in cash, we only have one million dollars' worth of gold. It's as if every two banknotes represent one unit of gold. Therefore, any cash printed without backing will harm the value of the currency.

Now that we understand how money came into existence, let's consider another question: Why does gold have value?

One reason is that gold has properties that make it useful in various applications. However, aside from its industrial uses, another reason for its value is its scarcity. Because it is rare and in demand, it holds value. If a large gold mine were discovered tomorrow, the value of gold would decrease. Similarly, if one day everyone woke up and decided they no longer accepted gold, demand would drop, and when demand decreases, people would no longer attribute value to gold, causing its worth to diminish even further (regardless of its physical properties).

³⁴*Inside the Bank of England's gold vaults* From World Gold Council on Youtube: <https://youtu.be/FVnxhsB92v4?feature=shared>

Bitcoin is Here

Bitcoin came along and said, “Anyone with a regular computer should be able to own and mine me.” That’s great! Everyone should have a share. But how do we distribute Bitcoin among people? Individuals literally need to put in the effort. Just like finding gold requires hard work, here too, people must prove that they have put in the effort, which is essentially what Proof of Work means.

So, what is one way to prove that work has been done? Can you guess? Yes, it’s calculating hashes! Bitcoin uses the SHA-256 hash function, which, as we’ve shown, can be calculated relatively quickly. That’s why Satoshi Nakamoto said that people need to keep calculating hashes until they find a certain number of leading zeros in their hash. This means they have to brute-force their way to find those specific zeros. Up to this point, everything seems normal.

However, things became unfair. Specialized devices were created (ASICs/FPGAs) solely for mining Bitcoin. We know that the computers we use can run games, browse the web, watch videos, and do a thousand other tasks. Their versatility means they can do everything, but not necessarily at very high speeds. But then, devices were developed that could only calculate hashes, and they did so at a much higher speed. These are called miners.

This is where the situation became unfair. Anyone with more money could buy special devices that could mine at a much higher speed. This meant that the rich got richer, and the poor got poorer, as the competition for calculating hashes was dominated by those who could afford to buy the devices. That’s why other coins emerged that used more resistant hashes against these devices, making it difficult to create specialized mining equipment for them. The cost of building those devices also increased significantly, which helped maintain a bit more fairness. In contrast, with a relatively low investment in Bitcoin mining, you could earn a lot more money.

What kind of hashes do you think these could be? Those which require more hardware resources. Password hashes! Yes, you can use password-related hashes. This is what some of the newer coins do:

- Litecoin (Scrypt)
- Zcash (Equihash)

Researchers say[14]:

A function that is memory-hard under this definition requires the adversary to use either a lot of working space or a lot of execution time to compute the function. Functions that are memory-hard in this way are not amenable to implementation in special-purpose hardware (ASIC), since the cost to power a unit of memory for a unit of time on an ASIC is the same as the cost on a commodity server. An important limitation of this definition is that it does not take into account parallel or multiple-instance attacks.

4.6 Applications of Hash Functions

Also, we have[15]:

GPU/FPGA/ASIC-unfriendly. Argon2 is heavily optimized for the x86 architecture, so that implementing it on dedicated cracking hardware should be neither cheaper nor faster. Even specialized ASICs would require significant area and would not allow reduction in the time-area product.

Warning

Bitcoin is not designed to make you anonymous. All transactions are recorded. Eventually, you need to convert that bitcoin into something else (money, goods, etc.). At that point, you will be identified. So, don't think that you can do anything with Bitcoin and remain anonymous. It is not true. There are other coins that are designed to make you more private. For example, Monero.

4.6.2 Cloud-base Applications

When you want to send a file to your friend in a regular Telegram chat, you might notice that as soon as you hit the send button, the progress bar jumps to 100%. The reason is that Telegram checks and sees, "Oh! The hash of the file you are sending matches the hash of one of the files on my server." So, why would you want to waste your internet data sending it? There's no need to send it! I can just send the file with the same hash from my server to the other person!

For example, if we want to find a copyrighted image, we can scan the entire phone. If we find a matching hash, we've located it! :)

Google Safe Browsing uses the hash of the URLs to check if the website is safe or not. If the hash of the URL is in the database, it is not safe.³⁵

4.6.3 Reading List

- Hash functions: Theory, attacks, and applications[16]
- SHA256 Algorithm Explained (A wonderful website that interactively shows how SHA-256 works)³⁶
- How the MD5 hash function works (from scratch)³⁷ (A wonderful video)

³⁵<https://security.googleblog.com/2022/08/how-hash-based-safe-browsing-works-in.html>

³⁶<https://sha256algorithm.com/>

³⁷From Rare skills Youtube Channel: video link

4.7 Encoding

Suppose you want to transmit your words via computers. Computers only know zeros and ones. What should you do then? A simple solution is to associate each character with a number. For example, we can associate 'A' with 1, 'B' with 2, 'C' with 3, and so on. Then we can represent each number in base 2 (binary). For example, 'A' is 1, which is 0001 in binary. 'B' is 2, which is 0010 in binary. 'C' is 3, which is 0011 in binary. Then we can transmit the binary numbers. This is what happens in computers. Each character is associated with a number, and then the number is represented in binary.

Definition 3. *The process of converting A to B is called encoding A with B.*

Definition 4. *The process of converting B to A is called decoding B with A.*

To have a common means of communication, we have to rely on a standard. ASCII (American Standard Code for Information Interchange) is one of the most common standards. It associates each character with a number. For example, 'A' is 65, 'B' is 66, 'C' is 67, and so on. (Search for ASCII table.) Unicode is another standard. You can search for it.

The important note is that encoding is not encryption. encoding is just a way to represent data. Please do NOT use words like *encoded*, *encoding*, *decoded*, or *decoding* when you mean *encrypted* or *decrypted*.

4.8 Encryption

Imagine I want to write a letter to someone. Well, it needs to be sent by mail. How can I be sure that the postal worker won't open it to read it? There's no guarantee that someone won't read it along the way!

This is where the concept of encryption comes in. It means that I can write a message in a coded form that only I and the recipient can understand. For example, what do you think the following text means?

uoy era woh

If you read it backwards, you'll see that it says:

how are you

This is a simple example of encryption. The message is changed in a way that only the recipient (who knows how to decode it) can understand. This is the basis of encryption. It's like a lock. You have the key, and only you can open it. If someone else tries to open it, they won't be able to. This is the essence of encryption.

Now what do you think about the following text?

ipx bsf zpv

It is a simple Caesar cipher. It means that each letter is shifted by a fixed number of positions down the alphabet.³⁸ For example, if the shift is 1, A becomes B, B becomes C, and so on. This is another example of encryption. Now shift the letters by 1 in reverse and you will see that it says:

how are you

- The original text before applying encryption is called plaintext or cleartext.
- The algorithm used for encryption and decryption is called a cipher.
- The process of converting plaintext to ciphertext is called encryption.
- The process of converting ciphertext back to plaintext is called decryption.
- The resultant text after applying encryption is called ciphertext or encrypted text.
- The original text is called plaintext or cleartext.

Remember: These terms are fundamental and will be used frequently throughout the text. Therefore, it is crucial to understand and remember them.

Definition 5. *The key principle in cryptography is this: It doesn't matter whether a message is being sent or not. What's important is that the actual meaning of our message should not be revealed. In other words, it's acceptable for an encrypted message to be transmitted and for someone to realize that a message has been sent; the only requirement is that the content must be encrypted so that the original intent and message remain unknown.*

4.8.1 Kerckhoff's Principle

In the 19th century, Kerckhoff said that the security of a cryptographic system should not depend on the secrecy of the algorithm. It should depend on the secrecy of the key. This means that the algorithm should be public. Everyone should see it. But the key should be secret.

Many of our previous examples (and other more complex hand-made ciphers) can be easily broken by computers³⁹. Do NOT rely on them.

³⁸See more at: https://en.wikipedia.org/wiki/Caesar_cipher

³⁹Hacking Secret Ciphers with Python: <https://inventwithpython.com/hacking/>

4.8.2 Steganography

Steganography is sometimes misunderstood as encryption. Although both help with security and privacy, they are different. What do you think the following text means?

jf@hdfs87@ijksd-isj!mjlsfj9\$bkjsd+akdlfj42fs%cjsdlf\$kjoi1

If you look at it, you will see that it is random text. But if you look at the first letter after each symbol, you will see that it says:

Hi I'm back

A similar thing can be done with a long letter: the first character of each sentence can be used to hide a message.

You can also hide a message in an image.⁴⁰ For example, you can change the least significant bit of each pixel to hide a message. Because only the LSBs are changed, the changes are very hard to detect by the human eye (but they can be detected by a computer).

Definition 6. *In steganography, the actual message must be hidden. This means that nobody should know that a message is hidden in something else.*

Sometimes steganography is used with encryption. The message is first encrypted and then hidden in something else. In this way, even if someone finds the hidden message, they can't understand it.

Reading List

- Steganography (Wikipedia)⁴¹
- Exploring Steganography: Seeing the Unseen⁴²
- Hide secret messages into a spammic message⁴³⁴⁴
- Printer Steganography

Function

A function is like a machine: you give it an input, and it gives you an output. For example, when we say $y = 2x$, it means that if you give 3 to the function, it

⁴⁰See also: <https://www.jjtc.com/stegdoc/sec313.html>

⁴¹<https://en.wikipedia.org/wiki/Steganography>

⁴²<https://www.jjtc.com/pub/r2026a.htm>

⁴³<https://www.spammimic.com/>

⁴⁴Be careful! It is a website. Don't use it for sensitive data. Never ever send a sensitive data to somewhere else. Even if they don't store it, it can permanently be stored in the logs or hard disks unintentionally. We will talk about swap/shadow/temp files later.

4.8 Encryption

will give you 6. If you give 4, it will give you 8. If $y = 2x - 3$ is our function, it means that if you give a number as x , it will double it and subtract 3 from it.

Functions can be used to encrypt data (though this is not really secure!). For example, your password is:

`kAmZ)!rf3V+qzQ2iVQfPVcXaE4`

Using a function, it can be written as:

`kAmZ)!rf5V+qzQ2iVQfPVcXaE8`

What function did I use? The function is $3x - 4$.

There are various ways to slightly alter or obscure the text so that no one notices. Of course, these methods do make it harder for a bad actor, but we shouldn't say they make it impossible (because they might be able to guess what function we used)!

For example, your password is:

`aWf8$dGH5b#jFRgT>st3ch:tDa7gS`

You can write it like this:

`aWf8$dGH5b#jFR2gT>st3ch:tD@a7gS`

And you know that you never use the number **2** or the **@** symbol in your passwords. So when entering it, you don't include those, but someone who gains access to the password won't know that they need to remove those characters, since we added them as distractions, so they won't be able to log in.

Or, for example, you can write:

`aWf8$dG!H5b#jFRgT>st3ch:tDa!7gS`

like this:

`sv#1af8ws2Ca:seaWf8$dG!H5b#jFRgT>st3ch:tDa!7gSE%dTrâxj7p`

And you keep in mind that your password is between e and E, so anyone reading the text won't understand what your password was.

When all these techniques are combined, they make it so that even if your password falls into someone else's hands, they will have to work harder to understand it. But remember, it's not impossible—it's just a bit harder.

Conclusion: Combine methods before entering your password into password managers. If your master password for the password manager is compromised and your passwords are leaked, it will be harder for the attacker to understand your passwords.

4.8.3 Number Station

Let's see some more interesting examples. Listen to the following audio:

<https://www.numbers-stations.com/english/e01-ready-ready/>

Strange, isn't it? Now listen to this one:

<https://www.numbers-stations.com/morse/m01/>

The latter one was stranger. But what are they? The second one was transmitting Morse code.⁴⁵ The first one was transmitting numbers. But why?

For example, this sequence has been transmitted:

A - F - H - H - P - X
34

These are also a form of encryption. Imagine a country has a spy in another country. They certainly can't send their messages in a way that anyone can understand. If they send them in a coded format, even if the government of that country does not understand the code, they might notice that every day at 9 AM, a certain individual receives an encrypted message. Naturally, the intelligence agency of that country would become suspicious about why this person is receiving encrypted messages at such a specific time. As a result, they might go and arrest that individual.

This led to the idea of a "numbers station," a public source where it is impossible to distinguish who is listening. For example, the spying country gives its spy a notebook and instructs them to write down the code provided and find the corresponding message in the notebook. The numbers indicate pages; for example, "34" means to look at page 34, and the random letters also have meaning.

The spy then matches those letters with a table to see what message they are meant to receive.

Letter	Meaning
A	R
P	O
H	N
F	U
X	W

⁴⁵If there are sounds, a short sound represents a "dot," while a long sound represents a "dash." Using light, a brief flash of light represents a dot, and a longer light represents a dash.

During the Vietnam War, when North Vietnam was capturing prisoners, they would torture them. North Vietnam arranged for a Japanese journalist to conduct an interview with a prisoner, aiming to portray that everything was fine and that the prisoners were treated well. However, the prisoner certainly couldn't say on camera that they were being tortured. So, he used a clever technique: he communicated in Morse code by blinking. A long blink indicated a "dash," while a quick blink represented a "dot." This individual managed to convey the message "T-o-r-t-u-r-e" to the world, meaning "torture." <https://www.military.com/video/operations-and-strategy/vietnam-war/pow-blinks-torture-in-morse-code/1381254901001>

So the message is “RUN, NOW!”

Important Notes

I) A pad should never be used more than once! II) The letters should not belong to any specific language. For example, if Persian letters are transmitted, it could indicate that the spy is Iranian or at least familiar with the Persian language.

- Why is this a problem?

- It makes it easier to identify the spy and the radio broadcaster. (We will discuss this further in the section on anonymity.)

- What if sending letters in Persian is a form of deception, making people believe the spy is Iranian?

- Imagine someone forgets how to write a particular Persian letter and searches for the Persian alphabet. By examining the search logs of people who looked up the alphabet, investigators can narrow down the list of suspects. (If every time the radio broadcasts, someone soon after searches for the Persian alphabet or the meaning of a word, it becomes obvious!)

Or suppose I want to set up a radio station; I certainly would not want to broadcast using my own voice. So what do I do? I search the internet for “text to speech.” Later, investigators can check the logs of people who searched for that and identify me more easily. Or, they might discover that these sounds were generated by a specific website, and then go to that website to request the IP addresses and device details of those who visited or requested that specific text-to-speech conversion. In this way, I could be exposed.

There have been many cases where people who wanted to remain anonymous were identified because, during certain events, they carelessly searched for topics using their real IP addresses or wrote about those subjects. This was the beginning of their identification. We will learn more about this in the **Dangerous behaviors lead to de-anonymizing** section.

4.8.4 What if using a pad several times?

Reading List

- Jadi’s Radiogeek, Episode 24⁴⁶
- Jadi’s Radiogeek, Episode 39⁴⁷
- Webdriver Torso YouTube mystery clips’ French connection⁴⁸

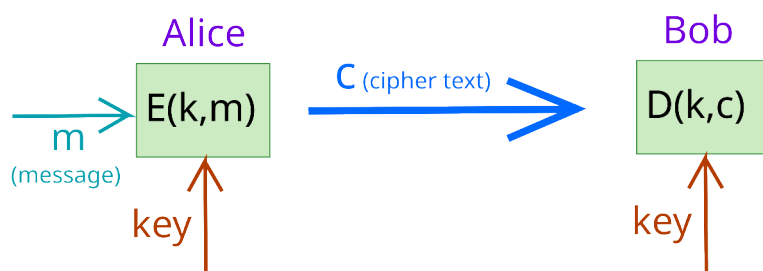
⁴⁶<https://jadi.net/2013/04/radiogeek-24-numbers-station/>

⁴⁷<https://jadi.net/2014/05/radio-geek-39-narenji-bleed/>

⁴⁸<https://www.bbc.com/news/technology-27238332>

4.9 Symmetric Encryption

Symmetric encryption is like a traditional lock: you can lock it with a key, and you can unlock it with the same key. For example, if you want to send a message to someone, you encrypt it with a key, and the recipient decrypts it using the same key. The Caesar cipher is an example of symmetric encryption; the key is the shift number. If you shift the letters by 3, you can decrypt it by shifting the letters back by 3.



In almost all modern algorithms, if you use the wrong key, the data is decrypted into random data. A good encryption algorithm is one where only a specific key can decrypt the data correctly; any other key will result in different random data.

If we have the key, decryption is very fast. But if we do not have it, **if the encryption algorithm is ideal**, we need to brute-force it. If the key is 256 bits, we need to try 2^{256} possible keys to find the correct one. But is it possible? *Bruce Schneier*, in chapter 7 of his book *Applied Cryptography*, explains [17] that it is not possible.

When we say an attack exists on an encryption algorithm, it means that in addition to brute-force, there is another way to recover the plaintext or key, which may take less or more time. A 2^{250} attack does **NOT** mean it is faster than brute-force. The operations may be different, but the total time may be longer. For example, if an attack requires 2^{250} operations but each operation takes 1 second, it requires 2^{250} seconds to break the encryption. But if brute-force requires 2^{256} operations and each operation takes 1 microsecond, it requires 2^{256} microseconds to break the encryption, which is 2^{250} seconds. So, the attack is not faster than brute-force; it is just another way to break the encryption.

Some papers may describe their attack as a *break*. Do not be confused. It is

4.9 Symmetric Encryption

not necessarily a break; it is just another way to attack the encryption, and it may not even be practical.

A larger key size does **NOT** always mean better security. It is important to stick to authorized recommendations. For example, an attack is possible on *AES-256* and *AES-192*, but not on *AES-128*.^[18] This does not mean that *AES-256* is insecure.⁴⁹

The important thing is to use a standard encryption algorithm. We discussed this in the *Use of Self-Invented Algorithms* section. The *Vigenère Cipher* shows us that knowing the length of the key should not result in a significant reduction in the time required to break the encryption.

4.9.1 Application of Symmetric Encryption

Some of you may have already used password-protected zip files. In order to prevent unauthorized access to the files, the data inside the file is encrypted⁵⁰ using a symmetric encryption algorithm. The password you enter is used as the key to encrypt the file. If the password is correct, the file is decrypted properly, and you can access the files. If the password is incorrect, the file is decrypted incorrectly into random data, and this is detected by the software, which tells you that the password is incorrect.

You may want to send a sensitive file by email. As we will discuss later, email is not a secure protocol for sending sensitive data (the email provider can see the actual data). You can encrypt the file using AES and send it to your friend. Your friend can use the same key that was previously agreed upon between you and them to decrypt it.

There are programs that securely encrypt files. For example, *gnupg* is a program that you can use to encrypt files. Always store your sensitive data in an encrypted format. This way, even if someone gains access to your hard drive, they won't be able to read the files without the key.

4.9.2 One-time Pad

There is a perfect encryption. If we use a key that is as long as the message, random, and use it only once, the encryption will be perfect. A simple *XOR* operation can be used:

$$\begin{aligned}E(m, k) &= m \text{ XOR } k \\D(c, k) &= c \text{ XOR } k\end{aligned}$$

⁴⁹Page 38, BSI TR-02102-1: "Cryptographic Mechanisms: Recommendations and Key Lengths" Version: 2024-1: <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.html>

⁵⁰It is important for the data to be encrypted. If this were only a software check without any encryption, anyone could read the data inside it with another program. Unfortunately, many software developers do not encrypt the data inside the file. They just check the password, and if it is correct, they show the data. This may include app lockers and gallery folder lockers.

Why is it unbreakable? Say someone gets `avfsg` as the ciphertext. Because the key is completely random, the resulting ciphertext (the result of the XOR operation) is also completely random. The attacker can't learn anything from the ciphertext. It could be `hello, world, night`, or anything else. If the key is **as long as the message**⁵¹, **completely random**, and **used only once**⁵², the ciphertext will be completely random. Any permutation of the characters will be a valid plaintext with equal probability.

- Why don't we use it?! It is perfect!

- Think of having a hard drive with 1TB of data. You want to encrypt it with a one-time pad. You need to generate a random key with 1TB of data. This means not only do you have to find a massive source of randomness⁵³, but you also have to store it somewhere. You need another 1TB hard drive to store the key.

Moreover, many applications, such as communication via the internet, require real-time transmission of data. Think about wanting to download a file that is 2GB. You somehow need to constantly generate random bits and exchange them with the other side. In the end, you've generated 2GB of random bits and exchanged them with the other side. This significantly increases the time required to download the file.

That was just for one website. Imagine your browser is open on 100 tabs. A huge source of randomness is needed to generate 100GB of random bits. This is not practical. We will see that generating random bits is very hard.

Conclusion: As *Bruce Schneier* says⁵⁴, many commercial products that claim to use one-time pads are not actually using them. They just use a symmetric encryption algorithm with a long key. But the key is not equal to the length of the message, nor is it random or used only once.

4.9.3 Analysis of the Basic Attack (Advanced)

The attack described in Section 5.1 of Biham and Shamir's paper, *Differential Fault Analysis of Secret Key Cryptosystems*, [19] is a powerful ciphertext-only attack that leverages a permanent hardware fault to deduce portions of the secret key. To fully understand its mechanics, we must first review the relevant structure of the Data Encryption Standard (DES) and then trace how the induced fault creates a vulnerability.

The Structure of the Data Encryption Standard (DES)

DES is a symmetric-key block cipher that operates on 64-bit blocks of data. Its core is a *Feistel network*, a design which splits the data block into two 32-

⁵¹Not half or 99% of it

⁵²Never using even a part of it twice

⁵³Generating random bits is very, very, very hard. We will talk about it later.

⁵⁴<https://www.schneier.com/crypto-gram/archives/2002/1015.html#7>

4.9 Symmetric Encryption

bit halves, a left half (L) and a right half (R), and processes them through 16 identical rounds of computation.

The Round Function. For each round i (from 1 to 16), the transformation is defined as follows:

$$\begin{aligned}L_i &= R_{i-1} \\R_i &= L_{i-1} \oplus F(R_{i-1}, K_i)\end{aligned}$$

where L_{i-1} and R_{i-1} are the outputs of the previous round, K_i is the 48-bit subkey for the current round, and F is the main round function, often called the Feistel function. The symbol $\oplus$ denotes the bitwise XOR operation. After 16 rounds, the final left and right halves are concatenated and subjected to a final permutation to produce the 64-bit ciphertext.

The Feistel Function The security of DES resides almost entirely within the non-linear F function. It takes a 32-bit data input (R_{i-1}) and a 48-bit subkey (K_i) to produce a 32-bit output. This process involves several steps:

1. **Expansion (E-box):** The 32-bit input R_{i-1} is expanded to 48 bits by duplicating certain bits according to a fixed permutation.
2. **Key Mixing:** The 48-bit expanded data is XORed with the 48-bit subkey K_i .
3. **Substitution (S-boxes):** The resulting 48-bit value is divided into eight 6-bit chunks. Each 6-bit chunk serves as the input to one of eight unique Substitution Boxes (S-boxes). **This is the origin of the 6-bit values central to the attack.**
4. **S-box Lookup:** Each S-box is a fixed lookup table with 4 rows and 16 columns, containing 4-bit values. A 6-bit input is used to select an output as follows:
 - The first and last bits of the 6-bit input determine the row number (0 to 3).
 - The middle four bits determine the column number (0 to 15).The 4-bit value at the selected table entry is the output. Thus, a 6-bit input yields a 4-bit output.
5. **Permutation (P-box):** The eight 4-bit outputs from the S-boxes are concatenated into a 32-bit string, which is then permuted by a P-box to produce the final 32-bit output of the F function.

The Mechanics of the Fault Attack

The attack assumes a specific, permanent hardware fault: the least significant bit (LSB) of the register holding the left half of the data is “stuck at zero.”

Exploiting the Fault. Let's consider the final round of DES (round 16). The governing equation for the right half of the output is:

$$R_{16} = L_{15} \oplus F(R_{15}, K_{16})$$

The attacker possesses the final ciphertext, which consists of L_{16} and R_{16} (before the final permutation). We know from the DES structure that $L_{16} = R_{15}$. By rearranging the equation, we can isolate the output of the F-function:

$$F(R_{15}, K_{16}) = L_{15} \oplus R_{16}$$

Now, we analyze the LSB of this equation. Due to the induced fault, we know with certainty that $LSB(L_{15}) = 0$. This simplifies the relationship dramatically:

$$LSB(F(R_{15}, K_{16})) = LSB(L_{15}) \oplus LSB(R_{16}) = 0 \oplus LSB(R_{16}) = LSB(R_{16})$$

Since the attacker has the ciphertext, they know the value of $LSB(R_{16})$. Therefore, the attacker knows the exact 1-bit LSB output of the F-function in the final round.

Targeting the Key. The design of the DES P-box is such that the LSB of the F-function's output is determined exclusively by the output of a single S-box: S-box 7 (S7). The input to S7 is formed by XORing 6 specific bits from R_{15} with 6 specific bits from the final subkey K_{16} . The attacker's goal is to find these 6 unknown key bits.

The attacker now has all the necessary components for a targeted search:

- **Known Data Input:** 6 bits from R_{15} (which is equal to the known L_{16}).
- **Known S-box Output Bit:** The LSB of S7's output (which is equal to the known $LSB(R_{16})$).
- **Unknown Variable:** The 6 bits of the subkey K_{16} that are fed to S7.

The Process of Elimination. There are $2^6 = 64$ possible values for the 6-bit portion of the key. The attacker can iterate through all 64 possibilities to find the correct one. For each 6-bit key guess:

1. Take the 6 known data bits from the ciphertext (L_{16}).
2. XOR them with the 6-bit key guess.
3. Use the 6-bit result as an address to look up the 4-bit output in the S7 table.
4. Check if the LSB of this 4-bit output matches the known target LSB from the ciphertext ($LSB(R_{16})$).

Any key guess that fails this check is definitively wrong and is discarded.

Why Approximately 6 Ciphertexts Suffice. A single ciphertext is not enough to uniquely identify the key. Because the S-boxes are designed to behave pseudo-randomly, a *wrong* key guess has a roughly 50% chance of producing the correct LSB just by luck.

This allows for a probabilistic process of elimination across multiple ciphertexts:

- **Start:** 64 candidate key values.
- **After 1st Ciphertext:** The single correct key will pass the test. About half of the 63 wrong keys will also pass by chance. This leaves $\approx 1 + 63/2 \approx 32$ candidates.
- **After 2nd Ciphertext:** The test is repeated on the remaining ≈ 32 candidates with a new ciphertext. Again, about half of the incorrect keys are eliminated, leaving ≈ 16 candidates.
- **After n Ciphertexts:** This filtering process continues, with the number of incorrect candidates being halved at each step: $64 \rightarrow 32 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$.

After analyzing approximately six different faulty ciphertexts, the set of candidates is statistically reduced to a single value. This value must be the correct 6-bit portion of the subkey, as it is the only one to have survived all six independent checks.

• Why cannot all 4-bits be recovered?

→ Because Only LSB of left side is set to zero

As the paper says:

The other bits of the last subkey can be found by inducing additional faults, or by enumerating and verifying additional key bits.

4.10 Symmetric Encryption Operating Modes

It is easier to design an algorithm that operates on a fixed length of data rather than variable length. As a result, we design algorithms that operate on fixed-length data (e.g., 256 bits). Thus, a protocol should be used to encrypt and decrypt the data using that. Each protocol has its own advantages and disadvantages. We want to know their pros and cons to choose the best one for our application. Some are:

- ECB: Electronic Codebook
- CBC: Cipher Block Chaining
- CFB: Cipher Feedback

4.10 Symmetric Encryption Operating Modes

- OFB: Output Feedback
- CTR: Counter

Definition 7. *Random access means that each block of plaintext can be encrypted or decrypted independently of the other blocks. This is a desirable property for applications that require the ability to access individual blocks of data without having to process or decrypt the entire message (e.g., video streaming, databases, disk encryption).*

Definition 8. Error Propagation: *The extent to which an error in the ciphertext (e.g., noise) affects the decryption of subsequent blocks.*

4.10.1 ECB (Electronic Codebook)

Divide the plaintext into blocks and encrypt each block separately.

Q: Why code book?

A: Because we can create a table of all the blocks and their corresponding ciphertexts (a code book) with a specific key.

In some applications, the ciphertext should replace the plaintext. Hence, the size of the ciphertext has to be the same as the plaintext. However, in ECB, dividing the plaintext into blocks may result in the last block being smaller than the block size. In this case, we need to pad the last block to make it the same size as the block size. In general, up to block size – 1 bits are added to the last block (as it can vary from 0 bits (no padding required) to block size - 1 bits (only 1 bit of data is in the last block)).

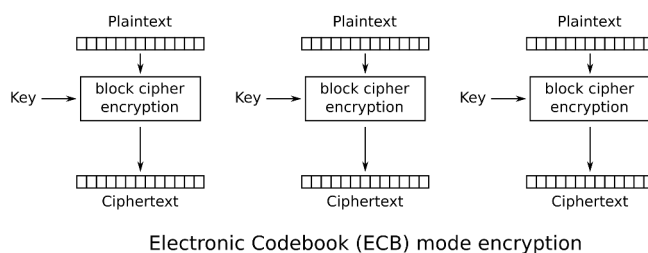


Figure 4.1: src: Wikipedia, Public Domain

Pros:

- Simple
- Parallelizable
- Random access (can decrypt any block independently)
- Error propagation is limited to one block

4.10 Symmetric Encryption Operating Modes

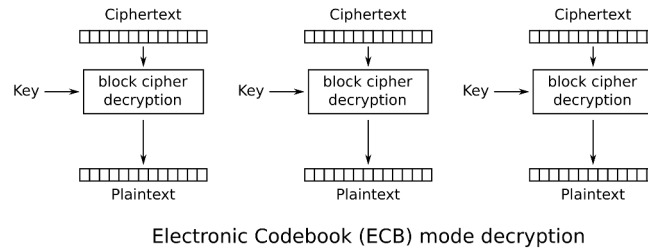


Figure 4.2: src: Wikipedia, Public Domain

- No need for initialization vector

Cons:

- Identical plaintext blocks result in identical ciphertext blocks (vulnerable to pattern analysis)⁵⁵
- Susceptible to replay/insertion/deletion attacks (does not provide integrity)
- ciphertext size = plaintext size + padding (up to block size - 1)

4.10.2 CBC (Cipher Block Chaining)

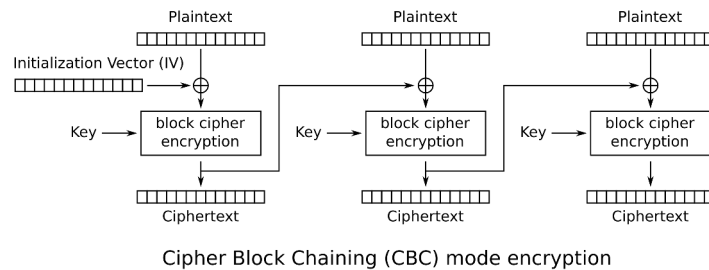


Figure 4.3: src: Wikipedia, Public Domain

<https://defuse.ca/cbcmodeiv.htm>

- XOR the plaintext block with the previous ciphertext block before encryption.
- The first block is XORed with an initialization vector (IV).

⁵⁵See image of Linux at [https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation#Electronic_codebook_\(ECB\)](https://en.wikipedia.org/wiki/Block_cipher_mode_of_operation#Electronic_codebook_(ECB))

4.10 Symmetric Encryption Operating Modes

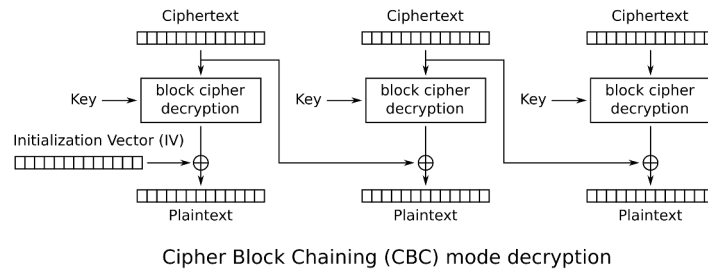


Figure 4.4: src: Wikipedia, Public Domain

- The IV should be random and unique for each message.
- The IV is sent along with the ciphertext.

The IV should be random, unique, and unpredictable, and protected against unauthorized modification. But it doesn't have to be secret.

Counter IV! DON'T use it.

Pros:

- Random access (can decrypt any block independently, because we only need the IV or previous ciphertext block)
- Error in a plaintext block is limited to that block (Although it affects the ciphertext of that block and all subsequent blocks, it doesn't affect the decryption as we only need the **same** previous ciphertext block. As long as we use the **same** previous ciphertext block, it doesn't matter if we use $1a2b$ or $1a2c$ for *XOR*.)
- Error in a ciphertext block garbles the entire corresponding plaintext block and the corresponding bit in the next block. (Again, the rest of the blocks are unaffected since we use the **same** previous ciphertext block for decryption)
- Identical plaintext blocks result in different ciphertext blocks (since IV is different for each message)

Cons:

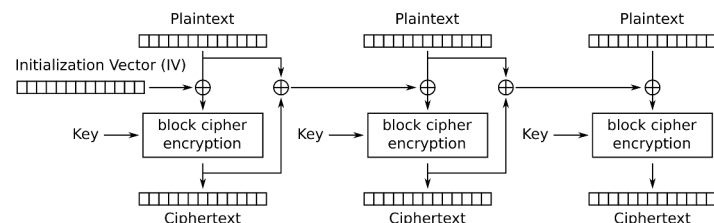
- Not parallelizable for encryption
- Needs an IV
- Susceptible to change/insertion/deletion of the last block
- Bit-flipping attack: If an attacker knows the structure of the plaintext, changing a bit in the ciphertext will change the corresponding bit in the plaintext.

4.10 Symmetric Encryption Operating Modes

- plaintext size $\neq$ ciphertext size
- Cannot recover from synchronization errors (loss/addition of a bit)

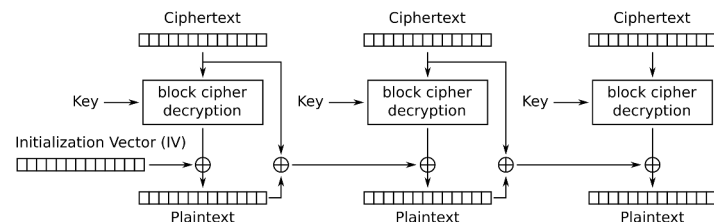
4.10.3 PCBC (Propagating Cipher Block Chaining)

This mode was designed so that small changes in the ciphertext propagate indefinitely during both encryption and decryption.



Propagating Cipher Block Chaining (PCBC) mode encryption

Figure 4.5: src: Wikipedia, Public Domain



Propagating Cipher Block Chaining (PCBC) mode decryption

Figure 4.6: src: Wikipedia, Public Domain

Pros:

- Semi-parallel for decryption (as the most expensive operation (block cipher
- Semi-parallelizable for decryption (since the most expensive operation, block cipher decryption, can be done in parallel)

Cons:

- No Random access (as we need the previous plaintext block)
- Needs using IV

4.10 Symmetric Encryption Operating Modes

- No random access (since we need the previous plaintext block)
- Requires an IV
- Swapping two ciphertext blocks results in garbled corresponding plaintext blocks, but the rest of the blocks are unaffected. [20]
- $C_0 = E_k(P_0 \oplus IV)$
- $C_1 = E_k(P_1 \oplus P_0 \oplus C_0)$
- $C_2 = E_k(P_2 \oplus P_1 \oplus C_1)$
- $P_0 = D_k(C_0) \oplus IV$ (I)
- $P_1 = D_k(C_1) \oplus P_0 \oplus C_0$ (II)
- $P_2 = D_k(C_2) \oplus P_1 \oplus C_1$ (III)

If we substitute (I) and (II) in (III), we get:

$$\begin{aligned} P_2 &= D_k(C_2) \oplus D_k(C_1) \oplus P_0 \oplus C_0 \oplus C_1 \\ &= D_k(C_2) \oplus D_k(C_1) \oplus D_k(C_0) \oplus IV \oplus C_0 \oplus C_1 \end{aligned}$$

If we swap C_0 and C_1 , in decryption we get:

- $P'_1 = D_k(C_0) \oplus P'_0 \oplus C_1$ (II)
- $P'_2 = D_k(C_2) \oplus P'_1 \oplus C_0$ (II)

If we substitute (I) and (II) in (III), we get:

$$\begin{aligned} P'_2 &= D_k(C_2) \oplus D_k(C_1) \oplus P'_0 \oplus C_1 \oplus C_0 \\ &= D_k(C_2) \oplus D_k(C_1) \oplus D_k(C_0) \oplus IV \oplus C_0 \oplus C_1 \end{aligned}$$

We see that $P_2 = P'_2$.

4.10.4 CFB (Cipher Feedback)

Simple variant of CFB decryption is almost identical to CBC encryption performed in reverse. IV is shifted to shift register and s bit is gone. s bit right is replaced by c_0

The IV is shifted into the shift register and the leftmost bit is removed; the rightmost bit is replaced by c_0 .

- plaintext size = ciphertext size (can be used for streaming)
- Same algorithm for encryption and decryption (Can be used to create special purpose hardware -> increases speed + avoids side-channel attacks)

4.10 Symmetric Encryption Operating Modes

A simple variant of CFB decryption is almost identical to CBC encryption performed in reverse.

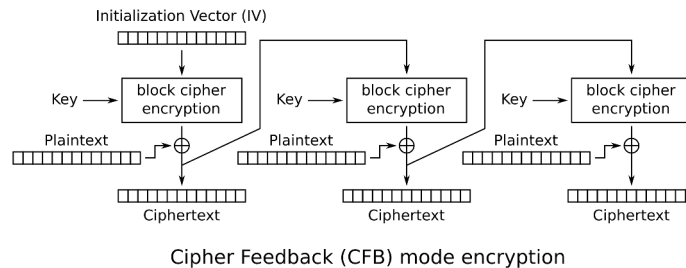


Figure 4.7: src: Wikipedia, Public Domain

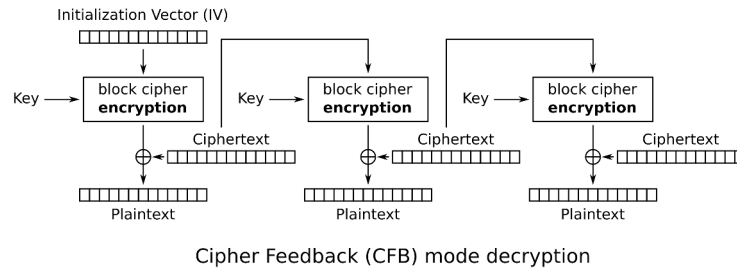


Figure 4.8: src: Wikipedia, Public Domain

- Random access and parallelizable for decryption
- Self-recover (partial noise-resistant) error in a ciphertext block changes the corresponding bit in the corresponding plaintext block and $B/O - 1$ blocks (B is the block size and O is the output size)?????!!!!!!!
- Self-recovering (partially noise-resistant): an error in a ciphertext block changes the corresponding bit in the corresponding plaintext block and in $B/O - 1$ blocks (where B is the block size and O is the output size).
- Self-recovering from synchronization errors

Cons:

- Not parallelizable for encryption
- Susceptible to bit-flipping attacks (the next block is garbled, though)

Pros:

- a little bit parallelizable for encryption (after calculating the expensive block cipher encryption operations in serial) and decryption

4.10 Symmetric Encryption Operating Modes

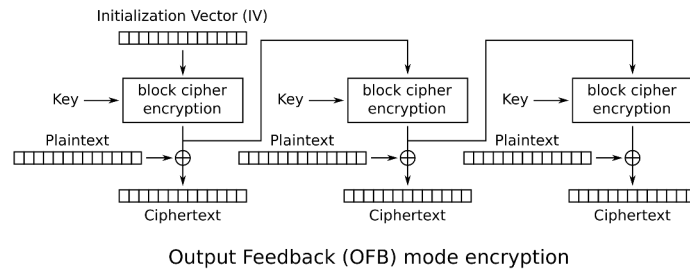


Figure 4.9: src: Wikipedia, Public Domain

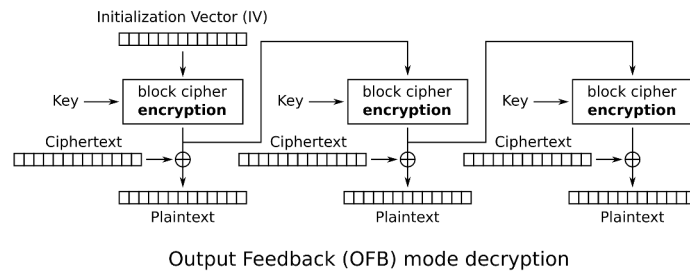


Figure 4.10: src: Wikipedia, Public Domain

- Slightly parallelizable for encryption (after calculating the expensive block cipher encryption operations in serial) and for decryption
- We can use CBC mode encryption with an input of all zeroes to create the key for the XOR in OFB mode (since $O_i \oplus 000...000 = O_i$), which is useful for using CBC hardware acceleration
- Noise-resistant
- Uses the same algorithm for both encryption and decryption
- Does not need padding (for the last block you can send only the ciphertext bits corresponding to the plaintext bits) [21]

Cons:

- No authentication code
- Bit-flipping attack

OFB has birthday problem, problem. (Same key after 2^{64} block) -> also cfb? Short cycle problem. OFB has a birthday problem (the same key may repeat after 2^{64} blocks), which can also affect CFB. There is also a short cycle problem.

4.11 ASCON

Is AES always the best choice for symmetric encryption? Let's consider a scenario where we need to encrypt data on a small IoT device with limited processing power and memory. In such cases, AES might be too resource-intensive, leading to slower performance and increased power consumption. This is where lightweight ciphers like ASCON[22] come into play. ASCON, NIST's choice for lightweight authenticated encryption, is designed to be efficient on constrained devices while still providing robust security.

Sometimes ChaCha20 is preferred over AES, especially in software implementations. ChaCha20 is a stream cipher known for its speed and simplicity, making it particularly effective on devices without hardware acceleration for AES. It also has a simpler design that can be easier to implement securely in software.

So, we choose different algorithms based on the specific requirements of the application.

End questionsss: 1. What happens if we use same IV for two different messages in CBC? It will result in the same ciphertext until the first different block.

4.12 Asymmetric Encryption

In symmetric encryption, the same key is used for both encryption and decryption. This means that anyone who wants to encrypt a message must have the key, and anyone who wants to decrypt it must also have the same key.

Now, imagine I want to visit a website. I can't simply send my encryption key over the internet to the site and say, "Here's my key; use it to decrypt my message." I would have to send the key in plain text, and anyone who intercepts my internet connection—such as people sharing the same Wi-Fi, the internet service provider (since my data passes through their servers), or anyone else along the communication path—could see the key in plain text. This means they could potentially decrypt my messages later on. This scenario introduces the need for a different type of encryption: *asymmetric encryption*.

Let me explain the method this way: imagine you have a box, a lock, and a pair of keys. The lock works in such a way that if you lock it with one key, it can only be opened with the other key.

One of the keys I keep with me at all times; I'll call it my private key. The other key can be given to anyone; this is known as the public key.

Now, if you want to send me a message, I send you the lock and my public key. You write your letter, place it in the box, lock the box, and secure it with my public key. Then you send the box to me. Since the lock is locked with my public key, only I can unlock it with my private key. This way, even if someone intercepts the box, they won't be able to open it because they don't have my

4.12 Asymmetric Encryption

private key. If I want to send you a message, I would ask you to send me your lock and your public key. I write my letter, place it in a box, lock it, and secure it with your public key. Since your private key is only with you, only you can unlock it!

- Why do we even need symmetric encryption? We can just use asymmetric encryption. Even remote communication is possible.

- Asymmetric encryption is very slow—thousands of times slower than symmetric encryption. We normally use asymmetric encryption to transmit or create a symmetric key on both sides. Then we use symmetric encryption to encrypt the message.

For now, assume that we can do two things:

- Encrypt the symmetric key with the public key and send it to the other side.
- Use an asymmetric algorithm to generate the symmetric key on both sides (Diffie-Hellman-Merkle Key exchange⁵⁶).

4.12.1 Meddler-in-the-Middle Attack (MitM Attack)

There is a problem with asymmetric encryption. Think about what can go wrong in the protocol. The issue is: how can you be sure that the public key you received is the real one? What if someone intercepts the connection and sends you their public key? The protocol goes like this:

1. Alice tells the server to send its public key.
2. The server sends its public key.
3. Mallory, who is sitting between Alice and the server, intercepts the connection, gets the public key from the server, and sends Alice her own public key.
4. Alice encrypts the message with Mallory's public key and sends it to Mallory.
5. Mallory decrypts the message with her private key and reads it.
6. Mallory encrypts the message with the server's public key and sends it to the server.
7. The server decrypts the message with its private key and reads it.

See? Mallory is now able to read the messages, while the server and Alice think that they are talking to each other. This is called a *Meddler-in-the-Middle Attack*⁵⁷. What do you think we should do to detect or prevent it?

⁵⁶Read more at: <https://skerritt.blog/diffie-hellman-merkle/>

⁵⁷While 'Man-in-the-Middle' (MitM) has been a widely recognized term for many years, I opted for 'Meddler-in-the-Middle' as a more inclusive alternative, as it more accurately describes someone who tries to involve themselves in situations where they do not belong, while promoting gender neutrality.

We know that public-key cryptography is very slow. Thus, if someone is intercepting, the connection should be slower than usual (because encryption is done twice and decryption is done once). So, one way is to use machine learning to detect unusual delays in the connection (considering the user's internet delay and packet timing).

However, the above solution will not always work. Delays vary, and the mechanisms may not detect every situation. We want something that will always work. A better way is to trust an authority to verify the public key. This is called a *Certificate Authority*. The authority signs the public key of the server. When you receive the public key, you also receive the signature. You can verify the signature with the public key of the authority. If it is correct, you can trust the public key. If it is not, you cannot trust it.

This method is used in HTTPS. Have you ever heard experts say that you should not install certificates from unknown sources? This is the reason. If you install a certificate from an unknown source, they can intercept your connection, and you won't know it. ⁵⁸⁵⁹

A country can promote its *National Browser*, which has a certificate from the government. If you install it, the government can intercept your connection.

There have been some cases where certificate authorities were compromised. See Diginotar⁶⁰.

This method needs a trusted authority. In many applications, we do not have a trusted authority. For instance, in a peer-to-peer network, WhatsApp, Signal, Telegram Secret Chat, etc., there is no authority to verify the public key. So, what do you think we should do?

We can build trust based on community agreement. For example, if Alice wants to talk to Bob, she can ask her trusted friends to send her Bob's public key. If all of them send the same public key, Alice can trust it. This is called a *Web of Trust*. It is used in *PGP* (Pretty Good Privacy) encryption. *PGP* is software that encrypts and decrypts messages and is used in email encryption.

Another approach is to generate a hash of both public keys ($H(\text{Public}_{\text{Alice}}, \text{Public}_{\text{Bob}})$) on both parties. The parties can exchange the hash over another channel that is already trusted. If the hashes are the same, it means that the public keys are the same:

- Alice side: $H(\text{Public}_{\text{Alice}}, \text{Public}_{\text{Bob}})$
- Bob side: $H(\text{Public}_{\text{Alice}}, \text{Public}_{\text{Bob}})$

Because if an attacker performs a MITM attack, the hash will be:

- Alice side: $H(\text{Public}_{\text{Alice}}, \text{Public}_{\text{Mallory}})$

⁵⁸Continuing to Protect our Users in Kazakhstan <https://blog.mozilla.org/netpolicy/2020/12/18/kazakhstan-root-2020/>

⁵⁹Apple, Google, Microsoft, and Mozilla ban Kazakhstan's MitM HTTPS certificate <https://www.zdnet.com/article/apple-google-microsoft-and-mozilla-ban-kazakhstan-mitm-https-certificate/>

⁶⁰<https://en.wikipedia.org/wiki/DigiNotar>

4.12 Asymmetric Encryption

- Bob side: $H(\text{Public}_{\text{Mallory}}, \text{Public}_{\text{Bob}})$

This is called *Fingerprint Verification*. It is used in *Signal*⁶¹, WhatsApp⁶², and Telegram’s Secret Chat⁶³.

Warning

As Techlore also suggests^a, if you want to verify the fingerprint and you do not trust the other party, do not send the fingerprint to them. Otherwise, the other party can simply resend it to you and claim theirs is the same as yours.

^aA Guide To Verifying Signal Safety Numbers: <https://discuss.techlore.tech/t/a-guide-to-verifying-signal-safety-numbers/2289>

Now let me ask you a question! Are those services that claim they offer a secure way for you to send a message “anonymously” to another person real? (Like those Telegram bots.) Of course not! They get your plaintext message and store it in their database. Then they send it to the receiver. The owner of the server/bot has access to all the messages (and may even be selling them). Be careful about them!

4.12.2 Digital Signature

Remember when I said that a message encrypted with one key can be decrypted by the other key? What if I encrypt a message with my private key? Others can try to unlock it with my public key. If it works, it means that I am the one who sent the message because only I have my private key. This is called a digital signature.

NOTE: It is not safe to use the same algorithm and key indiscriminately for both encryption and digital signatures [23]. Also, *NIST* says⁶⁴:

... the key used for key agreement shall be different from the ECDSA key used for digital signatures.

Remember we said asymmetric cryptography is slow? How can we use it for a digital signature? If we have a 10GB file, we would need to encrypt it with our private key. That is very slow. What should we do? **HINT:** To do it more quickly, we should encrypt less data. But how?

Instead of encrypting the whole file, we can just encrypt its fingerprint. Like humans, files can have a unique fingerprint. This fingerprint is called a *hash*. We can compute the hash of the file, encrypt it with our private key, and send

⁶¹<https://signal.org/blog/safety-number-updates/>

⁶²https://faq.whatsapp.com/1524220618005378/?cms_platform=web

⁶³<https://core.telegram.org/api/end-to-end>

⁶⁴5.6.2 Using Algorithm Suites and the Effective Security Strength, Page 59: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf>

4.12 Asymmetric Encryption

it with the file. The recipient can compute the hash of the file and decrypt the signature with our public key. If the decrypted hash is the same as the hash of the file, it means that the file is from us.

Legal issues are also important. You may need to change your algorithm for export to another country. Moreover, a country may only accept a digital signature under certain circumstances (e.g., acceptable protocols) and may not accept it in other cases in court.⁶⁵ What if a person claims that their private key has been stolen?

4.12.3 RSA

For asymmetric encryption, we seek a function such that if you know certain parameters, you can easily calculate the result. However, if you do not know the parameters, it is very hard to compute the result. For example, if you know two prime numbers p and q , you can easily find their product: $p \times q = n$. But if someone gives you n , it is very hard to find p and q . This is the basis of the RSA algorithm.

- Why is it hard? For example, if you give me 15, I'll easily tell you the two prime numbers behind it: 3 and 5!

→ For small numbers, yes, it is easy. But in RSA, the numbers are each 4096 bits! You cannot factor a number that has hundreds of digits!

To date⁶⁶, there has not been a **general algorithm** found for **classical computers** to factor an integer n in **polynomial time**.^{67,68}

- Wait a minute. I know a linear algorithm! Just start from 2 and go all the way up to the number. If it is divisible by a number, it is not a prime number. Even this easy algorithm is linear. How can you say that there is no polynomial algorithm?

→ Good question! In cryptography, we do not express the complexity in terms of the number itself. We express it in terms of the number of bits.

Any number can uniquely be expressed in base b like this:

$$n = a_0 + a_1 \times b + a_2 \times b^2 + a_3 \times b^3 + \dots + a_k \times b^k$$

Where b is the base and a_i are the coefficients with $0 \leq a_i < b$. If $a_n \geq b$, we can divide it by b and add the remainder to the next coefficient. Moreover, $a_n > 0$ because if $a_n = 0$, we could add $a_{n+1}b^{n+1}$ where $a_{n+1} = 0$ and continue adding zeros to the end of the number. In this way, the representation would not be unique unless we require the highest coefficient to be nonzero.

⁶⁵Almost all digital signatures have a timestamp.

⁶⁶It has not been proved that it cannot be found. It just has not been found until now. Some mathematicians believe that factoring is and will remain hard, but others think it may become easier as time passes. For now, it is hard.

⁶⁷Polynomial time algorithms are considered efficient for our computers.

⁶⁸Yes, there are algorithms that use certain properties of a number (e.g., *Pollard's p-1*) and can factor it easily. But there is no such general algorithm.

4.12 Asymmetric Encryption

Now we want to find the number of bits of a number. We know that:

$$\text{count} = \text{last} - \text{first} + 1$$

So:

$$\text{count} = n - 0 + 1$$

Thus, the number represented has $n + 1$ bits. We now need to find $n + 1$ in terms of the number (N). We know that:

$$b^n \leq N < b^{n+1}$$

If the number was less than b^n , it would be represented by a smaller power of b (b^{n-1}), and if it was greater than or equal to b^{n+1} , it would be represented by a greater power of b (b^{n+1}). We want to find $n + 1$. We can use logarithms to find it:

$$n \leq \log_b N < n + 1$$

Does this look familiar? A number that is greater than or equal to n and less than the next number ($n + 1$) matches the definition of the floor function. So we can write it as:

$$n = \lfloor \log_b N \rfloor$$

Thus:

$$n + 1 = \lfloor \log_b N \rfloor + 1$$

We can also say:

$$N \sim b^n$$

Now, is your algorithm polynomial? It is not. Something that is linear in terms of N is actually exponential in terms of the number of bits.

- Why, in algorithms, don't we express the complexity in terms of bits?
→ Because in general algorithms, we want to know how the algorithm behaves as the number itself grows. But in cryptography, operations are performed on (mostly) constant number of bits, so we measure complexity in terms of bits.

4.12.4 Low-entropy Message Attack

Think about a situation where by only know the public-key of a person and the ciphertext sent to them, you can find the plaintext message. How is that possible? (**Hint:** Think what we had done in hashing to find a password from its hash.)

4.13 Randomness

We can encrypt common sentences using the public key. If the encrypted message matches one of the transmitted ciphertexts, we will know what the original message was. That's where the name *low-entropy message attack* comes from. The message has low entropy (not much randomness). For example, if the message is "Yes" or "No", there are only two possible messages. An attacker can easily encrypt both messages and compare them with the ciphertext.

To prevent this attack, we use something similar to salting: we add random bits to the message before encryption. Here we call them *padding*.

4.12.5 Is RSA bulletproof?

It has not been proven that the plaintext cannot be recovered without factoring!

Moreover, in 1977, *Ron Rivest* (one of the inventors of RSA) said[5] that factoring a 125-digit number would require 40 quadrillion years of computing power. Yet, in 1994, researchers factored a 129-digit number![6]

An RSA-2048 key does not provide 2048 bits of security; it provides only about 112 bits. This means bandwidth and computational power are wasted. This is why we use elliptic curve cryptography (ECC). ECC provides the same level of security with much smaller key sizes. For example, a 256-bit key in ECC provides the same level of security as a 3072-bit key in RSA.⁶⁹ This means that ECC is faster and uses less bandwidth. A good explanation is available on YouTube by *Christof Paar*.

4.13 Randomness

Our computers are not designed to perform random actions. They are designed to run deterministic instructions. For instance, you can't ask a computer to generate a random number between 1 and 10. It says if you want me to do multiplication, give me two numbers and I will multiply them. If you want me to add, give me two numbers and I will add them. There is no instruction for generating random numbers. As a result, they cannot generate truly random numbers. Many libraries use pseudo-random number generators to produce numbers that are not suitable for cryptographic purposes.

According to *Bruce Schneier*[24]:

“ Random-number generators are another place where cryptographic systems often break. Good random-number generators are hard to design, because their security often depends on the particulars of the

— *Bruce Schneier*

⁶⁹NIST Special Publication 800-57 Part 1, Revision 5: Recommendation for Key Management: <https://nvlpubs.nist.gov/nistpubs/SpecialPublications/NIST.SP.800-57pt1r5.pdf>

“ hardware and software. Many products we examine use bad ones. The cryptography may be strong, but if the random-number generator produces weak keys, the system is much easier to break. Other products use secure random-number generators, but they don't use enough randomness to make the cryptography secure.

— Bruce Schneier

The `random` library from Python itself says⁷⁰:

“ However, being completely deterministic, it is not suitable for all purposes, and is completely unsuitable for cryptographic purposes.

— Python `random` library

Conversely, the `secrets` module provides better randomness:

“ The `secrets` module provides access to the most secure source of randomness that your operating system provides. [25]

— Python `secrets` module

And according to *PEP 506*:

“ secrets will be based on `os.urandom` and `random.SystemRandom`, which are interfaces to your operating system's best source of cryptographic randomness. On Linux, that may be `/dev/urandom`; on Windows, it may be `CryptGenRandom()`. See the documentation and/or source code for the detailed implementation details. [26]

— PEP 506

Warning

Don't use unmaintained libraries. For example, *pycrypto* is not maintained anymore.^a^b Unfortunately, many people blindly follow AI suggestions or tutorials that use such libraries. Always check the maintenance status of libraries before using them, especially in security-related applications.

^a<https://www.pycrypto.org/>

^b<https://github.com/pycrypto/pycrypto>

How are random numbers generated? There are two ways to generate ran-

⁷⁰`random` — Generate pseudo-random numbers: <https://docs.python.org/3/library/random.html>

dom numbers:

- Pseudo-random number generators (PRNGs)
- True random number generators (TRNGs)

4.13.1 Pseudo-random number generators (PRNGs)

PRNGs are algorithms that use mathematical formulas or precalculated tables to produce sequences of numbers that **appear** random. They are **NOT** truly random because their output is determined by an initial value, known as a seed. If you know the seed, you can predict the sequence of numbers that will be generated. This is why they are called pseudo-random. As an illustration, consider a simple linear congruential generator (LCG), which generates a sequence of numbers using the formula:

$$X_{n+1} = (aX_n + c) \bmod m$$

Where:

- X is the sequence of pseudo-random values
- n is the index of the current value
- a is a multiplier
- c is an increment
- m is the modulus
- X_0 is the seed or starting value
- $X_1, X_2, \dots$ are the generated pseudo-random numbers
- The choice of a , c , m , and the seed X_0 determines the quality and period of the generated sequence.

Take $X_1 = (2X_0 + 3) \bmod 5$ as an example. If we choose a seed of $X_0 = 3$, we get:

$$\begin{aligned} X_1 &= (2 \times 3 + 3) \bmod 5 = 4 \\ X_2 &= (2 \times 4 + 3) \bmod 5 = 1 \\ X_3 &= (2 \times 1 + 3) \bmod 5 = 0 \\ X_4 &= (2 \times 0 + 3) \bmod 5 = 3 \\ X_5 &= (2 \times 3 + 3) \bmod 5 = 4 \\ X_6 &= (2 \times 4 + 3) \bmod 5 = 1 \\ X_7 &= (2 \times 1 + 3) \bmod 5 = 0 \\ X_8 &= (2 \times 0 + 3) \bmod 5 = 3 \\ X_9 &= (2 \times 3 + 3) \bmod 5 = 4 \\ &\dots \end{aligned}$$

4.13 Randomness

You can clearly see that the sequence repeats after a few numbers: 3, 4, 1, 0, ...
(When the $\gcd(a, m) = 1$, the period is maximum $m - 1$.)

This periodic and dependency of the output on the previous values makes PRNGs predictable. A true random number generator would not have such patterns. Each bit is independent of the previous bits; with the probability of 0.5, it is either 0 or 1.

- Where are they used?

- They are fast. They are used in simulations, games, and other applications where true randomness is not required.

4.13.2 True random number generators (TRNGs)

TRNGs generate random numbers using physical processes rather than mathematical algorithms. They are based on unpredictable physical processes, such as electronic noise, radioactive decay, or thermal noise. These processes are inherently random⁷¹ and produce numbers that are truly random. TRNGs are used in cryptographic applications where true randomness is required.

- Why don't we always use them?

- They are slow. They are used in applications where true randomness is required, such as generating cryptographic keys, creating digital signatures, and securing communications.

Sometimes, multiple sources are combined to generate random numbers. For example, the Linux kernel uses a combination of sources, including keyboard and mouse activity, disk activity, network activity, light sensors (including webcams), and other sources to generate random numbers. This is why some people say that moving the mouse cursor around the screen can help improve the security of your system while generating keys.

Reading List

- [random.org](https://www.random.org/)⁷²
- [lotterycodex](https://lotterycodex.com/) (Full of analysis data)⁷³
- [Random number generator attack](https://en.wikipedia.org/wiki/Random_number_generator_attack)⁷⁴
- [Ensuring Randomness with Linux's Random Number Generator](https://blog.cloudflare.com/ensuring-randomness-with-linuxs-random-number-generator/)⁷⁵
- [Could RDRAND \(Intel\) compromise entropy?](https://crypto.stackexchange.com/a/10285)⁷⁶
- [PCG, A Family of Better Random Number Generators](https://www.pcg-random.org/)⁷⁷

⁷¹I am not an expert. Maybe there are some predictable patterns.

⁷²<https://www.random.org/>

⁷³<https://lotterycodex.com/>

⁷⁴https://en.wikipedia.org/wiki/Random_number_generator_attack

⁷⁵<https://blog.cloudflare.com/ensuring-randomness-with-linuxs-random-number-generator/>

⁷⁶<https://crypto.stackexchange.com/a/10285>

⁷⁷<https://www.pcg-random.org/>

4.14 An example of an E2EE system

Now that you know about encryption and hashing, let's see how they are used in a real-world application. Consider the example of the *Tuta* email service. Tuta provides end-to-end encryption (E2EE) for its users, meaning that only the sender and recipient can read the messages; no one else, not even Tuta, can access them. User passwords are combined with a salt⁷⁸ and hashed using a

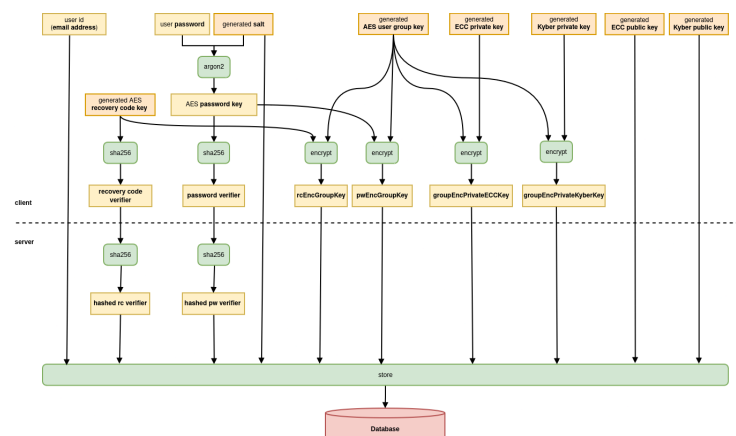


Figure 4.11: src: <https://tuta.com/encryption>

password hashing algorithm (Argon2). The hash output is 256 bits⁷⁹.

This hash is used as the encryption key for AES-256. The message is encrypted with this key. The salt is sent to the server. When you use another device to decrypt messages, the server provides the salt so you can combine it with your password for hashing.

So far, we have solved the problem of encryption. But what about authentication? A person might need to log in to the email service from different devices. How can we be sure that the person is the same one who created the account? We need to store a hash in the database. But wait! If we store the previously discussed hash in the database, anyone who has access to the database can use it as an encryption key and decrypt messages. So what should we do?

Tuta hashes this Argon2 hash with SHA2-256 and sends it to its servers. If someone gains access to this hash, they would need to brute-force 256 bits (since the key is 256 bits) and compare each attempt with the SHA2-256 hash. Since the key is random (remember, the Argon2 hash is a random mapping of inputs to outputs), it is impossible to brute-force 256 bits to match the SHA2-256 hash. Why would someone bother brute-forcing 256 bits to match SHA2-256 when they could try brute-forcing AES keys directly?

⁷⁸Remember, the salt must be completely random for each user, and its size should follow recommendations from authorities.

⁷⁹<https://datatracker.ietf.org/doc/html/rfc9106#name-argon2-inputs-and-outputs>

4.15 Post-Quantum Cryptography

Then, SHA2-256 is applied again to the SHA2-256 hash, and the result is stored in the database.

In this system, both ends are users. In messengers, the ends are also users communicating with each other.

Remember that developers should never access users' passwords. If something happens and hackers gain access to the server, they could use the passwords to log in to other services.

You may want to visit Bitwarden⁸⁰ for the solution they use.

You may also want to visit the Notesnook website⁸¹, which interactively shows how an end-to-end application works. It's excellent.

4.15 Post-Quantum Cryptography

Remember, we just said that for **classical** computers, no **general** algorithm has been found to factor an integer n in polynomial time. But what about **quantum** computers? They are based on quantum-mechanical phenomena, such as superposition and entanglement, which allow them to perform **some**⁸² tasks much faster than classical computers. One such task is factoring. There are some algorithms⁸³ that can factor a number in polynomial time on quantum computers. This means that RSA could be broken. Of course, there are some details that we do not cover here. To date, quantum computers are not powerful enough to break RSA (or ECC) in the near future. But who knows? Maybe in 20 years, they will be powerful enough to break it. This is why we need to find new algorithms that are secure against quantum computers. This is called *Post-Quantum Cryptography*.

Should we be worried? No, because NIST (and independent experts) are working on it. Post-quantum cryptography is hard. NIST itself says:

“ NIST anticipates that the evaluation process for these post-quantum cryptosystems may be significantly more complex than the evaluation of the SHA-3 and AES candidates. One reason is that the requirements for public-key encryption and digital signatures are more complicated. Another reason is that the current scientific understanding of the power of quantum computers is far from comprehensive. Finally, some of the candidate post-quantum cryptosystems may have completely different design attributes and mathematical foundations,

— NIST

⁸⁰<https://bitwarden.com/help/bitwarden-security-white-paper/#hashing-key-derivation-and-encryption>

⁸¹<https://vericrypt.notesnook.com/URL>

⁸²One common mistake is to think quantum computers are much faster and better at any task. No, they are not! They just solve certain problems in a different way.

⁸³e.g. Shor's algorithm

“ so that a direct comparison of candidates would be difficult or impossible.

— NIST

However, attackers and governments are collecting encrypted data and storing it. They may be able to decrypt it in the future. This is why we need to be prepared. Remember, anything that goes onto the internet stays there forever and may eventually end up in the hands of attackers.

However, our symmetric encryption algorithms will not be completely broken⁸⁴[27].

Hash functions are also less affected by quantum computers.[28]⁸⁵⁸⁶⁸⁷

Do not get overly excited about apps that claim to use post-quantum algorithms. They may be using weak algorithms. For instance, one of the candidates submitted to NIST was broken by classical computers!⁸⁸

As Bruce Schneier⁸⁹ and NIST say, post-quantum cryptography is hard and current knowledge is far from comprehensive. For now, it is better to use quantum-resistant algorithms as an **additional layer** to our current algorithms. The *Signal* app is one of the apps that uses post-quantum algorithms, but it uses them as an additional layer:

“ We believe that the key encapsulation mechanism we have selected, CRYSTALS-Kyber, is built on solid foundations, but to be safe we do not want to simply replace our existing elliptic curve cryptography foundations with a post-quantum public key cryptosystem. Instead, we are augmenting our existing cryptosystems such that an attacker must break *both* systems in order to compute the keys protecting people’s communications.^a

^a<https://signal.org/blog/pqxdh/>

— Signal

⁸⁴See Section 3, Symmetric Encryption Schemes from BSI TR-02102-1 “Cryptographic Mechanisms: Recommendations and Key Lengths” Version: 2025-1.: <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.html>

⁸⁵BSI TR-02102-1: “Cryptographic Mechanisms: Recommendations and Key Lengths” Version: 2024-1: https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf?__blob=publicationFile&v=7

⁸⁶<https://www.schneier.com/blog/archives/2022/08/nists-post-quantum-cryptography-standards.html>

⁸⁷<https://www.nist.gov/cybersecurity/what-post-quantum-cryptography>

⁸⁸<https://csrc.nist.gov/csrc/media/Projects/post-quantum-cryptography/documents/round-4/submissions/sike-team-note-insecure.pdf>

⁸⁹<https://www.schneier.com/blog/archives/2023/08/you-cant-rush-post-quantum-computing-standards.html>

4.15.1 Debunking RSA “Breaks”: The Reality Behind the Headlines

We see countless so-called “papers” that claim to factor RSA keys with today’s quantum computers. However, these claims often misrepresent the reality of the situation.

In the field of number theory, there are many *specialized* algorithms for factoring integers that are much faster than simple trial division. These algorithms, however, are not general-purpose; they work by exploiting specific structural properties of the number being factored. For example:

- **Pollard’s $p-1$** algorithm is highly effective at factoring a number n if one of its prime factors, p , has the property that $p - 1$ is “smooth” (i.e., it is composed only of small prime factors).
- **Fermat’s factorization method** is efficient when the two prime factors of n are close to each other.

While these methods are clever, they are ineffective against the numbers used in real-world RSA implementations. The security of RSA relies on a careful key generation process where two large, random prime numbers are chosen specifically to avoid these known weaknesses.

This brings us to the recent news. When a paper claims to have factored a large number (e.g., a 2048-bit number) with a quantum computer, it’s crucial to examine the details. Often, these experiments don’t break a true, randomly generated RSA key. Instead, they demonstrate their method on a specially constructed number that is vulnerable to their specific algorithm.

This is like someone claiming they bought a Mercedes-Benz, but when you ask to see it, they show you a toy car made by Mercedes. While technically true, it’s misleading. Similarly, factoring a carefully crafted number that is weak to a specific attack is not the same as breaking the RSA encryption that secures global communication. News outlets often publish these stories with sensational headlines like “RSA is broken, we’re all doomed!” to attract viewers, without conveying the critical nuance that the factored numbers are not representative of those used in practice.

See the funny article *Replication of Quantum Factorisation Records with an 8-bit Home Computer, an Abacus, and a Dog* [29].

4.16 MAC and HMAC

4.16.1 Message Authentication Code (MAC)

Suppose you want to send a message to somebody. How can your friend be sure that the message hasn’t been changed?

One way is to use a digital signature: hash the message, then encrypt the hash with your private key. On the other side, your friend decrypts the hash and compares it with the hash of the message that they calculated themselves. If they are equal, it means the message hasn't been altered. But what if you don't want to use a digital signature? We still need to use hash functions, as they can help us detect even small changes.

Someone might suggest sending the hash alongside the message. So instead of sending just the message, we can send: `m, Hash(m)`. How can this go wrong?

An attacker in the middle can change the message, recalculate the hash, then replace the old hash with the new hash and send it to the receiver. By doing this, the attacker successfully changes the message without the parties noticing. What can we do to prevent the attacker from being able to do this? (Hint: Change the hash mechanism so the attacker can't replace it with their own hash.)

We can use something similar to salting here. Instead of sending just the message, we can send: `m, Hash(secret || m)`⁹⁰, where `secret` is a shared secret between both parties. If an attacker wants to change the message, they need to know the secret in order to calculate the hash and replace it with the existing hash. Otherwise, the change will be detected.

One person might use `m, Hash(secret || m)`, while someone else might use `m, Hash(m || secret)`. To have a standard mechanism that prevents certain attacks (e.g., length-extension attacks), we can use a standard protocol for authentication. This is called *HMAC*.

4.16.2 Designated Verifier Signature

Suppose Alice is working for a corrupt company. She decides to blow the whistle on the corruption and tell the newspaper the whole story. The newspaper should know that the message came from Alice, but Alice should only be able to prove the authenticity of the message to the newspaper. She doesn't want the newspaper to be able to prove the authenticity of the message to anyone else. This is called a *Designated Verifier Signature*. It is like a digital signature, but the verifier is designated. The newspaper should not be able to tell others that the message is from Alice.

For this, we can use a symmetric MAC. The newspaper can be sure that the message is from Alice, but they can't prove it to others (since the MAC can be generated by both parties, as they both have the secret). See also *Stronger Security and Constructions of Multi-Designated Verifier Signatures*[30] and its presentation⁹¹.

⁹⁰ `||` means concatenation

⁹¹ IACR, Stronger Security and Constructions of Multi-Designated Verifier Signatures <https://youtu.be/tJqK9zJ10iA>

4.17 Homomorphic Encryption

Suppose you want to process your data, but you do not have enough resources. You want someone else to use their computational power to give you the answer. But what if the data is sensitive? Cryptography allows you to give them transformed data without revealing the actual data. They can compute the answer, and you can then transform the result back to obtain the actual answer.

For example, suppose you want to find the result of $100 + 200$. You apply the Caesar cipher with a shift of 40, so the two numbers become 140 and 240. You give these two numbers to a third party to perform the calculation. They compute $140 + 240 = 380$ and return 380 to you. You then subtract 80 (since each number was shifted by 40), so $380 - 80 = 300$. Now your result is ready! You gave someone a problem, they solved it without knowing the actual question, and you obtained the correct result. This is a simple example of *homomorphic encryption*.

Applications of homomorphic encryption include:

- Distributed computing on sensitive data: Surely you don't want to give health data to a third party to process it/learn on it. Homomorphic encryption allows you to give them encrypted data, and they can process it without knowing the actual data.
- Search on encrypted data: You can search for a keyword in an encrypted database without decrypting the entire database.

Do you remember privately outsourcing computation, which we discussed at the beginning of the chapter?

4.18 Shamir's Secret Sharing

Bruce Schneier, in his book *Applied Cryptography* [17] talks about a scenario:

“ You're setting up a launch program for a nuclear missile. You want to make sure that no single raving lunatic can initiate a launch. You want at least three out of five officers to be raving lunatics before you allow a launch. How can you set up a system that will allow a launch?

— Applied Cryptography

This is a classic example of secret sharing. In this case, the secret is the launch code, and it is shared among five officers. The secret is divided into five shares, and each officer is given one share. The secret can be reconstructed only when at least three officers come together and combine their shares. This is a simple example of secret sharing. We will discuss secret sharing in more detail later.

4.18.1 Secret Sharing

Secret sharing is a technique in cryptography in which a secret is divided into multiple shares, and each share is distributed among a group of participants. The secret can be reconstructed only when a certain number of participants come together and combine their shares. The secret cannot be reconstructed if the number of participants is less than the required threshold. This ensures that no single participant can reconstruct the secret. Secret sharing is used in various applications, such as nuclear launch codes, cryptographic keys, etc.

Definition 9. A (k, n) secret sharing scheme is a method to divide a secret S into n shares such that the secret can be reconstructed when at least k shares are combined.

In the physical world, secret sharing can be implemented using physical objects like keys, cards, etc. A lock can be opened if and only if a certain number of keys are used. In contrast, in the digital world, things are more complex.

4.18.2 Naive Approach

Definition 10. Encryption is the process of converting plaintext into ciphertext using a key. Only the person who has the key can decrypt the ciphertext and recover the plaintext.

Definition 11. Security bits is a measure of the strength of a cryptographic algorithm. It is the number of bits required to break the algorithm. A cryptographic algorithm with n bits of security requires 2^n operations to break.

A naive approach is to encrypt the secret using a key and then divide the key into shares. Shares can be distributed among participants. The secret can be reconstructed when the shares are combined and the key is recovered.

While this approach is easy to implement, it is completely insecure. According to *NIST*, if data should remain secure until 2030 and beyond, it should have at least 112 and 128 bits of security, respectively [31]. If our key has 128 bits of security, dividing it into 4 shares will reduce the security to 32 bits. Brute-forcing a 32-bit key can be done in a matter of seconds [17]. So, this approach is not secure.

4.18.3 Lagrange Interpolation

Suppose we have certain points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$. The Lagrange interpolation formula is used to find a polynomial that passes through these points. Lagrange interpolation does this by first creating a set of polynomials $L_i(x)$ such that

$$L_i(x_j) = \begin{cases} 1 & \text{if } i = j \\ 0 & \text{if } i \neq j \end{cases}$$

Because each Lagrange polynomial only contributes to its own point, we can multiply it by the corresponding y_i to get the final polynomial:

$$L(x) = \sum_{i=0}^n y_i L_i(x)$$

How can we create a polynomial that is zero at n points and 1 at the $(n+1)$ th point? Suppose we have points $x_0, x_1, \dots, x_n$. The polynomial that is zero at all points except x_0 is

$$L(x) = (x - x_1)(x - x_2) \dots (x - x_n).$$

Example 4.18.1. Find the polynomial such that $L_1(1) = 1$, $L_1(0) = L_1(2) = L_1(3) = 0$.

We can guess that the polynomial is

$$L_1(x) = a(x - 0)(x - 2)(x - 3).$$

At $x = 1$,

$$L_1(1) = a(1)(1 - 2)(1 - 3) = 2a.$$

So, $a = \frac{1}{2}$. The polynomial is

$$L_1(x) = \frac{1}{2}x(x - 2)(x - 3).$$

How can we generalize this to any set of points? (Because we need to find a general formula.) Let our new guess be:

$$L_1(x) = \frac{(x - 0)(x - 2)(x - 3)}{(1 - 0)(1 - 2)(1 - 3)}.$$

This will give us $L_1(1) = 1$ and $L_1(0) = L_1(2) = L_1(3) = 0$.

In this way, we can find a general formula for $L_i(x)$:

$$L_i(x) = \prod_{\substack{j=0 \\ j \neq i}}^n \frac{(x - x_j)}{(x_i - x_j)}.$$

Given $n + 1$ points $(x_0, y_0), (x_1, y_1), \dots, (x_n, y_n)$, the polynomial that passes through these points is:

$$L(x) = \sum_{i=0}^n y_i L_i(x).$$

With $n+1$ points, we can find a polynomial of degree n that passes through these points.

Proof. Suppose $L(x_i) \neq p(x_i)$ for some i . We know that:

$$p(x_i) = y_0 L_0(x_i) + y_1 L_1(x_i) + \dots + y_i L_i(x_i) + \dots + y_n L_n(x_i).$$

As $L_i(x_i) = 1$ and $L_j(x_i) = 0$ for $j \neq i$, we get:

$$p(x_i) = y_i L_i(x_i) = y_i.$$

So, $p(x_i) = y_i$ for all i . □

To prove that the polynomial is unique, we can use the following theorem:

Theorem 4.18.1. *Any polynomial of degree n has at most n roots.*

Suppose that $L(x)$ is the Lagrange interpolation polynomial for $n+1$ points $(x_0, x_1, \dots, x_n)$. Suppose there is another polynomial $q(x)$ of degree m where $m < n$ that passes through these points.

Then, $L(x) - q(x) = 0$ at all x_i for $i \in \{0, 1, \dots, n\}$, since both polynomials pass through the same points. But $L(x) - q(x)$ is a polynomial of at most degree n and has $n+1$ roots. This is a contradiction. Therefore, $q(x)$ cannot exist.

Theorem 4.18.2. *The Lagrange interpolation formula finds the lowest-degree polynomial that passes through the given points.*

4.18.4 Shamir's Secret Sharing

Background

- **Question:** How many lines pass through a single specific point?
- **Answer:** Infinitely many.
- **Question:** How many lines pass through two specific points?
- **Answer:** One.
- **Question:** How many degree 2 polynomials pass through two specific points?
- **Answer:** Infinitely many.
- **Question:** How many degree 2 polynomials pass through three specific points?

- **Answer:** One.
- **Question:** How many degree 3 polynomials pass through three specific points?
- **Answer:** Infinitely many.
- **Question:** How many degree 3 polynomials pass through four specific points?
- **Answer:** One.

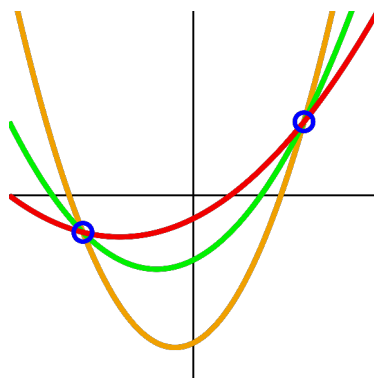


Figure 4.12: src: Vlsergey, 3 polynomials of degree 2 through 2 points, CC BY 3.0

We can see that with $n + 1$ distinct points, we can find a unique polynomial of degree n that passes through these points.

Shamir's Secret Sharing Scheme

Adi Shamir introduced a secret sharing scheme in 1979 [32]. For (k, n) secret sharing, a polynomial of degree $k - 1$ is created such that the secret is the constant term of the polynomial. The polynomial is evaluated at n points, and the shares are distributed among participants. The secret can be reconstructed when at least k shares are combined.

That's it! Simple! Imagine condensing a scientific breakthrough so brilliantly that two pages could spark a global research tsunami, generating roughly 21,000 academic citations by 2025.⁹²

Now let's go into the details. What coefficients should we use for the polynomial? Computers cannot store floating-point numbers precisely. (See Section 6.9.4) This may result in errors, so we should use integers for the coefficients. However, using integer coefficients can lead to a loss of perfect secrecy.

⁹²<https://scholar.google.com/citations?user=EHLBP20AAAAJ&hl=en>

Definition 12. *Perfect secrecy is a property of a cryptographic scheme in which the ciphertext reveals no information about the plaintext.*

Here, information about $k - 1$ points must not reveal any information about the secret. But using integers in the polynomial can result in information leakage. Consider the following example:

Example 4.18.2. *Let $f(x) = 1 + x$. The secret is 1. Points $(3, 4)$ and $(1, 2)$ are given to Alice and Bob, respectively. Alice cannot determine what the secret is, but she can determine what values it cannot be. She can find out that 2 is not the secret. If 2 were the secret, then the function would be $f(x) = 2 + \frac{2}{3} \cdot 3 = 4$. But the function does not have integer coefficients. So, 2 is not the secret.*

In this way, having partial information can result in information leakage.

Finite Fields

Finite fields have certain properties that make them suitable for cryptographic applications. The proof of the perfect secrecy of finite fields is beyond the scope of this book.

Definition 13. *A finite field is a field that has a finite number of elements on which operations like addition, subtraction, multiplication, and division can be performed. Properties of finite fields:*

- *Closure under addition, subtraction, multiplication, and division. $(a, b \in F \implies a + b \in F)$*
- *Associativity of addition, subtraction, multiplication, and division. $(a + (b + c) = (a + b) + c)$*
- *Commutativity of addition and multiplication. $(a + b = b + a)$*
- *Distributive property. $(a(b + c) = ab + ac)$*
- *Existence of additive and multiplicative identity. $(a + 0 = a \text{ and } a \times 1 = a)$*
- *Existence of additive and multiplicative inverses. $(a + (-a) = 0 \text{ and } a \times a^{-1} = 1)$*

Shamir's Secret Sharing in Finite Fields

Instead of using the infinite field of integers, which requires infinitely many bits and can result in information leakage, we can use finite fields.

Definition 14. *A finite field of order p is a field that has p elements, where p is a prime number, and is denoted by $\mathbb{Z}_p$.*

Example 4.18.3. $f(x) = 1 + x \pmod{7}$, which is a polynomial in $\mathbb{Z}_7$.

Theorem 4.18.3. *If a and m are relatively prime, then a has a multiplicative inverse modulo m .*

A prime number is used as the order of the finite field because the multiplicative inverse exists for all elements in the field:

$$ax \equiv 1 \pmod{p}$$

Example 4.18.4. *Find the multiplicative inverse of 3 in $\mathbb{Z}_7$.*

We need to find x such that $3x \equiv 1 \pmod{7}$. $3 \times 5 = 15 \equiv 1 \pmod{7}$. So, the multiplicative inverse of 3 in $\mathbb{Z}_7$ is 5.

4.19 Zero-Knowledge Proofs

Think of a scenario where you want to prove that you know a secret without revealing the secret. For instance, you want to prove that you know the password to a system without revealing the password, or you know the solution to a Sudoku puzzle⁹³ without revealing the solution. This is where zero-knowledge proofs come into play.

Imagine we have a cave with a secret door in the middle. Alice knows the secret to open the door, while Bob doesn't know the secret. Alice wants to prove to Bob that she knows the secret without revealing it. This is a zero-knowledge proof. Alice goes to the cave and randomly chooses one side. Then Bob enters

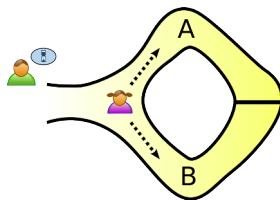


Figure 4.13: src: Dake, Zkip alibaba1, CC BY 2.5

the cave and chooses a side for Alice to come out from. For example, here Bob chooses side A. If Alice was at side A, she can come out from side A. If she was at side B, she must know the secret to open the door and come out from side A.

In this way, Alice can prove to Bob that she knows the secret without revealing it. However, Bob may think that Alice simply entered from side A and, without knowing the secret, can come out. There is a 50% chance that Alice was already on the correct side. Bob can ask her to repeat the process. If Alice can repeat the process multiple times, the probability that she does not know

⁹³Interactive Sudoku Zero-knowledge Proof: <https://manishearth.github.io/blog/2016/08/10/interactive-sudoku-zero-knowledge-proof/>

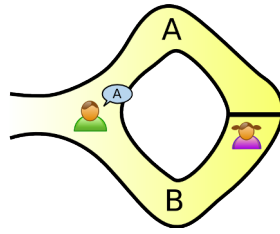


Figure 4.14: src: Dake, Zkip alibaba1, CC BY 2.5

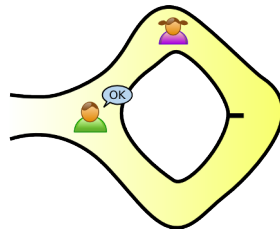


Figure 4.15: src: Dake, Zkip alibaba1, CC BY 2.5

the secret and can still come out from the side Bob chooses decreases. The first time, the probability is $1/2$. The second time, it is $1/2 \times 1/2 = 1/4$. The third time, it is $1/8$. After n repetitions, the probability is $1/2^n$.

If we do it 20 times, the probability that Alice doesn't know the secret and can come out from the side Bob has chosen is $1/2^{20} = 1/1048576$. The probability is very low. Thus, Bob can be sure that Alice knows the secret.

Zero-knowledge proofs have many applications. For instance, anonymous coins use ZKP. Or[33]:

Currently, when a security analyst discovers a vulnerability in critical software system, they must navigate a fraught dilemma: immediately disclosing the vulnerability to the public could harm the system's users; whereas disclosing the vulnerability only to the software's vendor lets the vendor disregard or deprioritize the security risk, to the detriment of unwittingly-affected users. ...by using Zero Knowledge (ZK) protocols that let analysts prove that they know a vulnerability in a program, without revealing the details of the vulnerability or the inputs that exploit it.

4.20 Bit-commitment

Read about Bit-commitment in section "4.9 Bit-commitment" of the book "Applied Cryptography" by Bruce Schneier[17].

4.21 Combining Multiple Block Ciphers

If sensitive or important data should be encrypted for a long time, it is better to use multiple block ciphers. For instance, you can encrypt the data with AES, then encrypt the result with TWOFISH, and then encrypt the result with Serpent. This way, if one of the algorithms is broken, the data is still secure. This is called *cascade encryption*. *cascade encryption* is an example of adding layers of defense. See *Prevention* from 3.1.2.

The high-level explanation I gave is not enough to fully understand how cascade encryption works. It is a complex topic that requires a deep understanding of cryptography and block ciphers. If you do it incorrectly, you may end up with a system that is less secure or at best only as secure as a single algorithm.

For instance, certain requirements must be met for cascade encryption. For example, the keys should be independent. This topic is highly complex and beyond the scope of this book. I recommend that you read about it from reputable sources.

4.22 What to know?

- SHA2, SHA3: Used for authentication, check for similarity
- Argon2, Bcrypt, PBKDF2 Used for generating symmetric encryption key
- Symmetric Encryption: Used for fast encryption of data. Example: AES, TWOFISH, Serpent
- Asymmetric Encryption: Used for transferring/generating symmetric keys for both sides. Example: RSA, ECC, Diffie-Hellman

Cryptography is very hard. Let me quote a paragraph from *Cryptography Engineering*[1] book:

“ That makes this a very dangerous book. Some people will read this book, and then turn around and design a cryptographic algorithm or protocol. When they’re finished, they’ll have something that looks good to them, and maybe even works, but will it be secure? Maybe they’ll get 70% right. If they’re very lucky, they may get 90% right. But there is no prize for being almost right in cryptography. A security system is only as strong as its weakest link; to be secure, everything must be right. And that is something you simply can’t learn from reading books.

— *Cryptography Engineering*

For instance, when working with encryption keys, many things can go wrong. Operating systems use temporary files, hibernation files, and swap files to keep

track of changes in case of power outages or for RAM management. An encryption key might be stored on the hard disk and never deleted! You might use an encryption key in a function and not clear its location in RAM before the function exits. This is very dangerous. That memory location could be given to another process, which could then access the encryption key stored there. With high-level programming languages, it is sometimes impossible to clear RAM correctly. With languages like C, you can use something like the `memset()` function. But wait! What if the compiler sees that an array is zeroed and never used again, and optimizes away the zeroing? If you are writing code for critical systems, you should read the assembly code generated by the compiler. Consider the following code snippet in C:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
int get_random_128bit_key(unsigned char key[16]);
void encrypt_decrypt(const unsigned char *input, unsigned char *output,
                    unsigned char key[16], size_t input_len);

int main()
{
    const char *plaintext = "This_is_a_secret_message.";
    size_t len = strlen(plaintext);
    unsigned char key[16];
    if (get_random_128bit_key(key) != 0) {
        fprintf(stderr, "Failed_to_get_random_key\n");
        return 1;
    }

    unsigned char ciphertext[len];
    encrypt_decrypt((const unsigned char *)plaintext, ciphertext, key, len);
    printf("Ciphertext:_");
    for (size_t i = 0; i < len; i++) {
        printf("%02x", ciphertext[i]);
    }

    printf("\n");

    unsigned char decrypted_text[len + 1];
    encrypt_decrypt(ciphertext, decrypted_text, key, len);
    decrypted_text[len] = '\0';
    printf("Decrypted_text:_%s\n", decrypted_text);

    for (size_t i = 0; i < 16; i++) {
        key[i] = 0;
    }
}
```

```
    return 0;
}

int get_random_128bit_key(unsigned char key[16])
{
    FILE *fp = fopen("/dev/urandom", "rb");
    if (!fp) {
        return -1;
    }
    if (fread(key, 1, 16, fp) != 16) {
        fclose(fp);
        return -1;
    }
    fclose(fp);
    return 0;
}

void encrypt_decrypt(const unsigned char *input, unsigned char *output,
                    unsigned char key[16], size_t input_len)
{
    for (size_t i = 0; i < input_len; i++) {
        output[i] = input[i] ^ key[i % 16];
    }
}
```

Just look at the the zeroing `for` loop at the end of the `main()` function. The compiler might optimize it away! You should check the assembly code generated by the compiler to ensure that the zeroing is not optimized away. You can use the `-S` flag with `gcc` to generate assembly code. For example:

```
gcc -O3 -S your_code.c -o your_code.s
```

Or use the online Compiler Explorer⁹⁴.

We can see that no code is generated for the zeroing loop. The compiler optimized it away! To prevent this, a person should have the knowledge about GCC compiler! Or to be more precise, one should read the **specific documentation**⁹⁵ for that **specific version**⁹⁶ of GCC they use.⁹⁷ Moreover, they should be aware about security implications of using anything they're using! Only after that, one should implement the code they were intended to do.

We can change the loop to:

⁹⁴<https://godbolt.org/>

⁹⁵As we discussed in subsection ??, reading documentation and therefore not relying on trial-and-error is important in security-related tasks.

⁹⁶This is important! Different versions of GCC may have different behaviors.

⁹⁷<https://gcc.gnu.org/onlinedocs/gcc-15.2.0/gcc/Volatiles.html>

```
volatile unsigned char *vkey = key;
for (size_t i = 0; i < 16; i++) {
    vkey[i] = 0;
}
```

This way, the compiler will not optimize it away. But wait! What if the compiler is buggy? What if the version of the compiler you're using has a bug that causes it to optimize away volatile writes? You either have to read all the bug reports for that version of the compiler, which is impractical or may not be discovered yet, or you have to read the Assembly code generated by the compiler to ensure that the volatile write is not optimized away.

Who has access to encryption keys? The CEO? Does the CEO have enough knowledge about security? Can the CEO protect the key well enough? What if the CEO dies? Often, parameters that can be used to recreate the encryption key are distributed among several people (e.g., Shamir's Secret Sharing).

Imagine a situation where, during the encryption process, a crash occurs. Your data may be half-encrypted or become corrupted. You may not be able to decrypt it.

There are many things that can go wrong in cryptography and security. Do **NOT** implement your own security mechanisms. Simply don't!

Reading List

- *cryptocoding v2* Presentation⁹⁸ by JP Aumasson
- Cryptocoding⁹⁹ ("coding rules" for implementations of cryptographic operations, and more generally for operations involving secret or sensitive values.)
- Are garbage-collection programming languages inherently unsafe for use in cryptography¹⁰⁰ (Didn't fully read it but seems related)
- Cryptofails¹⁰¹ (Really interesting)

4.23 End of Chapter Questions

Problem 4.1 Why do we prefer to use different symmetric keys for each session (a session is a period of time during which two systems communicate with each other) instead of using the same key for all sessions? (Hint: Read subsection "Heartbleed" 7.4.2.)

Problem 4.2 Quantum computers are becoming increasingly powerful, raising

⁹⁸https://www.aumasson.jp/data/talks/cryptocodingv2_zn14.pdf

⁹⁹<https://github.com/veorq/cryptocoding>

¹⁰⁰<https://crypto.stackexchange.com/questions/113619/are-garbage-collection-programming-languages-inherent>

¹⁰¹<https://www.cryptofails.com/archive>

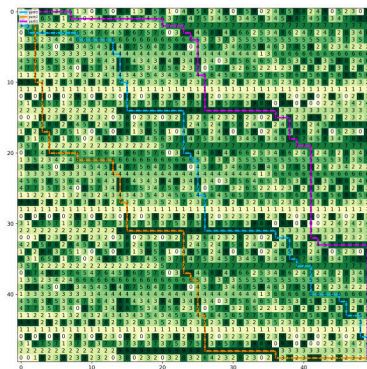


Figure 4.16: Random numbers generated by random.randint in a grid

concerns that current public-key cryptographic algorithms will soon be broken. The National Institute of Standards and Technology (NIST) is seeking a standard replacement for global use. If you were tasked with designing such an algorithm, what three key aspects would you prioritize to ensure it could become a worldwide standard?

Problem 4.3 In the final round of NIST’s competition to choose the Advanced Encryption Standard—held nearly twenty years ago—three algorithms were shortlisted: Rijndael, Serpent, and Twofish. The experts agreed that all three were secure, and on paper Serpent even offered the largest safety margin. Yet for various practical reasons, NIST ultimately selected Rijndael as AES. Now imagine one of your engineers argues that you should adopt Serpent, while another insists that you stick with AES for security reasons. Why might the second engineer recommend AES—even though Serpent appears stronger on paper?

Problem 4.4 A system uses a AES with a 256-bit key to encrypt data. The asymmetric algorithm used, RSA, has a 4096-bit key. The hashing algorithm used is SHA-256. What is the security level of the system in bits?

Problem 4.5 What is the difference between coding and encryption? With an example, illustrate the difference.

Problem 4.6 The following numbers (in each cell) are created by random.randint. can you spot the period?

Problem 4.7 As we discussed earlier, for one with high threat model, it is important to assume that any system is compromised. In such a scenario, how can one prevent password brute-forcing with the assumption of that system uses SHA-256 for hashing passwords?

4.24 End of the Chapter Solutions

Solution 4.1 If one session's key is leaked (e.g., a hacker steals it), the other sessions' keys remain secure. It is important that, in case of a problem, past and future data cannot be decrypted.

Solution 4.2

- **Security:** The algorithm should be resistant to quantum attacks, ensuring that it remains secure against future quantum computing capabilities.
- **Easy-to-Audit Design:** The algorithm should have a simple and transparent design, making it easy for security experts to audit and verify its correctness.
- **Performance and Cost:** The algorithm should be efficient in terms of computational resources, allowing it to run on a wide range of devices, from low-power smart cards to high-end servers.
- **Hardware-Friendly Implementation:** The algorithm should be designed to be compatible with hardware acceleration, allowing for efficient implementation on various platforms.

Solution 4.3 One of the following responses is acceptable:

- a) AES has been a formal standard for years and has undergone far more review by security experts than Serpent. In cryptography, it's vital that algorithms be vetted by many specialists.
- b) A much larger proportion of Serpent's libraries and implementations have gone unmaintained and outdated for a long time.
- c) AES benefits from hardware acceleration on many CPUs (dedicated instruction-set support), giving it higher throughput and built-in resistance to certain side-channel attacks.
- d) Standardized algorithms almost always come with extensive, trusted documentation and usage guidelines, which greatly reduces the risks that arise when developers rely on less-documented, ad-hoc implementations.

Solution 4.4 The security level of the system is determined by the weakest link. AES-256 has a security level of 256 bits, SHA-256 has a security level of 128 bits, and RSA-4096 has a security level of above 128 bits. The weakest link is SHA-256, so the security level of the system is 128 bits.

Solution 4.5 Coding is the process of converting data from one format to another. For example, converting a string to a number or vice versa. Encryption is the process of converting data into a format that is unreadable by anyone who does not have the key to decrypt it. For example, encrypting a message using AES-256.

Example 4.24.1. $\text{code}(\text{"hello"}) \rightarrow 72\ 101\ 108\ 108\ 111$ (*ASCII values*)
 $\text{AES-256}(\text{"hello"}) \rightarrow (16F5AAC06C0E96E8B60723639DBC7492405F7D38)$

Solution 4.6 See line 11 and 28. Or 12 and 29.

Solution 4.7 To prevent brute-forcing, ensure the password has sufficient entropy, ideally around 128 bits. This makes brute-forcing infeasible with current technology (see 4.9). Note that each ASCII character does not contribute a full 8 bits of entropy, as not all bits are entirely random.



5.0 Hardware Security

Your software runs on hardware. If the hardware is compromised, your software is compromised. Let's assume you're a developer. Did you know that if your computer is compromised, all the software you write is compromised? An attacker can add a backdoor to your software without you knowing it! Even if you use code signing, an attacker can steal your code signing key and sign the backdoored software with it! So, hardware security is very important.

“ No operating system, not even Qubes, can help you if you're installing it on hardware that is already compromised. This includes CPUs, GPUs, SSDs, HDDs, the motherboard, BIOS/EFI/UEFI, and all relevant firmware. Unfortunately, in today's world of undetectable supply chain attacks, there are no easy solutions.^a

^a<https://doc.qubes-os.org/en/latest/user/downloading-installing-upgrading/install-security.html>

— *Qubes-OS Security Guide*

5.1 Physical Access

The most important thing to remember is that physical access means being hacked. If I have physical access to your computer, I can do anything with it. I can simply download and install a backdoor!

- How can you download one?

- I can upload a password-protected zip file to cloud storage and download it from your computer. (Why a zip file? I'll explain after I talk about encryption.)

- No. My system requires a password to install software.

- For most operating systems, there are portable versions of software. I can use that feature!

- No! My system doesn't have an internet connection!

- I can use a USB drive!

5.1 Physical Access

- I have an antivirus!

- Many targeted malware¹ are not detected by antivirus programs. Some are embedded into the firmware and cannot be removed even after formatting! If the target is worth \$100,000 and the malware costs \$10,000 to develop, it is worth it!

- I can quickly code a program to spy on you. Many systems have some programming language compilers or interpreters installed.

- My system is locked. You cannot run portable software.

- Malware is not the only way to hack. Most hacks are done via vulnerabilities. I can use a vulnerability to get access to your system.

- I can use a live USB to boot your system and access your data. Many operating systems have a feature to boot from a USB drive. This means I can download it and boot your system from it. It is like a normal operating system, but it runs on your USB drive. The only thing you may need to change is the boot device priority in the BIOS. With this, I can access the data on your hard drive.

- I have my BIOS encrypted.²

- No worries :) I can physically remove your hard drive and connect it to another computer. I can access the data on it.

- I have encrypted my important files.

- How about your swap files, temp files, log files, and other hidden files? Full disk encryption must be used alongside file encryption. If only file encryption is used, the data in the hidden files can be accessed. If only full disk encryption is used, after decrypting the disk (even if the decryption is on-the-fly), data could be in danger.

- I have encrypted my hard drive.

- Maybe your hard drive password is the same as your login password. I can use that password to access the data.

- No. I have never used this password for anything else. It is long and random.

- I can use a hardware keylogger. It is a small device that is connected between the keyboard and the computer. It records all the keystrokes and sends them to me via a SIM card.

- I don't let anybody access my computer or attach anything to it.³

- You may leave your laptop in the hotel when you're going out. The hotel

¹If you don't know what malware is, just read it as a virus. We will cover it later.

²See also: BIOS Password Backdoors in Laptops: <https://dogber1.blogspot.com/2009/05/table-of-reverse-engineered-bios.html>

³It's important not to attach anything you're not sure about to your computer. Some public charging stations have a device that can install malware on your phone. Search for it on the web.

5.1 Physical Access

staff can access your data.⁴

→ How are you sure that no hidden camera is installed in your home or hotel room?

→ I can gift you a gaming keyboard for your birthday. It has a feature to record keystrokes! I can use that to find your password!

● No, I don't accept gifts from strangers.

→ How are you sure that none of your friends would do it? How are you sure that the keyboard you bought from the store or online shop hasn't targeted you?!

→ I can invite you to a café that belongs to my friend. I can later watch the camera footage to find your password. Not only that, many cameras are insecure and can be accessed by hackers. This includes your CCTV camera. So you should not type your password where there is a camera. Even your home camera can be accessed by hackers.

→ I can act as a support agent and ask you to install software to help you (mostly remote access software). I can use that software to access your data. If I cannot do anything, I can simply destroy your computer.

You might delete a sensitive file and think it's gone, but it is not! Only the file's pointer in the operating system (OS) is deleted. This means that the OS no longer knows if a file is there or where it is. However, if we scan the disk with special software, we can find and recover the file. It's not that hard. Files have specific structures. For example, opening a file with a text editor reveals its structure. Software can search for these structures and recover them.

So, how can we prevent this? Have you ever tried to cover up something you've erased by scribbling over it with a pencil, writing random things to hide what was originally there? It's similar in the world of computers. When you understand how hardware writes data to a hard drive, you can create algorithms that repeatedly write and erase data in the areas where files have been deleted, making it difficult to recover the original file!

Yes, researchers have developed algorithms and software to securely delete files. Perhaps one day you will be one of them or someone who gives recommendations for secure deletion of files.

● How should we overwrite it?

→ I don't have enough knowledge (nor do many people who falsely claim they do). In some old devices, a single pass of zeros is not enough. See section 2. *Methods of Recovery for Data stored on Magnetic Media* from [34]. However, we read from section 2.4 *Trends in Sanitization* from *NIST SP 800-88 Rev. 1*:

⁴See also: When in China, don't leave your laptop alone <https://www.infoworld.com/article/2279646/data-security-when-in-china-don-t-leave-your-laptop-alone-2.html>

“ For storage devices containing *magnetic media*, a single overwrite pass with a fixed pattern such as binary zeros typically hinders recovery of data even if state-of-the-art laboratory techniques are applied to attempt to retrieve the data.

— NIST SP 800-88 Rev. 1

See also *Can data be recovered from a zero-filled hard drive?*⁵, which mentions quotes from different sources. However, the only way to be sure is to destroy the hard drive into really small pieces. Hard disks are not expensive. If your data is worth ten thousand dollars, you are better off spending twenty dollars to buy a new hard drive than living with the risk of someone recovering your data.

5.1.1 Cold Boot Attack

The code is: Your operating system has several features to prevent one process from accessing the memory of another process. Normally, I cannot access the memory of another process to steal its data. However, if I have physical access to your computer, I can bypass these protections.

DRAM (Dynamic Random Access Memory) is a type of memory that stores data in capacitors. These capacitors can hold a charge for a short time (several seconds [35]) after power is removed. If I cool the RAM chips using liquid nitrogen, I can extend the time that the data is stored in the capacitors. This means that, using another computer, I can dump the contents of RAM into a file and analyze it to find sensitive data, such as encryption keys or passwords.

Removing the RAM chips from the computer is not necessary. I can simply turn off the computer and then quickly turn it back on and boot it into a live operating system. Since the RAM contents are not lost, I can access them.

- We know that encryption keys are short and random. How can those keys, which are stored in RAM (not in the TPM⁶), be found?

→ You said it yourself! They are random and therefore have high entropy. I can search for high-entropy data in the RAM dump or data with certain properties (as public keys have a specific structure). You will see they become distinguishable from other data.

Of course there are several ways to prevent this attack. See section *Effective countermeasures* from Wikipedia⁷

⁵https://tinyapps.org/docs/recovering_data_from_zero_filled_hard_drive.html

⁶Assuming the encryption key won't be loaded into RAM

⁷https://en.wikipedia.org/wiki/Cold_boot_attack#Effective_countermeasures



Figure 5.1: High entropy random data is distinguishable from low entropy random data. src: By Nicko van Someren - Nicko van Someren, CC0, Wikipedia

5.2 What to do?

- Set a password for your BIOS.
- Use secure boot.
- Encrypt your SSD/hard drive.
- Use a strong password.
- Don't type your password where there is a camera.
- Disable your computer ports.
- If you have a CCTV camera, be careful because many of them use weak protocols and can be hacked.
- Don't let anybody access your computer, even for one second. (They may be forced to do bad things.)
- Use tamper-evident technologies. ⁸
- Don't perform encryption or decryption of sensitive data in public (because of side-channel attacks. We will talk about this later.)
- Some security mechanisms are implemented in hardware. For example, AES-NI is a set of instructions that can be used for encryption and decryption. This prevents some side-channel attacks.
- Many employees are careless. They may throw away sensitive data (including passwords) in the trash. Some hackers search the trash to find sensitive data. Always shred sensitive data.

5.3 Hardware Attacks

⁸See also: https://en.wikipedia.org/wiki/Tamper-evident_technology

5.3.1 A simple Backdoor with Mux

Let's say we have a CPU that executes instructions one by one. We can add a covert multiplexer (mux) before the instruction decoder. The mux chooses between the normal instruction and a backdoor instruction based on a secret key. If the key is provided, the mux selects the backdoor instruction, which can perform malicious actions. Otherwise, it selects the normal instruction. The key can be anything (e.g., a specific sequence of bits in a certain register or a specific, rare sequence of inputs). This way, the backdoor remains hidden until the key is used.

5.3.2 Rowhammer Attack

First read its Wikipedia page⁹.

Then read the paper *Flipping Bits in Memory Without Accessing Them: An Experimental Study of DRAM Disturbance Errors* [36].

Now only read the section "Data-Dependent Bit Flips" from *RAMBleed: Reading Bits in Memory Without Accessing Them*. [37]

Now think how we can use the property of "Data-Dependent Bit Flips" to read one process's data from another process's memory. The problem is answered in their presentation.¹⁰ Such a clever attack!

5.3.3 Fault Attack

Read section 5.1 from paper *Differential Fault Analysis of Secret Key Cryptosystems*. [38]

5.3.4 Side-channel Attacks

Let's start with a real-world example.

Have you ever seen in movies when someone tries to open a safe by placing a stethoscope on it? They listen for the sounds of the tumblers inside the safe to determine when the correct combination is found. This is similar to how a door lock works.

A door lock consists of several pins attached to springs. When the key is inserted, the pins move up and down so that their cut sections align with the rotating part of the lock. When the key is turned (because the pins are positioned correctly), the lock opens.

When they place the stethoscope on the safe, they are essentially listening to see which pins move up or down and where they need to be positioned. When a pin is correctly aligned, it makes a tiny "click" sound, indicating that they've

⁹https://en.wikipedia.org/wiki/Row_hammer

¹⁰"RAMBleed: Reading Bits in Memory without Accessing Them" from "IEEE Symposium on Security and Privacy" Youtube channel: https://youtu.be/Y5iZJ12_HK4

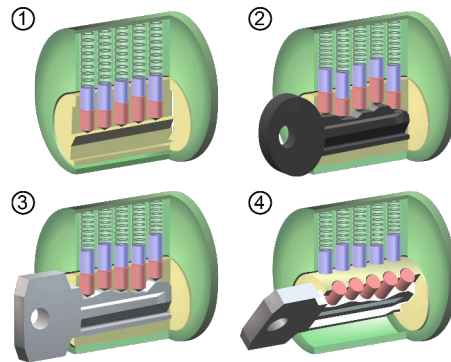


Figure 5.2: How a lock works

found the right position. They listen to each pin one by one, adjusting them until all are in the correct position.

Now, imagine the safe's combination is four digits. For each digit, they test numbers from 0 to 9 to see which one produces that specific sound. Once they hear the sound for the first digit, they move on to find the next three.

Instead of randomly guessing the combination, which would require 10,000 attempts (from 0000 to 9999), they can use the stethoscope to listen for the clicks. They use their knowledge of how the safe works and analyze the sounds to extract information from the system.

Now, let's translate this into the computer world:

The most fundamental aspect of a computer is its hardware. Everything runs on hardware. If the hardware is not secure, it's like having a poorly built foundation for your house; eventually, it will collapse, and your data could be compromised due to hardware vulnerabilities.

Do you agree that hardware consists of electronic components? Do you also agree that electronic components emit noise and sound when they operate and electricity passes through them?

I can use special devices to capture these noises and sounds and identify patterns. For example, I might say that if the sound is like this, it means the CPU is doing a certain operation. If the sound is like that, it means the CPU is doing a different operation. By analyzing the sounds, I can determine what the CPU is doing and what data it's processing. For instance, I could potentially discover someone's password as they type it in! [2]

Or, for example, by analyzing the amount of energy consumed by the CPU and the differences in energy usage for various tasks and inputs provided to the system, one could gain insights into what's happening inside!

These types of attacks are known as *side-channel attacks*. Forms of side-channel attacks include:

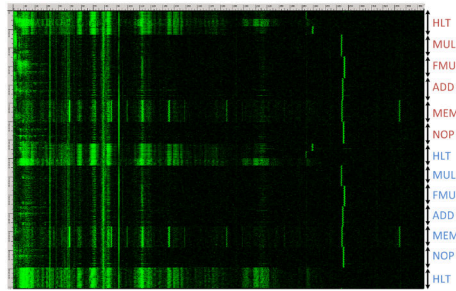


Figure 5.3: “Acoustic measurement frequency spectrogram of a recording of different CPU operations. The horizontal axis is frequency (0-310 kHz), the vertical axis is time (3.7 sec), and intensity is proportional to the instantaneous energy in that frequency band.” Image and description from [2]

- **Timing attacks:** These attacks exploit the time it takes for a system to respond to different inputs. By measuring how long it takes for a system to perform operations, an attacker can infer information about the data being processed.
- **Power analysis attacks:** These attacks analyze the power consumption of a device during cryptographic operations. By observing variations in power usage, attackers can extract secret keys or sensitive information.
- **Electromagnetic attacks:** These attacks capture electromagnetic emissions from devices to gather information about their internal operations. This can include data being processed or cryptographic keys.
- **Acoustic attacks:** These attacks use sound waves emitted by devices to extract information. For example, the sounds produced by a keyboard can be analyzed to determine which keys were pressed.
- **Optical attacks:** These attacks involve capturing light emissions from devices, such as LEDs or screens, to gather information about their state or the data being displayed.

What does a security expert do? They try to prevent these attacks. For example, they might use special materials to shield the device from electromagnetic emissions or design the hardware in a way that minimizes power variations during operations, making it harder for attackers to extract information.

You may have heard of the *TPM (Trusted Platform Module)*, which is required for Windows 11. This chip is designed to prevent certain physical attacks. Secret keys are stored in this chip, which is specifically designed to resist tampering.

5.4 Career Opportunities

Cache Side Channel Attacks

See “Cache Side Channel Attack: Exploitability and Countermeasures” presentation on “Black hat” Youtube channel.¹¹

Spectre and Meltdown

Spectre and Meltdown are two famous vulnerabilities that leverage side-channel attacks in modern CPUs by exploiting microarchitectural state leakage.

First watch minute 6:57 to 12:58 from “Spectre Attacks Exploiting Speculative Execution” presentation on “IEEE Symposium on Security and Privacy” Youtube channel.¹²

Then you may read its paper. [39]

Then read the paper *NetSpectre: Read Arbitrary Memory over Network*. [40]

For Meltdown, read their presentation “Meltdown: Basics, Details, Consequences” on “Blackhat” Youtube channel.¹³

5.4 Career Opportunities

A security professional might evaluate the server room’s physical safeguards by testing lock-picking techniques or employing social-engineering methods to convince staff to grant access.

For example, you can design hardware in such a way that extracting information is not possible.

- What is needed for this part?

→ Understanding hardware! If you don’t know how computer hardware works, how the components interact, and how information is exchanged, how can you understand its security? So, the first step is to gain knowledge and understanding. (You need to comprehend how the CPU operates, how data is transferred between the CPU and other components, the protocols for data transfer, and how to bypass currently implemented security measures and identify potential data leaks.)

There are no shortcuts to becoming a cybersecurity expert. Anyone who tells you to just learn this or that to become a hacker is misleading you! Security requires an understanding of:

1. How a system works.
2. How to make it malfunction or how to extract information from a functioning system.

¹¹<https://www.youtube.com/watch?v=DkWqLDSqHm8>

¹²<https://www.youtube.com/watch?v=zOvBHxMjNls>

¹³<https://www.youtube.com/watch?v=UTSJf05pw-0>



Figure 5.4: Physical security

3. How to fix the problem once you identify it.

You need to broaden your perspective. Hacking Wi-Fi or Instagram accounts won't get you anywhere. Right now, you might be young and seeking excitement, thinking, "Wow, I did this cool thing!" But it's not impressive! These activities won't provide you with a sustainable livelihood. You need to find a real job—one that will support you when you're forty years old. At that point, no one will be covering your expenses; you'll have to take care of yourself! Hacking Instagram or Wi-Fi is trivial, illegal, and has no future. Instead, focus on gaining the knowledge required to become a security engineer—an ethical and well-respected profession.

So, videos that claim to teach you how to hack Instagram or have titles like "Become a Hacker" or "Let me teach you how to hack a website" are likely just offering a few exciting tidbits that won't lead to a real career. Instead, take a structured approach and read the right books. Not every book on security is good, especially those in Persian that may have poor translations or are just poorly copied versions of foreign texts without proper explanations.

Yes, the path to becoming a security engineer is long, and you won't get there in two or three months. You always need to stay updated. If you don't keep up, those who do will take your place. You don't need artificial intelligence to replace you; people with more knowledge will take your spot.

5.5 End of Chapter Questions

Problem 5.1 Why we prefer disk-encryption over file-encryption?

5.6 End of the Chapter Solutions

Solution 5.1 Although file encryption is useful and is good to do so, it is not enough. Because you can't be sure about temp files, swap files and other important hidden files.

5.6 End of the Chapter Solutions



5.6 End of the Chapter Solutions

6.0 Operating System (OS) and Software Security

6.1 What is an Operating System?

Think of a big restaurant. You have many robots that are chefs. If two robots try to access the refrigerator at the same time, they will collide. So you need a manager to coordinate them—deciding when to give access to the refrigerator to one and when to give it to another. This is the job of the operating system.

You have hardware. Programs need to access the hardware. But if two programs want to access the hardware at the same time, what should you do? When should you give the CPU to one and when to another? This is the job of the operating system.

Operating systems manage the memory (RAM). They control which part of the memory should be used by which program. They free the memory when a program is closed.

The OS provides an API (Application Programming Interface) to programs. This means that programs don't need to know how to access the hardware; they just need to call the OS functions. The OS will take care of the rest. It also helps to make programs portable, so they don't need to know the hardware details. In addition to functionality, it also provides security. It doesn't allow programs to access the hardware directly. Accessing the hardware directly is dangerous. Programs might inadvertently or intentionally harm the hardware. They might access the data of other programs (which may be sensitive).

6.2 OS security

The operating system is the core of a computer's functionality, so its security is very important.

I am not going to talk about the security of the operating system itself. The topic is very broad, and I don't have enough knowledge to discuss it in detail. Instead, I am going to talk about the security of the operating system from the

6.3 What OS to use?

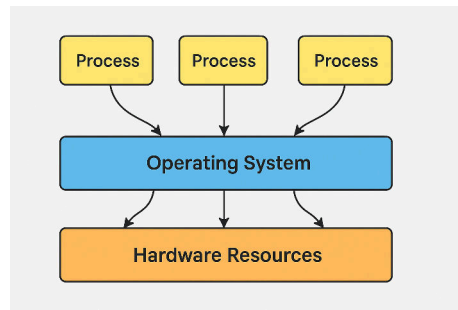


Figure 6.1: How OS manages hardware

user's perspective, or features and programs that can help you secure your data related to the operating system.

There are many operating systems. The most famous ones are Windows, macOS, GNU/Linux, BSD, Android, iOS, etc. Each of them has its own security features.

If you want to improve your security mindset, you have to think like an attacker. Now think of yourself as an attacker. Would you write a malicious program for Windows or for GNU/Linux? Would you rather target hundreds of millions of users or a few million users? You would probably choose Windows because it has more users. Also, many Windows users are average people who don't know much about security (if they know at all!). So it is more likely that you can find a victim.

This is one of the reasons people mostly say Windows is insecure. I don't have enough knowledge to talk about the security of the operating system itself, and neither do most people who blindly say Windows is insecure. They just say it because they have heard it from others. Just ask them a simple question like "how Address Space Layout Randomization (ASLR) works", and you will see that they can't explain it in depth. Only take advice from those who are truly experts in the field.

Windows **home users** are the most targeted for **common attacks**. This doesn't mean GNU/Linux is secure; it means that it is less targeted for **common attacks**.

6.3 What OS to use?

Before that, let's talk about updating programs. Many people ask, "I updated WhatsApp, but nothing changed. Why should I update?"

Think of it this way: you have a house. The person who built it regularly comes and checks the house and fixes any problems. Many times, you don't see the issues or changes, but the builder does and fixes them. It's the same

with programs. The developers regularly check and fix problems in the software. Often, you don't notice the changes, but the developers do and resolve them. So, you should update your programs regularly. Updates may also bring new features, like changing the color of the main page of a program. This is something you will notice.

- I am afraid that the new version has a bug and doesn't work properly.

- It is possible. I'm not saying it can't happen. But it is less likely because the developers have tested the program before releasing it. Also, if a bug is found, they will fix it in the next version. But if you don't update, you will be vulnerable to bugs that have already been fixed, like a thief who knows you have a hole in your house and can enter it.

Some people say to use Windows 7 because it has received many updates over the years. But they don't consider new bugs! What if a new bug is found? It won't be fixed. You will remain vulnerable forever.

Have separate accounts for your work and personal life. Don't use the same account for both. Some people use just one account (an admin account) for everything. This is dangerous. If a program is infected, it can access all your data.

Don't give admin accounts to employees. Employees should not use company computers for personal tasks. Even a simple, quick payment of bills may become a security problem for the company.

6.4 Malware: The Most Common Threat

In the beginning, because viruses were the first type of malware, the term "virus" became very well-known. This led to the creation of antivirus software. However, it's more accurate to use the term "malware," as viruses are actually a specific type of malware. Malware is a general term that refers to "any software intentionally designed to cause disruption to a computer, server, client, or computer network, leak private information, gain unauthorized access to information or systems, deprive access to information, or which unknowingly interferes with the user's computer security and privacy."¹

Why is knowing about malware important? Because it is the most common way of harming a computer and stealing data (e.g., banking information).

Furthermore, certain countries have been affected by malware more than you think. Every year, Kaspersky releases statistics on the users of its antivirus software, and each year, Iran ranks as the country most affected by mobile

¹From Wikipedia Creative Commons Attribution-ShareAlike 4.0 License

6.4 Malware: The Most Common Threat

malware! 2017², 2018³, 2019⁴, 2020⁵, 2021⁶, 2022(3rd)⁷

This is due to several reasons (I will explain more in the future):

- The use of insecure marketplaces
- Downloading apps from unofficial sources, such as Telegram channels
- Value-Added Services (VAS) that are designed to infect users' devices

NOTE

When I mention the names of different malware, I do not want you to try to memorize them. My main goal is for you to understand their functions so you can recognize what might threaten you. Memorizing names is not useful; we need to know how they operate. What is important is to identify that malware can do this, that, and other things, which means you need to be cautious in these areas. Additionally, most modern malware is a combination of several types, such as being a keylogger, ransomware, and spyware all at once. Names are more relevant for those who want to delve deeper into research or whose work is specifically related to this field.

6.4.1 Virus

A virus is a type of malware that can replicate itself and spread to other computers. It can attach itself to a program or file, and when that program or file is executed, the virus is activated. It can spread through email attachments, infected files, or even USB drives.

A prime example of a virus is the *Stuxnet* virus. It was designed to target Iran's nuclear program. It spread through USB drives and infected the systems that were connected to them.

Stuxnet is a sophisticated computer virus discovered in 2010, designed to target Iran's nuclear facilities, particularly the centrifuges at Natanz. Developed by the U.S. and Israel, it was based on footage taken during a past visit to the site. The virus did not operate in any system except the one it was designed for. I highly recommend reading Langer's analysis⁸.

The lesson we can learn from Stuxnet is that when dealing with sensitive systems, no information about them should be disclosed. In the case of Stuxnet, even background noise could reveal the rotation speed of the motors. Any

²<https://securelist.com/mobile-malware-review-2017/84139/>

³<https://securelist.com/mobile-malware-evolution-2018/89689/>

⁴<https://securelist.com/mobile-malware-evolution-2019/96280/>

⁵<https://securelist.com/mobile-malware-evolution-2020/101029/>

⁶<https://securelist.com/mobile-malware-evolution-2021/105876/>

⁷<https://securelist.com/mobile-threat-report-2022/108844/>

⁸<https://www.langner.com/stuxnet/>

After Stuxnet, there was significant media frenzy. Amidst this, scammers attempted to gain positions and promotions by making false claims. For instance, the news about the discovery of the Stars virus was one such example, which is claimed by Webamooz to be a scam⁹.

6.4.2 Ransomware

Background (Encryption)

For instance, take a look at the image below:

[illegible]

Figure 6.2: Binary data behind a picture

- ⁹<https://t.me/webamoozir/4286>, webamooz.com/?p=20294

6.4 Malware: The Most Common Threat

numbers and understands how to display the image accordingly.

Instead of opening the image in a hexadecimal viewer, I can open it in a text editor. By doing this, the computer tries to read the image as text. Normally, double-clicking opens the image, but I can right-click and choose to open it with Notepad. You probably won't see anything meaningful, because that is the information for the pixels.

But what does this have to do with encryption? Think about it! If I change this text, the computer will not be able to understand what the original image was anymore because the data has been altered. For example, instead of each letter, I can write the next letter in the alphabet. So, A becomes B, B becomes C, and so on. Now, when I open the image, it will be completely different! This is the basis of encryption. It's like a secret code that only you and the computer know. If someone else tries to read it, they won't understand anything!

That's why the altered files won't open. This is exactly the technique that ransomware uses. Ransomware is a type of malware that enters your computer and changes the data behind your files. Essentially, it encrypts them, making your files inaccessible! Then it demands, for example, \$300 to give you the key to unlock your files. For instance, they might display messages like the one below:

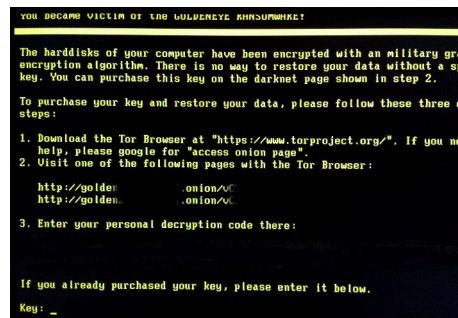


Figure 6.3: src: BlueBreezeWiki, Goldeneye-ransomware-161212, CC BY-SA 3.0

Or, for example, they might not show a message; instead, they may place a TXT file next to your files. When you open it with Notepad, you can read the text inside.

Sometimes, they lie and claim to be the police or the FBI to scare you into paying quickly. They may say that your computer has been locked due to illegal activities or illegal downloads, and that you need to pay a fine:

Or sometimes, they say your files are locked because you have not paid for licensing.

Sometimes, they do not encrypt files immediately after entering your computer. Instead, they first steal your files and then threaten to publish them if

6.4 Malware: The Most Common Threat

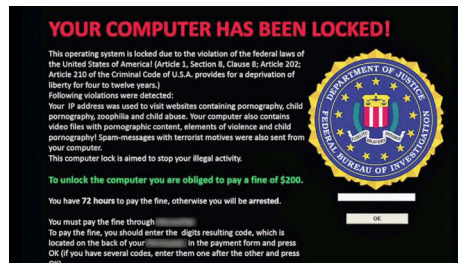


Figure 6.4: src: Motormille2, Ransomware-pic, CC BY-SA 4.0

you do not pay.

But it does not stop there! They also set a deadline for you. For example, they might say that if you do not pay within a week, all your data will be deleted or leaked. Many companies have faced situations where an attacker demanded money not to publish sensitive data.

- Should I pay to get my files back?
- Absolutely not!

How can you be sure that after paying, the attacker will actually fulfill their promise and provide you with the decryption key and tools? How do you know that someone who has already encrypted your files and demanded money will not use that decryption tool to install spyware on your system?¹⁰ What if they say, "Well, I have also planted malware, so you need to pay for that too!" Would you trust someone who is an attacker and has already harmed you?

Moreover, this leads to the proliferation of ransomware. If people pay, attackers will continue to use this method. If people stop paying, attackers will stop using this method. Why? Because they see that they can make money this way, so they are more likely to continue. If they knew that no one would pay the ransom, why would they waste their time creating ransomware when it would not generate any income?

This is serious. We have something called *ransomware as a service*. Some will create ransomware and sell it to others so they can infect computers. They will take a percentage of the ransom. This is a serious problem. It is like a business.

The best way to counter ransomware is to have backups.

¹⁰Sometimes, the decryption tool they provide is actually just another piece of malware! You cannot trust someone who is an extortionist!

That is why companies that fall victim to ransomware, even if they manage to obtain a decryption tool, first test it on a specific computer to ensure it is not malware. They simulate the environment on a series of computers that resemble normal, real systems, rather than using an isolated computer. This is because some types of malware check for certain system parameters (like system sensors and files) to see if they are running in an isolated environment. If they detect that they are in an isolated environment, they may not perform any harmful actions to avoid detection.

- Well, I have a backup! So they can do nothing!

- First, your backup needs to be reliable. It should not be the case that you just say, "I have some copies of my files on my hard drive, and I call that a backup!" because that could also be infected! Second, this is not enough. The attacker can still threaten to publish your data.

Your sensitive data should be encrypted. So if they steal it, they cannot read it. **Some Tips About Backups**

- Any digital device might stop working at any moment, so backups must be regular. Backing up once a month is not enough.
- Data should be encrypted before being backed up.
- Backups should be stored in different locations. For example, keep one at home and one at your office. If your home burns down, you can still access your data.
- Backups should be in different formats. For example, keep one on a hard drive and one in cloud storage.
- If you run a business, you should be able to recover your data quickly. For example, if your data is deleted, you should be able to restore it within a few hours. (Backup power lines should also be different. This way, if one area loses power, another area will still have electricity.)

What to do if you're infected with ransomware? Do NOT trust random decryption tools. Many videos on YouTube recommend downloading a tool and running it to get your files back, but many of them are actually malware! Look for recommendations from security experts and antivirus companies.

*No More Ransom*¹¹ is an initiative that brings together law enforcement agencies and cybersecurity companies to assist individuals and organizations affected by ransomware attacks. By providing valuable resources, including decryption tools and guidance on how to respond to such incidents, No More Ransom aims to empower victims to recover their data without paying ransoms.

- How can they decrypt the data?

- Many ransomware programs use poor cryptographic implementations. This means the encryption is not strong enough and can be broken. Some use weak keys. Some do not clear encryption keys from memory, so the key can be recovered. This is why we sometimes suggest not turning off your computer. If you turn it off, the key will be cleared from memory (since RAM is volatile and loses its data when the power is off).

¹¹<https://www.nomoreransom.org/>

A Message to Those Who Create Ransomware

If you are a ransomware creator, I want to talk to you. You may think that you are doing a good job. You may think that you are making money. But you are not! You are harming people.

Ransomware can destroy files that represent years of hard work and dedication for individuals and businesses. This loss can have devastating consequences, not just financially but also emotionally, as people may lose irreplaceable memories, important documents, or critical data. Additionally, in sectors like health-care, ransomware attacks can have life-threatening consequences. For example, if a hospital's systems are locked, patients may not receive the care they need, and lives could be lost.

Crimes like creating ransomware leave a lasting mark in law enforcement databases. No matter how clever you think you are at hiding your tracks, the reality is that law enforcement agencies are continuously improving their methods and technologies to track down cybercriminals. Eventually, the evidence will catch up with you, and you will be held accountable for your actions.

Even if you use cryptocurrencies and Tor, it is still possible for law enforcement to track you down. Just as there are search engines like Google, there are also powerful tools specifically designed to locate cybercriminals. Organizations and individuals dedicated to tracking down cybercriminals—beyond just law enforcement—have financial incentives to do so. When they successfully monitor and identify you, they often receive increased funding for their efforts. They can analyze even the smallest clues to trace your activities and ultimately identify you. Engaging in cybercrime is a risky endeavor, and the chances of being caught are higher than you might think. You have to live every day in fear of being caught. The golden age of cybercrime is over. Right now, you might be around 18 or 19 years old and feel invincible, not fully understanding the consequences of your actions. But when you reach 40, you'll realize the impact of those choices and how they can affect your life in ways you never anticipated. I will show you how people are caught in the future. (See Section Tor)

6.4.3 Trojan

One of the most legendary wars involving the Greeks was the siege of the city of Troy, which lasted for ten years. Eventually, the Greeks devised a clever plan: they built a massive wooden horse and hid a few soldiers inside it. The rest of the army pretended to retreat. The people of Troy, believing they had won, thought the giant wooden horse was a gift from the Greeks and brought it into their city. Later that night, the soldiers hidden inside the horse quietly emerged and opened the gates, allowing the Greek army to enter and conquer Troy.

In the world of computers, a Trojan horse operates in a similar way. It often appears to be a legitimate program, tricking users into thinking it is safe. For example, it might look exactly like Firefox, but it secretly performs malicious actions in the background. Essentially, a program is created that mimics



Figure 6.5: Trojan horse (AI)

Firefox, but with additional harmful code embedded within it, allowing it to carry out destructive tasks without the user's knowledge. That is why we advise downloading programs from official sources. If a program has a checksum (which we discussed in the hash section), make sure to verify it to ensure that the file has not been tampered with.

Reading List

- GhostBuster-Detecting arbitrary persistent and stealthy software¹²
- Forged Memory-A scary development in rootkits¹³

6.4.4 Keystroke Logger (Keylogger)

Imagine there's a program that records everything you type and sends it to a hacker. This poses significant risks, especially when you're entering sensitive information. For instance, if you type your password, that password gets recorded and sent to the attacker. This is extremely dangerous! It could even capture your bank password, leading to serious financial consequences. These types of programs are known as keyloggers.

Keyloggers primarily target physical keyboards, which is why, during online payments, websites connected to payment gateways often provide a virtual keyboard with random numbers. They recommend using this virtual keyboard because if you have a keylogger monitoring your physical keyboard, it won't be able to capture what you enter on the virtual keyboard. (This doesn't mean that virtual keyboards are 100% secure; they can also be monitored by malware, but it's less likely.)

Don't use keyboard software that you're not sure about, as it can be a keylogger. For example, if you're using software that changes the color of your keyboard, it might be a keylogger.

¹²<https://www.schneier.com/blog/archives/2005/02/ghostbuster.html>

¹³https://www.schneier.com/blog/archives/2011/05/forged_memory.html

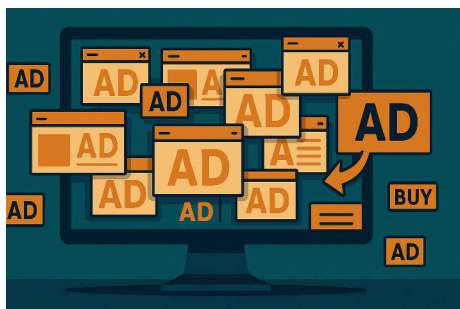


Figure 6.6: Adware (AI)

6.4.5 Adware

Adware is a type of malware that displays unwanted advertisements on your computer. It's like a person who follows you around all day, constantly showing you ads. It's very annoying! Adware can also slow down your computer, making it difficult to work efficiently. It can consume a lot of your internet bandwidth, which can be a problem if you have a limited data plan.

Sometimes ads are fake, and if you click on them, you might be redirected to a malicious website.

How can you know you're infected? Seeing ads that you didn't see before is a sign of adware. Also, if you notice a lot of pop-up windows or changes in your browser, like a new homepage or a new search engine, that could indicate an infection.

6.4.6 Spyware

Spyware can capture a wide range of personal information, not just photos or screenshots of your chats. It can track your location, record your conversations through the microphone, take pictures using your front camera, and invade your privacy in many ways. Any activity that violates your personal space and intrudes into your private matters can be considered spying.

Attention! Having access to certain features does not necessarily mean that an app is spying on you. For example, a photography app obviously needs access to the camera; without that permission, it would not be able to take photos at all.

However, if a calendar app requests access to your camera, it raises a red flag. Why would a calendar app need camera access? This is why we need to be cautious about granting unnecessary permissions to applications.¹⁴

Developers should clearly explain in their app descriptions why they need

¹⁴Doctor Web: Android Trojan controlled via Telegram spies on Iranian users: <https://news.drweb.com/show/?i=11331>



Figure 6.7: Spyware (AI)

specific permissions. For instance, if an app requires camera access for scanning QR codes, it should state that explicitly. Transparency about permissions helps users make informed decisions and protects their privacy.

You might think that since an app is reputable, it is safe to grant it all permissions. However, even well-known companies have violated people's privacy in the past. This brings us back to the principle of "my data, my ownership." Your data belongs to you, and there is no reason to give it away unnecessarily.

For example, if you do not plan to scan a QR code, why should you grant camera access? Allowing access just for the sake of it means that the camera could sit idle, potentially being misused later.

6.4.7 Botnet

Sometimes, you might become infected with malware that effectively turns your computer into a "zombie."

What does that mean? Think of a zombie as a creature that lacks independent thought and is controlled by someone else. It simply follows commands without any reasoning. Similarly, when your computer is infected with malware, it can be manipulated to perform actions you do not want it to.

On a more technical note, the servers that the malware connects to for receiving commands and reporting back to the hacker are known as "Command & Control Servers (C&C)." Typically, the victim's computer connects to these servers and informs them that it is compromised, asking, "What should I do now?" The server then sends back instructions, telling the infected computer what actions to take.

So, what characteristics should these servers have? When a program con-

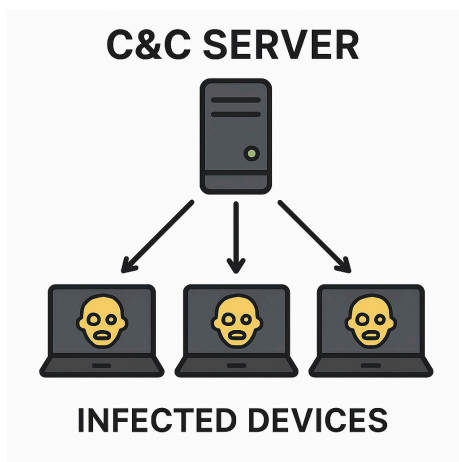


Figure 6.8: C&C Server and Botnet (AI)

nects to a server, monitoring tools can track which website it connects to. This poses a significant risk for malware developers. Just as there are thieves, there are also police, along with malware analysts, security researchers, and law enforcement agencies. These security researchers analyze network packets to find ways to stop malware and trace it back to its original creator.

If a malware developer directly embeds their personal server address in the code or connects to their own server, it becomes easy for authorities to track them down. They could be questioned about why commands are being sent from their server to infected computers, instructing them to perform certain actions.

Additionally, antivirus software will block that server, and services like Google Safe Browsing or DNS providers may also prevent access to it. This is where malware developers employ various techniques to obscure their server's identity and avoid detection. They might use a series of *compromised* servers, known as a "botnet," to relay commands. This way, even if one server is blocked, the malware can still receive commands from another server in the botnet.

If the C&C server always has a fixed address, it would quickly be detected and blocked by antivirus software and web browser protections. Therefore, the C&C address needs to change frequently. However, if the malware developer includes a list of addresses in the code, those would also be discovered and blocked. So, the address must be dynamic and constantly updated. But where do they get this changing address from?

One approach is to create a specific website where the current C&C server address is posted and regularly updated. The malware would then only need to include this single website address in its code, and it would go to that site to retrieve the latest C&C server address.

However, you might wonder how this is different from directly embedding

6.4 Malware: The Most Common Threat

the C&C server address in the code. In essence, it's not much different, as that website could also be blocked easily.

This is where malware developers employ a clever technique. They might place the address of the website in the comments of a popular post, like one about Britney Spears. The malware would then search through the comments to find the C&C server address.

But wait! It's not that simple! You might ask how the malware knows which address to extract. This is typically done using a predefined format or pattern that the malware is programmed to recognize. For example, the malware could be designed to look for specific keywords or structures in the comments that indicate the presence of the C&C server address. This way, even if the address changes frequently, the malware can still locate it by searching for the right cues in the comments.

In this method, the malware doesn't directly place the address in the comments (because the page admin would likely delete it). Instead, it hides the address within a text that, when a specific hash (not a cryptographic hash, but think of it more like a hash table) is calculated, results in the value 183.

For example, the comment text might be:

```
#2hot make loved to her, uupss #Hot #X
```

The custom hash for this text is 183.

The victim's computer will then go through the comments under the Britney Spears post and calculate the hash for each one (using that specific hash function). If the hash equals 183, the comment is found. Now, how do we convert this text into a link?

This is where regular expressions (Regex) come into play. To briefly explain, Regex is used to search for a string within another string. For example, if we write:

```
gr(e|a)y
```

We're looking for strings that start with "gr," followed by either "e" or "a," and ending with "y." This would find both "grey" and "gray."

The Regex used here is:

```
(?:\u200d(?:#|)(\w))
```

This searches for the Unicode character `u200d` (known as a "zero-width joiner," which is used to separate special characters without creating a space) or the characters `@` or `#`. After that, it looks for a character that is part of the set `[A-Za-z0-9_]`.

Now, let's see how this works with the comment text:

```
#2hot make loved to her, uupss #Hot #X
```

In the computer, it appears as:

```
#2hot ma<200d>ke lovei<200d>d to <200d>her, <200d>uupss  
<200d>#Hot <200d>#X
```

The malware is supposed to look for the characters u200d, @, or #. So, when it searches through the text:

```
#2hot ma<200d>ke lovei<200d>d to <200d>her, <200d>uupss  
<200d>#Hot <200d>#X
```

It finds the u200d characters and takes the letters that follow them. This results in:

```
2kdhuHX
```

The malware also initially adds the following text on the victim's computer:

```
http://bit.ly/
```

Now, let's see the complete link:

```
http://bit.ly/2kdhuHX
```

Congratulations! Your link has been created! This is how they cleverly obfuscate and hide information. In essence, obfuscation is the practice of making something unclear or difficult to understand. In this case, they are hiding the C&C server address in a way that is not immediately obvious. We can also obfuscate our code to make it harder for others to understand what it does.

- Russian APT Groups Continue Their Stealthy Operations¹⁵
- Satellite Turla: APT Command and Control in the Sky¹⁶
- Using Google Docs as a C2 proxy with a headless browser¹⁷

¹⁵<https://www.infosecinstitute.com/resources/threat-intelligence/russian-apt-groups-continue-stealthy-operations/>

¹⁶<https://securelist.com/satellite-turla-apt-command-and-control-in-the-sky/72081/>

¹⁷https://frereit.de/google_doc2/

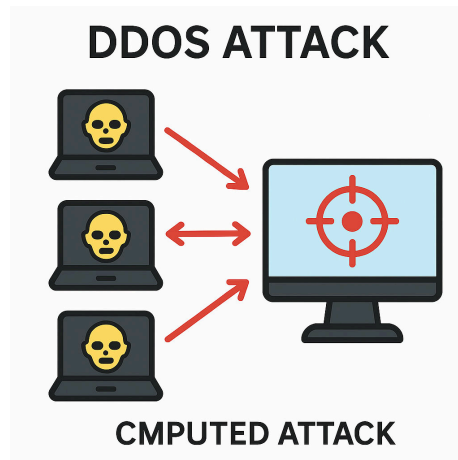


Figure 6.9: DDoS Attack. The attacker might wait for a response from the server.

6.4.8 Denial-of-service attack (DoS) & Distributed Denial-of-service attack (DDoS)

One type of attack that can be launched against a server is a DoS or DDoS attack, which stands for Denial of Service or Distributed Denial of Service. As the name suggests, it aims to deny users access to services.

Imagine you are an employee assisting clients. When someone approaches, you greet them and ask how you can help. Now, if someone wanted to disrupt your performance, they could send a thousand people to surround you, and each one would just say hello and shake your hand before leaving. This would create such a crowd that you wouldn't be able to attend to your actual clients.

In a DDoS attack, a similar scenario occurs. When a computer communicates with HTTPS websites, it first sends a request to the server, essentially saying, "Hello, are you there?" The server responds, "Yes, I am here." This initial exchange is known as a handshake, much like a greeting.

Now, imagine if millions of "hello" requests are sent to the server. The server has to respond to each one, saying, "Yes, I am here." This flood of requests overwhelms the server, leaving it too busy to handle legitimate tasks. As a result, the website becomes extremely slow and may even become inaccessible. This means that legitimate customers cannot access the service, and you might also struggle to maintain a connection with your server. This is why websites often slow down during sales or promotions: many people (in this case, legitimate users) are trying to access the site at the same time.

DDoSing is a very harmful act and is considered a crime. Engaging in such activities can lead to legal consequences, including being taken to court and facing lawsuits. It's important to understand that this behavior is not only un-

6.4 Malware: The Most Common Threat



Figure 6.10: src: This work is in the public domain in the United States because it is a work prepared by an officer or employee of the United States Government as part of that person's official duties under the terms of Title 17, Chapter 1, Section 105 of the US Code.

ethical but also illegal.

- My friend gave me permission to DDoS their website.

→ The website is hosted on a server, and the server is owned by someone else. So, even if your friend gives you permission, you are still attacking someone else's server. This is illegal!

The FBI has arrested individuals and shut down websites that provide DDoS services and tools under the "Computer Fraud and Abuse Act."

Sometimes, you may receive messages demanding a ransom, threatening to launch a DDoS attack against you if you don't pay, for example, \$500 by tomorrow. They might even say, "To prove we're serious, we'll launch a small DDoS attack at 4 PM today so you know what to expect."

It's crucial to remember: never pay the ransom, whether it's in the case of ransomware or these types of threats. Instead, report the incident to the police. They can provide assistance and help you deal with the situation. Paying the ransom only encourages further criminal behavior and does not guarantee that the attackers will follow through on their promises. Always seek help from law enforcement in these situations.

In fact, this is an underground business where infected computers are bought and sold! For example, the person who controls these botnets, which have infected numerous computers, might say, "Pay me \$20,000, and I'll launch a DDoS attack on that target for you!"

CAPTCHA

With the advancement of technology, robots have been developed to replicate tasks that humans perform and to carry them out on their behalf. However, this has led to a problem: these robots can execute malicious actions, such as brute-force attacks or DDoS attacks. To combat this issue, researchers created

a mechanism known as CAPTCHA.

As the name suggests, CAPTCHA is designed to distinguish between humans and computers. Whenever suspicious activity is detected—such as receiving ten requests in one second or repeated actions that resemble robotic behavior—a CAPTCHA test is implemented to determine whether the request is coming from a human or a bot. Tests like “I am a human” or “I am not a robot” are all forms of CAPTCHA.

The idea is to leverage the differences between human cognition and robotic processing. For instance, humans can read and interpret the characters in a CAPTCHA, while bots struggle to do so. Therefore, these tests are used to validate requests.¹⁸

Now, what happens when your computer is used for attacks like DDoS? Typically, the security systems of websites will notice that your computer is sending requests to many different sites. This indicates suspicious activity, especially if your IP address has been involved in multiple recent attacks (for example, if your IP was part of eight out of ten recent attacks). As a result, they may conclude that your device has become part of a botnet and block your access. This is a clear sign that you may be infected when websites start implementing CAPTCHAs for your requests.

You may have noticed this when using Tor or a VPN. Because these services provide anonymity and many users share the same range of IP addresses, websites may mistakenly think you are a bot, prompting them to present a CAPTCHA. So, whenever you encounter a CAPTCHA, don’t panic.

The story behind the creation of reCAPTCHA is quite fascinating. The developers realized that there are countless books and documents that have not been digitized, and hiring people to type them all out would be prohibitively expensive and time-consuming. Instead of employing typists, reCAPTCHA proposed a clever solution: they would separate words into pairs and present them to people on the internet to type. This way, they could collect the typed results and combine them, effectively digitizing millions of images containing text.

How do they ensure that the typed words are correct? They give the same two words to multiple users—typically around ten—and the correct result is determined by the majority’s input. By aggregating the results from many users, they can accurately reconstruct the text.

Now, Google is using a similar approach with its Google Street View images. They extract elements like postal codes and license plates¹⁹ and present them to users for typing. The typed information is then incorporated into Google Maps. This demonstrates how technology can be leveraged effectively: instead of randomly typing words, millions of books can be transformed into text files, and street maps can be constructed. It’s truly impressive how innovative ideas like these can lead to such significant advancements!

¹⁸See images: <https://commons.wikimedia.org/w/index.php?search=captcha&title=Special%3AMediaSearch&fulltext=Search>

¹⁹See image: <https://techcrunch.com/wp-content/uploads/2012/03/recaptcha-collection.jpg>

How can you bypass CAPTCHA? There are services that solve CAPTCHAs for you. They use AI and machine learning to do so. Not only that, but there are centers where people solve CAPTCHAs for you. Most of them are in countries where the minimum wage is low, like India.

6.4.9 Backdoor

We have discussed it before. So we will not repeat it here.

6.4.10 Fileless malware

Fileless malware is a type of malware that does not rely on files to infect a system. Instead, it resides in the system's memory, making it more difficult to detect and remove. This is because **some** (not all) antivirus prevention mechanisms are based on scanning files, not memory.

That is why the NSA²⁰ and Kaspersky²¹²² recommend restarting the device weekly. Many devices have a built-in feature to do this automatically. When you restart the device, the memory is cleared and the malware is removed.

6.4.11 Coinminer

We know that mining cryptocurrencies requires significant computational power. Some thieves think, "Why should we invest in all this hardware to mine when we can exploit other people's computers to do the mining for us?"

In this scenario, malware is used to hijack your CPU or GPU for mining purposes. Inexperienced and greedy malware developers often consume a large portion of your CPU and GPU resources. This leads to your computer being overworked, causing it to slow down and become less responsive. As the hardware works harder, it generates more heat, which in turn causes the fans to spin faster. You might notice these signs and start to suspect that your computer has been compromised.

However, more sophisticated malware developers take a different approach. Instead of consuming a large amount of processing power, they only use a small fraction—say, 3%—to avoid detection. This way, your computer doesn't overheat, the fans don't run constantly, and you remain unaware of the mining activity. Additionally, they often name the malware something similar to well-known processes, like "Microsoft Service," so that if you open the Task Manager to check which programs are running, you won't easily recognize the malicious software.

²⁰NSA Mobile Device Best Practices: https://media.defense.gov/2021/Sep/16/2002855921/-1/-1/0/MOBILE_DEVICE_BEST_PRACTICES_FINAL_V3%20-%20COPY.PDF

²¹<https://www.kaspersky.com/blog/how-to-protect-from-pegasus-spyware/43453/>

²²<https://arstechnica.com/information-technology/2023/06/clickless-ios-exploits-infect-kaspersky-iphones-with-never-before-seen-malware/>



Figure 6.11: Coinminer

But one important thing to keep in mind is that websites can also engage in this practice. When you visit a website, it might use your computer's processing power to mine cryptocurrencies. This can happen in two ways:

- **Transparent Mining:** The website informs you that they are using your computational power for mining, explaining that it helps them earn money and allows you to support their services. They will ask for your permission and approval.
- **Covert Mining:** In this scenario, the website does not inform you and secretly uses your resources for mining. This is considered malicious behavior and is unethical.

Note: If the website is mining, you might notice in the Task Manager that, for example, Firefox is consuming 90% of the CPU. This is certainly a red flag (though it may not necessarily indicate malware; it could simply be a bug in Firefox itself).

To prevent this kind of resource exploitation, you can take several steps:

- **Use Browsers with Built-in Protection:** Popular browsers like Firefox have features that include a cryptocurrency miner blocker. While this is a good start, it may not be sufficient on its own.
- **Script Blocking:** A more effective method is to use script-blocking extensions like *NoScript*. This extension allows you to control which scripts run on the websites you visit, effectively blocking unwanted mining scripts and other potentially harmful content.²³

²³However, it's important to note that using script-blocking extensions like NoScript may impact your web browsing experience. Some useful elements on websites, such as interactive features or certain content, might also be blocked along with the unwanted scripts. This can lead to a less functional or more cumbersome experience on some sites.

You may need to manually allow specific scripts or features on trusted websites to ensure they work properly. Balancing security and usability is key, so it's essential to be mindful of which sites you choose to allow scripts on while maintaining your online safety.

6.5 What to Do?

Some cracked software also contains coin miners. Surely, the person who cracks the software needs money to live. Why should they work for you for free? Some may include malware in the cracked software.

Other signs of infections may be:

- You cannot access antivirus websites. (Malware does this to prevent you from downloading an antivirus.)
- Unintentional changes (deletion of files, UI changes, etc.)
- More internet charges²⁴
- Battery drain²⁵

6.5 What to Do?

One way that apps generate revenue is by entering into agreements with companies or individuals. They might say, "When someone installs our app, if they also install your app, pay us, for example, 1 cent for each installation." The other party agrees to this arrangement. Therefore, it's important to be cautious when installing apps and to check the boxes you select to avoid installing any additional programs. An app might also violate your privacy, and the more apps you have, the higher the likelihood of encountering bugs. For instance, if you have 200 apps, the chances of a security breach are greater compared to having just 10. It's clear that with 200 apps, each one could potentially have its own vulnerabilities!

²⁴Some malware consumes a lot of internet bandwidth for downloading things or uploading and stealing your data.

²⁵When you become infected with malware, a portion of your hardware is dedicated to the operation of that malware. This is because malware uses your hardware resources to function, much like a car that works harder the more passengers it has. When part of your hardware is occupied by a process, what happens? Naturally, that process requires energy, which means your battery will drain faster than usual. You might suddenly notice that your battery is depleting more quickly after a few days. (Not every instance of battery drain is related to malware! For example, a 5-6 year old phone battery will naturally drain faster than when it was new. In the context of malware, "draining faster" means you notice a sudden decrease in battery life over a short period.)

When hardware is used more intensively, what happens? The device heats up, causing the fan to spin faster to cool it down. Since a portion of your system's capabilities is allocated to the malware, your device's overall performance decreases, making it less able to handle its regular tasks, which can lead to freezing or lagging.

Well, if there were no malware and your phone was still draining battery quickly or overheating, it's worth considering what might be causing this. One possibility is that we know from previous discussions that apps can have issues that are resolved through updates, and no app is perfect. It's possible that the developer didn't optimize the app well for your specific device, leading to poor performance.

If your phone is overheating, it could be due to an app that isn't compatible with your device or an unstable update that doesn't work well with your system. In such cases, you can uninstall the problematic app and wait for a new, improved version to download after it has been updated.

6.5 What to Do?

Don't open or download files sent via email. Also, make sure to disable automatic downloads. Sometimes, just downloading certain files can cause issues. There might be a vulnerability leading to malicious actions. That's why we always recommend staying updated! The sooner vulnerabilities are patched, the sooner the risk of exploitation is eliminated.

In your phone's settings, disable the installation of apps from unknown sources. Also, look for device admin apps. Normally, there should not be any app there. Additionally, check for accessibility software. Many of them can read everything that appears on the screen. Disable those you don't want.

Imagine a hacker trying to hack into your system. To do this, they would typically need to provide you with a malicious link or a harmful file. Since you are aware that you shouldn't click on every link or open every .exe or .apk file, how would they manage to get through your defenses?

Malware developers have come up with a sneaky tactic: they take advantage of the fact that many users have file extensions hidden. They think, "If someone can't see file extensions, we can create a misleading filename that makes it look harmless, like an image." For instance, they might name a file `screenshot.jpg.exe`. If you have file extensions hidden, you would only see the part that looks like an image (the name): `screenshot.jpg` and assume it's safe to open, which could lead you to fall for their trick. That's why it's essential to enable file extension visibility on your device!²⁶

One of the tricks hackers use is to provide you with a legitimate-looking app, but when you try to open it, you receive a message saying that a certain program, like Google Play Services, is outdated. It prompts you to click a button to install the latest version. Be aware that these are actually malware, disguised with fake icons to resemble well-known apps, trying to convince you that they are the real deal. Always make sure to update your apps only from official sources (e.g., Google Play Store).

Virustotal is a wonderful website where you can upload suspicious files for analysis. An important note is to not upload sensitive files, as *Virustotal* sends the data to multiple parties.

If you get infected, search for *Safe Mode*, *emergency kits*, and *second opinion malware scanners*. If you suspect that an app is malicious, it's best to uninstall it while in Safe Mode (not in normal mode). Why Safe Mode? Because third-party apps are disabled in this mode. This means that if you try to uninstall a problematic app in normal mode, it might not be completely removed, or you may not be able to uninstall it at all.

²⁶If you don't know where to enable it, just search for it!



Figure 6.12: A house with many IoT devices. (AI)

6.6 The Dark Side of Connectivity: Security Risks in the Internet of Things

The Internet of Things (IoT) is a network of interconnected devices that communicate with each other. These devices can be anything from smart home devices like smart TVs, smart refrigerators, and smart thermostats to smart cars, smart watches, and even smart toys. The IoT is a double-edged sword. On the one hand, it makes our lives easier and more convenient. On the other hand, it poses a significant security risk. The more devices we connect to the internet, the more vulnerable we become to cyberattacks.

For example, I wake up in the morning, and with a click on my phone, it tells my water heater to start because I want to take a shower. When I'm done, it automatically sends a message to my oven to preheat for breakfast. When I start my car, it notifies my smart door to unlock because I'm heading out. After I leave, the door informs my smart vacuum cleaner to start cleaning the house so that when I return, everything is clean.

In short, in our quest to showcase progress and convenience, we are connecting everything we see to the internet. This is a very dangerous trend. It increases our attack surface. Additionally, security is often a secondary concern,

6.6 The Dark Side of Connectivity: Security Risks in the Internet of Things

no one is really interested in it. Employers might say, "What? You need a month to enhance security? Forget it! We need to get the product to market before our competitor does. Just skip that month and focus on adding new features, like making the lights change color. We need to cut costs to be able to sell our product at a lower price than our competitors." IoT devices must be cheap to be affordable for everyone. This means that the manufacturers cut costs in every possible way.

Many IoT devices are hard to update, and some are not updated at all. Many use default passwords for all users.²⁷ For example, if you buy a smart camera, the password might be "admin" for everyone. This is a huge security risk. If a hacker knows the default password, they can easily access your device. This is why it's crucial to change the default password to a strong, unique one. How many of you have changed the default password on your router? I bet 90% of you haven't. :)

If you're a developer, please, please don't implement a system with a fixed default password.²⁸

Many IoT devices have a minimal OS. Think about what can go wrong. Many of these OSs have little to no security features.

In summary, not much attention is paid to security. No one cares about the security protocols of these IoT devices. Now, think about what problems this could create.

- Imagine I write malware to hack your washing machine and make it run with the door open, creating a mess in your home. :)
- Or I create ransomware that infects your front door. Then I demand \$10 to unlock it. You're in a hurry, so instead of calling a repairman and paying \$50 for their service, you say, "Okay, fine!" and pay up. :)
- I could hack your TV and listen in on all the conversations happening in your home through its microphone.

One of the things I've noticed is that in the future, there might be microchips injected into the body that continuously release necessary medications over a month. Now, imagine in the distant future, a company wants to create a luxury drug and connect it to the internet or a communication protocol that monitors the body in real-time. That communication protocol will likely be full of vulnerabilities. It could be hacked, and they might demand a ransom, saying, "If you don't pay, we'll release the entire dosage of the medication all at once!"

If you're working in the IoT field, you need to be very careful about security. Explain to your employer why security is important and convince them of its significance. If you're involved in research, focus on the security of IoT protocols.

²⁷Many VPNs also do this!

²⁸UK Becomes First Country to Ban IoT Devices with Default Passwords: <https://www.bitdefender.com/en-us/blog/hotforsecurity/uk-becomes-first-country-to-ban-iot-devices-with-default-passwords>

6.7 Isolation from the Actual System

If you're a user, be careful about the devices you connect to the internet. Don't connect everything you see to the internet. Don't buy a smart device just because it's smart. Think about whether you really need it. Use a firewall to block unwanted traffic.

Reading List

- Spying with your robot vacuum cleaner: eavesdropping via lidar sensors[41]
- Keyhole Imaging[42]

6.7 Isolation from the Actual System

Remember we discussed about designing software in a way that if one part gets compromised, the rest of the system remains unaffected? This is a crucial concept in cybersecurity. This idea can be extended further: what if we could run potentially harmful programs in a relatively isolated environment, separate from our main system? This is where the concept of a sandbox comes into play.

6.7.1 Sandbox

Sometimes, we need to open files or programs that we are unsure about in terms of their safety or functionality. However, if these files contain malware or harmful activities, opening them on our main system could put our computer at risk. To mitigate this, we can use a sandbox to create a relatively isolated environment. This way, if something malicious occurs, it remains confined to the sandbox and does not affect our computer.

Think of it as having a room where you can open unknown items, but instead of doing it in the main room, you use a special box within that room. If something goes wrong, any harmful actions occur inside the box rather than in the main area of your computer.

In a sandbox, we restrict the program's access. We isolate it from other processes and resources. For example, we might allow it to access only a specific part of the computer's memory, permit only certain actions, or limit its access to specific files. By doing this, we minimize its permissions, preventing it from accessing the main computer and interfering with other processes, thus keeping our system safe.

So, what is the purpose of a sandbox, and who might need it?

Frequent Web Surfers: For those who spend a lot of time browsing the internet, a sandbox can be invaluable. The web is a common entry point for malware, as it allows various content to enter your device. If someone frequently visits untrustworthy sites, they can use a sandbox to isolate their browsing activities.

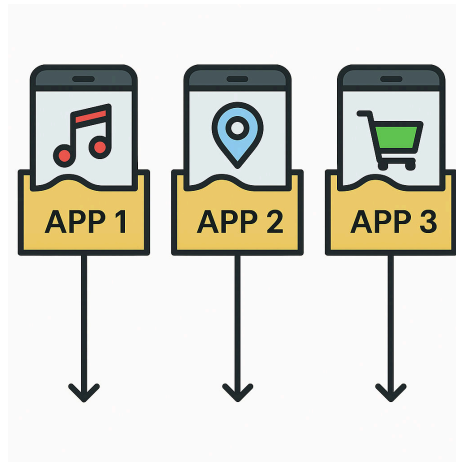


Figure 6.13: Sandbox. (AI)

Users Opening Various Files: People who receive numerous emails or download different files can also benefit from a sandbox. Emails can be a gateway for malicious files, so it is wise to open email attachments and use email programs within an isolated environment.

Malware Analysts and Security Professionals: These are individuals whose job is to analyze malware behavior. If they were to work with malware on their main system, they would risk getting infected. By using a sandbox, they can safely study malware without compromising their primary system. However, it's important to note that for malware analysis, the sandbox must be properly configured; otherwise, there is still a risk of infection.

In summary, a sandbox is a useful tool for anyone who interacts with potentially harmful content, providing an extra layer of security and isolation.

- Is a sandbox completely safe? Does having a sandbox mean we won't get infected at all?

→ Think of it this way: is a seatbelt completely safe? Yes, it prevents many injuries, but does wearing a seatbelt make you invulnerable? No! The same principle applies in the world of computers. There's a saying: "No system is safe." This means that no system is entirely secure. Even the most secure systems can have vulnerabilities that can be exploited. Software also has its flaws, and there are always potential entry points for attacks. Nothing is 100% fool-proof!

It's important to configure sandboxes correctly, especially for malware analysis or when running programs of uncertain safety. Otherwise, issues can arise. It's like trying to keep a snake in a box; if the box isn't secure, the snake will escape! If there are gaps at the edges, the snake will find a way out.

There are various sandbox programs available. For general use, the standard

settings are usually sufficient. However, for those looking to perform professional tasks like malware analysis, it's essential to customize the settings according to their specific needs.

● Great! We can analyze malware using this method. For example, we can run a program we're unsure about in a sandbox. If it doesn't perform any malicious actions, we can conclude that the program is completely safe and not malware! → It's not that simple! While it's true that antivirus programs often use sandboxes to run suspicious applications and observe their behavior, malware has its own tricks. For instance, some malware tries to detect whether it's running in a sandbox.

Think about it: a computer with no files, no internet connection, and nothing else—doesn't that seem suspicious? Doesn't it look like a sandboxed environment? Malware can check to see if it's running in a sandbox to avoid detection by antivirus software or analysts. Some sophisticated malware, especially those created by professionals or used in state-sponsored attacks (where one government tries to hack another or specific targets), perform initial checks to see if they're in a sandbox. If they are, they may refrain from executing their malicious actions to avoid being discovered.

Take the Stuxnet virus, for example. It spread to thousands of computers but did not activate in those systems because it was designed to target specific nuclear facilities. It would verify if it was in the correct environment before executing its malicious tasks.

Therefore, for those aiming to analyze a company's malware, their sandbox environment should closely resemble the company's actual systems. This involves creating files that mimic the company's files but contain false information and configuring the isolated system settings to match the main systems.

There's also a concept called a honeypot. These are systems set up with misleading information to lure hackers into attempting to breach them, enabling their capture. It's like placing a jar of honey to attract insects; they come for the food but end up getting trapped. In this case, it's a trap designed to catch hackers! Some honeypots are located in the weakest links. Since hackers often target the most vulnerable systems, these honeypots are intentionally made weak to attract them.

- Advanced Browser Security with Firejail - A Hands On Guide²⁹
- Sandboxie (Windows sandbox app)³⁰
- Windows Sandbox (Official solution for Windows)
- Dangerzone³¹³² (Take potentially dangerous PDFs, office documents, or images and convert them FIREJAILto safe PDFs)

²⁹<https://firejail.wordpress.com/2021/11/11/advanced-browser-security-with-firejail-a-hands-on-guide/>

³⁰<https://sandboxie-plus.com/>

³¹Yes PDF and other files can also be dangerous. (e.g. Malware in Image: <https://blog.reversinglabs.com/blog/malware-in-images>)

³²<https://dangerzone.rocks/>

- Qubes OS (A reasonably secure operating system)³³

Reading List

- Safe PDF viewers³⁴
- Mozilla Firefox sandbox³⁵³⁶
- EdgeSpot detects PDF samples tracking users who use Google Chrome as local PDF viewer³⁷
- Improving Malicious Document Detection in Gmail with Deep Learning³⁸
- Chrome 0-day exploit CVE-2019-13720 used in Operation WizardOpium³⁹
- Snap and Flatpak sandboxing method
- SELinux & AppArmor

6.7.2 Virtual Machine

A virtual machine is like creating a new computer within your main operating system. You can install a separate operating system on this virtual machine.

For example, let's say your CPU has 6 cores, your storage is 512 GB, and you have 16 GB of RAM. You can allocate resources from your main system to create a virtual machine. You might decide to set aside 2 of those 6 CPU cores, 50 GB of your 512 GB storage, and 6 GB of RAM. From that point on, these resources function as if they belong to a separate computer, allowing you to treat it as an independent system. This is done using software known as hypervisors, such as VMware or VirtualBox (the latter being open-source!).

This provides better⁴⁰ virtualization compared to regular sandbox applications. I recommend using a virtual machine for your web browsing activities.

If the main operating system is Arch-based with a KDE environment, and the virtual machine is Debian-based with a GNOME environment, is this setup more secure than having both systems use the same GNOME environment? I believe that if there's a vulnerability in Debian, it might not exist in Arch. Thus, one system could become compromised while the other remains safe. It's like having two different filters instead of two identical pieces of fabric.

³³<https://www.qubes-os.org/>

³⁴<https://malwaretips.com/threads/safe-pdf-viewers.109756/>, archived:
<https://web.archive.org/web/20220526230914/https://malwaretips.com/threads/safe-pdf-viewers.109756/>

³⁵<https://hacks.mozilla.org/2021/12/webassembly-and-back-again-fine-grained-sandboxing-in-firefox-95/>

³⁶<https://hacks.mozilla.org/2021/05/introducing-firefox-new-site-isolation-security-architecture/>

³⁷<https://blog.edgespot.io/2019/02/edgespot-detects-pdf-zero-day-samples.html>

³⁸<https://security.googleblog.com/2020/02/improving-malicious-document-detection.html>

³⁹<https://securelist.com/chrome-0-day-exploit-cve-2019-13720-used-in-operation-wizardopium/94866/>

⁴⁰No app is completely bug-free!

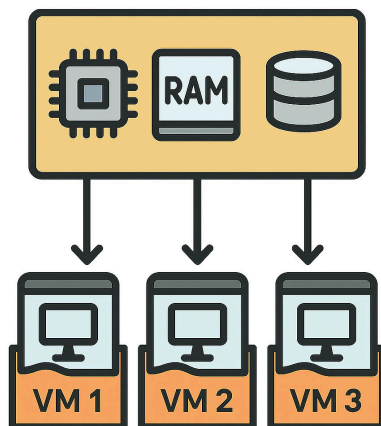


Figure 6.14: A computer with three virtual machines (AI)

6.7.3 Isolated Environments: The Key to Resilient Software Design

To now, we have discussed sandboxing and virtual machines as methods to isolate potentially harmful software from the main system. However, there's a broader concept known as compartmentalization that plays a crucial role in enhancing software security. Compartmentalization involves breaking down software into isolated components or modules, each with its own set of permissions and access controls. This approach minimizes the potential damage that can occur if one part of the software is compromised. For instance, web browsers like Firefox and Chrome have implemented sandboxing techniques to isolate web pages from each other. This means that if you open multiple tabs, each tab operates in its own sandboxed environment. If one tab gets compromised by malware, it cannot easily affect the other tabs or the main browser process. In essence, compartmentalization is done at the software design level.

6.7.4 See Also

- See quotes on this topic from the Linux foundation ⁴¹

6.8 Anti-virus

An important note is that an antivirus is not a magical solution that will protect you from all threats. It's like a seatbelt in a car; it can prevent many accidents, but it doesn't make you invulnerable. Always consider it as a second layer of

⁴¹<https://www.linuxfoundation.org/press/linux-foundation-launches-the-open-robust-compartmentalization-alliance-orca->

defense. The first layer is you! You should always be cautious and aware of what you're doing.

6.8.1 Signature-based detection

Have you ever noticed that some people identify someone's nationality based on their facial features, like Chinese or Japanese? In computers, we do something similar with malware. They analyze it and notice that certain elements are common among a series of malicious programs. These common elements can be considered shared features. So, whenever we see them, we can say that it's malware from that family. This means we create a signature based on certain binary patterns, and whenever we see that pattern, we identify it as malware. This is called signature-based detection. It is like a fingerprint.

Signatures can be created based on:

Byte sequence

Researchers often extract a unique sequence of bytes from the malware binary that rarely appears in other programs. This could be a specific function call, a distinctive piece of code, or a payload-embedded data structure. For example, the binary:

```
0x68 0x65 0x6C 0x6C 0x6F 0x20 0x65 0x78 0x61 0x6D 0x70 0x6C 0x65 0x2E 0
```

is the hex representation of the string "hello example.com". This may indicate that the malware is trying to connect to a specific domain (C&C server) to download additional payloads or exfiltrate data.

In addition to signature detection, antivirus programs check which company has signed the file. If the file is signed by a company that is known to be legitimate, it is considered safe. However, this creates a problem. What if the private key of the company is stolen? Then the attacker can sign the malware with the company's key, and the antivirus will consider it safe. There have been cases where the private key of a company was stolen, and the attacker signed the malware with it and targeted major companies.

Furthermore, certain sequences of bytes representing sequence of operations, API calls with certain parameters can be used to create a signature.

What is the advantage of signature-based detection?

- It is easy to implement.
- It is fast.
- Can be used when scanning a file. (Before executing it)

What is the disadvantage of signature-based detection?

- It is not effective against new malware.
- It is not effective against polymorphic malware. (Malware that changes itself to avoid detection)

6.8.2 Behavioral-based detection

Behavioral-based detection is a technique used to identify malware based on its behavior. It examines what the malware does. For example, if a program attempts to access the webcam without the user's permission, it is likely to be malware. Malware often employs various techniques to conceal its presence and evade detection. Here are some suspicious behaviors specifically related to hiding malware:

- **Code Injection:** Malware often injects its code into legitimate processes to avoid detection. This technique enables the malware to run within the context of a trusted process, making it harder to detect.
- **Process Hollowing:** This technique involves replacing the memory of a legitimate process with malicious code. By doing so, the malware can execute its code within the context of a trusted process.
- **Steganography:** Some malware uses steganography to hide its code within other files, such as images or audio files. This technique allows the malware to evade detection by security tools that only scan for known executable file types.
- **Encrypted Payload:** If a malware sample contains encrypted payloads⁴² and decrypts them at runtime, it may indicate malicious behavior. Why is this significant? Because many antivirus programs scan files statically, meaning they analyze the file without executing it. If the payload is encrypted, the antivirus program won't be able to detect it.
- **Running Certain Commands:** Some malware may execute specific commands to manipulate system behavior or gather information.
- **Delayed Execution:** Some malware may delay its execution to avoid detection during initial installation. This can involve waiting for a specific event or time before activating its malicious functions.
- **Environmental Checks:** Malware often performs checks to determine if it is running in a sandbox or virtual environment. If it detects that it is being analyzed, it may alter its behavior to avoid detection. Some malware is designed to harm other countries; if it infects its own country, it will be caught by its own country's security systems. For example, malware from Russia will check whether the system it is targeting has the Russian language or timezone. If it does, it will not perform any malicious actions.

⁴²A payload is the part of the malware that performs the malicious actions.

6.9 Code Security

- **Self-Destruction:** Some malware is designed to delete itself if it detects that it is being analyzed or if certain conditions are met, making it harder to study and understand.
- **Manipulating System Logs:** Malware may attempt to erase or modify system logs to cover its tracks and prevent detection of its activities.
- **Unusual Network Activity:** Unusual outbound connections to unknown IP addresses or domains, especially if they occur at odd hours or in large volumes.

6.8.3 Anti-virus bypass methods

TODO...

Reading List

- Writing YARA Rules⁴³ (A tool to create signatures)

6.9 Code Security

6.9.1 Input Validation

Consider the following Python code:

```
1 num = int(input())
2
```

Listing 6.1: Getting an integer from the user

Explanation: The Python interpreter, processes the innermost parentheses first. It takes user input, which is initially a string, and then attempts to convert it to an integer using the `int` function.

Now, suppose a user enters a word instead of an integer. What happens? The program will throw an error such as:

```
1 ValueError: invalid literal for int() with base 10: 'hello'
2
```

Listing 6.2: Error when entering a word instead of an integer

In other words, this results in an error, causing the software to crash.

The basic idea behind fuzz testing is to provide invalid or unexpected input data to the system to identify crashes or potential security vulnerabilities.

Consider the following scenarios:

⁴³<https://yara.readthedocs.io/en/stable/writingrules.html>

6.9 Code Security

- If the program asks for a number, what happens if I enter a negative number, zero, or a very large positive number? (The idea is to test how the program handles unexpected numerical inputs.)
- If the program expects an age between 1 and 100, what happens if I input a value like 200? (The idea is to enter data that falls outside of the acceptable or expected range.)
- If the program requires a selection between options 1 to 5, what happens if I choose an invalid option like -1?
- If the program expects a website URL (for example, `https://example.com`), what happens if I mistakenly provide `https:/example.com` (using one slash instead of two)?
- If it requires a file input, what happens if I supply an incorrect or oddly formatted file (for instance, with a strange file extension)?

The core idea is to test the program by deviating from its expected flow. In security testing, not only should you understand how the system is supposed to work, but you should also explore ways of interacting with it that the programmer did not anticipate. This can reveal vulnerabilities where the system might crack, allow unauthorized access, or even lead to data breaches.

• But my program is large and complex. I can't test all possible inputs!

→ That's true! You can't test every possible input. But you can use your programming skills to write a program that generates random inputs. This way, you can automate the process of testing various inputs against the system. This highlights the importance of coding skills in security testing. The process of giving random inputs is called **fuzzing**.

If you are familiar with languages like C that have `int` and `long int`, consider the following case:

```
1 int res;
2 scanf("%d", &num1);
3 scanf("%d", &num2);
4 res = num1 * num2;
5
6 // or
7 int a = 1000000;
8 int b = 1000000;
9 int c = a * b;
10
11 // or
12 int x;
13
14 for(x = 0; x <= 3000000000; x++)
15     printf("%d ", x);
16 putchar('\n');
17
18 // instead of:
19 int mid = (low + high) / 2;
```

6.9 Code Security

```
20 // use:
21 int mid = low + (high - low) / 2;
22
23 // But...
24 char c1 = 100;
25 char c2 = 100;
26 char c3 = (c1 * c2) - 9935; // no overflow! integer promotion
27 printf("%c", c3);
28
29 // or
30 // Do mathematics with number theory basics (modular arithmetic)
   to avoid overflow:
31 (a+b) % m = ((a % m) + (b % m)) % m // also for '-' or '*'
32 (a*b) % m = ((a % m) * (b % m)) % m
33
```

Listing 6.3: C code that multiplies two integers

If the product of `num1` and `num2` becomes too large to fit within the range of an `int`, an overflow may occur. It might even underflow, resulting in a very small or very negative number that does not fit within the range of `int`. The second case suggest that many programmers forget to check variables before multiplying them.

Solution?

- Never trust user-provided data! Always validate critical inputs.
- Always check user data for validity before storing it. For example, if a number is expected but the user enters something like 14212a1643, it is clearly invalid!

6.9.2 Sign and Unsigned Comparison

Do not compare signed and unsigned integers.

6.9.3 Reach the End of Function Without Return

In C, if you have a function that is supposed to return a value, make sure that all code paths lead to a return statement. If you reach the end of the function without returning a value, it can lead to undefined behavior. For example:

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int add(int a, int b) { int c = a + b; }
5
6 int main()
7 {
8     int ret = add(10, 20);
9     printf("%d\n", ret);
10
11     return 0;
12 }
```

13

In this code, the function `add` is supposed to return an integer, but it does not have a return statement. Because an integer should be returned, the value of `eax` (the register that holds the return value) will be returned.

Another example would be:

```
1  int foo(int age)
2  {
3      if (age > 0 && age <= 18)
4          return age;
5      else if (age >= 19 && age <= 35)
6          return age + 1;
7      else if (age >= 36 && age <= 50)
8          return age + 2;
9      else if (age >= 51 && age <= 200)
10         return age + 3;
11 }
12
13 int main()
14 {
15     int age = 210;
16     printf("%d\n", foo(210));
17     if (foo(age)) {
18         /**/
19     }
20
21     return 0;
22 }
23
```

Listing 6.4: No else statement in C code

6.9.4 Floating-Point Comparison

Background (IEEE 754)

Floating-point numbers are represented in computers using a standard called IEEE 754. This standard defines how floating-point numbers are stored in binary format, including the allocation of bits for the sign, exponent, and mantissa (or significand or fraction). A floating-point number is typically represented as:

$$(-1)^{\text{sign}} \times 2^{\text{exponent} - \text{bias}} \times (1 + \text{fraction})$$

Where:

- **Sign bit:** A single bit that indicates whether the number is positive (0) or negative (1).
- **Exponent:** A set of bits that represents the exponent of the number, which determines the scale or range of the number. (To translate to base 10, look at it as unsigned integer)

- **Bias:** A constant value subtracted from the stored exponent to get the actual exponent. This is done to allow for both positive and negative exponents.
- **Fraction (Mantissa):** A set of bits that represents the significant digits of the number. It is normalized, meaning there is an implicit leading 1 before the binary point. To calculate the actual value of the fraction, you sum the values of each bit multiplied by 2^{-n} , where n is the position of the bit after the binary point: $\sum_{i=1}^k b_i \times 2^{-i}$, where b_i is the i -th bit of the fraction.

for example, lets consider the 32-bit single-precision floating-point representation:

01000001101111011000000000000000

- **Sign bit:** 0 (indicating a positive number)
- **Exponent:** 131 (in decimal)
- **Bias:** 127 (for single-precision)
- **Fraction:** 10110000000000000000

To calculate the actual value:

$$(-1)^0 \times 2^{131-127} \times (2^{-1} + 2^{-3} + 2^{-4} + 1) = 23.6875$$

Comparison Issues

The representation allows for a wide range of values, but it also introduces some limitations in precision. Therefore, when comparing floating-point numbers, be cautious. Due to the way floating-point numbers are represented in binary, they can have precision issues when you use **the result of a calculation for comparison**. For example, the following code may not behave as expected:

```
1 printf("%d\n", 0.3 == 0.1 * 3);  
2
```

Listing 6.5: Floating-Point Comparison

6.9.5 Timing Attack

Do you remember what the ==operator does in Python (or in other languages like Java)? It checked whether the two items provided were exactly equal. For example:

```
1 'abcd' == 'bbbb'  
2
```

Listing 6.6: Python code that checks if two strings are equal

Python starts comparing the two strings character by character, beginning with the first character. It checks whether the first character of the first string is equal to the first character of the second string. If they are not equal, it immediately concludes that the two strings are not equal—even if the rest of the characters might match. There is no need to continue checking, as the mismatch of one character is enough to declare that the strings are different; this avoids unnecessarily slowing down the program.

Now, consider the case where the first characters of two strings are the same:

```
1 'abcd' == 'aaaa'
2
```

In this case, Python will check the first character, find it equal, and then move on to the second character. It will continue checking until it finds a mismatch or reaches the end of the string. This means that if the two strings are equal, Python will take longer to check them than if they are not equal.

This type of vulnerability is known as a *timing attack*. An attacker can use this information to determine whether two strings are equal or not based on how long it takes for the program to respond. For example, if the attacker provides a string and receives a response quickly, they know that the first character is not equal. But if it takes longer, they know that the first character is equal and can continue checking the next characters.

To prevent timing attacks, use constant-time comparison functions that execute in the same amount of time, regardless of whether the strings are equal.

Now, suppose the password on a website is the string '123456789'. If I want to guess the password, how might I approach it? I might try the following sequences in order:

'000000000', '111111111', '222222222', ...

Why would this work? If you look at the code, you notice that Python compares the strings character by character. It stops as soon as it finds a mismatch. Now, suppose the test takes 1 millisecond for '000000000', 2 milliseconds for '111111111', and 1 millisecond for '222222222', and so on. The longer duration for '111111111' suggests that the first character is correct and that Python took additional time to check the second character as well. In effect, you would have determined the first character of the password.

Thus, with about 10 tests (for each possible digit) you could determine the first character. In total, for each character in the password you only need about 10 tests.

For a 10-character password, that would be approximately $10 * 10 = 100$ tests instead of testing all 10^{10} possibilities. This is a significant reduction in the number of tests required to guess the password.

These types of attacks are called **timing attacks**. They exploit the time it takes for a system to respond to different inputs. By measuring the time it takes for a system to perform operations, an attacker can infer information about the data being processed.

6.9 Code Security

- Does adding random delays help?

→ No! It doesn't fully mitigate the problem. An attacker can still measure the time and determine the average time it takes for the system to respond. They can then use this information to make educated guesses about the data being processed.

To prevent timing attacks, use constant-time comparison functions that execute in the same amount of time, regardless of whether the strings are equal. This way, an attacker cannot gain any information about the data being processed based on the time it takes for the system to respond. A simple way to compare is:

```
1 # Assuming the size of both strings is the same
2 def compare_strings(s1, s2):
3     result = 0
4     for i in range(len(s1)):
5         result |= ord(s1[i]) ^ ord(s2[i])
6
7     return result == 0
8
```

Listing 6.7: Python code that compares two strings in semi-constant time

This function compares two strings in constant time, regardless of whether they are equal or not. It uses the XOR operator to compare each character and returns true only if all characters are equal.

A better way to do this is to use *C* and check for compiled assembly code to know the code is actually constant time or not.

To fully mitigate the problem of timing attacks, hardware should be designed to prevent timing attacks. For example, the CPU could be designed to take the same amount of time to perform operations regardless of the data being processed. Here also you need to have a good understanding of hardware and how it works. This is why security experts need to have a deep understanding of both software and hardware.

Timing attacks are not limited to input validation. Let's consider a more complex example. You use WhatsApp, right? When you send a message, it shows a single tick mark, indicating that the message has been sent. When the recipient receives the message, it shows two tick marks. But how does the tick marks appear? When you send a message and the recipient receives it, a special packet is sent to you indicating that the message has been received.

A hacker mindset would be to think, "Hmm, I wonder if I can gather information about the recipient's location based on the time it takes for the message to be delivered." See *Timing Attacks on WhatsApp, Signal, and Threema can Reveal User Location*[3] for more information. ⁴⁴

⁴⁴Or you may just read the news: <https://cyberinsider.com/timing-attacks-on-whatsapp-signal-threema-reveal-user-location/>

Reading List

- Configuring Workloads for Microarchitectural and Side Channel Security⁴⁵
(from intel. Make sure to read cited linked in the *Background* section.
They are **super important!**)

6.9.6 Careless programmers/Faster is not always better

Have you ever heard of the phrase “faster is not always better”? This is especially true in programming. Sometimes, programmers try to optimize their code to make it run faster, but this can lead to security vulnerabilities.

When you crop an image, many image-editing software programs do not remove the pixels. Instead, they use flags to indicate which pixels are part of the image and which are not. For example:

- Original image: `sdklfjsadfjfdslkweruoisdsklfjsadfjfdslkweruoiIEND`
- Cropped image: `sdklfjsadfjfdslkweruoiIENDsdklfjsadfjfdslkweruoi`

In fact, the cropped image is not really cropped; it’s just a flag that indicates which pixels are part of the image and which are not. This optimization is often implemented to save time and improve performance.

But a hacker might think, “Hmm, I see the size of the cropped image is not that much smaller than the original image. I wonder if there would be the original image in the cropped image.” So they open the cropped image as binary, remove the flag, and voila! They have the original image.

• Wow! Does this applies to all image-editing softwares?

→ No! Some have this problem.

Similar is true for drawing. For example, if you have a picture of a person and you want to hide their face with drawing on it, the software might not actually remove the pixels; it just added a layer of drawing on top of the original image. This means that if someone knows how to remove the drawing, they can see the original image.

This shows that programmers need to be careful when optimizing their code. They should not sacrifice security for speed. The code should be secure, even if it takes a little longer to run. Security is more important than speed. If the code is not secure, it can lead to serious vulnerabilities that can be exploited by hackers.

What should you do? The best way to avoid this is to take a picture of the edited image with another camera. This way, you can be sure that the original image is not in the edited image. If no other camera is available and you’re not sure about the editing app, use solid black color to cover the face. (This is a

⁴⁵<https://www.intel.com/content/www/us/en/developer/articles/technical/software-security-guidance/best-practices/securing-workloads-against-side-channel-methods.html>

better way as it prevents **some** changing the exposure of the image and making the color color transparent.)

Blurring works by applying a filter (see it as a calculation) to the image. Unfortunately, many filters are reversible. This means that if you know the filter, you can reverse it and get the original image.

Reading List

- aCropalypse (How someone noticed⁴⁶)
- https://www.reddit.com/r/technology/comments/3v0j5x/whatsapp_security_flaw_can_reveal_your_location/

6.9.7 Race Condition

Let your CPU has only one core and you have multiple programs running on it. Does the CPU run all of them at the same time? No! It runs one program at a time. But how does it do that? It uses a technique called **multitasking**. The operating system (OS) switches between different programs quickly, giving the illusion that they are running simultaneously. This switching is done very quickly, so you don't notice it. For example, when you open a web browser and a text editor at the same time, the OS switches between them, doing a little bit of work on each program before switching to the next one. This is called **context switching**.

Now consider the following instructions:

Listing 6.8: Instructions taken for applying a discount coupon code

```
Get the code from user
Check validity
Apply code
Validity = False
```

This code is used to check if a discount coupon code is valid. The first line gets the code from the user. The second line checks if the code is valid. The third line applies the code. The fourth line sets the validity to false. Now, suppose two users are trying to apply their codes at the same time. The OS switches between them, and the following happens:

Listing 6.9: How the OS may switch between two processes

```
Get the code from user 1
Get the code from user 2
Check validity of code 1
Check validity of code 2
Apply code 1
```

⁴⁶<https://issuetracker.google.com/issues/241936678>

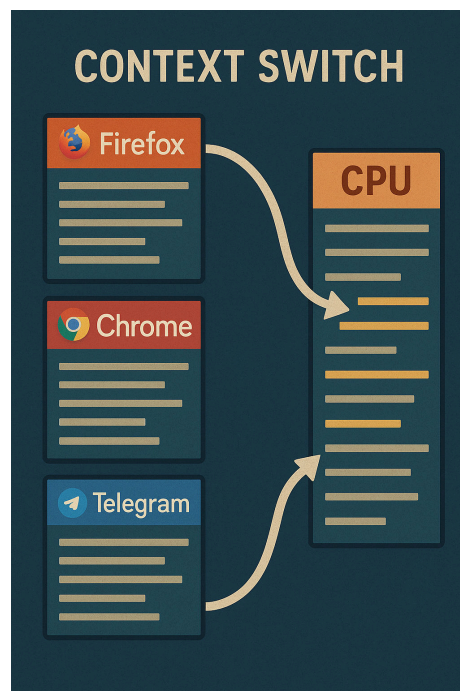


Figure 6.15: Context Switching

6.9 Code Security

```
Apply code 2
Validity = False
Validity = False
```

or

```
1 if coupon_validation == True:
2     enable_discount() # (I)
3     coupon_validation = False # (II)
4
```

Listing 6.10: Python code that applies a discount coupon code

If two user try to apply their codes at the same time, the switching of the OS can cause a problem. As you can see, OS does a little bit of work on each program before switching to the next one. However, it may cause both code to be applied. Because two process can enter the block before the validity is set to false. This issue is an example of a *race condition*, a type of concurrency vulnerability.

The same issue can occur if two requests to transfer money are sent simultaneously. Consider the following code:

```
1 my_balance = 100 # Get the balance from the bank API (I)
2 transfer_amount = 100
3 his_balance = 0
4 if my_balance >= transfer_amount:
5     his_balance = his_balance + transfer_amount
6     my_balance = my_balance - transfer_amount # (II)
7
```

Listing 6.11: Python code that transfers money

If two requests are sent at the same time, the OS will switch between them. The first request will check the balance and find that it is sufficient. Then, the second request will also check the balance and find that it is still sufficient. Both requests will then proceed to transfer the money, resulting in a double transfer. By this, the hacker can transfer more money (200) than they actually have (100).

Have you spotted other potential problems? Yes! Input validation is not done. The code does not check if the transfer amount is valid (for example, if it is a negative number). This could lead to other issues, such as transferring a negative amount (resulting in a positive balance for the sender)!

Race condition may also occur in other scenarios. For example, two account can be created with the same email address.

Now let's consider the following code:

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <pthread.h>
4
5 int x = 0;
6
```

6.9 Code Security

```
7 void *raceCondition(void *vargb);
8
9 int main()
10 {
11     pthread_t t1, t2; // thread id
12
13     pthread_create(&t1, NULL, raceCondition, NULL);
14     pthread_create(&t2, NULL, raceCondition, NULL);
15
16     pthread_join(t1, NULL);
17     pthread_join(t2, NULL);
18
19     printf("%d", x);
20     return 0;
21 }
22
23 void *raceCondition(void *vargb)
24 {
25     for (int i = 0; i < 1000000; i++)
26         x++;
27     return NULL;
28 }
29
```

Listing 6.12: C code that increments a variable by two threads

The code above creates two threads that increment the variable `x` one million times each. However, does it always increment `x` to 2000000? Try it out! You will see that `x` becomes something less than 2000000. What has gone wrong?

First let's look out `x++`. The assembly code for `x++` is:

```
1 mov eax, DWORD PTR x[rip] ; // Load the value of x into eax
2 add eax, 1 ; // Increment eax by 1
3 mov DWORD PTR x[rip], eax ; // Store the updated value back to x
4
```

Listing 6.13: x86 assembly of `x++`

The first line loads the value of `x` into the register `eax`. The second line adds 1 to the value in `eax`. The third line stores the updated value back into `x`.

Now, consider the following scenario:

- Thread 1 loads the value of `x` (let's say it is 0) into `eax`.
- Thread 2 also loads the value of `x` (still 0) into its own `eax`.
- Thread 1 increments its `eax` to 1 and stores it back to `x`.
- Thread 2 increments its own `eax` to 1 and stores it back to `x`.
- Thread 1 stores the value 1 back to `x`.
- Thread 2 stores the value 1 back to `x`.
- Now, both threads have stored 1 in `x`.

The switching of the OS between the two threads causes a problem.

Reading List

- Towards Systematic Black-Box Testing for Exploitable Race Conditions in Web Apps [43]

6.9.8 An attempt to backdoor the kernel

The following code snippet was intended to be added to the Linux kernel. Assume that the id of the root user in Linux is 0. Think you can spot the problem?

```
1 #define __WCLONE 1
2 #define __WALL 2
3 #define EINVAL 1
4 int options = 2;
5 int retval;
6 int currUid;
7
8 int fun()
9 {
10     if (((options == __WCLONE) || (options == __WALL)) && (
11         currUid = 0))
12         retval = -EINVAL;
13 }
```

Listing 6.14: Checking for root access?

This code was an attempt to be added to the Linux kernel.⁴⁷ Can you spot the problem? The code attempts to check if the user is root (user ID 0), but it mistakenly uses a single equal sign (=) for assignment instead of a double equal sign (==) for comparison. This means that instead of checking if the user is root, it assigns the value 0 to `currUid`. As a result, if a program does something to cause the routine to be executed, it will become root (since `currUid` is set to 0) and gain unauthorized access.

This is a classic example of a *backdoor*: a hidden vulnerability that allows unauthorized access to a system, often with triggers that are not obvious to regular users. In this case, the backdoor allows any user to gain root access by exploiting the assignment operator mistake.

Various tools, such as static analysis tools, are available to detect such issues in code. For instance:

- **cppcheck:** A static analysis tool for C/C++ code that can detect various types of bugs, including the misuse of assignment and comparison operators.
- **SonarQube:** A platform that provides continuous inspection of code quality, including security vulnerabilities.
- **Pylint:** A static code analysis tool for Python that can identify potential linting issues and some security vulnerabilities.

⁴⁷<https://lwn.net/Articles/57135/>

In this case, however, the problem was not detected by *cppcheck*.⁴⁸

6.9.9 Out-of-Bound Access

Example 1

Let's return to the code for checking if two strings are equal. What problem do you think the following C code would have (aside from the timing issue we discussed)?

```
1 #include <string.h> // For strlen
2
3 int isPassValid(char pass[], char inp_pass[])
4 {
5     // Assuming pass is a null-terminated string.
6     // strlen(pass) gives the length of the correct password.
7     for (int i = 0; i < strlen(pass); i++)
8     {
9         // If inp_pass is shorter than pass, this can read out of
10        bounds.
11        if (pass[i] != inp_pass[i])
12            return 0; // Not equal
13    }
14    // This part is only reached if all characters up to strlen(
15    pass) matched.
16    // It doesn't check if inp_pass might be longer than pass.
17    // A more robust check would also ensure inp_pass[strlen(pass
18    )] == '\0'
19    // or strlen(pass) == strlen(inp_pass).
20    return 1; // Equal up to length of pass
21 }
```

Yes! An "out-of-bounds read." If the user-provided `inp_pass` is shorter than the actual `pass` (e.g., `inp_pass` has 2 characters and `pass` has 10), the loop continues as long as `i < strlen(pass)`. When `i` exceeds the length of `inp_pass` (e.g., `i` becomes 2), `inp_pass[i]` attempts to read memory outside the allocated space for `inp_pass`. This means you are reading memory locations that do not belong to `inp_pass`, which can lead to crashes, incorrect behavior, or information disclosure.

Example 2

To avoid accidentally accessing an incorrect index when working with arrays (or lists in Python), pay close attention to your `while` or `if` conditions. For example, consider this Python code:

```
1 size = 5
2 arr = [1, 2, 3, 4, 5]
3 # This loop intends to print elements in reverse
4 while size >= 0:
```

⁴⁸As of Dec 2025.

6.9 Code Security

```
5 print(arr[size-1]) # Accesses arr[4] down to arr[-1]
6 size -= 1
7
```

Listing 6.15: Python code that prints elements in reverse

This code has a problem. Let's trace its execution:

- When `size` is 5, it prints `arr[4]` (which is 5). `size` becomes 4.
- When `size` is 4, it prints `arr[3]` (which is 4). `size` becomes 3.
- When `size` is 3, it prints `arr[2]` (which is 3). `size` becomes 2.
- When `size` is 2, it prints `arr[1]` (which is 2). `size` becomes 1.
- When `size` is 1, it prints `arr[0]` (which is 1). `size` becomes 0.
- When `size` is 0, it prints `arr[-1]`. In Python, `arr[-1]` is a valid way to access the last element of the list (which is 5). `size` becomes -1.

The loop then terminates. The output sequence is 5, 4, 3, 2, 1, 5. So, the value 5 is printed twice. While Python handles negative indexing by wrapping around, in languages like C or C++, accessing `arr[-1]` would be an out-of-bounds memory access, likely causing a program crash or undefined behavior. The condition `while size >= 0` combined with `arr[size-1]` is tricky. If the intent was to iterate through valid positive indices from `size-1` down to 0, the condition should be `while size > 0`. Always be careful to define your loop conditions and array/list indexing correctly to ensure you only access memory within the allocated bounds.

Example 3

Now, what problem do you think the following C code has? (The logic is: it takes an integer `length` as input from the user, then reads `length` characters from the input and stores them in the `str` array.)

(P.S.: For this example, assume `scanf` and `getchar` are functioning correctly as input mechanisms, and disregard other potential security concerns related to them.)

```
1 #include <stdio.h>
2 #include <stdlib.h>
3
4 int main()
5 {
6     int length;
7     scanf("%d", &length);
8     getchar(); // To consume the newline character left by scanf
9
10    // Allocate memory for 'length' characters
11    char *str = (char *)malloc(sizeof(char) * length);
12    if (str == NULL) {
```

```
13     // Handle memory allocation failure
14     return 1;
15 }
16
17 int i;
18 for (i = 0; i < length; i++)
19 {
20     str[i] = getchar(); // Read characters
21     if (str[i] == EOF) { // Handle premature end of input
22         break;
23     }
24 }
25 // Attempt to null-terminate the string
26 str[i] = '\0';
27
28 // printf("String: %s\n", str); // Using the string
29 free(str);
30
31 return 0;
32 }
33
```

Listing 6.16: Reading a string from user input

The problem here is an "out-of-bounds write." The code allocates memory for `length` characters using `malloc(sizeof(char) * length)`. The `for` loop then reads `length` characters into `str[0]` through `str[length-1]`. After the loop completes (assuming no EOF is encountered prematurely), the value of `i` will be `length`. The line `str[i] = '\0';` attempts to write the null terminator at `str[length]`. However, the allocated memory block only has valid indices from 0 to `length-1`. Writing to `str[length]` means writing one byte beyond the allocated buffer. This can corrupt adjacent memory, lead to unpredictable program behavior, or cause a crash. To correctly store `length` characters plus a null terminator, you need to allocate `length + 1` bytes: `char *str = (char *)malloc(sizeof(char) * (length + 1));`.

6.9.10 Memory-Safe Languages

First we should know what a vector is. A vector is a dynamic array that can grow and shrink in size as needed. It means that if the initial size of the vector is not enough to hold all the elements, it will automatically allocate more memory to accommodate the new elements.

However, not always will the vector be able to allocate more memory in the same location. In such cases, it will allocate a new block of memory and copy the existing elements to the new location. This means that any pointers pointing to the old location will become invalid. But C++ does not automatically update those pointers. This can lead to undefined behavior if you try to access the vector using those stale pointers.

Consider the following C code.

```
1 #include <iostream>
```

6.9 Code Security

```
2 #include <vector>
3
4
5 int main()
6 {
7     std::vector<int> vec(10);
8     int *p1 = vec.data();
9
10    for (int i = 0; i < 1000; ++i) {
11        vec.push_back(i);
12    }
13
14    p1[800] = 42;
15    std::cout << "Value at p1[800]: " << p1[800] << std::endl;
16
17    return 0;
18 }
19
```

Listing 6.17: C code that modifies a vector and accesses it via a pointer after modification

The problem here is that after pushing back 1000 elements into the vector, the original memory location where the vector was stored, depending on the implementation, may have changed. (In STL, when the vector needs more space, reallocation happens, and the old memory location is no longer valid.)

The pointer `p1` still points to the old location, which may no longer be valid. Accessing `p1[800]` could lead to:

- reading 42 (if we are lucky and the program could allocate more memory ensuring the old location. If you get 42, consider increasing the number of elements pushed back to see the problem; something like 100,000.) In STL, this is not the case.
- reading 42 but other values between 11 and 1000 are garbage values (since after 10, the vector location changed, and the old location has garbage values)
- crash (if the old location is no longer accessible by our program address space)

In this case, since STL reallocates memory in a way that the old location is no longer valid, accessing `p1[800]` leads to undefined behavior. This is because `p1` is now a stale pointer that points to a memory location that may have been freed or reallocated for other purposes.

To understand this better, consider the following code:

```
1 #include <stdio>
2 #include <iostream>
3 #include <vector>
4
5 int main()
6 {
```

```
7  std::vector<int> vec(16);
8  int *p1 = &vec[0]; // get pointer to the underlying array
9
10 vec.push_back(0); // vector size usually doubles here
11 int *p2 = &vec[0]; // get new pointer to the underlying array
12
13 vec.push_back(0);
14 int *p3 = &vec[0]; // get another pointer to the underlying
    array
15
16 printf("%p %p %p\n", static_cast<void *>(p1), static_cast<
    void *>(p2),
17         static_cast<void *>(p3)); // Pay attention to this.
    Using C-style code in C++. I'll tell you later in a warning
    box.
18
19 return 0;
20 }
21
```

Listing 6.18: C code that shows pointer invalidation with vector reallocation

When you run this code, you will see that the address of `p1` and `p2` are different, indicating that the vector has reallocated its memory. This shows that `p1` is now a stale pointer and accessing it can lead to undefined behavior!

However, the value of `p2` and `p3` is likely the same, since when a vector reallocates memory, it typically doubles its capacity. In this case, after the first `push_back`, the vector's capacity increases from 16 to 32. The second `push_back` does not require another reallocation since the current capacity (32) is sufficient to accommodate the new element. Therefore, both `p2` and `p3` point to the same memory location.

Programming languages such as C and C++ give the responsibility of memory management to the programmer. This means that (mostly) the programmer has take care of such issues. This is why C and C++ are called *memory-unsafe* languages.

Now what could we do? Use memory-safe languages like *Rust*.

Rust

Let's look at a few quotes.

“ Many vulnerabilities have a well defined scope of impact. For example, a permissions bypass vulnerability generally grants access to a specific set of information or resources and is generally only reachable if code is already running on the device. Memory safety vulnerabilities tend to be much more versatile. Getting code execution in a process grants access not just to a specific resource, but everything that that process has access to, including attack surface to other processes. Memory

— Google

6.9 Code Security

“ safety vulnerabilities are often flexible enough to allow chaining multiple vulnerabilities together. The high versatility is perhaps one reason why the vast majority of exploit chains that we have seen use one or more memory safety vulnerabilities.^a

^a<https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html>

— Google

“ approximately 70% of security vulnerabilities that Microsoft fixes and assigns a CVE (Common Vulnerabilities and Exposures) are due to memory safety issues.^a

^a<https://msrc-blog.microsoft.com/2019/07/16/a-proactive-approach-to-more-secure-code/>

— Microsoft

“ The Chromium project finds that around 70% of our serious security bugs are memory safety problems.^a

^a<https://www.chromium.org/Home/chromium-security/memory-safety/>

— Chromium

After seeing these quotes, do you think memory safety is important? Yes! In essence:

“ Safety or quality cannot be “tested into” programs. It must be designed in from the start. Choosing to implement with a safer or more secure language or language subset can entirely avoid whole classes of weaknesses.^a

^a<https://www.nist.gov/itl/ssd/software-quality-group/safer-languages>

— NIST

This is why languages like Rust are gaining popularity. Rust is a memory-safe language that guarantees both memory safety and thread safety, at compile-time, without requiring a garbage collector.⁴⁹

You may ask why don't we just use modern C++? Modern C++ has many features that help with memory safety, such as smart pointers and the standard library containers.

Firstly:

⁴⁹<https://www.nist.gov/itl/ssd/software-quality-group/safer-languages>

Warning

Unfortunately, many developers write C++ code in a C-like style, neglecting the memory safety features that modern C++ provides. Simply using C++ does not ensure memory safety; you must actively leverage features such as smart pointers and standard library containers to benefit from its safety improvements.

But, the inherent problems of C and C++ still exist. Microsoft states:

“ While many experienced programmers can write correct systems-level code, it’s clear that no matter the amount of mitigations put in place, *it is near impossible to write memory-safe code using traditional systems-level programming languages at scale.*

In our next post^a, we’ll explore why we think the Rust programming language is currently the best choice for the industry to adopt whenever possible due to its ability to write systems-level programs in a memory-safe way.^b

^a<https://msrc-blog.microsoft.com/2019/07/22/why-rust-for-safe-systems-programming/>, Archived: <https://web.archive.org/web/20221007190843/https://msrc-blog.microsoft.com/2019/07/22/why-rust-for-safe-systems-programming/>

^b<https://msrc-blog.microsoft.com/2019/07/18/we-need-a-safer-systems-programming-language/>, Archived: <https://web.archive.org/web/20221007191107/https://msrc-blog.microsoft.com/2019/07/18/we-need-a-safer-systems-programming-language/>

— Microsoft

Moreover, *Safety measures make memory-unsafe languages slow*:

“ Mobile devices have limited resources and we’re always trying to make better use of them to provide users with a better experience (for example, by optimizing performance, improving battery life, and reducing lag). Using memory unsafe code often means that we have to make tradeoffs between security and performance, such as adding additional sandboxing, sanitizers, runtime mitigations, and hardware protections. Unfortunately, these all negatively impact code size, memory, and performance.^a

^a<https://security.googleblog.com/2022/12/memory-safe-languages-in-android-13.html>

— Google

The Tor Project states:

“ Why rewrite Tor in Rust? Because despite (or because of) its maturity, the C Tor implementation is showing its age. While C was a reasonable choice back when we started working on Tor 2001, we’ve always suffered from its limitations: it encourages a needlessly low-level approach to many programming problems, and using it safely requires painstaking care and effort. Because of these limitations, that pace of development in C has always been slower than we would have liked.^a

^ahttps://blog.torproject.org/arti_100_released/

— Tor Project

“ At every stage, we’ve encountered way fewer bugs than during comparable C development. The bugs that we have encountered have almost all been semantic/algorithmic mistakes (real programming issues), not mistakes in using the Rust language and its facilities. Rust has a reputation for being a difficult language with a picky compiler - but the pickiness of the compiler has been a great boon. Generally speaking, if our Rust code compiles and passes its tests, it is much likelier to be correct than our C code under the same conditions.^a

^ahttps://blog.torproject.org/arti_100_released/

— Tor Project

“ Since 2016, we’ve been tracking all the security bugs that we’ve found in Tor, and it turns out that at least half of them were specifically due to mistakes that should be impossible in safe Rust code.^a

^a<https://blog.torproject.org/announcing-arti/>

— Tor Project

So, does Rust is replacing C and C++? Not really! Rust is being used in conjunction with C and C++. For example, the Linux kernel is primarily written in C, but it has started to include Rust code for certain components. Primarily, for new components where memory safety is critical.

“ We feel that Rust is now ready to join C as a practical language for implementing the kernel. It can help us reduce the number of potential bugs and security vulnerabilities in privileged code while playing nicely with the core kernel and preserving its performance characteristics.^a

^a<https://security.googleblog.com/2021/04/rust-in-linux-kernel.html>

— Google

Rust is being used in:

- Linux⁵⁰ (kernel)
- Android⁵¹
- Firefox⁵²
- Tor⁵³
- Cloudflare⁵⁴
- Chromium⁵⁵
- Apple⁵⁶
- Discord⁵⁷
- Dropbox⁵⁸

Reading List

- Memory Safe Languages: Reducing Vulnerabilities in Modern Software Development (Joint NSA and CISA document about memory safety)⁵⁹
- MISRA C: Guidelines for the use of the C language in critical systems⁶⁰
- Safe C++⁶¹

6.9.11 Telegram Emoji Suggestion

One day, I was chatting in a secret chat on Telegram and noticed that it suggested emojis for what I was writing.

At first, I thought that since it offers emoji suggestions in different languages and for various words, it must be sending my words to a server. The server would then determine which emojis to display and send that list back to me, which the Telegram app would show as suggestions.

⁵⁰<https://security.googleblog.com/2021/04/rust-in-linux-kernel.html>

⁵¹<https://security.googleblog.com/2021/04/rust-in-android-platform.html>

⁵²<https://hacks.mozilla.org/2017/08/inside-a-super-fast-css-engine-quantum-css-aka-stylo/>

⁵³<https://blog.torproject.org/announcing-arti/>

⁵⁴<https://blog.cloudflare.com/cloudflare-workers-as-a-serverless-rust-platform/>

⁵⁵<https://security.googleblog.com/2023/01/supporting-use-of-rust-in-chromium.html>

html

⁵⁶<https://preettheman.medium.com/this-is-what-apple-uses-rust-for-37ddfb9e9237>

⁵⁷<https://discord.com/blog/why-discord-is-switching-from-go-to-rust>

⁵⁸<https://blogs.dropbox.com/tech/2016/06/lossless-compression-with-brotli/>

⁵⁹https://media.defense.gov/2025/Jun/23/2003742198/-1/-1/0/CSI_MEMORY_SAFE_LANGUAGES_REDUCING_VULNERABILITIES_IN_MODERN_SOFTWARE_DEVELOPMENT.PDF

⁶⁰<https://www.misra.org.uk/>

⁶¹<https://safecpp.org/>

6.10 End of the Chapter Questions

But then I realized that this doesn't make sense! Secret chats on Telegram are designed to ensure that messages can only be read by me and the other person involved. Messages shouldn't be sent to a server! So, this seemed like a security issue.

To confirm whether my messages were actually being sent to the server, I disconnected my internet and searched for something I had never looked up before (to ensure it wasn't in the cache). I found that it still provided emoji suggestions.

That's when I understood that the processing was happening on my device, and nothing was being sent to the server. (Of course, a better way to check would have been to analyze the connections and packets.)

This is an example of the hacker mindset I mentioned earlier—being able to understand events and identify potential security and privacy issues that may arise.

6.10 End of the Chapter Questions

Problem 6.1 A program access to many files at once and after that, entropy of the files increases. What is the program doing? Might it be malicious?

Problem 6.2 An online website, provides free obfuscation of JavaScript code. A user uploads a JavaScript file to the website, and the website returns an obfuscated version of the code. The user then includes the obfuscated code in their web application. Is there any security concern with this approach? Explain.

Problem 6.3 One threat to future security is the rapid expansion of the Internet of Things (IoT). Why might such accelerated growth in the IoT sector become a serious challenge for society? Also, please provide one concrete example of a problem that could arise.

Problem 6.4 In a sensitive system, people can recover their account if they forget the login password with a secret and a code sent to their phone number using an SMS. Is there any wrong security implementation here?

```
1 def recovery(inpUserName, inpSecret):
2     if (isExist(inpUserName) and inpSecret == secret):
3         sendSMS()
4     else:
5         print('User Not found')
6
```

Listing 6.19: Forget password code

Problem 6.5 You are the security code checker of an open-source project. The following code is submitted, do you accept it? Explain.

```
1 def check_credential(name: string, pass: string, parameter="":  
    string)  
2 {  
3     // previous codes ...  
4     if name == "Jason" and password == "Wild":  
5         run(parameter)  
6     // rest of the codes ...  
7 }  
8
```

Listing 6.20: credential check

Problem 6.6 Does this code implement something in an insecure way? Explain.

```
1 void foo()  
2 {  
3     int length;  
4     scanf("%d", &length);  
5  
6     char inp[length];  
7     scanf("%s", inp);  
8  
9     char *tmp = (char *)malloc(length);  
10    memcpy(tmp, inp, length); // tmp <- inp  
11    printf("%s\n", tmp);  
12 }  
13
```

6.11 End of the Chapter Solutoins

Solution 6.1 Encryption produce random outputs (high entropy). The program is likely doing encryption (May be ransomware)

Solution 6.2 Yes, there are several security concerns with this approach:

1. Malicious Code Injection: The obfuscation service could inject malicious code into the obfuscated JavaScript. Since the user is including the obfuscated code in their web application, any malicious code added by the service would run in the context of the user's application, potentially leading to data theft, unauthorized access, or other malicious activities.

2. Trustworthiness of the Service: The user is relying on a third-party service to handle their code. If the service is compromised or operated by malicious actors, they could exploit this trust to introduce vulnerabilities or backdoors into the user's application.

3. Lack of Transparency: Obfuscated code is difficult to read and understand. This lack of transparency makes it challenging for developers to review the code for security vulnerabilities or malicious behavior before including it in their application.

4. Potential for Data Leakage: If the obfuscation service collects data about the code being uploaded (e.g., through logging or analytics), sensitive

information contained in the JavaScript file could be exposed to unauthorized parties.

Overall, using a third-party obfuscation service introduces significant risks that could compromise the security of the user's web application.

Solution 6.3 Because companies fiercely compete to launch smart products quickly and cheaply—and to claim market leadership—they often ship devices using unvetted, proprietary, and insecure protocols.

For example, you wake up, shower, and expect your smart kettle to have brewed your tea, only to find the device locked by ransomware and unusable—and your home's power outlet won't even turn it on.

In another scenario, many Internet-connected gadgets are poorly secured and become part of malware networks and botnets, which attackers exploit to launch larger, more powerful DDoS campaigns.

And since nearly every appliance today is always online, malware can spread rapidly: it finds one vulnerable device, compromises it, and then uses that foothold to infect others in turn.

Solution 6.4 First of all, SMS is not a secure way to send sensitive data. The SMS protocol is inherently insecure. Furthermore, SMS can be via malware.

The problem of the code is about timing attack. A brute-force attack with measuring time difference (because `isExist()` function) can find the existed usernames.

When we are working with sensitive data, we should do things in a constant time. So regardless of correctness of username and secret, we should say that if the data is correct, code is on the way. A safer way to do is:

```
1 def def recovery(inpUserName, inpSecret):
2     flag = true
3     flag = flag and isExist(inpUserName)
4     flag = flag and safeCompare(inpSecret, secret)
5     # start a thread for sendSMS(flag)
6     # start a thread for foo()
7
8 def sendSMS(flag):
9     if flag:
10         sendCode()
11
12 def foo():
13     print('If the information you have provided is correct, the
14         code is on the way!')
```

Listing 6.21: Safer Forget password code

Solution 6.5 Again timing attack. But there is something much bigger here. This code checks secret data hardcoded directly in the source code. anyone read the source code, can find the actual login credential.

Additionally, this code is likely a backdoor. Because it can let anyone who

6.11 End of the Chapter Solutoins

have this credential, run something. Never ever let users run something on your computer.

Solution 6.6 `int` is not suitable for `len`! `size_t` is. (+0.25 bounus)

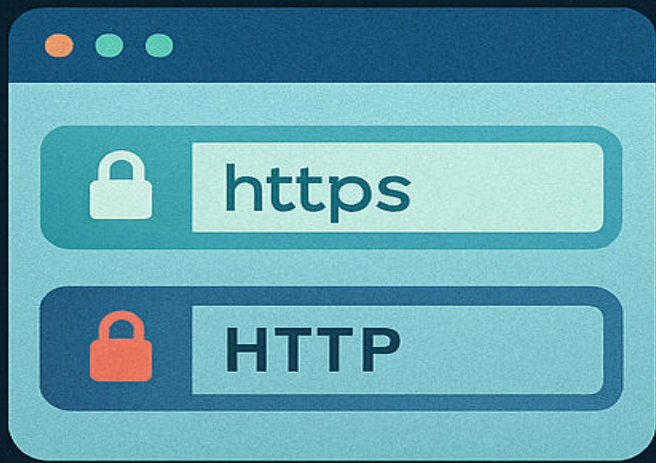
`scanf("%d", &length);` → subjected to non-valid input. (-1, integer overflow, or non-integer input) → 0.25

`scanf("%s", inp);` → gets input to the end of line. This causes buffer overflow (if line length is larger than `length`. → non-valid input → 0.25

`memcpy(tmp, inp, length);` → Think about a situation where `inp` is only 10 character (the end of line occurred after 10 characters in `scanf()`) and given `length` is 100. This functions copies 100 character from start of `inp` into `tmp` which leaks the other 90 characters in the memory. (Characters following the end of `inp`) → 0.5

```
1 void SecureFoo()
2 {
3     size_t length;
4     // securely read the length
5     secure_scanf("%zu", &length, min_, max_);
6
7     char *inp = (char *)malloc(length);
8     // get input of length 'length'
9     size_t size = secure_scanf("%s", inp, length);
10
11     char *tmp = (char *)malloc(length);
12     secure_memcpy(tmp, inp, size); // tmp <- inp
13     printf("%s\n", tmp);
14 }
15
```

6.11 End of the Chapter Solutoins



6.11 End of the Chapter Solutoins

7.0 Network

7.1 Look-alike Domain

Imagine you're trying to access your bank's website, and you type in the URL. However, you make a typo and end up on a site that looks identical to your bank's website. You enter your username and password, and the site accepts them. You think you've logged in, but in reality, you've just given your credentials to a malicious actor.

This is a common tactic used by cybercriminals to steal your information. They create websites that look identical to legitimate ones, tricking you into entering your sensitive data. This is known as a look-alike domain attack.

Sometimes they use tricks like offering a discount to prevent you from carefully checking the URL and instead clicking on the link.

- You can't be phished with a one-time password!

→ That's not true!

- To determine if a page is fake, enter incorrect card information; if you don't get an error, it's legitimate!

→ That's not a valid approach!

Please don't get caught up in these incorrect methods! The best way is to check the domain (though there are conditions, such as using a reliable browser, which I'll explain later). It's all about the website's domain. For example, in Iran, all payments must be made through the Shaparak system (**shaparak.ir**). If you see a website that doesn't use the Shaparak system, it's likely a scam. Always check the domain!

7.1.1 How to determine the domain of a website?

The domain is the last part before the .com, .ir, or .org, etc., and also the part just before the last dot. In other words, it's the last segment before the character "/" and the part just before the final dot.

What does this mean? Let's look at some examples:

```
https://www.eff.org/about
```

7.1 Look-alike Domain

The domain is `eff.org`. Why? Because it ends with `eff.org` followed by the character `/`.

`https://www.eff.org/about/contact`

The domain here is still `eff.org`.

`https://supporters.eff.org/donate/join-eff-4`

The segment just before the last dot (the last dot is highlighted in red, which is the dot after `supporters`).

When we say the Shaparak domain, we mean something like this:

`https://sep.shaparak.ir/`

This is the Shaparak domain.

The domain of the bank payment gateway page must end with `shaparak.ir`. Here are some examples:

`https://www.bmi.shaparak.ir/`

In each example, I highlighted the domain in green and the dot from which the domain starts in red. It is almost always something like `.shaparak.ir`. Anything else is a phishing attempt:

`https://www.khariid.ir/`

I said it must be Shaparak, not `khariid` or any other domain! So this is phishing!

`https://www.shaparak.xyz/`

I said it should be `shaparak.ir`, not `.xyz`! So this is phishing.

`https://www.bmi-shaparak.ir/`

I said it should be something like `.shaparak.ir`. That's it! Here, a hyphen was used instead of a dot, which makes it phishing.

`https://www.bmi.shapalak.ir/`

In this case, a lowercase `l` was used instead of `r`, which is phishing. You might not pay attention to the address and read it incorrectly.

`https://www.bmi.shaqarak.ir/`

7.1 Look-alike Domain

Again, a technique was used to confuse us. A **q** was placed instead of a **p**, which is phishing.

`https://www.bmi.snaparak.ir/`

Here, an **n** was used instead of an **h**, which is phishing.

`https://www.shaparak.lr/`

Here, a lowercase **l** that looks like a capital **I** was used instead of an **i**. That's why they always say not to click on links. It looks like Shaparak, but when you read it, you might think it's a capital **I** when it's actually a lowercase **l**. Never, ever, ever click on links. Instead, type the address letter by letter into your browser.

1. Never download programs from untrusted sources. Hackers often try to lure you into installing software from unreliable sources, such as links sent via messages. For example, they might send a text saying, "A complaint has been registered against you in the government system. Follow the link below for more information," encouraging you to click!

Do not click on such links under any circumstances. Similarly, avoid clicking on links in messages claiming "free internet," "tax reduction tricks," and so on. Malicious software can easily steal the one-time password sent to you via SMS or other formats.

- Some banking apps are not available on the Google Play Store. What should we do?

- Download them from the official website of the bank. (Pay attention to the bank's domain!) Ask someone knowledgeable about what the bank's official domain is. You can also contact the bank's customer service.

Sometimes, malicious software is sent to you via files through WhatsApp or other applications. Under no circumstances will any government agency send you a message through WhatsApp. For instance, someone might falsely pose as a postal worker and claim, "I have brought a notification letter from the judiciary, and since you weren't home, I had to open it and provide you with the tracking code so you can follow up online using the link below."

They will give you a phishing link. When you enter the site, it will ask you to pay a fee of \$2. While \$2 may not seem like a lot, you may not realize that the actual amount being charged is \$200.

2. Always check that the beginning of the website address starts with HTTPS. If it starts with HTTP, it indicates a phishing attempt. However, just because it starts with HTTPS doesn't necessarily mean it's safe.

You can also enable the HTTPS-Only mode in your browser settings to ensure that you only visit secure websites. You will receive a warning if you try to visit an insecure site.

7.1 Look-alike Domain

Some people use their main card, which has a significant amount of money, for online purchases. However, this is completely wrong. Instead, do the following:

- One bank account should hold a significant amount of money but should not have a debit card or online banking password.
- Another account can have a debit card but no online banking password, and it should contain a small amount of money for everyday expenses like buying fruits and groceries.
- A third account should have an online banking password and a small balance for online purchases.

This way, if you fall victim to skimmers (devices that copy your banking information and later use it to withdraw from your account) in the real world, you won't lose much since that account doesn't hold a lot of money! Yes, scams can happen in the real world too!

Similarly, if you get scammed during an online purchase, since your card is separate and only a small amount of money is in that account, you likely won't lose much!

Whenever you need to make a larger purchase, transfer the amount you want to spend to the card you use for online shopping. This way, there will always be just enough money on that card for your purchases, but not a large amount!

4. Do not make payments on public computers, such as those in internet cafes! There is a very high likelihood that their systems are filled with malware (viruses)! I say this seriously! Public computers, including university systems, are often infected. Instead, use your personal phone for payments whenever possible.

5. Use a reliable and updated browser. Good browsers include Firefox and Brave. Chrome is also good, but not for privacy. It is good for security, but not for privacy. Safari is also a good browser. However, I recommend Firefox and Brave. They are both open-source and have good privacy settings.

These browsers are secure and provide good protection against attacks. For example, Firefox (not Brave) makes it very easy to identify the domain. Look at the images below! Firefox highlights the domain and dims the rest of the website address, making it significantly easier to identify the domain: The domain in



Figure 7.1: Firefox's domain highlighting feature

figure 7.1 is `listings-94915329.com` and not `airbnb.com`.

Here is another example: The domain in figure 7.2 is `eff.org`.

6. Use Private Window or Incognito Mode. In Private Window mode in Firefox and Brave, some (not all) malicious capabilities related to certain things

7.1 Look-alike Domain

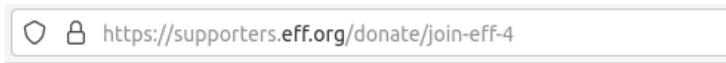


Figure 7.2: Firefox’s domain highlighting feature

(like extensions) are disabled or not allowed to use Private Window at all! This might help you **a bit**.¹

7. Avoid installing unnecessary extensions. Extensions can be dangerous. For example, they can read your browsing history, steal your passwords, and more.

You can also keep the browser you use for payments separate from others. For instance, you might use Firefox exclusively for online transactions.

8. Using a virtual keyboard can be helpful. As mentioned earlier, keyloggers primarily target physical keyboards (though not always).

9. Pay attention to the information in the SMS containing the one-time password. When the one-time password arrives, it will indicate the amount and the company involved, for example, “An amount of X for Y company.” Ensure that the amount matches the website and that the name of the company you are making the payment to is correct and consistent!

(Note that the one-time password generation apps provided by banks are more secure than SMS-based one-time passwords. Many phishing attempts occur because malware (viruses) can steal the SMS containing the one-time password. This is why we advise against installing apps from sources other than the Google Play Store and similar platforms. Additionally, SMS is an inherently insecure protocol.)

10. Use antivirus software. It can help you detect malware. However, it is not a panacea. It is not a complete solution. It is just a tool to help you. It is like a seatbelt—it can prevent many injuries, but it doesn’t make you invulnerable. The same applies to antivirus software. It can help you, but it is not a complete solution.

11. On Samsung phones, go to: **Settings => Accessibility => Installed Services**

Settings => Biometrics and security => Device admin apps

Check to ensure that no suspicious apps are enabled in this section. If they are, they can perform various actions, such as reading and stealing your dynamic password SMS.

On Xiaomi phones, go to: **Settings => Privacy Protection => Special permissions => Device admin apps**

12. Most phishing sites have a short lifespan. This means that after a few hours or days, they are added to a blacklist, and both antivirus software and browsers will warn you that the site is phishing. Therefore, one way to check is

¹Private Window or Incognito Mode won’t hide your IP address or search history from your ISP (Internet Service Provider) or the websites you visit.

7.1 Look-alike Domain

by looking at the domain age. For example, the payment domain of a certain bank should not have been created just a month ago. You can check the domain age by searching for the site's WHOIS information.²

13. Never click on links. Instead, type the address directly into your browser. Have you noticed that sometimes when you enter a website, it says something like:

"Click here to see more content." "Linux is actually a kernel."

In the first case, by clicking on the word "here," and in the second case, by clicking on the word "Linux," you are taken to a website where the link is hidden beneath the words "here" and "Linux." If you want to see the link to that website, you can hover your mouse over it and look at the bottom of the page to see the actual address. Additionally, you can right-click on the link and select "Copy link address" to see the actual address. This way, you can see where the link will take you before clicking on it.

Additionally, phishing can occur through QR codes. For example, you might see a QR code on a poster that says, "Scan this code to get a discount." When you scan the code, it takes you to a phishing site. So, be careful with QR codes as well. Always use tools that allow you to see the actual address before clicking on it.

Moreover, when you type the address yourself, you can avoid phishing with Unicode characters.³ We have different characters that look the same. For example, the letter a can be written in different ways (different languages have similar characters that can be used to trick you. You might think you're clicking on the right website when in fact you're not).

14. See *Phishing campaigns are using AMP URLs to avoid detection*⁴

15. See <https://web.archive.org/web/20220429195039/https://www.youtube.com/watch?v=vPJ6irUDmHI>

This is a very important topic. Therefore, I will pose a few questions. Please think about them before reading the answers below.

I have entered the bank payment gateway. Which addresses do you think, if present on my payment page, would indicate that it is not a phishing attempt?

- 1) <https://www.sep-shaparak.ir/>
- 2) <https://www.sep.shapatak.ir/>
- 3) <https://www.sep.shaparak.ir.khariid.ir/>
- 4) <https://www.sep.shaparak.jf/>
- 5) <https://www.sep.shparak.ir/>
- 6) <https://www.sep.shaparak.ir.shaparak.jr/>
- 7) <https://www.sep.shaparak.ir/>

²However, some hackers, especially those who are more professional, can buy old domains to avoid this check.

³Phishing with Unicode Domains: <https://www.xudongz.com/blog/2017/idn-phishing/> -> However, the address bar of browsers is now fixed and shows the associated ASCII characters.

⁴<https://www.threatdown.com/blog/phishing-campaigns-are-using-amp-urls-to-avoid-detection/>

7.1 Look-alike Domain

- 8) <https://www.sep.pardakht-shaparak.ir/>
- 9) <https://www.sep.shaparak-es.ir/>
- 10) <https://www.sep.shaparak.ir.xyz/>
- 11) <https://www.sep.shapraak.ir/>
- 12) <https://www.sep.shaparak3.ir/>
- 13) <https://www.pytnxjwsafe.ir/>
- 14) <http://www.shaparak.ir.snaparak.ir/pay/hsju26>
- 15) <https://www.pardakht-hesab.ir/shaparak.ir>
- 16) <https://www.shaparak-ir.ir/>
- 17) <https://www.shaparakir.ir/>
- 18) <https://shaparak.ir.kdsfk.sfjdsalfsaf.whierlw.asovlwervn.gnbv.xsulgvvf.com/>
- 19) <https://www.sep.shaparak.ir/>

Answer

Only the last one is not phishing; the rest are all phishing attempts. Let's review them one by one (to help you become more familiar with domains, I will highlight the domain in green and the dot where the domain starts in red):

1. We said it should be something like `.shaparak.ir`, but this is `.shaparak-ir`. A hyphen?! That's incorrect!

`https://www.sep-shaparak.ir/`

2. `shapatak??` It should be `shaparak`, but they replaced the letter `r` with `t` to trick us!

`https://www.sep.shapatak.ir/`

3. This is `khariid.ir`, not `shaparak`! Notice that the last part before `.ir`, `.com`, or `.info` is the domain!

`https://www.sep.shaparak.ir.khariid.ir/`

4. Why `jf`? It should be `.ir` instead!

`https://www.sep.shaprak.jf/`

5. `shparak.ir`? Where did the `a` after `sh` go? It should say `shaparak.ir`!

`https://www.sep.shparak.ir/`

6. Again, pay attention to the domain. The domain of the site is `shaparak.jr`! `.jr`? It should be `.ir`, meaning `shaparak.ir`!

`https://www.sep.shaparak.ir.shaparak.jr/`

7. Why `shaprak.ir`? Where did the `a` after `p` go?! Look! The `a` is missing after `p`, so it's incorrect! It should be `shaparak.ir`, not `shaprak.ir`!

7.1 Look-alike Domain

<https://www.sep.shaprak.ir/>

8. The domain is `pardakht-shaparak.ir`. They used a hyphen to trick us into not noticing!

<https://www.sep.pardakht-shaparak.ir/>

9. The domain is `shaparak-es.ir`. What is `es-?`?! The domain should be `shaparak.ir`, that's it!

<https://www.sep.shaparak-es.ir/>

10. Why `.ir.xyz`? That's incorrect!

<https://www.sep.shaparak.ir.xyz/>

11. Why are there two `as` after the `r`? Why `shapraak.ir`? Two `as` are a mistake!

<https://www.sep.shapraak.ir/>

12. Shaparak 3? We never had a number! What kind of domain is `shaparak3`? That's incorrect!

<https://www.sep.shaparak3.ir/>

13. What kind of domain is this? It's not even `shaparak`! So it's completely wrong! What does `pytnxjwsafe.ir` even mean?

<https://www.pytnxjwsafe.ir/>

14. This has several issues: One: Why is it `http` and not `https`? The fact that it's `http` indicates that it's phishing! That's enough to realize it's a scam! Two: Why is the domain `snaparak.ir`? Why isn't it `shaparak`? They used a technique to replace the letter `h` with `n`!

<http://www.shaparak.ir.snaparak.ir/pay/hsju26>

15. The domain is `pardakht-hesab.ir`, and it's not `shaparak` at all! - Oh, but what about that `shaparak.ir` at the end? + That's after the character `/` and doesn't matter!

<https://www.pardakht-hesab.ir/shaparak.ir>

16. The domain is `shaparak-ir`! Did you notice that a dot is different from a hyphen? Here, they tried to trick us with a hyphen!

<https://www.shaparak-ir.ir/>

17. The domain is `shaprakir.ir`! It's not `shaparak.ir`!

7.1 Look-alike Domain

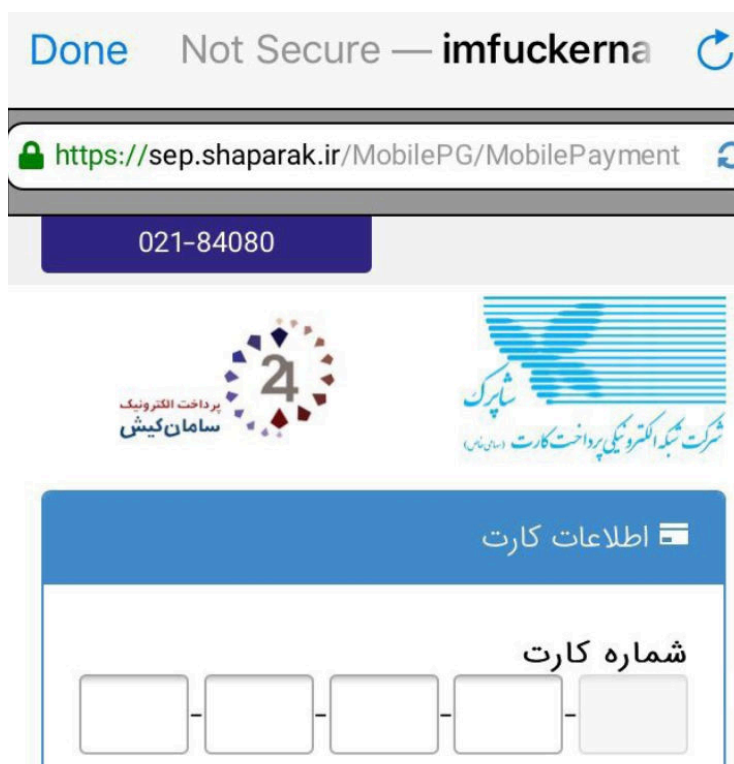


Figure 7.3: A phishing site

<https://www.shaparakir.ir/>

18. This one is deliberately long to fit in the browser's address bar on mobile devices and not show completely, making you mistakenly think it's really shaparak, when it is not!

<https://shaparak.ir.kdsfk.sfjdsalfsaf.whierlw.asovlwervn.gnbv.xsulgvvf.com/>

19. This one is completely correct! It is something like sep.shaparak.ir. The domain is entirely valid and is shaparak.ir.

<https://www.sep.shaparak.ir/>

See figure 7.3 The designer of this site was really creative. They placed a fake image to make you think you're on the Shaparak site, but in reality, you didn't pay attention to the address bar of your browser and only saw the fake image.

Sometimes instead of text, an image containing text is sent to your email. This method bypassed many spam filters which operate based on text.

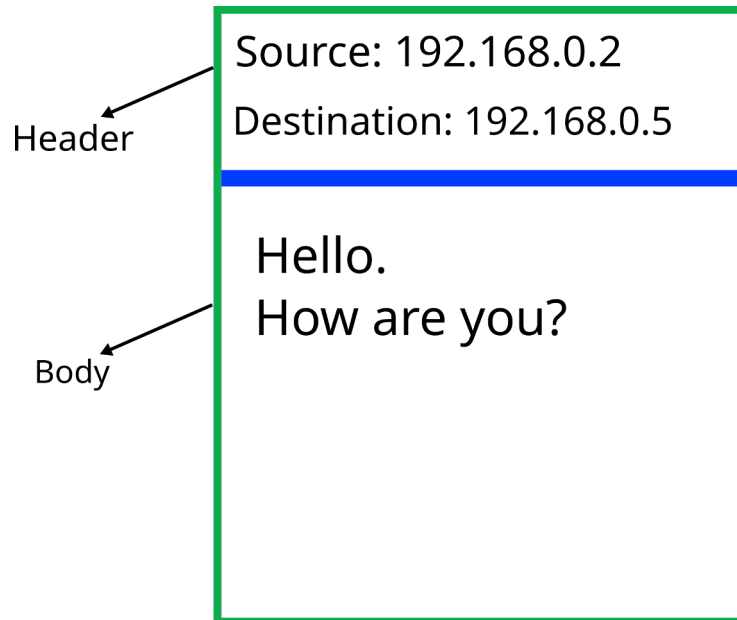


Figure 7.4: A packet

7.2 DNS

The internet is a space where everyone is connected. If I want to send a message to someone in the physical world, I need to go to the post office, write a letter, and specify the sender's and recipient's addresses. On the internet, it's quite similar. Everyone has an address. If I want to connect to Google, I find Google's address and send a message to that address! This address is called an IP address, or Internet Protocol Address.⁵

Each packet has a header and a body. The header includes the sender's and recipient's addresses, the protocol used, connection parameters, and the packet size. (Remember this! We will come back to it later.) The body contains the actual data being sent. The header is like the envelope of a letter, while the body is like the content of the letter itself.

But here's the thing! Remembering the addresses of all websites is really hard, right? Also, addresses on the internet look something like this:

61.156.22.11

It's quite difficult to memorize all those numbers for every site, isn't it? Well, here they tell us that you don't need to memorize numbers! Instead, just

⁵That's why it's said that it's better for no one to know your IP address, because it's like knowing your home address!

7.2 DNS

remember the English names. You can write something like `signal.org`, and I will find out what the address behind `signal.org` is! In other words, we assign a series of names to it, and later we go find it ourselves.

- How does it find that?

→ It's like a phone book. There are certain places that connect these alphabetical addresses to their numerical addresses. It's something like this:

Alphabetical Address	Numerical Address
<code>signal.org</code>	<code>123.123.123.123</code>
<code>google.com</code>	<code>12.34.124.123</code>
<code>eff.org</code>	<code>45.123.11.14</code>

Table 7.1: A simple example of a DNS table

Note: To obtain the IP address of a server (if it is not behind proxies), you can use the command `ping`. For example:

```
ping signal.org
```

This is why a request called DNS is sent, asking for the IP address of a site. For example, "I want to access the Mozilla site. Please give me its IP." The other side accepts the request and sends it back.

- Who provides us with this IP address for Mozilla?

→ We'll get to that later!

However, there's a problem! This DNS request is sent in plain text, meaning it's transmitted in a completely normal format using English letters, without any encryption. This means that others (anybody in the same network as I am and ISP and routers) can see which website I intended to visit or which server I wanted to connect to. It's important to note that it's not just about accessing a website; sometimes, an application may need to communicate with a server as well.

This situation poses a threat to individuals' privacy because ISPs and other parties can see which sites I wanted to access and even alter that request.⁶

We can use other DNS resolvers that encrypt the DNS communication. (e.g. DNS over HTTPS) For example, we can use `9.9.9.9` or `1.1.1.1`.

When you use HTTPS, the body of the request is encrypted.

But should we use any provider?

No! For instance, if we use Google, we are essentially providing them with a list of all the sites we visit. Let's examine one of these providers.

⁶One of the methods used for filtering and banning websites is manipulating the DNS request, resulting in us receiving an incorrect IP address. For example, in Iran, we might receive the IP address of a filtering page, such as `10.10.34.35`, which leads us to a filtering page instead of the intended website. (Peyvandha page: https://fa.wikipedia.org/wiki/%D8%B5%D9%81%D8%AD%D9%87_%D9%BE%DB%8C%D9%88%D9%86%D8%AF%D9%87%D8%A7)

7.2 DNS

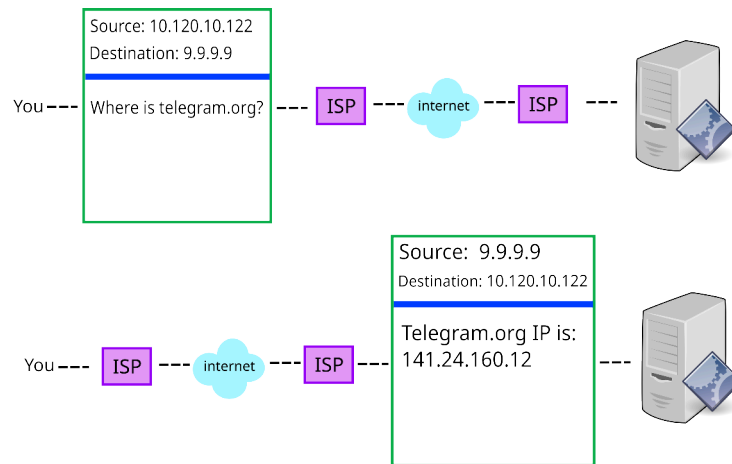


Figure 7.5: A DNS request

The information I gathered about Quad9 is sourced from their website and is licensed under the CC BY-NC SA 4.0 License (Attribution-NonCommercial-ShareAlike 4.0 International). You can find more details on their official website at <https://www.quad9.net>!

Furthermore, these DNS resolvers block requests that lead to websites containing malware or harmful content. This provides a level of protection against malware, helping to keep you safer online:

“ Quad9 routes your DNS queries through a secure network of servers around the globe. The system uses threat intelligence from more than a dozen of the industry’s leading cybersecurity companies to give a real-time perspective on what websites are safe and what sites are known to include malware or other threats. If the system detects that the site you want to reach is known to be infected, you’ll automatically be blocked from entry - keeping your data and computer safe.

— Quad9

You may ask how much of an impact does it have? → A study has shown that:

“ DNS firewalls could have mitigated one-third of the incidents we studied^a

^ahttps://www.globalcyberalliance.org/reports_publications/

— Global Cyber Alliance

7.2 DNS

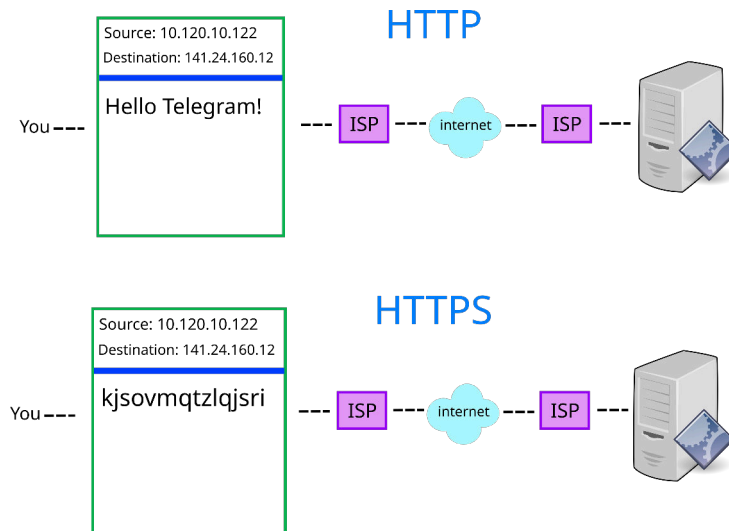


Figure 7.6: Using a encrypted DNS over HTTPS

“ [measuring-the-economic-value-of-dns-security/](#)

— Global Cyber Alliance

Also we have:

“ Quad9 commits to obey the law in any country in which it operates. Therefore, it will only operate in countries with a rule of law compatible with Quad9’s ethics and moral duty to protect users. If a government were to use national law to attempt to force Quad9 to act in a way that would harm users (such as collecting information that might de-anonymize an at-risk individual), Quad9 would withdraw from operations in that country. This does not mean users within that country would be prevented from using the Quad9 service (unless the country itself prevented them); the service will operate from locations in nearby countries.

— Quad9

One of the good things I noticed in the FAQ section on the Quad9 website is that they addressed a question that many people might have: how do they benefit from this service? This is something that should always be considered in discussions about privacy. They provided an answer to this concern:

7.3 Virtual Private Network (VPN)

“ Why do threat intelligence (TI) providers share their data with Quad9, and what do they get out of it?
Quad9 gives anonymized telemetry back to the TI providers only for the malicious domains they share with Quad9. This telemetry never includes the source IP information of the user.

— Quad9

“ When your devices use Quad9 normally, no data containing your IP address is ever logged in any Quad9 system. Connections can employ encryption if your system supports it, and the entire Quad9 platform has been designed to be GDPR-compliant since its first public announcement in 2017.

— Quad9

Don't forget to change your DNS settings on your router, as well as on your operating system and browser.

● Does this prevent filtering?

→ No! It doesn't prevent filtering. Eventually, the packets will have the IP address of the filtered site in their header. The ISP will see that and filter it. Blocking users from accessing a site is done in several ways. One of them is filtering the DNS request. Another is dropping the packets that have the IP address of the filtered site in their header.

7.3 Virtual Private Network (VPN)

Many of us may have used VPNs with names like:

- Super VPN
- Super Fast VPN
- Super Fast Unlimited VPN
- Turbo Fast VPN
- Free VPN
- Free Proxy

However, we may not realize that almost all of these VPNs are insecure!

7.3.1 Why should I be careful when choosing a VPN?

For example, let's learn about *Betternet*.

7.3 Virtual Private Network (VPN)

It had an IP leak issue, which is one of the worst problems a VPN can have!⁷

“ Two VPN apps (Flash Free VPN [18] and Betternet [19]), which combined have more than 6M installs, have the highest number of embedded tracking libraries: 11 and 14 respectively.^a

^a<https://research.csiro.au/isp/wp-content/uploads/sites/106/2016/08/paper-1.pdf>

— Tor Project

Is that all? No! What could be worse than this?! Yes, 13 antivirus programs have identified Betternet as malware.

Some VPNs/software are created by governments to spy on their citizens.⁸

7.3.2 Why should we even use a VPN?

Generally speaking, when you visit a website, the government, your ISP (Internet Service Provider), and the website itself can automatically gather the following information: (Of course, they can learn other things through various methods, but these are the most obvious ones!)

If the website is using HTTP and you are not connected to a VPN, they can see everything! Literally, everything! It's as if they are sitting right next to you, watching everything you do. They can even alter the information you see on your screen!

In the case of an HTTPS site: They can see the address of the website (not the exact page on the website) you want to visit. (We discussed this in detail in the DNS section.)

The big question is: how can you determine if the connections within an app to its website are HTTP or HTTPS? Normally, you can't!

- So why is it bad for websites to see where we want to go?

→ Because people's browsing habits are unique and specific!^[44] Let me clarify this for you. Think about your daily routine in the real world. For example, you usually wake up at 7 AM, then go to work at 8 AM. You do certain tasks at work, and finally, you come home around 5 PM, have something to eat, do a few things, and go to bed at a certain time.

Now, let's say you put on a mask and change your clothes. But does that make you anonymous? No! Everyone knows that the person whose life follows that routine is you. So, if someone is familiar with your lifestyle, even if you go to Austria, they can still figure out that the person who wakes up at 7 AM and does certain things is you.

⁷<https://restoreprivacy.com/vpn/reviews/betternet/>

⁸See "Tech Tuesday: Eye Spy - The Dangers of Legal Malware": <https://blackpointcyber.com/resources/blog/eye-spy-the-dangers-of-legal-malware/>

7.3 Virtual Private Network (VPN)

The same applies to browsing the web. Our browsing habits are unique. For instance, if I visit a specific website every morning, even if I change my laptop and internet connection tomorrow, my browsing habits will still reveal that I am the person visiting those particular sites!

There are very few people in the world who browse the web like I do, visiting the same sites and going to specific websites at certain times of the day. You can see this in detail in the research from Mozilla.[44]

See the image below:

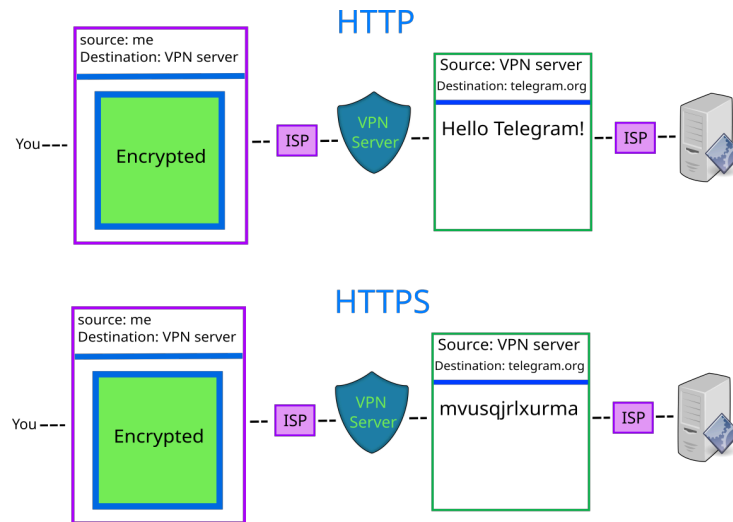


Figure 7.7: How packets look like when using VPN

In the case of HTTP without using a VPN: Explanation: The government, your ISP, and anyone else on your network (like Wi-Fi) can read your messages! They can see the exact content of your messages.

In the case of HTTP with a VPN: Your message is placed in a secure box that no one can read! This box is sent to the computers of the VPN provider. From there, since things are not filtered, you connect to the website without any restrictions. Essentially, the VPN takes your data and communicates with that website on your behalf. Since there are no filters on that end, we can think of it as a “filter breaker”—something that bypasses censorship! However, from the VPN provider’s computers onward, everything remains as it was before.

It prevents interference with your privacy (spying) by the government and your ISP. When they try to read your message, they’ll say, “Oh! We can’t read what’s in that box!” (Of course, this is true only if you use a suitable VPN, which we will explain later.)

Better Explanation: Your message is encrypted on your device, and then this encrypted message leaves your device and goes to the VPN server. Along the way between you and the VPN provider’s servers, because the data is encrypted,

7.3 Virtual Private Network (VPN)

it prevents interference from the ISP and some attacks like man-in-the-middle attacks! (Not all types, but some!) Now, at the VPN servers, this encryption is decrypted, and the message continues on its way to the destination you want.

However, the VPN must be trustworthy! Otherwise, just as the ISP and the government could alter our messages or steal our information, a VPN could do the same! While it prevents ISP interference, what if the VPN itself is untrustworthy? That's why we're providing guidance on which VPNs are reliable and trustworthy!

Now, what if the website is HTTPS?

In the case of an HTTPS website without using a VPN: **Explanation:** The government and your ISP cannot read the message. This is because HTTPS acts like that secure box.⁹ However, they can still know which website we are visiting!

In the case of HTTPS with a VPN: The government won't see which website we want to visit! Once the data reaches the VPN server and is decrypted, the HTTPS encryption still protects our data until it reaches the website! It can see the amount of data you transmitted and how much time you were connected to the VPN though.

From us to the VPN server:

`VPN_Encryption(HTTPS(plain_text))`

From the VPN server to the desired website:

`HTTPS(plain_text)`

Now, if you use an untrustworthy VPN, the information we mentioned can be accessed by the VPN provider, just like the information you would give to your ISP and the government.

While a VPN prevents the ISP and the government from seeing what you are doing and is often used to connect to HTTP sites to prevent ISP spying, what happens if the VPN itself is untrustworthy and decides to act maliciously? That's why it's crucial to choose a reliable VPN!

7.3.3 What Data Can They Collect

The information you typically provide when installing an app includes various device details. This includes hardware information, unique device identifiers, and other related data.

- What to do?

- You can use apps that collect less information, as experts believe these options are better.

- Some providers offer a file called a config file. You can use this file to establish your VPN connection through other applications. This way, you can utilize

⁹The government can launch various attacks. However, by default, it is secure, and they cannot read the messages.

7.3 Virtual Private Network (VPN)

clients that either collect no data or significantly less data.

7.3.4 When choosing a VPN (or generally an app), what should you pay attention to?

An important note is that you should not rely solely on one factor. For example, if a VPN is open-source, it doesn't necessarily mean it's good. You need to consider multiple factors together. Here are some of the most important ones:

Don't be fooled by reviews!

Many people are unaware that reviews can be manipulated. There are services that allow companies to buy fake reviews. For example, you can pay a few dollars to get 1000 positive reviews written by real people. Those people may be infected with malware. Additionally, there are centers in countries with low wages where people are hired to write fake reviews. They are paid a few cents for each review they write. So, if you see a VPN with 1000 positive reviews, it doesn't necessarily mean it's good. It could be that the company paid for those reviews. Look for other indicators of trustworthiness.

What version should you use?

It's best to use the official version of the VPN. For example, if you want to use ProtonVPN, download it from the official website. Don't download it from other sources. If you download it from a third-party source, you might be downloading a modified version that collects data.

Additionally, which version should you use? Alpha, beta, or stable? Alpha is the most unstable version, followed by beta, and then stable. Programmers use alpha and beta versions to test their software. If you use an alpha version, you might encounter bugs or security problems (as program testing is still ongoing).

The creators should be trustworthy

Do you buy something from an untrustworthy person? Of course not. The same principle applies to software.

Does the company have a good track record when it comes to privacy? For instance, if a company with a history of selling data has developed this VPN, it cannot be trusted.

What if the company has no history at all? Sometimes, you might notice an unknown VPN suddenly gaining popularity overnight, which can be suspicious.

Perhaps the goal is to gather information from users. For example, if a government is blocking VPNs, but one VPN is still operational and everyone is talking about it, promoting it, or if someone messages you saying: "Hi, I've

7.3 Virtual Private Network (VPN)

been following you, and you're a great person. I found a VPN that connects easily and rarely disconnects. You should install it." You should consider the possibility that this VPN might have been created with the intention of collecting data.

- But wait, a well-known TV network is advertising it! A network that operates against a certain government. Or someone with 200,000 followers is promoting it. They're an influencer! It's impossible that they're in relation with those who want to collect information.

- Most TV networks and influencers will promote anything if it's somewhat popular or if you pay them well. They might bring in an expert who is not credible and promotes bad products. The network itself may lack the knowledge to determine whether the information being presented is accurate.

Every government, especially dictatorial regimes, has certain individuals who may appear to oppose them on the surface. However, their main goal is to engage in media manipulation and gather information about people.

These individuals enter the scene during critical moments to divert attention from specific issues. In fact, some of those with a large following accept offers to collaborate with security organizations for financial gain or cooperate to continue their work.

Transparency and Open-source Nature are Essential.

The service must be clear about what it does, what it claims, and how it operates. This transparency in applications is tied to being open-source. However, simply being open-source is not enough; it must be verified. Open-source means the code is accessible, but it doesn't necessarily mean it's good!

It should be validated by experts, meaning that independent organizations or researchers have conducted security assessments. Additionally, bug bounty programs¹⁰ can be a good indicator of a service's reliability.

A good VPN provides clear information about the protocols and algorithms it uses¹¹ and does not hide this information. In contrast, unreliable VPNs typically do not explain the algorithms they employ or how they are implemented. They may simply claim, "Trust us for your privacy; we are great." That's fine, but I need to see how you implement your service! Where is your code?

A reputable provider offers a transparent picture of what they have done and

¹⁰We even have a profession called "bug bounty hunter." For example, if I have knowledge about application security and I discover a bug, I report it according to the company's guidelines. They might say, "Great job! You didn't exploit the bug and instead reported it to us so we can enhance our security. Here's a \$100 reward." For instance: ProtonMail Bug Bounty Program. However, be aware that you can't just go out and find bugs randomly! You need to have the necessary knowledge and not engage in unauthorized activities. Follow the established rules; otherwise, if you exploit a bug, you could be committing illegal hacking and end up in jail!

¹¹e.g. <https://protonvpn.com/secure-vpn/strong-protocols>

7.3 Virtual Private Network (VPN)

what they plan to do. For example, you can see Proton VPN's 2023 roadmap.¹²

They explain what they have accomplished. If you visit their website, you can see what they plan to do in the future. This way, they present a transparent picture of their operations, which is a sign of professionalism. (Not every company that does this can be labeled as professional, but it is certainly a good practice that others should follow. It allows them to create an accurate representation of their past activities and a more organized vision for the future by outlining their goals. Additionally, customers feel more satisfied when they see the improvements that have been made and the new features that are set to be added. As a customer, I can understand that this company is making an effort and has accomplished certain things.)

Country of Origin

It's important to consider that while the individuals or team behind a VPN may be trustworthy, they might be legally obligated to comply with certain regulations. For instance, in some countries, the government may require the VPN provider to retain user data or share it under specific circumstances.

- Can the VPN provider do anything about this?

- Sometimes, they can! For example, they can choose not to store any data. If there's no data retained, then even if the government requests it or a hack occurs, there's nothing to leak!

This is often referred to as a "No-log policy."¹³¹⁴ (However, it's worth noting that some providers falsely claim to have a No-log policy while actually storing data.¹⁵ This is why it's crucial to choose a provider that is genuinely trustworthy!)

- So, if they don't store data, the government can't do anything, right?

- Not necessarily!

Group 1: In some countries, it is difficult to set up services that protect privacy to the extent that they can prevent government access. If you can, you often have to cooperate, and this cooperation sometimes lacks a specific legal framework! This means they can seize data based solely on accusations, without a court trial.

These countries also share their data with their allies. For example, if countries X and Y are allies, and you use a VPN from country X, your own country, Y, can request information about a specific individual.

Group 2: In some countries, like the United States, you are allowed to set up these services, but sometimes you are required to log who is using them. If they want information about specific individuals, they can obtain the logged data with a court order. If you refuse to cooperate, you might have to shut

¹²<https://protonvpn.com/blog/proton-vpn-roadmap-spring-2023/>

¹³Mullvad VPN was subject to a search warrant. (Customer data not compromised)

¹⁴OVPN wins court order

¹⁵<https://cyberinsider.com/vpn-logs-lies/>

7.3 Virtual Private Network (VPN)

down your business. You may face some difficulties, but compared to countries in Group 1, it is much easier to separate yourself and say, “I won’t cooperate,” and you can bring media attention to the issue so that the government cannot pressure you too much. However, in Group 1 countries, this usually does not happen. The pressure is often so intense that you are forced to comply, and you do not have the freedom to publicize the situation.

For example, a notable case is Lavabit, which faced pressure to hand over information about Edward Snowden. They stated they were willing to shut down their service rather than provide the data.¹⁶

Group 3: In some countries like Switzerland and Sweden (and the European Union), which have strong privacy protections, the government does not force you to log data. Generally, the government cannot exert pressure. They can only request logs in specific cases, and this requires approval from a court and a judge within the country. They cannot simply request logs based on suspicion; they must (at least mostly) prove that you are a criminal. Only then can they obtain logs to track you down, as was the case with ProtonMail.¹⁷ Typically, these countries have better privacy protections and respond to requests from other countries with caution:

“ **In general, Swiss authorities do not assist foreign authorities from countries with a history of human rights abuses.^a**

^a<https://web.archive.org/web/20230121203459/https://proton.me/legal/transparency>

— ProtonMail

That is why the origin country is important.

- Why do I hear more about privacy violations in Europe than in certain other countries?

→ Because in countries with freedom of speech, these issues are taken very seriously. People have the right to defend their rights and make them public, and there are even researchers who actively look for instances of privacy violations to address them. In contrast, in other countries, people do not have the right to speak out about these issues, and there is not enough freedom for investigative journalists.

¹⁶<https://web.archive.org/web/20130809031439/https://lavabit.com/>

¹⁷<https://restoreprivacy.com/protonmail-logs-users/>

NOTE

When news about theft, corruption, and accountability for wrongdoers diminishes or disappears, it indicates a high level of corruption in that place, as there is no sensitivity to holding wrongdoers accountable. Corruption never truly disappears; it exists everywhere. However, when there are no reports of high-ranking officials being dismissed, it suggests that everyone is colluding and engaging in corrupt practices together.

Privacy Policy

Another thing we should consider when looking for a desired program is its privacy policy. A privacy policy is a document that explains how the company will handle your data, including how it uses, stores, and shares your data. It's important to read this document carefully. Generally speaking, if a company has a very complicated privacy policy, it's a red flag. Some companies make their privacy policy so complicated that you can't understand it. This is a sign that they are trying to hide something. A good privacy policy is clear and easy to understand.

- Do they always tell the truth in their privacy policy?
- Not always! But depending on the country, if they don't, you can sue them.

Protocol in Use

In a VPN, there are various connection methods (protocols) available. However, not all of these methods are secure or reliable. Good protocols include:

- OpenVPN
- Wireguard

An example of a bad protocol is PPTP.

- I see some protocols doesn't work in countries like Iran and China. Why is that?
- Remember when we talked about the structure of packets? (Figure 7.4) The header of the packet contains information about the protocol used. Bits used in the header of OpenVPN packets are different from those used in Wireguard packets. So, if the government wants to filter a specific protocol, they can do so by filtering the packets that have that specific header. They can fingerprint the packets and say, "If I see packets with this type of header, I will drop them." This is another method of filtering and blocking access.

Encryption

For apps, you should pay attention to how they protect your data. What algorithms for symmetric encryption, asymmetric encryption, hash functions, and key exchange mechanisms do they use?

Plaintext data should be encrypted before being sent over the network. The encryption key must not leave the device. The key should be generated on the device and not transmitted to the server.

Moreover, data should be stored encrypted on the device. For example, some apps (e.g., messaging apps, note-taking apps) store messages in plain text on the device. This is a security risk because if someone gains access to your device, they can read your messages through the app's database (which is stored in plaintext in different files on the device). Unfortunately, many apps we use, even those that claim to be secure, store data in plaintext on the device. Data security starts from the device.

- If the data is encrypted on the device, how can we enable features like search? Many features require access to the data.

→ There are solutions out there. See how Tuta does that.¹⁸

Desktop app or Browser Extension or web apps?

Desktop apps can be more secure than browser extensions in some cases. Since they have more access to the system, they can provide more security features (e.g., through the enforcement of security features).

Browser extensions are limited in terms of what they can do. They can't access the system as much as desktop apps can. They can only access the browser and the data that the browser provides.

Another advantage is that open-source desktop apps are mostly tested and analyzed by experts. You can download them and use them. But how can you be sure that the JavaScript code you get is the same JavaScript code others get? The service may collaborate with security agencies to provide you with specific code that uses weak encryption.

On the other hand, desktop apps can collect significantly more data.

What login method should I use?

It is best to use a method that does not require you to provide your email address. If you use your email address, it can be linked to other services. For example, if you use the same email address for your VPN account and your email account, they can be linked together. Using a different email address for each service makes it harder to link the accounts.

¹⁸<https://tuta.com/blog/first-search-encrypted-data>

7.4 Vulnerabilities Related to Network and Connection

Using a username and password is more difficult.¹⁹

Although biometrics may seem convenient, they are sometimes not recommended. Some biometric options are not very secure. Most face recognition systems can be fooled with a photo.

You can be forced to unlock your device with your fingerprint or face. In some countries, police can force you to unlock your device with your fingerprint or face, but they are not allowed to force you to unlock your device with a password.

Additionally, if biometric data is stolen, it cannot be changed. You cannot change your fingerprint or face. That is why I recommend not enabling biometrics for cheap IoT devices, as they are more likely to have less security.

A disadvantage of using a password is that it might be leaked through security cameras. It is really hard to always be aware of your environment and ensure no one is watching you. Eventually, there will be a situation where you inadvertently enter your password in front of a camera.

7.4 Vulnerabilities Related to Network and Connection

7.4.1 Leak of Traffic During System Shutdown and Reboot

Imagine that your VPN disconnects due to some issue. Normally, this would mean that your data is transmitted outside the secure VPN tunnel, potentially exposing your IP address. To address this, there is a feature called a “kill switch.” This feature disconnects your internet if the VPN connection drops, preventing data from being transmitted outside the secure tunnel. It will only allow data transfer once the VPN is reconnected.

However, consider the scenario where you turn off the device. If the program is not well-designed, the kill switch process might be terminated before the system shuts down. This creates a window of time between the VPN disconnection and the termination of the kill switch process, during which data could be transmitted outside the VPN while the internet is still active.

Alternatively, if you have the “auto-connect” and “launch app on start-up” options enabled, the VPN should start automatically when the system boots up, preventing any data from leaking outside the VPN. However, due to various factors, there might be a brief moment before the VPN establishes a connection, during which data could be transmitted outside the VPN, revealing your real IP address.

In essence, in the realm of security, we focus on identifying vulnerable points and edge cases. We ask ourselves, “What happens if we encounter a sensitive

¹⁹But not impossible, as many people use similar usernames for different services. There are OSINT tools that can help find usernames on different platforms.

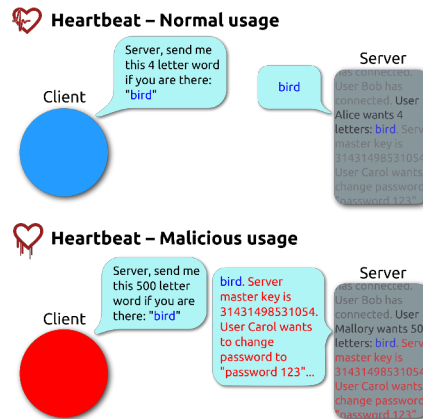


Figure 7.8: FenixFeather, Simplified Heartbleed explanation, CC BY-SA 3.0

situation at that critical moment?"

Want a real-world example? See "MUL22-02" from *Mullvad VPN Platform Security Assessment*²⁰²¹ by "Atredis Partners".

7.4.2 Heartbleed

Opening HTTPS connection takes time. So, to speed up the process, the server and client keep a connection open for a while. This way, they don't have to establish a new connection every time they want to send data.

The process of keeping the connection open is called a **heartbeat** (does the heart of the server beat? Is it alive?). But the problem arises when the server doesn't check the length of the data being sent. For example, suppose I send a message saying "bird" and claim that the length of the message is 500 bytes. All data is stored in RAM (including the word "bird"). The server will read the first 500 bytes from the location where "bird" is stored in RAM. The problem is that the server doesn't check if the length of the data is correct. So, instead of reading only 4 bytes (the length of the word "bird"), it reads 500 bytes. This means it will read data stored in RAM after "bird." This data could be anything, including sensitive information like passwords, credit card numbers, and so on.

A security expert would check protocols and find out where things might go wrong. In this field, you need to read and understand how the protocols work.

²⁰<https://static1.squarespace.com/static/576323cfd482e984e113fe9c/t/635053e31bf41d56a97a920d/1666208739842/Atredis+Partners+-+Mullvad+VPN+Platform+Security+Assessment+-+Report+v1.0.pdf>

²¹<https://mullvad.net/en/blog/2022/10/21/security-audit-report-for-our-app-available>

7.4.3 Crawling

For example, suppose you want to find the site owner's login page (the admin page). One way is to manually test a series of addresses:

```
admin.example.com/  
example.com/admin  
example.com/administrator  
example.com/login
```

But this method is tedious. When you're testing a website, you have to perform dozens of different tests. You shouldn't waste your time. Otherwise, you won't find vulnerabilities in time, and others will find and report them sooner. That's why you need to use automation. This means performing these tasks automatically by creating tools and programs. Let the program do the searching for you.

Or, imagine you've found a database like this:

```
example.com/database/1.png
```

You can access other images in the same folder on the server by changing the number 1. For example:

```
example.com/database/2.png  
example.com/database/3.png  
example.com/database/4.png  
...
```

Suppose you find a folder containing images of national ID cards. This way, you can download all the images in the folder! Congratulations! You now have all the national ID cards. In fact, you should do all of this with automation.

Essentially, try to learn how to write a program to solve various problems. That is, try to understand how to write programs to address different issues.

7.5 End of the Chapter Questions

Problem 7.1 Identify the domain of each of the following websites:

- (a) `https://mail-google.com/getemail`
- (b) `https://mail.google.com/getemail`
- (c) `https://mail.getmail.mailbox.get.google.com/getemail`
- (d) `https://mail.getnum.std-google.com/getemail/google.com`

Problem 7.2 Suppose you want to communicate with an anonymous person, but the internet connection is weak. The person suggests enabling the peer-to-peer (no server in the middle) call option. Would you do it? Does this pose any privacy or security concerns? Why?

Problem 7.3 Consider a network where data must pass through three nodes to reach its destination. To send data, it is encrypted three times on the sender's device before being transmitted. At each node, one layer of encryption is removed. In the first node, the outermost layer is decrypted; in the second node, the second layer is decrypted; and in the final node, the last layer is decrypted. Eventually, the data reaches its destination.

Now, suppose someone connects to your website using the university's Wi-Fi and this three-node network, sending you a message. However, the DNS request is mistakenly sent outside this network in plain text (since DNS over HTTPS is not used). Additionally, the way the individual connects to this three-node network differs from a standard HTTPS connection.

How could you reasonably deduce who sent you the message? (1.5 points)

Problem 7.4 A company wants to secure data sent between its client and server. They are currently using an HTTP connection. Their plan is to hash the data on the server before sending it to the client. Is this mechanism a secure and correct way to protect the data? Why or why not?

7.6 End of the Chapter Solutions

Solution 7.1

- (a) mail-google.com
- (b) google.com
- (c) google.com
- (d) std-google.com

Solution 7.2 No, because the other party can discover your IP address (via the source IP address in the packet header).

Another example: the ISP can directly determine with whom you communicated and for how long.

Solution 7.3 Public Wi-Fi networks, such as those in universities, often log user activity. (0.25)

Since the way devices connect to these networks differs from HTTPS, the packets sent can be fingerprinted based on their structure. By analyzing these packets, you can determine who was connected to the network at a specific

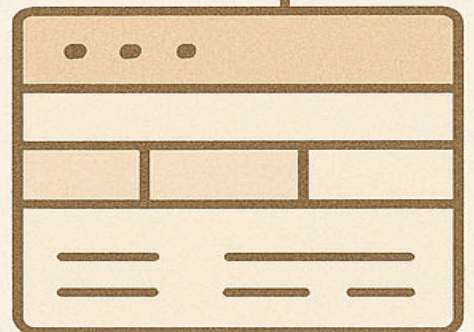
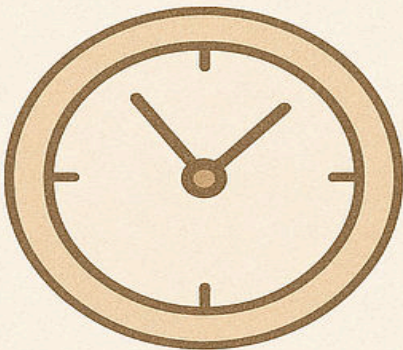
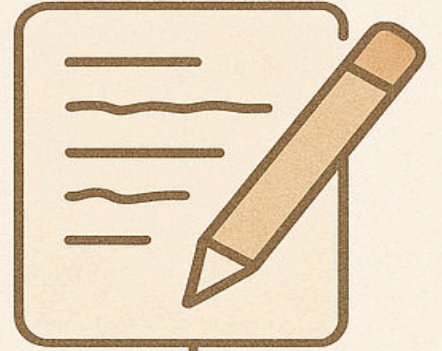
time. For example, you might find that 10 people were connected during that particular period. (0.75)

To narrow down the identification further, you can examine DNS requests. Since DNS requests are sent in plain text, the content and destination of the requests can be identified. This allows you to see who sent DNS requests to the target website at that specific time. (0.5) The overlap between individuals whose DNS requests were directed to the target website at that moment and those connected to the three-node network can help you uniquely identify the individual (or at least reduce the pool of candidates to a very small number).

Solution 7.4 No, this mechanism is neither secure nor correct for protecting data in transit.

Hashing is a One-Way Street: The core issue is that hash functions are designed to be one-way. You can easily compute a hash from data, but you cannot reverse the process to get the original data back from the hash. This makes hashing unsuitable for encrypting data that the recipient needs to read in its original form.

Sending a table that maps hashes to the original sentences at the start of the connection essentially creates a simple substitution cipher. This is highly insecure. Anyone eavesdropping on the connection (which is easy with unencrypted HTTP) can capture this initial table. Once they have the table, they can "decrypt" all subsequent hashed messages by simply looking up the hash values.



7.6 End of the Chapter Solutions

8.0 Fingerprint

While many resources focus on trackers, cookies, and other methods of tracking, we rarely see discussions about fingerprinting. For this reason, I decided to dedicate a separate section to this topic.

In this section, we will learn that privacy-based tools are not necessarily good! They might set us apart from others, which can be suspicious. For example, if only 10 people are using a specific privacy-focused system, it clearly makes us identifiable! Others may not know what we are doing, but it's obvious that we might be engaging in something considered "bad." Therefore, not every anonymity-based operating system is beneficial! Instead, it should not make our fingerprint unique on the web!

8.1 What is Fingerprinting?

In the real world, you stand out from others based on certain characteristics. In other words, these traits help identify you. For example, if a friend were to describe you, they might say: "Yeah, She's of average height. Her skin tone is light brown. She has black eyebrows and an average nose. Her eyes are brown and large. Her hair is black. She usually wears green clothes and typically has white or black pants. She wears glasses."

So, people recognize you by these features. You know how, when they want to identify a criminal, they use facial recognition? Well, when you go online, your device sends and shares certain information that can describe you. There used to be a saying: "On the internet, nobody knows you're a dog." This means that no one knows if a dog is sitting behind the computer or a person. However, we need to revise this thinking! You are not anonymous on the internet! When you enter a website, your device automatically tells that site (or makes some traces based on your behavior), "I'm a laptop with a Windows operating system, from Tehran. I'm browsing with Firefox version X, I have these fonts installed, my timezone is X, these extensions are installed (with the help of investigating the behavior of the browser based on some elements), my window size is 1920x1080¹,

¹See also: *How CSS Alone Can Help Track You:* <http://web.archive.org/web/20230424015823/https://matt.traudt.xyz/posts/>

8.2 Narrowing Down: How Characteristics Lead to Identification

and my hardware information is this." So, if you visit that site again tomorrow, it will recognize you because of this information. In fact, finding you online is quite easy!

With this information, they can track you on the internet, and even if you change your IP address a hundred times, they can still figure out who you are. This raises serious questions about anonymity on the internet.

- Why is this information sent about me? Why does it say I'm using a certain browser and that my operating system is Windows?

→ I'll provide the answer in the User-Agent section.

- Why wasn't the internet designed from the beginning in a way that doesn't require this information to be sent?

→ Unfortunately, the internet wasn't built with privacy and security as its top priorities! In fact, things are usually created first, and then security measures are added later. Initially, workers were just doing their jobs, and then they realized, "Oh! We need something called safety gloves and helmets." Cars were introduced first, and then seat belts came along, followed by airbags. There were no airbags from the very beginning! Essentially, we first develop a new technology or feature, and later we realize, "Oh! This part needs to be secured!" So, we think about securing it afterward.

- Is there a way to reduce the amount of data sent, or at least send false or misleading data so that they don't know what my device really is? For example, can I pretend I'm using Linux and Brave instead of Windows? Can't we trick websites by sending incorrect data?

→ Yes, there are some solutions for that. We can discuss them in detail later.

8.2 Narrowing Down: How Characteristics Lead to Identification

How can this data and fingerprinting lead to my identification? Let's create a scenario in the real world to illustrate how you could be identified:

- Which country are you from? Let's assume you're from Iran. This narrows it down from 8 billion people to 85 million.
- What is your height? This might reduce the group to, say, 20 million people.
- What is your skin color? This could further narrow it down to around 5 million.

8.3 Does This Fingerprinting Have Any Positive Uses?

- What is the color of your eyes? This might reduce the number to about 4 million.
- Do you wear glasses? If you do, that could narrow it down to approximately 500,000 people.
- What color do you usually wear? This might limit it to around 50,000 individuals.

This process continues, with each characteristic further narrowing the number of potential matches until you are identified! There are thousands of other factors that can also play a role, and some of these factors can distinguish you very quickly.

On a computer, you have specific characteristics that serve as your "fingerprint" or, more accurately, your features (device characteristics and behavior patterns). When these elements are combined, they tighten the identification loop around you, making it easier to recognize you.

8.3 Does This Fingerprinting Have Any Positive Uses?

8.3.1 Detecting Suspicious Behavior

Imagine you always log into your bank account using your laptop. Now, if one day someone compromises your account (the term "hacking" is often used, but "cracking" is more accurate) and tries to access it, the bank's identification mechanisms, designed with automated programs, will check and realize, "Wait! Someone is trying to log in from a device that you've never used before." This prompts the bank to suspect that someone might be trying to access your account from a different location. As a result, they may block your account and notify you of the incident, asking if it was you who attempted to log in.

For instance, when you log into your Google account from a new device, Google wants to ensure that it's really you. It uses the fingerprint of the new device to determine whether this is a device that hasn't been used before. Therefore, it might not be you at all! This is why they want to verify that the right person is accessing your account.

Additionally, if a website requires each person to have only one account, fingerprinting techniques, including cross-browser and cross-device fingerprinting, can help prevent the creation of multiple accounts by the same individual.

Furthermore, fingerprinting can be used to detect bots. For example, if a website receives a large number of requests from a single device in a short period, it might suspect that a bot is trying to access the site. In this case, the website can block the bot from accessing the site. This also helps to prevent DDoS attacks, as the fingerprint of requests from a bot is different from that of a real person's browser.

8.4 Behavioral Fingerprinting

Moreover, fingerprinting can be used to detect whether a real person has clicked on ads or whether a bot has clicked on them. This is important for advertisers, as they want to ensure that their ads are being seen by real people and not bots.

Nevertheless, fingerprinting can also be used for malicious purposes. For example, if a website wants to track you, it can use fingerprinting to identify you and monitor your activities. This is why it's important to be aware of fingerprinting and take steps to protect your privacy.

- Is it accurate enough to identify me?

→ Yes[45].

8.4 Behavioral Fingerprinting

8.4.1 Authorship Identification

You write a post or you write a message to someone. The way you write, the words you use, and the way you express yourself are all unique to you. Someone born in Texas might write differently than someone from New York. This is similar to how people have different writing styles.

- No, this is not possible!

→ Yes, it is! Read research about the topic.

8.4.2 Coding Style

You hack into a website and use your own script to do something. But little do you know that the style of your code is unique to you. How you write comments, the way you name variables, and the flow of your code all reflect your personal style. This is similar to how people have different writing styles. For example, some people write in a very organized manner, while others are more chaotic.

8.4.3 How You Type

How you type can also serve as a fingerprint. For example, the speed at which you type and the time intervals between keystrokes can be used to identify you. I highly recommend reading Whonix's article on this topic.²

8.4.4 How You Use Your Phone

Perhaps one day you'll be identified by how you use your phone. For example, the way you swipe on your phone, the speed at which you scroll, and the way you hold your phone can all be used to identify you.

²https://www.whonix.org/wiki/Keystroke_Deanonimization

8.5 Device Fingerprinting

What device are you using? What is the name of the device? Are you on a desktop or a mobile phone? What operating system are you using—Windows, GNU/Linux, BSD-based, Mac, Android, iOS, or something else? All of these can help narrow down your identification.

What is the name of the device you are using to connect to the internet? This name can be seen by your ISP. Unfortunately, the default name of each device is often the exact model of your smartphone! For example, for a Samsung A73, it appears as SM-A736. This means the ISP can see that SM-A736 is connected.³

- Should we change it?

- Changing it would create a unique fingerprint, because most people use default names. Changing it would make you stand out. While SM-A736 might blend in with others, the downside is that if someone knows your device model (or the apps you're using), they can also understand its vulnerabilities! Knowing the exact model of your phone can be the first step toward potential risks. The truth is, I can't give a definitive answer. What do you think?

What is your time zone? It can likely help determine your geographical area.

- What is a time zone?

- The Earth is divided into different sections to synchronize time across regions. This allows me to understand what time it is in places like Switzerland, for example. We have a reference point known as Greenwich Mean Time (GMT). For instance, Iran is 3 hours and 30 minutes ahead of Greenwich (UTC+3:30).

It's not always necessary for someone to know your time zone directly! They can infer it from your activity patterns. For example, I can determine where you are based on the times you post tweets. Generally, people sleep at night and are awake during the day. By monitoring your activity, I can see when you are online and active, which helps me estimate your time zone.

For instance, if I notice that you're not online on Fridays and Thursdays, it might indicate that you're Iranian, as those days are typically weekend days in Iran, and you might not be working then. Conversely, if you are more active on Thursdays and Fridays, it could suggest that those are your days off, giving you more time for online activities.

Additionally, there might be certain times of the year when you are less active, perhaps due to celebrations like the New Year. All of these patterns can provide clues about your geographical location and time zone without you

3

- Where can I find out what this name is?

- Let's follow a logical approach. The device name should typically be in the settings! So, let's go to the settings. You'll see a series of options. The device name is usually found under "About Device." There, you can see the name of your device.

8.6 Browser Fingerprinting

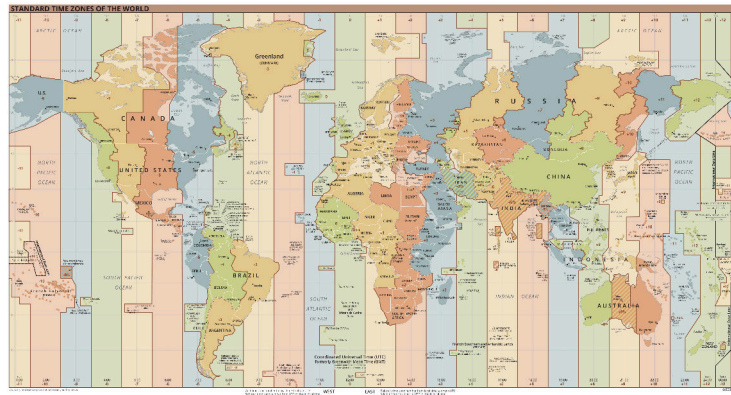


Figure 8.1: Time Zones src: This work is in the public domain in the United States because it is a work prepared by an officer or employee of the United States Government as part of that person’s official duties under the terms of Title 17, Chapter 1, Section 105 of the US Code.

explicitly stating it.

For this, Tails OS (an operating system that focuses on privacy and anonymity) sets the time zone to GMT.⁴ What is installed on your operating system? This includes both applications⁵ and various other elements like languages, fonts, settings you’ve configured, and even your wallpaper! Yes, even your wallpaper can serve as a fingerprint!⁶

Warning

If you believe you are a target and need to remain anonymous, keep in mind that every additional item or customization can serve as a fingerprint! Almost anything can contribute to your unique identification, even things you might not consider significant.

This method can lead to your identification across different browsers because the information about your device remains consistent.

8.6 Browser Fingerprinting

⁴https://tails.boum.org/doc/first_steps/desktop/time/index.en.html

⁵Some Android apps gather the list of installed apps and use it to track you.

⁶<https://fingerprint.com/blog/how-android-wallpaper-images-threaten-privacy/>

8.6.1 User-Agent (HTTP Header)

The Mozilla website⁷ provides a very comprehensive yet simple explanation of what a User-Agent (UA) is. In summary, a browser sends a string of text to a website to describe itself, ensuring that it receives the most compatible version of the site. This allows the website to display correctly, ensuring that buttons work properly and the layout looks good. For example, the browser might say, "I am Firefox version X on Windows."

While this is beneficial for user experience, it poses privacy issues. This information can serve as a distinguishing feature that sets you apart from others. Each piece of data contributes to creating a unique fingerprint for you, making it easier for websites to identify and track you across sessions.

- Why should we differentiate and say that if you come in with Firefox, the site looks one way, and if you come in with Brave, it looks a bit different? Why not just provide the same version for everyone?

- You're right! It would be better if that were the case. However, browsers have some differences, which creates issues. Due to these variations, it is sometimes necessary to make certain adjustments to ensure compatibility.

There have been various efforts to standardize different aspects of web development, but when things vary between browsers, achieving that standardization becomes challenging. Each browser may interpret and render web content differently, which is why developers often need to make specific changes to accommodate these differences.

8.6.2 Accept Header

The Accept header is another part of the HTTP request that the browser sends to the server. These headers specify the types of content the browser can accept. For example, the browser might say, "I can accept mp3, mp4, and webm files." This information can also contribute to your fingerprint. See also: Accept-Encoding⁸, Accept-Language⁹.

8.6.3 Extensions and add-ons

The extensions and add-ons you have installed can also serve as a fingerprint. For example, if you have an ad-blocker with specific settings, this can help identify you. Because your browser behaves differently from others, it can be used to distinguish you from other users.

For example, some extensions block trackers based on your behavior. They monitor which websites you visit. If a website you visit tries to make a connection, the extension allows it. However, if a website you have not visited (or a

⁷<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/User-Agent>

⁸<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Accept-Encoding>

⁹<https://developer.mozilla.org/en-US/docs/Web/HTTP/Headers/Accept-Language>



Figure 8.2: A world where everyone is the same. (AI)

website in a different category) tries to make a connection, the extension blocks it.

Nevertheless, this behavior can be used to identify you. For example, if you have an extension that blocks connections to a specific website, this can be used as a fingerprint and can leak your browsing behavior. See [46] and EFF Privacy Badger changes^{10, 11}.

8.7 Solutions

First, I should note that there is no perfect solution! However, we can take some steps to reduce the effectiveness of fingerprinting. The Internet was not designed to be private, and it's not easy to be completely anonymous. Still, we can take steps to lessen the effectiveness of fingerprinting (some may cause compatibility issues). Here are some solutions:

8.7.1 Blending In

Imagine a world where every person is just like any other person. They all have the same height, weight, hair color, eye color, and even the same face! In such a world, it would be impossible to identify anyone. This is the concept of blending in. If everyone is the same, no one stands out, and no one can be identified.

To achieve this, we need to make our fingerprint similar to others. For instance, the Tor Browser¹² is one of the best tools for blending in. It's designed to make all users look the same.¹³ People using the Tor Browser appear to be

¹⁰<https://www.eff.org/deeplinks/2020/10/privacy-badger-changing-protect-you-better>

¹¹See also:

DataSpii: The catastrophic data leak via browser extensions: <https://securitywithsam.com/2019/07/dataspii-leak-via-browser-extensions/>

Avast and AVG collect and sell your browsing history: What you need to know: <https://www.tomsguide.com/news/avast-avg-data-collection>

¹²<https://www.torproject.org/>

¹³<https://blog.torproject.org/browser-fingerprinting-introduction-and-challenges-ahead/>

using the same browser, with the same settings, extensions, and even the same operating system! This makes it difficult to distinguish one user from another (not impossible, but difficult!).

The Tor Browser is slow (I will talk about it later). If you want a regular browser with similar features, you can use the Mullvad Browser¹⁴. It is designed in partnership with the Tor Project and Mullvad VPN. Generally, it is the Tor Browser without the Tor network. It is designed for better privacy (not anonymity).

8.7.2 Randomizing

Randomizing is another method to reduce the effectiveness of fingerprinting. For example, you can use a tool that changes your fingerprint every time you visit a website. This way, you won't always appear as the same user. You can also randomize your time zone, language, and other elements that contribute to your fingerprint. See how the Brave Browser implements this.¹⁵

Warning

Some people might use test websites to check if their fingerprint is unique. However, if that website is malicious, it can store your fingerprint and track you across the web! Be careful when using such websites. The EFF^a tool is a better option if you want to check your fingerprint.

^a<https://coveryourtracks.eff.org/>

Cover Your Tracks from EFF gives you an output like this:

```
only one in X browsers have the same fingerprint as yours
```

Generally speaking, the smaller the number, the better your fingerprint is. This is because it means we need to check fewer browsers to find one that is similar to yours. Conversely, if the number is high, it means that your fingerprint is so unique that we need to check many browsers to find a similar one. You're not blending in.

8.7.3 Browser Isolation

If your browsing data is separated, it becomes harder to link different browsing data to one another. For instance, use one browser for your school stuff, another for work stuff, and so on.

However, this doesn't prevent device fingerprinting (device-specific information). Additionally, don't use the email or phone number you used in one

¹⁴<https://mullvad.net/en/browser>

¹⁵<https://brave.com/privacy-updates/4-fingerprinting-defenses-2.0/>
<https://github.com/brave/brave-browser/wiki/Fingerprinting-Protections>

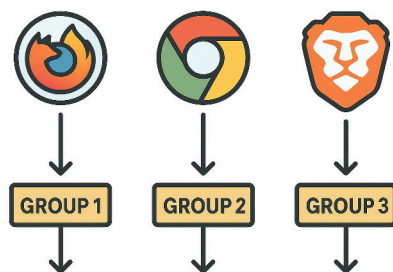


Figure 8.3: Browser Isolation

browser in another browser, as this can link your browsing data to one another. For your privacy, use fake email addresses and phone numbers. Phone numbers are dangerous; they are directly linked to you.

As a result, some people have different devices/OSs for different tasks.

8.7.4 Reading List

- A history of Fingerprinting protection in Firefox¹⁶
- Demo: Disabling JavaScript Won't Save You from Fingerprinting¹⁷
- Mitigating Browser Fingerprinting in Web Specifications¹⁸

¹⁶<https://www.ghacks.net/2018/03/01/a-history-of-fingerprinting-protection-in-firefox/>

¹⁷<https://fingerprint.com/blog/disabling-javascript-wont-stop-fingerprinting/>

¹⁸<https://w3c.github.io/fingerprinting-guidance/>



9.0 Web Browsing

We've reached the web browsing section. Just like in the previous parts, where I first introduced malware and then discussed countermeasures, in this section I want to start by outlining the threats before explaining how to deal with them.

- Why do you do this?

→ Because you need to understand what poses a threat to you. If I simply give you a list of dos and don'ts to memorize, you'll quickly forget them because you won't understand the reasoning behind each instruction. For example, why shouldn't you do a certain action? Why should you avoid accessing an HTTP site? If you don't understand the reasoning, it will slip your mind. Moreover, new and more complex threats are emerging every day, and it's not always someone else's job to teach you. You need to be your own teacher so that when new threats arise, you can learn how they operate and how to counter them.

In the future, I won't be here to guide you, so I'm teaching you how to fish now so that you can catch bigger fish on your own later.

9.1 Web Browser

A browser is software that allows you to access the internet. It's like a car that takes you to different places on the internet. If your car had a camera to watch you while you drive, you wouldn't like it, right? Similarly, browsers can watch you while you browse the internet. They can track you, record your activities, and even send this information to others. This is why it's important to choose a browser that respects your privacy.

I recommend:

- *Firefox* and *Brave* for regular use
- *Mullvad* for better privacy (better privacy comes with some compatibility issues for some websites)
- *Tor Browser* for anonymity¹

¹If your anonymity is vital for you, the Tor Browser alone is not enough. Use anonymity-focused operating systems like Tails OS.

9.1.1 Chromium-based or Non-Chromium based?

Chromium is an open-source project that serves as the foundation for many browsers, including Google Chrome, Microsoft Edge, and Brave. These browsers are created by making changes to the code of Chromium. They modify some functionalities and add new features to create a unique browser. However, the core of these browsers is still Chromium. While Chromium is open-source, Google Chrome and Microsoft Edge are not.

I do not have enough knowledge about the security of browsers. Some say the sandbox of Chromium is better than that of Firefox. Others argue that Firefox is better because some of its code is written in Rust. Of course, none of these can be a complete reason. We should consider everything. If you can help in this section, please let me know.²

Here, I want to discuss why you should also use Firefox-based browsers.

Currently, all major browsers except Firefox and Safari are based on Chromium. The entire web is becoming dominated by Chromium³⁴, which is concerning. This trend reduces innovation and creativity, putting the web in the hands of a single entity.

Mozilla's revenue primarily comes from its user base. For instance, Google pays Mozilla to make its search engine the default in Firefox.

- Why?

- This allows Google to maintain its market share, keep users within its ecosystem, and gather more data about individuals. Remember when we discussed how important information and presence in an ecosystem are? In 2021, Google paid Apple around \$15 billion to ensure that its search engine is the default for Safari, Mac, and Siri!⁵

For Mozilla, this figure is about \$400 to \$450 million annually from 2021 to 2023⁶, which accounts for roughly 90% of its revenue. If Mozilla's user base declines, its income will decrease, leading to potential layoffs and a decline in the organization. This would be detrimental to the web, as we would lose out on the valuable technologies that Mozilla brings to the table. It would also mean losing an organization that prioritizes our privacy.

You can customize Firefox using the settings available on the *about:config* page. Alternatively, you can use browsers like *Mullvad* that are configured out of the box. *Librewolf* is another Firefox-based browser that is worth mentioning. The Tor Browser is also Firefox-based.

²Might be good: <https://madaidans-insecurities.github.io/firefox-chromium.html>

³<https://gs.statcounter.com/browser-market-share>

⁴<https://data.firefox.com/dashboard/user-activity>

⁵<https://www.forbes.com/sites/johanmoreno/2021/08/27/google-estimated-to-be-paying-15-billion-to-remain-default-search-engine-on-safari/>

⁶<https://www.pcmag.com/news/mozilla-signs-lucrative-3-year-google-search-deal-for-firefox>

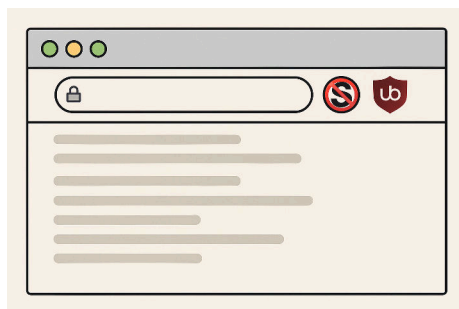


Figure 9.1: Extensions and add-ons

9.1.2 Extensions and addons

You can use extensions to enhance your browser’s privacy.

NoScript is a fantastic extension that lets you control which scripts run on a website. While it can be a bit annoying at first, it significantly improves your privacy.

uBlock Origin is another great extension that blocks content you don’t like (such as ads and trackers). I personally don’t like to block ads, as I believe in supporting websites that provide valuable content. Without ads, many websites would struggle to survive since you don’t pay for their services.

An interesting idea to hide your real browsing history is to generate a lot of fake browsing history. This way, it becomes difficult to identify which browsing history is real and which is fake. See *Adnauseam*⁷ and *TrackMeNot*⁸. (I am not aware of their effectiveness, but the idea is interesting.)

9.1.3 Search Engines

The problem is that most search engines log your searches for advertising purposes, improving their services, or for warrant requests⁹ or tracking purposes. This includes keywords you type but don’t hit enter. It’s important to choose a search engine that respects your privacy.

OK, that’s enough for now.

- Why so little information?

→ For more up-to-date and more precise information, I recommend you read their documentation.¹⁰

⁷<https://adnauseam.io/>

⁸<https://www.trackmenot.io/>

⁹Exclusive: Government Secretly Orders Google To Identify Anyone Who Searched A Sexual Assault Victim’s Name, Address Or Telephone Number: <https://www.forbes.com/sites/thomasbrewster/2021/10/04/google-keyword-warrants-give-us-government-data-on-search-users/>

¹⁰Brave, Firefox

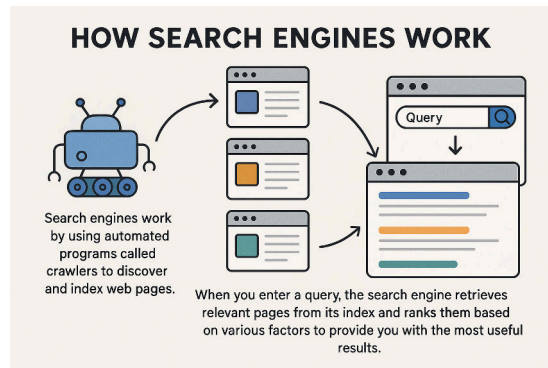


Figure 9.2: Search engines work by using automated programs called crawlers to discover and index web pages. When you enter a query, the search engine retrieves relevant pages from its index and ranks them based on various factors (your search history, location, gender, etc.). The search engine then displays the results to you.

9.1.4 Reading List

- PrivacyTests.org (Tests different browsers for privacy)¹¹

9.2 Deepweb and Darkweb

9.2.1 What is the Deep Web?

The deep web is the part of the internet that is not indexed by search engines. It includes things like your emails, bank accounts, and other private information that you do not want to be publicly accessible. The deep web is not inherently bad; it is just not indexed by search engines. It's like a private room in your house that only you have access to.

The dark web refers to a part of the deep web that is intentionally hidden. It is sometimes used for illegal activities, such as selling drugs, weapons, stolen data, and hacking tools. The website addresses on the dark web are made up of random characters, making them difficult to find by brute force. People use special browsers like Tor to access the dark web. You need to get the address of the website you want to visit from a trusted source, as there are many scams on the dark web.

- Is most of the dark web illegal?
→ No.[47]

¹¹<https://privacytests.org/>

9.3 The Onion Router (Tor)

As I showed in figure ??, everything on the internet is transferred via packets. These packets are like letters; they have a sender and a receiver. Everyone in the middle can see the packet. However, when we talk about anonymity, we want to hide the packet content, sender, and receiver. When using a VPN, the packet content is hidden from the ISP. However, the VPN provider can see:

- Packet content
- Sender
- Receiver

We can't change the sender and receiver for the entire communication (otherwise, communication can't be done!). So we somehow need to split this data into different parties, ensuring that no party has total access to all the information.

9.3.1 Tor comes in

According to a *Top Secret* document leaked from NSA, Tor is:

“ The King of High-Secure, Low-Latency Internet Anonymity. There are no contenders for the throne in waiting.^{a,b}

^a<https://www.scribd.com/doc/173441295/Tor-The-king-of-high-secure-low-latency-anonymity>

^b<https://www.theguardian.com/world/interactive/2013/oct/04/tor-high-secure-internet-anonymity>

— NSA

“ We still never be able to de-anonymize all Tor users all the time. With manual analysis we can de-anonymize a very small fraction of Tor users, however, no success de-anonymizing a user in response to a TOPI request/on demand.^a

^a<https://web.archive.org/web/20190228101152/https://edwardsnowden.com/wp-content/uploads/2013/10/tor-stinks-presentation.pdf>

— NSA

See how the wonderful guide “How Does Tor Really Work? The Definitive Visual Guide (2023)” explains Tor^{12,13}. (It is required to read the guide before continuing this section.)

The primary idea is to let each node know only partial information:

Node 1:

¹²<https://skerritt.blog/how-does-tor-really-work/>

¹³<https://support.torproject.org/https/>

9.3 The Onion Router (Tor)

- Nothing about data (as data is encrypted three times)
- Knows the sender (Because it gets the data from the sender)
- No information about the receiver (As it only passes data to node two)

Node 2:

- Nothing about data (as data is encrypted two times)
- No information about the sender (Because it gets the data from node 1)
- No information about the receiver (As it only passes data to node three)

Node 3:

- Knows the data (As it is the last node and needs to send the data to the receiver). However, if you use HTTPS, the data is encrypted and node 3 can't see the data. Only metadata is visible.
- No information about the sender (Because it gets the data from node 2)
- Knows the receiver (As it passes the data to the receiver)

This way, no node has access to all three pieces of information we discussed.

• What if all three nodes are controlled by the same entity?

→ This is a valid concern. If all three nodes are controlled by the same entity, they can see all the information. However, the probability of this happening is very low. The Tor network consists of thousands of nodes, and it's highly unlikely that all three nodes will be controlled by the same entity.

We do not even need all three nodes; only the first and the last one are necessary. The first node knows the sender, and the last node knows the receiver. To be able to map the packet in the first node to the corresponding packet in the third node, an attacker can use timing, packet size, and other features. This is called a *Correlation Attack*.

Nodes of the Tor network are called *Relays*. There are three types of relays: *Guard Relay*, *Middle Relay*, and *Exit Relay*. All relays are public¹⁴ and anyone can run a relay.

Some malicious actors run exit nodes to monitor and change unencrypted traffic (mainly¹⁵ HTTP traffic)¹⁶. However, Tor regularly checks the exit nodes to see if they are doing something bad. If they are, they are removed from the network.¹⁷¹⁸ This is why it is important to use HTTPS.

¹⁴Except for bridges, which we will get to later.

¹⁵There is other unencrypted traffic also

¹⁶<https://arstechnica.com/information-technology/2007/09/security-expert-used-tor-to-collect-government-e-mail-passwords/>

¹⁷<http://gitlab.torproject.org/legacy/trac/-/wikis/doc/ReportingBadRelays>

¹⁸<https://web.archive.org/web/20220416013422/https://matt.traudt.xyz/posts/2019-10-17-you-want-tor-browser-not-a-vpn/>

The Tor Browser is a modified version of Firefox that is designed to work with the Tor network. It is configured to use the Tor network by default, and it has additional privacy features to help protect your anonymity.

9.3.2 Timing Attacks

One drawback of using Tor is that your ISP can see that you are using it. This is because the Tor nodes are public. Additionally, the fingerprint of a Tor connection is different from that of a regular HTTPS connection. Think about a situation where you're a journalist: you're the only one using Tor in your area, and you publish a news article related to your area. Although you are anonymous and nobody sees who published it, you can easily be de-anonymized. This is because the ISP sees that, for example, at 12:03, someone in area A connects to Tor, and at 12:05, a document related to that area is published. So it is highly likely that you are the one who published that document. This is called a *Timing Attack*.

Additionally, security organizations might even collaborate with your ISP to drop your connection and see if that anonymous individual goes offline as well. If they do, it becomes clear that you were the one hiding behind that anonymous persona. When your power goes out or your internet connection drops, that person will also disconnect, revealing the link between you two. They could also perform a DoS attack on that Tor node or on you, and if you get disconnected, it might indicate that it was you.

See also "Correlation Counting Attacks" from "Hitchhiker's Guide"¹⁹.

Timing attacks are serious. There is no way to avoid them unless more people start using Tor. If more people use Tor, it becomes difficult to identify who published that document. This is the exact reason that Tor was made public by the US Navy; they wanted to hide their communication in the crowd. The more people use Tor, the more secure it is. This is why it is important to use Tor even if you don't need it—to help others.

Traffic correlation will never be completely solved. If a global collaboration between countries occurs, in theory, it would be possible to identify individuals.

Another scenario is that if I want to find out who someone is on Tor, I might send them a video link from a well-known site. If that person accesses the site without using Tor, their IP address gets exposed. Since it's a popular site, they might not think it could be a trap to gather their information.

When someone sends you a link, don't click on it right away. It becomes very obvious that you were the one who accessed the website just seconds after receiving the message.

¹⁹<https://anonymousplanet.org/guide/>

Bomb In Harvard

The story goes that a Harvard student wanted to have his final exam canceled. He sent an email to the university using a disposable email service called “Guerillamail” and claimed that there was a bomb in the university. The email was sent through Tor:

shrapnel bombs placed in science center, sever hall, emerson hall,
thayer hall, 2/4. guess correctly. be quick for they will go off soon.^a

^a<https://www.justice.gov/usao-ma/pr/harvard-student-charged-bomb-hoax>

Keep in mind that when you send an email, the sender’s IP address is included in the header.

- How should I know that?

→ That’s why I say that without sufficient knowledge, if you take action, the chances of making a mistake are high! If you understand how email works, you would realize this. At the very least, you could have read the privacy policy and documentation of such email services!

In fact, this means that the IP address from Tor was sent and received by the university. So far, everything might seem normal since it’s a Tor IP, and there shouldn’t be a problem.

That’s true, but here’s the catch! The FBI noticed that an email had been received from Tor. They suspected that the sender might be at the university at the time the email was sent. The FBI looked into the university’s Wi-Fi network to see who was connected to Tor at that time. One of those individuals was named “Eldo Kim.” The FBI approached him and asked if he was the one who sent the email, and he confessed that it was indeed him.

As we mentioned, when you connect to Tor, it’s evident that you are using it, but they can’t tell where you went or what you did. In this case, the FBI wasn’t sure if this person had actually sent the email because the Tor network obscures that information. If many people had been using Tor at that time, the FBI wouldn’t have been able to make everyone confess.

He could also have not used Tor in the university. As almost all universities, do log the activities of their students, they could have seen that he was using Tor. Additionally, it is a high chance that you would get de-anonymized if you use Tor in a network that you want to connect to that same network. (e.g., using Tor in university to send the message to the university).

- Should we use VPN with Tor? To hide the fact that we are using Tor?

→ Generally speaking, No. VPNs are not for anonymity. They may record your activities and help deanonymize you.²⁰

²⁰Might be helpful: <https://gitlab.torproject.org/legacy/trac/-/wikis/doc/TorPlusVPN>, <https://tails.net/support/faq/index.en.html#vpn>, <https://matt.traudt.xyz/posts/2016-11-12-vpn-tor-not-net-gain/>

Since the fingerprint of a Tor connection is different from HTTPS and Tor relays are public, some countries are able to block Tor. However, there are some Tor relays that are private and not public. These are called *Bridges*. They are mostly used in countries that block Tor. If a country blocks Tor, you can use bridges to access it. However, if the country knows the IP addresses of the bridges, it can block them. This is why bridges are not public. For example, meek-google is a bridge that makes Tor traffic look like Google traffic. This is why it is hard to block.²¹ Or, see Snowflake.²² Or, see “Roger Dingledine - The Tor Censorship Arms Race The Next Chapter - DEF CON 27 Conference” from “DEFCONConference” YouTube channel²³ which describes how Tor is blocked in different countries.

9.3.3 Dangerous behaviors lead to de-anonymizing

In this section, we will explore a number of actions you should avoid while using Tor, as they can lead to the exposure of your true identity. Unfortunately, for years, the conditions for using Tor have not been clearly communicated; instead, it has simply been promoted. This misleading promotion causes users to engage with Tor without proper knowledge, leaving them unaware that they can still be identified!

Don’t use Brave, Firefox, or any other browser instead of the Tor Browser for anonymity

You might think about using Firefox while routing your data through Tor and assume that it’s as if your browser is Tor, believing you are anonymous. While it’s true that your data passes through Tor, many issues can arise. These browsers are not designed for anonymity! You can easily expose yourself in various ways.

One of the smallest issues that can occur is a DNS leak. This means that while your regular data is routed through Tor, your DNS requests are sent normally, revealing your original IP address!

These browsers do not have certain anonymity features enabled. For instance, some settings may inadvertently contribute to creating a unique fingerprint for you. Therefore, it’s best to use only the Tor Browser, avoiding unofficial versions or other browsers.

One day, while browsing the Tutanota website, I noticed a suggestion that if you encounter an error with the Tor Browser, you could use another browser that is proxied through Tor.

²¹<https://blog.torproject.org/how-use-meek-pluggable-transport/>

²²<https://support.torproject.org/censorship/what-is-snowflake/>

²³https://www.youtube.com/watch?v=ZB80Dpw_om8

9.3 The Onion Router (Tor)

“ If you are getting this warning on the Tor browser, you can Launch another browser, using the running Tor instance as a proxy.^a

^a<https://web.archive.org/web/20230823193652/https://tutanota.com/blog/best-encryption-with-kdf>

— Tutanota

This is a very dangerous recommendation! Especially coming from Tutanota, a service that focuses on protecting people's privacy! It really needs a warning stating that this practice is not suitable for maintaining anonymity, and if you want to stay anonymous, you should avoid it!

For example, the Brave browser displays a message when you open a window that routes data through Tor, stating:

“ With Tor connectivity, it becomes more difficult^a for sites to see your true IP address and for network observers to see what sites you visit. However, if your personal safety depends on remaining anonymous, use the Tor Browser instead.

^aNot impossible

— Brave

Tor itself says:

“ We strongly recommend against using Tor in any browser other than Tor Browser. Using Tor in another browser can leave you vulnerable without the privacy protections of Tor Browser.^a

^a<https://support.torproject.org/tbb/tbb-9/>

— Tor Project

Download OR Opening Files

Absolutely avoid downloading any files unless it is absolutely necessary!

- Well, I'll download it, but I'll scan it with antivirus software before opening it! Or I'll even check it on VirusTotal to make sure it's not malware!

→ First of all, no antivirus software can detect all malware. Second, sometimes your system may have vulnerabilities that can be exploited simply by downloading a file. In the past, there have been vulnerabilities that allowed infections just from downloading a file.

Third, why are you downloading a file from Tor in the first place? Surely it's to remain anonymous. However, scanning it with antivirus software could actually lead to your identification.

9.3 The Onion Router (Tor)

- How so?

→ Remember when I mentioned that most antivirus programs send file hashes, or sometimes the files themselves, to the company for analysis? It can be determined that someone downloaded a specific file from Tor, and that file was later scanned by an antivirus program. This could lead to your identification.

- Alright! I'll open the file in a virtual machine or a sandbox environment!

→ The Tor Browser can only keep the data passing through it anonymous. However, sometimes when you open certain files, they may establish a connection to a website and send or receive information, which can lead to your real IP being exposed outside the browser. It doesn't matter if it's in a sandbox or elsewhere because the connection is made outside of Tor, revealing your IP.

Or the file could even be malicious, like a harmful image, and opening it could lead to your exposure! ²⁴ Tor Project says²⁵:

“ Tor Browser will warn you before automatically opening documents that are handled by external applications. DO NOT IGNORE THIS WARNING. You should be very careful when downloading documents via Tor (especially DOC and PDF files, unless you use the PDF viewer that's built into Tor Browser) as these documents can contain Internet resources that will be downloaded outside of Tor by the application that opens them. This will reveal your non-Tor IP address. If you must work with files downloaded via Tor, we strongly recommend either using a disconnected computer, or using *dangerzone*^a to create safe PDF files that you can open. Under no circumstances is it safe to use BitTorrent²⁶ and Tor together, however.

^a<https://dangerzone.rocks/>

^a<https://blog.torproject.org/bittorrent-over-tor-isnt-good-idea/>

— Tor Project

Don't use the Tor browser alone!

If a connection is made outside of Tor, your real IP address can be exposed. There are some operating systems that are designed to prevent any connection outside of Tor. All data passes through Tor. Additionally, no software is free of bugs²⁷. If a website can exploit a vulnerability in your browser, it can de-anonymize you. This is why we prefer to use sandbox environments. Read about Qubes OS, Tails OS²⁸, and Whonix.

²⁴interesting: <https://blog.reversinglabs.com/blog/malware-in-images>

²⁵<https://support.torproject.org/>

²⁷Firefox, Google Chrome

²⁸Tails is mostly used in situations where no data should remain on disk after the session.

9.3 The Onion Router (Tor)

Don't use Windows or macOS

Windows and macOS are not designed for privacy. No matter how much effort you put into disabling data collection features, there is still telemetry and other features that can leak your information. Additionally, they are closed source. This means that you can't see what is happening in the background. There is a higher chance that there are backdoors in these OSs.²⁹ This is why we prefer to use open source operating systems. Read about Qubes OS, Tails OS, and Whonix for anonymity.

Don't install any other add-ons or change any settings

As I mentioned in the fingerprinting section, any change can be used to identify you. This is why we prefer to use the Tor Browser as it is. It is designed to make all users look the same. If you change something, you are no longer blending in.

The only change you're allowed to make is to change the protection level ("Standard", "Safer", "Safest") in the Tor Browser.

It is also obvious that you should not use an outdated version of the Tor Browser.

Do NOT do illegal things

Doing bad things is wrong! Don't do it! Would you want to harm yourself or others? No! So, don't engage in such actions! Even if it seems funny or entertaining, it's not worth it! You will understand this when you get older and face years in prison. It's not worth it! You can't be anonymous forever. You will be caught. It's just a matter of time.

Security organizations regularly monitor websites that are known for illegal activities by using vulnerabilities³⁰ or dealing with the creator of the website. Some are even created by them as honeypots.

Some scenarios

Never share any information that you think might identify you. This includes providing your phone number, address, country, or even the continent you live on.

Scenario 1: For example, imagine you are a journalist working on a story. You might inadvertently write something like: "Today, I was passing by a certain place when an incident occurred." This essentially reveals where you live, and they could check the cameras to see who was there at that time!

Well, I won't describe myself! I'll write: "Today, I was informed that..."

²⁹Why did I say more? Because no one has read all lines of Linux.

³⁰FBI Admits It Controlled Tor Servers Behind Mass Malware Attack: <https://www.wired.com/2013/09/freedom-hosting-fbi/>

9.3 The Onion Router (Tor)

Eventually, you might slip up. For instance, if I want to find you, I might write: “By the way, a certain car came by, but it didn’t fire any shots.”

Then, if you see I’m mistaken, you might correct me by saying: “No! That car came by, but it fired a couple of shots.”

In doing so, you inadvertently reveal that you were the one who witnessed the scene! And whether you were there or not, they will definitely check the cameras and so on.

Scenario 2: For instance, suppose you want to leak a sensitive document, but you don’t realize that only a limited number of people had access to that specific document. Once it gets leaked, they will know that three people were aware of it. They can then check who has accessed that document recently and track you down.

Keep in mind that company logs are usually very precise. They clearly show which individual accessed which document, when they accessed it, or even if they copied or sent it! So, they can trace you from that as well!

Not only that, some companies change the document slightly for each person they want to share it with. This way, they can identify who leaked the document.³¹

Scenario 3: For example, suppose you upload a screen recording showing that you visited a certain website and performed specific actions, and you share this video on Tor, unaware that you accessed the website using a regular internet connection! This means I can check the logs of that website to see who accessed that particular section. I could potentially find your real IP address.

- Well, you might not know when I accessed that website. Which day’s log do you want to retrieve?

- First of all, the IPs of exit nodes are public and identifiable, so I just need to filter out the IPs associated with Tor. Additionally, with the video you recorded, I might be able to determine your operating system and apply filters to narrow down your identity even further!

- How can you tell which operating system I’m using?

- If you’re on Windows or Mac, it’s obvious. That leaves BSD-based and Linux-based systems. I can often tell which operating system you’re using based on certain elements, like the wallpaper, since most operating systems come with customized wallpapers that are included with each new version.

- I can even figure out whether you’re using a desktop computer or a laptop.

- How can you do that?

- If there’s a battery icon in the taskbar, I know it’s a laptop (or at least a portable device). Also, you might not have noticed that at some point, the website or the taskbar at the top or bottom of your screen displayed the date

³¹How Apple catches leakers: From color changes to comma placement: <https://9to5mac.com/2023/05/11/how-apple-catches-leakers/>

9.3 The Onion Router (Tor)

or time... that makes it much easier! I can check the logs for that specific day or time.

Scenario 4: They post a Photoshopped image of a person doing something bad to shame you. You respond to them, saying that the picture is fake, using a fake account registered on the same device or network as your other identity. This is a big mistake! You are linking your two identities together.

Don't use a session for two purposes

Scenario 1: For example, if you log into a website using two different accounts, those accounts can be linked together, revealing that both belong to you. To prevent this, Tor has a feature called "New Identity," which effectively changes the circuit to avoid such connections.

Remember, I've always emphasized that one of the most important things is to read the documentation of the service you are using to understand how to work with it and grasp its nuances! For instance, Tails states that when you use the "New Identity" feature, only the Tor Browser changes, while the operating system retains the previous circuit, which is not ideal. Therefore, it's better to restart Tails.³²

Be aware that engaging in risky behaviors, such as logging into a website with one identity and then using "New Identity" to log in again shortly after with a new identity, can raise red flags. It's clear that you might be the same person who just changed your connection. For example, if ten people are logged in through Tor and one person logs out, then another person logs in right after.

So, keep in mind that while you may have changed your identity, with a little attention, it can be deduced that the person who was active ten minutes ago and performed certain actions is the same person returning after ten minutes to perform similar or even different actions.

Scenario 2: You might enter a website using Tor and send the link to someone else within the same Tor network through a secure chat room. Everything happens within Tor. After you send the link, that person logs in, and if I am the owner of the website, I would notice that another Tor IP address entered immediately after yours. This suggests that they are likely a friend or associate of yours, or at least connected to you in some way. Therefore, I can link you two together.

This likelihood increases significantly for websites with fewer visitors, as the chances of two people being on my site at the same time, both using Tor and both navigating to the same page, are quite low!

³²<https://tails.boum.org/doc/about/warnings/identity/index.en.html#contextual>

9.4 Metadata

Use Tor consistently

Even if you do not use Tor just once, you can be de-anonymized. This is further discussed in the “Silk Road” section.

Check configurations every time after an update

After an update, the settings might be changed. So, it is important to check the settings after an update.

Be careful about people and cameras around you

If you are in a public place, there might be cameras that can see you.

9.3.4 Reading List

- Hitchhiker’s Guide on Digital Footprint³³ (Highly recommended)
- Hitchhiker’s Guide on IRL and OSINT ³⁴
- Tails OS documentation³⁵
- Whonix documentation³⁶
- Search for blogs of people who are behind Tor³⁷

9.4 Metadata

Each file on a computer has a certain structure. The computer interprets the data and knows how to display it to you. For example, for a photo, the data indicates the color of each pixel. Alongside this data, there is metadata. Metadata includes additional information about the file. For example, for a photo, metadata includes the date and time the photo was taken, the camera model, the location, and even the name of the person who took the photo. This information is stored in the file itself. When you share a photo, you also share this metadata. This is why it is important to remove metadata before sharing a file.

For example, John McAfee was hiding from the police. During an interview with a journalist, the journalist took a photo of him and shared it on Twitter. The police were able to track him down by analyzing the metadata in the photo.

³³<https://anonymousplanet.org/guide/#your-digital-footprint>

³⁴IRLandOSINT

³⁵<https://tails.net/doc/>

³⁶<https://www.whonix.org/wiki/Documentation>

³⁷<https://www.torproject.org/about/people/>, e.g. <https://matt.traudt.xyz/>

9.4 Metadata

The metadata revealed the location where the photo was taken, leading to his arrest.³⁸

Almost Any file you share can contain metadata. This includes photos, videos, documents, and even PDFs. For example, one technique that the police can use to locate a stolen camera is to check the metadata of suspicious photos to see if the camera in question can be identified. This is possible because we know that photos taken with any camera include information about the camera's specifications, model, and the location of where the photo was taken in their metadata.³⁹

Consider whether you want to know if a student created a project by themselves or copied it from the internet. You can check the metadata of the file to see when the file was created. If the file was created before the project was assigned, it's likely that the student copied it from the internet. Or you can check who created the file. If the file was created by someone else, it's likely that the student copied it from the internet.

Metadata can be found in various contexts. For instance, the main content of an email consists of its text and data, but when we say that a service collects email metadata, we refer to information such as the email's subject, sender and recipient details, sending and receiving times, email size, and so on.

It's important to note that metadata might not seem very relevant at first glance (like the time a message is sent), but when combined, it can reveal a wealth of information. For example, if you message someone every day, that person is likely a close friend or family member, or perhaps a work colleague.

This can lead to the creation of a network showing how close you are to certain individuals. For instance, if a terrorist consistently sends messages to someone before carrying out their plans, that person is likely an accomplice. Even without the actual content of the messages, a lot can be inferred from the metadata.

For example, consider an individual involved in a covert operation. While their activities may be hidden, we can observe that every time this person enters a specific area, an email is sent to a particular address. From this temporal correlation, we can deduce that it is likely this individual who is sending the email.

Well, we've talked a lot about metadata. Is it really that impactful? If you haven't realized its significance through our discussion so far, let me draw your attention to two quotes:

“ We kill people based on metadata. –Micheal Hayden^a

^aRetired United States Air Force four-star general and former Director of the

— *Micheal Hayden*

³⁸<https://www.npr.org/sections/thetwo-way/2012/12/04/166487197/betrayed-by-metadata-john-mcafee-admits-hes-really-in-guatemala>

³⁹<https://web.archive.org/web/20110220205639/http://www.stolencamerafinder.com/>

9.4 Metadata

“ National Security Agency, Principal Deputy Director of National Intelligence, and Director of the Central Intelligence Agency: from WikipediaCreative Commons Attribution-ShareAlike 4.0 License

— Micheal Hayden

“ Metadata absolutely tells you everything about somebody’s life. If you have enough metadata you don’t really need content.^a –Stewart Baker^b

^a<https://www.wired.com/2015/03/data-and-goliath-nsa-metadata-spying-your-secrets/>

^bformer General Counsel of the National Security Agency

— Stewart Baker

9.4.1 What to do?

Remember, metadata is stored in the binary of the file. Therefore, we can use tools to delete or change that binary.

The first point is that the more complex a file is, the less likely it is that its metadata can be removed. Some types of metadata cannot be deleted at all. Therefore, it’s advisable to use simpler formats like `.txt`, which contain minimal metadata.⁴⁰

The second point is that even if you remove metadata from a file, it’s possible that the file itself contains information that can be used to identify you. For instance, a photo of you in your room can be used to identify you, even if the metadata is removed.

LibreOffice

In LibreOffice, you can go to `Tools => Options...` => LibreOffice to specify which metadata should be saved. Keep in mind that, due to the complexity of document files, removing metadata from their components can be a bit challenging. Therefore, it’s best to delete the metadata from each individual file and image used in your document beforehand.

Linux Tools

- `mat2 file_name` – for most common file formats
- Metadata Cleaner⁴¹ (uses `mat2`)
- `exiftool -all= file_name`

In discussions about privacy and security, always be cautious about trusting others! The person may not have sufficient knowledge. So, ask them why they

⁴⁰https://tails.boum.org/doc/sensitive_documents/index.en.html

⁴¹<https://metadatacleaner.romainvigier.fr/>

9.4 Metadata

claim that mat2 and Metadata Cleaner can effectively remove metadata. Is it really effective? What is their source? Are they just talking nonsense?

I learned about these two tools from the Tails website.⁴²

Android

Scrambled Exif removes image metadata.

9.4.2 Reading List

- What is Metadata (with examples)⁴³ A very good website to see real examples of metadata.

9.4.3 Silk Road

The story revolves around a website called Silk Road, which was one of the largest drug marketplaces on the dark web. The FBI had been searching for its creator for a long time. However, as you may know, the Tor network concealed both the identities of its users and the real IP addresses of its servers.

In an effort to find the creator, the police began searching the internet for old discussions about the website. They hoped that by connecting with people who were among the first to talk about Silk Road, they might get closer to identifying its creator.

They came to a post⁴⁴ by “Altoid” on a website about mushrooms:

I came across this website called Silk Road. It's a Tor hidden service that claims to allow you to buy and sell anything online anonymously. I'm thinking of buying off it, but wanted to see if anyone here had heard of it and could recommend it. I found it through silkroad420.wordpress.com, which, if you have a tor browser, directs you to the real site at <http://tydgccykixpbu6uz.onion>. Let me know what you think...

They get suspicious. They searched about “Altoid”. They got several results:

⁴²https://tails.boum.org/doc/sensitive_documents/metadata/index.en.html#index2h1

⁴³<https://dataedo.com/kb/data-glossary/what-is-metadata>

⁴⁴<https://www.shroomery.org/forums/showflat.php/Number/13860995>

Result 1

What an awesome thread! You guys have a ton of great ideas. Has anyone seen Silk Road yet? It's kind of like an anonymous amazon.com. I don't think they have heroin on there, but they are selling other stuff. They basically use bitcoin and tor to broker anonymous transactions. It's at <http://tydgccykixpbu6uz.onion>. Those not familiar with Tor can go to silkroad420.wordpress.com for instructions on how to access the .onion site.^a

^a<https://web.archive.org/web/20150319080002/https://bitcointalk.org/?topic=175.70\protect\leavevmode\ifvmode\kern+.2777em\relaxwap2>

Result 2

IT pro needed for venture backed bitcoin startup
...If interested, please send your answers to the following questions to rossulbricht at gmail dot com^a

^a<https://bitcointalk.org/index.php?topic=47811.msg568744#msg568744>

Result 3

How can I connect o a Tor hidden service using cURL in PHP?^a

^a<https://stackoverflow.com/questions/15445285/how-can-i-connect-to-a-tor-hidden-service-using-curl-in-php>

The email behind this post was 'rossulbricht@gmail.com'; he later changed his username on StackOverflow to 'Frosty'.

The pieces of the puzzle are starting to come together. Ross, who is interested in Silk Road, is looking for someone to help with his business who is an expert in Bitcoin. Meanwhile, our main character is facing some issues with the hidden service...

At one point, when they suspected Ross Ulbricht, an undercover FBI agent was investigating him and stumbled upon his YouTube channel. He noticed that Ross's YouTube username was familiar: 'ohyeaross'.⁴⁵ The agent recalled that whenever he chatted with the admin of Silk Road, that person would write 'Oh yea' instead of 'Oh Yeah,' omitting the final 'h.' This became another reason for them to be more certain that Ross was indeed the admin of the site.

NOTE: Try to avoid unnecessary chatting, as your writing style might reveal your true identity. This is especially important in a world where artificial intel-

⁴⁵<https://www.youtube.com/@ohyeaross/about>

ligence can scan the entire web for writing patterns similar to yours, potentially leading to your identification on the clear net. If it's essential to communicate, make sure not to use your real-world writing style at all. How you code is also a part of your writing style. See also [48].

Due to a problem with the website's configuration, the IP address of the main Silk Road server was exposed. The FBI agents went to the server and noticed that several admin IPs had connected without using Tor⁴⁶. In fact, they discovered that one user had accessed it from a coffee shop.

Can you guess where that coffee shop was? That's right, it was near Ross's home! It seems that the puzzle is becoming more and more complete.

Although the evidence was strong, they still wanted to catch Ross in the act. They wanted to catch him while he was logged in to the website.

One day, while Ross was sitting in a café and doing his work, the police saw that the admin was logged in. Subsequently, they staged a fake fight behind him. When he turned to see what was happening, one person grabbed his laptop while another apprehended him. This is how Ross was arrested.

Warning

The lesson here is: don't try to start something that strongly requires anonymity until you have gained enough knowledge. Even a small mistake in the beginning can lead to your downfall.

When the police go bad!

Remember when we were discussing privacy and its importance, and we asked who watches the watchers? We mentioned that those in power might abuse it. This is a perfect real-life example! The police go rogue, and their focus shifts to all the money they can seize!⁴⁷

- Why are all the examples you give about people who did something illegal and got caught by the FBI? We're not doing anything wrong, so we're safe!

→ No! Look, security organizations certainly won't announce, "Hey, look, we just arrested a journalist who was criticizing us!" They won't say that. But when they catch bad people, they make a big deal out of it, saying, "Hey everyone, look, we're ensuring your safety!" This way, they can justify asking for more funding and power.

It's very important not to share sensitive information with anyone. For example, Ross confided in his girlfriend, and she, in turn, told her friend. There was even a time when she was upset with Ross and posted on Facebook that

⁴⁶Which is something wrong they did.

⁴⁷Stealing bitcoins with badges: How Silk Road's dirty cops got caught: <https://arstechnica.com/tech-policy/2016/08/stealing-bitcoins-with-badges-how-silk-roads-dirty-cops-got-caught/>

9.5 Behavior on Social Media

people would find it very interesting to know he was running a drug website. This small detail could have easily exposed Ross.

Additionally, towards the end, Ross had become overconfident, thinking that since they hadn't been able to catch him for such a long time, they would never be able to do so.

You can read the full story in the book 'American Kingpin: The Epic Hunt for the Criminal Mastermind Behind the Silk Road.'^[49] If you're a Persian speaker, I highly recommend you to listen to ChannelBPodcast.⁴⁸

9.4.4 Reading List

- Thirteen Years of Tor Attacks^[50]
- Speculative Tor Attacks⁴⁹
- Multi-hop VPN⁵⁰
- Combating Traffic Correlation Attacks - Decoy Traffic Mode⁵¹

9.5 Behavior on Social Media

When we want to join a social media platform, forum, website, or email service, the first step is to register. However, there are some important points we should keep in mind during this process.

9.5.1 Registration

Only Register When Necessary

Don't register unless you have to. There's no need to sign up for every site you visit. Some websites may not be secure and could misuse your cookies, IP address, email address, and password to compromise your other accounts (or attackers may hack into their accounts and use the information). This is especially true for many Iranian websites, which often lack proper security measures.

⁴⁸1: <https://channelbpodcast.com/archives/3209>

2: <https://channelbpodcast.com/archives/3220>

3: <https://channelbpodcast.com/archives/3289>

4: <https://channelbpodcast.com/archives/3453>

⁴⁹https://www.whonix.org/wiki/Speculative_Tor_Attacks

⁵⁰Proton VPN: <https://protonvpn.com/support/secure-core-vpn/>

⁵¹<http://windscribe.com/blog/combating-traffic-correlation-attacks-decoy-mode/>

Avoid Using Personal Information for Registration

When registering, avoid using your birth date, city name, or similar personal information in your ID, username, or account name. For example, names like Alex1994 or AlexLondon2000 can be easily traced back to you.

Try to choose a username that is longer and more difficult to guess, making it harder for someone to use brute-force methods to access your account. Often, people looking to find you on other social media platforms will try names similar to yours or variations of your existing accounts using scripts (automated codes that perform tasks).

Password

Never use the same password for different accounts. If a website does not securely store your password (e.g., not using strong hash algorithms) and its data is compromised, attackers can gain access to your other accounts that share the same password.

Change your passwords regularly. If a website is hacked and your password gets leaked, changing it will likely (though not always, as many websites do not log out other devices that are already logged in) stop attackers from accessing your account. However, if you do not change it, they can continue to access your account even after the website has been hacked.

While we recommend changing your passwords every few months, this does not mean you should get lazy and revert to using old passwords. For instance, some people might change their GitHub password to the same one they used for their Stack Overflow account, or they might just add a letter or two at the end. This is a mistake. If your password has been leaked on another site, attackers can use automation to test variations of it by simply adding extra characters.

For example, if your password is "AlexJia," they might try "AlexJia2003" as a variation. Just adding numbers or letters to the end of an old password is not a secure approach. Make sure to change your password completely and create a new one.

Do not get too creative^{52, 53}. There are extensive lists of passwords based on popular songs, well-known words from various languages, and different dialects. The best approach is to use a completely random and long password.

If you find it hard to remember such passwords, consider using a password manager. This way, you only need to remember one random password, and with a little practice, you will be able to memorize it in just a few days.

If you are a developer, please avoid implementing unnecessary restrictions, such as disallowing special characters in passwords. Such limitations reduce the entropy of the password, making it less secure.

⁵²e.g., mir is not more secure than amir

⁵³See *Bad Idea!!! - Lyrics, Quotes & Phrases as PASSWORDS*: <https://passwordbits.com/bad-idea-lyrics-quotes-phrases-as-passwords/#more-1439>

9.5 Behavior on Social Media

Do not use websites that provide feedback for your password. For example, some websites offer a service where you can check if your password has been leaked or if it is strong enough. This is a bad idea because you are sharing your password with them, and they might not be trustworthy. Even if they claim to be secure, you cannot be sure. It is better to use a password manager that does not require you to share your password with them.

9.5 Behavior on Social Media

10.0 Security in Action

This section is small because we have discussed most of these topics in previous sections. Here, we will cover some additional practical points.

10.1 Log Analysis

A log is a record of events that occur in a system or network. For example, see the following log entry:

Listing 10.1: Log entry example

```
2025-10-01 12:34:56 [INFO] User 'admin' logged in from IP address
2025-10-01 12:35:00 [ERROR] Failed login attempt from IP address
2025-10-01 12:35:05 [INFO] User 'admin' logged Out
2025-10-01 12:35:10 [INFO] User 'admin' accessed file 'sensitive_data.txt'
```

Logs show which files were accessed, when they were accessed, and by whom. This information can be used to identify unauthorized access to sensitive files or data.

Logs can also be used to investigate security incidents after they occur. By analyzing the logs, security teams can determine what happened, how the incident occurred, and what steps need to be taken to prevent it from happening again.

Furthermore, if lots of failed login attempts are detected from a single IP address, it may indicate that someone is trying to attack the system.

A security expert would check write a program to analyze the logs and find out if there are any unusual patterns.

To analyze logs, you need to be able to code and use terminal commands. In many servers, only terminal commands are available. You need to be able to write scripts to automate the process of analyzing logs.

- What are terminal commands?

→ When you use a computer, you usually use the graphical user interface (GUI) to interact with the system. However, there is another way to interact with the system using text-based commands. This is called the command line or terminal. In the terminal, you can type commands to perform various tasks, such as

creating files, moving files, and running programs.

10.2 Bug Bounty

Companies often reward individuals for finding bugs, especially security vulnerabilities. The company announces that user security is important to them and organizes a program where, if someone finds a security issue in their services, they should report it. In return, the company rewards the person who found the issue. It's a win-win situation: the company becomes more secure, and the security expert earns money.

You might wonder why companies pay for bug discovery. First, a company might genuinely care about its users' security and privacy, viewing it as a user right, and thus invests money in it. Second, in countries where user rights are highly valued, such as in Europe, if user data is compromised, the company faces very heavy fines, potentially tens of millions of dollars. Therefore, a company might reason that spending, for example, two million dollars on bug bounties is preferable to risking a fifty-million-dollar fine. So, even if a company isn't inherently motivated, they are often compelled to do so.

Another question might be: doesn't the company have its own security team to check the programs? Why expose themselves to unknown individuals for testing? Couldn't an unknown individual, after finding security issues, hide the problem, not report it, and sell it to malicious actors?

Indeed, every company has security specialists who regularly conduct tests and reviews to ensure the system is secure. However, this is not always sufficient. Everyone has a different perspective, and no one knows everything. So, while you might have ten people conducting tests, isn't it better to have hundreds of people testing? Certainly, more bugs will be found, and you will become more secure. Also, someone within your company is salaried and may lack the "thirst" for finding bugs. They might think, "I get my salary anyway. I'll find whatever vulnerabilities I can." But they may lack that intense drive. In a bug bounty program, however, individuals are eager to find more bugs. You only pay them when they find a bug. It's a win-win. The more they secure you, the more money they earn, so they have a strong motivation.

But can unknown individuals be trusted? What if they find a bug and don't report it? Well, when you conduct tests yourself subsectionSHA-3 SHA-3 is the latest member of the Secure Hash Algorithm family. It was released by NIST and originated from a competition called the SHA-3 competition. The goal was to find a new secure hash algorithm. First, some workshops were held to discuss the requirements for the new algorithm. Then, NIST invited the world to submit their algorithms for the competition. After five years of competition, Keccak was selected as the winner. It is now known as SHA-3. Hooray! We have a new secure hash algorithm.

Now you see why we say that self-created hash functions (and any security

protocols) are not necessarily secure: five years of research went into this!

Let's delve deeper into the SHA-3 project.¹

“ As was the case for the AES competition, security is the most important factor when evaluating the candidate hash algorithms. However, there remains significant disagreement within the cryptographic community at large over what security definitions should be used to evaluate hash algorithms. While initially proposed for use in digital signatures, cryptographic hash algorithms are used in a wide variety of applications, including message authentication codes, pseudorandom number generators, key derivation, and one-way functions for obfuscating password files. All of these applications have different security requirements.

— NIST

It is obvious that a hash should be secure. A hash that does not have the properties we discussed is not secure.

“ FRN-Nov07 identified cost as the second-most important criterion when evaluating candidate hash algorithms. In this case, cost includes computational efficiency and memory requirements. Computational efficiency essentially refers to the speed of an algorithm. NIST expects SHA-3 to offer improved performance over the SHA-2 family of hash algorithms at a given security strength. Memory requirements refer both to code size and random-access memory (RAM) requirements for software implementations, as well as gate counts for hardware implementations.

— NIST

If something is to be used worldwide and by any device, it must be fast. Don't just consider your laptop; think about a smartwatch or a smart card. They have very low computational power. It must be fast for them too. Additionally, it must be memory efficient.

“ The SHA-3 competition has received many candidate algorithms with new and interesting designs, and with unique features that are not present in the SHA-2 family of hash algorithms. Candidate algorithms with greater flexibility may be given preference over other algorithms. This includes algorithms capable of running efficiently on a wide variety of platforms, as well as algorithms that use parallelism or instruction

— NIST

¹<https://csrc.nist.gov/Projects/hash-functions/sha-3-project>

“ set extensions to achieve higher performance. In addition, simple and elegant designs are preferable, in order to encourage understanding, analysis, and design confidence.

— NIST

As we discussed regarding falsifiability, in order to analyze and implement different attacks on an algorithm, it must be simple and elegant. If it is complex, it is hard to understand and analyze. It is hard to find vulnerabilities and design attacks on it or hire a company for penetration testing, they too might find a critical bug and not report it, instead selling it on the black market or the dark web. Even your own employees might do this. They’ve received their salary, say \$20,000 for testing. They might report four out of five bugs found and keep the last one to sell on the black market for additional profit.

In bug bounty programs, however, individuals are motivated to report bugs. The more they report, the more money they earn. Also, suppose one person decides not to report and tries to sell it in illegal markets. Okay, but there are other ethical individuals who will likely find that vulnerability and report it sooner. There’s competition. If you hold onto it, someone else might find it and get the reward. So, you’d prefer to report it yourself. Reporting bugs also builds your resume. If you want to work for a company later, you can show your resume detailing how many important vulnerabilities you’ve found. The recruiter will then realize you have good knowledge because you’ve reported so many bugs and will hire you.

- Wow, that’s great! So I’ll go learn about different vulnerabilities to get rich.
→ Hold on! It’s not that simple. Like any other job, this work has prerequisites (we’ve discussed the need for knowledge). And there are rules. You can’t just go around conducting tests. If you do that, you’ll likely do something illegal, be considered a criminal, and end up in jail.

They specify that you can conduct tests on certain parts of the website or application under specific conditions. You must not operate outside these boundaries. If you find a particular type of vulnerability, they might offer, for example, \$100, and for another type, \$200, and so on.

So, you write a detailed report according to their **guidelines** and submit it. They review it to determine if it’s valid. If it is, they give you the reward. (Sometimes the reward might not be monetary; for instance, they might offer a one-year subscription to their site.)

- What if we report the bug directly to them? Won’t they later claim they already knew about it, or that someone else had reported it earlier, and therefore your report isn’t valid, just to avoid paying?
→ Typically, if the bounty program is legitimate and the company is reputable, no. But yes, sometimes such situations can occur with some companies. Especially in countries where the law is not very strong, companies might try to avoid paying.

This is where platforms that act as intermediaries come into play. Essentially, some companies say, “Companies, come list your rules on our site and announce your presence so individuals can conduct tests. Whoever finds a vulnerability should report it to me. I will verify its validity and then pass it on to the company. I’ll collect the payment from the company and give it to the hacker.”

In effect, they act as intermediaries to ensure fairness and to check whether a bug is valid. For example, HackerOne and Bugcrowd are well-known international platforms.

Remember when I said that competition in bug bounties is high? The competition is so intense that if you find a bug and don’t report it, someone else might find and report it within a few hours! Yes, the competition is that high and intense. (This is why beginners might struggle to compete in this field. That’s why it’s often suggested to start with local platforms, which are simpler, have less competition, and where much simpler bugs can be found.)

This is also why I emphasized learning programming in a tool-oriented way, so you can automate tasks and find vulnerabilities faster than others.

- Oh, so I should go install Kali Linux (an OS which contains many security tools) and a thousand tools to find vulnerabilities.

→ Note that standard tools don’t make you a hacker. It’s your core knowledge that transforms you into a security engineer. Without knowledge, even with a million tools, you’ll at best find a couple of simple vulnerabilities and remain at that level. However, someone who acquires deep knowledge and uses a combination of standard tools and tools they’ve written with their knowledge will be successful.

Also, no one installs Kali on their main system. Since Kali is for testing, its settings must also be configured for testing tasks. Therefore, Kali is installed on a virtual machine (to avoid fingerprinting your default system).

Don’t experiment on friends’ machines without a solid understanding of what you’re doing. For example, you might deploy a piece of malware just to “see how it works,” not realizing that your friend’s laptop is on a shared Wi-Fi network—and now you’ve just infected other devices on that network.

Warning: A company’s public-relations address or general email inbox is not the right place to report a security flaw. First, they probably won’t understand what you’ve found, and second, if you post it there you may be blamed for leaking a vulnerability.

Warning: If you discover a vulnerability, don’t rush to publish it. Wait until it’s fixed—otherwise attackers will learn about it and exploit it. I know you’re excited to share your discovery, but first check the vendor’s disclosure policy and get their permission. Just getting it fixed is not enough; ask exactly what you’re allowed to say publicly and when.

Warning: Steer clear of untrustworthy bug-bounty programs. Some sites advertise huge payouts but either never pay or treat researchers badly. Watch

out for vague promises like “earn up to \$100,000 for finding a bug.” That range is meaningless: professional programs specify exact rewards (e.g., “\$100 for low-risk bugs, \$500 for medium, \$5,000 for critical”). If they only say “up to X,” they may pay you almost nothing.

10.3 End of the Chapter Questions

Problem 10.1 Hey, hope you’re doing well! I found a new site that tracks live gold and dollar rates—similar to the popular ones out there. They’re running a referral program: when someone signs up with your code, both you and I get bonus credit.

I’m really strapped for cash right now, so I’d appreciate it if you could help me by using my referral code: FREEGWT. It only takes a minute—just register with your own phone number and you’ll automatically receive the bonus.

When you have signed up, please let me know. Thanks so much!

10.4 End of the Chapter Solutions

Solution 10.1 The request to “please let me know when you have signed up” is likely so they can examine the site’s logs and identify the exact moment you registered. From those logs they can pull your email address, phone number, IP address and any other information you provided at signup. So if you thought you were pseudo-anonymous under your Telegram account, your information is now revealed.

10.4 End of the Chapter Solutions

11.0 Online Scams

11.1 Ponzi Schemes

11.1.1 How They Deceive You

Let's get familiar with a type of scam that's become increasingly common these days. It comes with attractive names and promises: you invest your money, and you might even double or triple your initial investment! But make no mistake—it's a scam.

These schemes often use appealing titles like:

- Investing in the US Nasdaq Stock Market
- Cloud Mining
- Cryptocurrency Trading AI

- Wait, is it really a scam? How can it be a scam if there's a chance to multiply my money? What kind of scam is this? + Stick with me, and I'll explain exactly how it works.

This scam works like this: someone comes and claims, for example, "I know how to make money," (In **any way**, it doesn't matter)

They say, "Just give me your money, and I'll pay you a profit." For example, they might claim, "Since I'm in America, I'll invest your money in the US Nasdaq stock market." Or, "I'm an expert at trading—I know exactly when to buy a cryptocurrency at a low price and when to sell it high. I'll trade for you and give you a share of the profits." Sometimes, they say, "We have a team and an AI-powered trading bot that will trade on your behalf." Or, "We know how to do something special that you don't, so let us handle it and we'll pay you returns." In short, they promise to use their supposed expertise or technology to make money for you—if only you trust them with your funds.

This is the essence of Ponzi schemes: they take your money and, using fancy names, promise you profits and income. Yes, they do pay out profits—but it's a scam!

- Why do they use such fancy names?

11.1 Ponzi Schemes

→ Because when we don't know much about something, it's easy for people to impress or deceive us by throwing around technical terms and jargon, making themselves sound like experts. For example:

The basis of trading is analyzing different coins based on the protocols they use. For instance, a certain coin uses a specific cryptography algorithm. But why is Bitcoin successful? Because it uses Elliptic-curve cryptography on Secp256k1 and also SHA2-256 for its hash. So, we look for coins that use Secp256k1 and Secure Hash Algorithm 2, or SHA-2. Not SHA-1! SHA-2, because SHA-1 has been broken and is no longer secure or recommended.

With SHA-1, you can generate a specific hash for any input you want.

See? I just wrote a paragraphs and threw in terms like Secure Hash Algorithm and Elliptic-curve cryptography. For someone without the background knowledge, it might sound impressive and make me seem like an expert. But really, I just said a bunch of nonsense! It's like if I don't know anything about carpentry, someone could easily sell me a table or chair at a higher price just by using technical terms. The same thing happens with cryptocurrency. Most of us don't have deep knowledge, so it's easy to be fooled by this kind of talk and think, "Wow, this person really knows their stuff!" and fall for their scam.

Often, these scammers promise you very high returns—sometimes even as much as 1% profit per day, or other unrealistic rates. They might say things like, "Invest with us and get 30% profit per month", or "We guarantee you a fixed daily income". These high returns are used to lure people in, but in reality, such profits are not sustainable or realistic.

● But wait! 1% profit per day is not much!

→ Are you sure? Let's play with the math

Think you want to invest \$100. At the end of the first day, you get \$1 profit. So now you have \$101. You can invest that \$101 again. for the second day, 1% of \$101 is \$1.01. So now you will get \$102.01. And so on. If you keep doing this for only a year, you will get:

$$\$100 \times (1 + 0.01)^{365} = \$3778.34$$

If you keep doing this for only 5 years (5 years is not much in the world of finance), you will get:

$$\$100 \times (1 + 0.01)^{(5 \times 365)} = \$7,700,291,275$$

Great! You just turned \$100 into \$7.7 billion in 5 years! What about just 10 years? You will get:

$$\$100 \times (1 + 0.01)^{(10 \times 365)} = \$5,93 \times 10^{17}$$

Congratulations! You are now able to buy several countries!¹ You tell me; It this even possible?

¹Ponzi Scheme Calculator: <https://ponzi.vercel.app>

11.1 Ponzi Schemes

If this was possible, governments would be doing it, and everyone would be rich. But the truth is, no one can guarantee such high returns consistently.

- Some may say that you can't use this formula because you can't reinvest the profit. Each account has a limit. For example, you can only invest \$1000 in one account.

→ No worries! You can still use this formula. Just reinvest the profit in another account. The problem is not about one individual account. The problem is that the whole system is a scam.

Consider the following table, which simulates a Ponzi scheme. In this example, we assume that each person pays \$10 to join the scheme, and the profit is \$1 per person per month. The scheme starts with one person, and each month, the number of new entries doubles. The scheme continues until the fourth month.

Time	New Entries	Total People	Money Entered by New Entries	Leftover from Previous Month	Profit to Pay (1 per person)	Leftover Money After Paying
End of Month 1	1	1	10	0	1	9
End of Month 2	2	3	20	9	3	$29 - 3 = 26$
End of Month 3	4	7	40	26	7	$66 - 7 = 59$
End of Month 4	8	15	80	59	15	$139 - 15 = 124$

Table 11.1: Simulation of profit and entries in a Ponzi scheme

As you can see, the scheme starts with one person. When the person gets their profit, they are happy and tell their friends about it. The next month, two more people join, and the scheme continues to grow.

The system continues to work, as long as new people keep joining. The more people join, the more money is available to pay profits. However, the scheme will eventually collapse when there are no new entries. By this point, the creator of the scheme takes the money and disappears. The system that was paying profits until yesterday suddenly falls apart.

This makes it incredibly challenging to convince people that they are being scammed and should not fall for it. Everyone is receiving profits and seems happy, like guests at a feast, oblivious to the fact that one day the scammer will clear the table and disappear with everything. When you warn them it's a scam, they often retort, "How can it be a scam if it's paying profits?" and refuse to believe you.

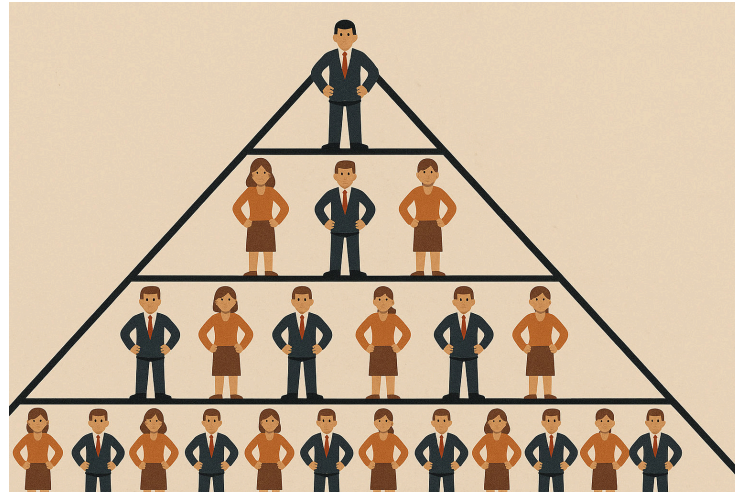


Figure 11.1: A simple pyramid scheme of a Ponzi scheme

For instance, someone might recoup their entire initial investment through profits after just a few months. If you then tell them it's a scam, they might say, "I've already made back all the money I put in. Even if it stops paying profits tomorrow, it doesn't matter to me since I've recovered my principal investment."

However, they are unaware of the source of these profits: the money is taken from newer investors and paid to earlier ones. By participating, they have inadvertently helped build this pyramid structure, keeping the scheme alive and allowing it to defraud even more people. When the scammer eventually disappears with the money, those early investors become complicit in sustaining the system. The newer investors, who are often numerous and have not yet recovered their initial investment through profits, end up losing everything.

In a Ponzi scheme, those who join early are likely to recover their entire investment as profit, and may even earn multiples of what they put in. However, investors in the lower tiers, particularly recent joiners, inevitably lose money because they don't receive enough profit to cover their principal. Consequently, the vast majority of participants suffer losses. Only the earliest investors profit, but this comes at the expense of everyone else's money.

Even if the creator is arrested, they have no money to give back to the people, because the initial investors each received several times their money in profit, and in practice, they themselves had become scammers.

As the system continues to work as long as new people keep joining, they try to attract more people by using various methods. Mostly by creating a False Sense of Legitimacy.

Exploiting Your Greed and The “Traditional Markets are Rigged” Narrative

Scammers try to evoke a fear of missing out (FOMO) by making enticing promises. For example, they might say: “Remember when Bitcoin was cheap or even given away for free? Those who got in early made a fortune. Now, my new coin is also free at the start. Grab some now, and it will become as valuable as Bitcoin, making you rich.” Or they might claim, “My coin will be the next Bitcoin. Bitcoin was also cheap initially, and now it’s worth a fortune. So, buy my coin now.”

In short, Bitcoin is unlike any other coin, and you should not compare any other coin to Bitcoin.

They say that traditional stock markets are designed to benefit large institutions rather than individual investors. They suggest that your money might be better placed in community-focused or alternative projects where you have more direct control or understanding of where your funds are going. This way, they try to convince you that investing in their scheme is a better option than traditional stock markets, which they claim are rigged against individual investors.

The Success Stories

Scammers often showcase testimonials from people who claim to have made significant profits through their schemes. These testimonials are usually fabricated or exaggerated, featuring individuals who appear to be successful investors. They might share stories of how they turned a small investment into a fortune, further enticing potential victims to join the scheme.

- The CEO has bought a new Lamborghini by using the same techniques that he/she teaches us!

→ Of course! He/she has a lot of money *from the people who have fallen for his scam*. He/she can buy whatever he/she wants. But that doesn’t mean he/her’s a successful entrepreneur or that his methods are legitimate. Renting a Lamborghini for a day or two and taking photos with it is easy. Many scammers do this to create an image of success and attract more victims.

Warning

Be cautious of success stories that seem too good to be true. If someone claims to have made a fortune overnight or with minimal effort, why would they share this information with you? In business, no one shares their secrets to success for free! There’s always a catch which you might not see at first glance.

Maintaining a Legitimate Side Business

Another tactic scammers employ to appear reputable is to run a legitimate side business alongside their fraudulent schemes. By being known for a genuine enterprise, they can gain public trust, which they then exploit to lure victims into their scams. People might think, "This person runs a successful, known business, so their other ventures must be trustworthy too." This makes it easier for the scammer to deceive them. Similarly, you are more likely to trust someone who works at Google (even though working at Google isn't necessarily a huge deal; like any other company, Google also needs ordinary employees) than an ordinary person. This principle of trusting someone based on their association with a known, reputable entity is often exploited by scammers.

Additionally, one very common technique they use is that they may donate to charities, sponsor events, or engage in community activities to further enhance their image.

We are doing something new and different, We are not like others, We are the best!

Scammers often portray themselves as highly sophisticated and engaged in cutting-edge ventures to lure victims. They employ various enticing narratives, such as:

- **Advanced Trading Claims (AI, Bots):** They might claim to possess a super-fast trading bot or an AI system that outperforms human traders. The pitch is simple: "Give us your money, and our advanced technology will generate substantial profits for you."
- **Investing in Prestigious Foreign Markets (e.g., Nasdaq):** A common tactic is to promise investments in high-return foreign stock markets like the US Nasdaq. They might say, "Why bother with your local currency income when it's devaluing daily? We'll invest for you in the US market and give you profits in dollars!"

It's crucial to be skeptical. For instance, an individual from a sanctioned country typically cannot easily buy shares in the US Nasdaq market directly while residing in the US. How can you believe someone is genuinely investing for you under such circumstances, especially if their "proof" consists of a few photos or documents?
- **Cloud Mining:** They might claim to offer cloud mining services, where you can invest in mining cryptocurrencies without needing expensive hardware. However, these operations are often fraudulent, and the promised returns are unrealistic.

One of the factors that people fall for these scams is the use of technical terms and jargon. Scammers often throw around complex-sounding phrases to make themselves appear knowledgeable and trustworthy. This can create a false sense

of security, leading people to believe that they are dealing with experts who know what they are doing. Like the beginning of this section, I mentioned that they might use terms like “Elliptic-curve cryptography” or “SHA-2” to impress people. This is a common tactic in Ponzi schemes and other scams.

Renting an Office and Holding Events

To appear more official and trustworthy, scammers often rent physical office spaces and establish a seemingly formal business presence. They might organize conferences or seminars, inviting potential investors to events that project an image of a professional and successful company.

Furthermore, they may celebrate milestones, such as a “one-year anniversary,” complete with cakes and festivities. These actions are all designed to create a facade of a legitimate and established company, making it harder for people to realize they are dealing with a scam. People might think, “If they have an office and hold events, they must be a real company.”



Figure 11.2: Scammers often rent physical office spaces to appear more legitimate.

They may organize Conference in big halls, with a lot of people, and they may even invite some celebrities to speak at the event. All halls, I mean all of them, can be rented!

Fake Social Media Presence

Their social media accounts might have millions of followers, and they might even have a Wikipedia page. They might claim to have been featured in reputable news outlets, but these articles are often paid for and lack genuine credibility. They might even have a Wikipedia page, but this can also be manipulated by creating fake accounts or paying someone to write it.

- They claim that their website has been operational for 10 years, and they have a lot of experience.

11.1 Ponzi Schemes

→ Check if an archived version of their website exists on the Wayback Machine.² If it does, you can see how long they have been operating. minusOur website is translated into 20 languages, and we have a lot of users around the world. → How long does it take to translate a website into 20 languages? It's not that hard. You can use Google Translate or other translation services to do it quickly. Many of website users' trackers and analytics tools can be easily manipulated to show fake numbers (including fake comments). They might claim to have millions of users, but in reality, they might only have a few hundred or even less. Still, having millions of real users is not a guarantee of legitimacy. Ponzi schemes can attract a large number of people (in both countries with strong and weak regulations), and system can be operational for a long time, but that doesn't mean it's a legitimate business.

Fake Licenses and Registrations

Registering a company in a particular country? That doesn't automatically prove legitimacy. Let me give you an example: Mohammad Jorjandi, through his platform Webamooz, is known for issuing warnings about scams, explaining how they work, and investigating various schemes. If he identifies a scam, he alerts the public. (I personally learned about Ponzi schemes from Webamooz.)

Interestingly, scammers actually went ahead and registered a company under Mohammad Jorjandi's name.³ This demonstrates that it's possible for one person to register a company using another individual's identity! Therefore, simply having a registered company does not equate to having a legitimate license to operate, nor does it imply trustworthiness.

- Is registering a company with the same name as a well-known company possible?

→ Yes.⁴

Most licenses can be obtained easily, especially in countries with weak regulations. Scammers might claim to have licenses from various organizations, such as the US Ports and Fisheries, or even the US Securities and Exchange Commission (SEC). However, these claims are often fabricated or exaggerated. They might show you a document that looks official, but it's likely just a piece of paper they created themselves. Always ask a lawyer (you can't just verify it by yourself) to verify the legitimacy of any license or registration they claim to have.

²<https://archive.org/>

³<https://find-and-update.company-information.service.gov.uk/company/13101026>

⁴See https://find-and-update.company-information.service.gov.uk/officers/3nweUIV6JITEz0-NfioIu_KNuRc/appointments

The Expert or Guru Appeal with Fake Credentials

Another tactic scammers use to attract people is by bestowing upon themselves titles like “Dr.” or “Expert.” To achieve this, they often purchase fake degrees. However, most people don’t bother to verify whether these credentials are legitimate. Many fake degrees come with a code that can be entered on a specific website to supposedly confirm their authenticity. Unbeknownst to the victims, this verification website is often created by other scammers and is not a credible source.

You might wonder why these websites aren’t shut down and why the police don’t apprehend them. Firstly, they might be operating in a different country where there’s no police cooperation.

Secondly, a tactic some of these individuals use is to provide fake credentials, like a PHD, to a few high-ranking officials. (After all, these officials also like to advertise themselves, saying, “I’m a doctor, vote for me.”) This way, if any problems arise, they have the support of those officials. Because, ultimately, the high-ranking official doesn’t want it to be revealed later, if the individual is arrested, that they too were given a fake PHD by this person. So, they will do everything to protect them.

Warning

Never Ever invest in a project where you cannot legally sue the company or individual behind it. This is especially important if you’re living in a country where the legal system is weak or corrupt and no police cooperation exists.

Lavish Advertisements and Sponsored Content

Everyone has a way of earning income, and news websites are no different; they don’t produce news for free. Many news sites derive their revenue from advertising, whether it’s through the ads you see in the corners of articles or as separate posts introducing a product or business.

Due to a lack of proper oversight on these advertisements in certain countries, websites often publish whatever content they receive without verifying its legitimacy. They might only add a tiny disclaimer at the end stating that it’s an advertisement. Scammers exploit this by advertising extensively across numerous sites. They portray an individual as a “doctor,” “engineer,” “professor,” “top entrepreneur,” a recipient of “success medals and certificates of appreciation,” with “five years of experience and 200 years of expertise,” and so on. The message is to go to this person to learn how to invest, or that this benevolent individual will take your money and give you profits.

Many people fall for this because they see the same advertisement on multiple sites, leading them to believe that if so many reputable sites are advertising it, it must be legitimate. They think, “If it were a scam, why would all these sites

promote it?"

Additionally, sponsored content is used for other purposes. Scammers might anticipate that people will search online for terms like “[Scammer’s Name] scam” or “Is [Scammer’s Company] legitimate?”. To counter this, they purchase sponsored articles with titles such as “Is [Scammer’s Name] a Fraud?” or “The Truth About [Scammer’s Company]”. The aim is to push genuine negative reviews or warnings further down in search engine results, making them less visible to potential victims.⁵ This same tactic of flooding search results with positive or neutral sponsored content can also be used to build a seemingly legitimate online presence, sometimes even to obtain verification badges on social media platforms. They might release a song, create a Wikipedia page (which can also be manipulated), and buy advertisements on several sites to bolster their perceived credibility.

A technique you can use to identify such scams is to seek for their historical reviews. If a person claims to be expert, they should have a history of providing valuable content, such as articles, videos, or other educational materials. If you can’t find any such content, and the only thing you can find is their new website or social media account, it’s a red flag. Genuine experts usually have a track record of sharing their knowledge and expertise over time.

For instance, in *Wiseling* project, *Matias Lappo*, The CEO had no expertises background.

11.1.2 If We Got Scammed, What Should We Do?

First, you should tell your friends and family about the scam. Warn them not to fall for it. Write a post on social media, explaining how you got scammed and what the scammer did. Share your experience so that others can learn from it. By this, we prevent more people from falling for the scam.

- I’m afraid they’ll say how stupid I was to get scammed.
- Let them say it. You can’t always please everyone. But telling your story helps prevent others you care about from falling for it. This is helping those who are important to you.

Even if you didn’t lose much money, you should still report the scam to the authorities. This is important because it helps them track down the scammers and prevent them from scamming others. Don’t say it’s just a small amount of money, so it doesn’t matter. Every little bit helps, and the more reports they receive, the better chance they have of catching the scammers.

⁵See <https://web.archive.org/web/20220807100119/https://www.eghtesadonline.com/%D8%A8%D8%AE%D8%B4-%D9%81%D9%86%D8%A7%D9%88%D8%B1%DB%8C-64/427660-%D8%A2%DB%8C%D8%A7-%D8%A2%DB%8C-%D9%85%D8%A7%DB%8C%D9%86%D8%B1-%D9%82%D8%A7%D9%86%D9%88%D9%86%DB%8C-%D8%A7%D8%B3%D8%AA-%D8%A8%D8%B1%D8%B1%D8%B3%DB%8C-%D8%B3%D8%A7%DB%8C%D8%AA> or <https://web.archive.org/web/20220807092819/https://vigiato.net/p/70898> or <https://web.archive.org/web/20220807094524/https://techrato.com/2020/04/30/review-iminer/> for examples. (They’re in Persian.)

Complaint Threshold

There is a concept called the “complaint threshold.” This means that if a scammer steals only a small amount from each person, most victims will not bother to file a complaint. The time, effort, and hassle required to pursue legal action are not worth it for such a small loss. As a result, the scammer can collect small amounts from many people without facing serious consequences.

Don’t let scammers to benefit from this. If you have been scammed, file a complaint, even if it’s just a small amount.

Sometimes, collective complaints are very important. Because if complaints are scattered, the scammer will pay each person individually and ask them to drop the complaint, and the system will remain operational. But when everyone is united, this doesn’t happen. (Ask a lawyer for help with this.)

Report quickly. Don’t fall for:

- We are updating our system to provide better services. Please be patient.
- We are working on a new version of our platform, and we will be back soon.
- We are experiencing technical difficulties, and we will resolve them as soon as possible.
- We are currently under maintenance, and we will be back online shortly.
- The Website got hacked, and we are working to fix it. (They may also add a fake message from the hacker.)

Be Careful Not to Fall for These Scams Again!

One of the victims of a previous scam, having realized that these sites pay early investors, might come forward and say, “Okay, I understand now!” They might tell other victims of the previous scam, “Let’s join this new site; its profits are more reasonable. It’s not like the last one that promised 30-40% profit; this one offers lower, more logical returns!” They try to draw others into the new site, saying, “Our money is gone! At least let’s join this new site to recoup some of our losses and get our money back so we don’t lose out completely!” Others might trust them again because someone like themselves, a victim of the previous scam, has introduced the new site with explanations, claiming the profits are lower and more sensible. They join the new site, only to be scammed again! But this time, they were fooled by someone who was just like them! So please, don’t get caught in this situation and be tricked into escaping one pitfall only to fall into another.

11.2 A Common Misconception

Some people say, “Forget it. Anyone who falls for a promise of a 30% profit is foolish and deserves to lose their money.” But this perspective is flawed. Scammers are thieves, and their job is to deceive. What seems obvious to you—something you might dismiss as “Who would fall for that?”—can be entirely unfamiliar to someone without the necessary knowledge.

For instance, in the field of computer security, it’s obvious to me that using a weak password can lead to account breaches. But for someone who has never seen or understood the consequences of a hacked account, it might not seem like a big deal. Should I then say, “Anyone whose account gets hacked deserves it”? Or should I say, “Anyone who refuses to get vaccinated and falls ill deserves it”? Of course not!

We must strive to educate and improve society. This includes patiently explaining even the most basic concepts repeatedly. For example, reminding people to turn off unnecessary lights at home or to turn off the tap while brushing their teeth. Awareness and education are key to building a better community.

Instead of shaming victims (which often discourages them from reporting due to fear of ridicule and embarrassment), we should focus on shaming scammers and their fraudulent activities.

11.3 Why Do Nigerian Scammers Say They are From Nigeria?

You’ve probably heard scammers make outrageous claims, and you might think to yourself, “What is this? They could have said something at least a bit more believable. Who would actually fall for this?”

The reason they make such unbelievable claims is to filter out everyone except the most gullible individuals. This way, they don’t waste their time on people who are less likely to be deceived. They are looking for those who are easiest to fool.

When you deceive truly gullible people, they will do whatever you want without thinking. But if you deceive knowledgeable and intelligent people, they will look for a way to get their rights back and might catch you. So, it’s better to deceive the gullible ones to be at ease.

According to *Why Do Nigerian Scammers Say They are From Nigeria?* [51]:

Since gullibility is unobservable, the best strategy is to get those who possess this quality to self-identify.

According to *Dr. Azarakhsh Mokri*, this is a comforting thought. It reminds us that when we see an influencer with tens of millions of followers spouting

11.4 Illegal Activities (Mainly Illegal Gambling)

nonsense, not all those millions of followers actually believe them. Only a small fraction does. The rest are often there just for entertainment.

11.4 Illegal Activities (Mainly Illegal Gambling)

11.4.1 Why Illegal Gambling Sites Shouldn't Be Compared to Legal Betting Sites

When we say that illegal gambling sites are problematic, some people respond, "What's the big deal? These sites operate abroad. It's the same thing. Anyone who wants to can play, and those who don't, don't have to."

However, when a site is legal and operates under the laws of a country, it is required to adhere to certain regulations. For instance, if a game is based on chance, it must genuinely be random (the random number generator must be reliable), and there is oversight in place.

On the other hand, gambling sites that operate from Turkey and target Iran are not under any regulatory authority. The Turkish government states that as long as they are not operating within Turkey, they won't interfere much since their activities are directed elsewhere. This lack of oversight allows them to engage in various unethical practices, such as phishing or manipulating outcomes. Players might initially win small amounts, leading them to believe they are skilled, but as time goes on, they end up losing and feel compelled to keep playing to recover their losses, trapping them in a vicious cycle.

Worse still, when these networks operate illegally and evade taxes, the odds of winning are significantly lower, leading to substantial financial losses. This creates an environment ripe for corruption. What does this mean? It means that individuals within these networks may engage in activities that harm society. When we talk about large sums of money, we're talking about around one billion dollars in Iran. That's a staggering amount.

For example, if the police call to freeze a certain bank account, you might find that just ten minutes before the freeze, the account has been emptied. Why? Because a bank employee has become involved in the corrupt network and has alerted others to withdraw their funds quickly before the account is frozen. They may even corrupt police officers to avoid investigations or influence lawyers to assist them in legal matters. This creates a cycle of corruption.

Additionally, these illegal networks have indirect effects on society. By offering large sums for advertising, they monopolize the advertising space. Someone might wonder why they should settle for one dollar for a simple ad when they could earn ten dollars for each ad. This is why we see many Telegram channels solely promoting gambling, stifling the growth of legitimate businesses.

When there's a lot of money involved, spending it becomes easy. They can easily threaten individuals and journalists who write against them. Do you think it's difficult for them to pay someone thirty thousand dollars to harm a

journalist?

11.5 End of the Chapter Questions

Problem 11.1 A Ponzi scheme scammer say: "If I am a scammer, why hasn't anyone sued me?" How would you respond to this question?

11.6 End of the Chapter Solutions

Solution 11.1 Because in Ponzi scheme scams, the system continues to work as long as new people keep joining. The scammer pays profits to everyone as long as there are new entries. The scammer will only disappear when there are no new entries, and by that time, the scammer has already taken the money and disappeared!

They say, "I have a bot that can make money for you." They might even show you some results, like "Look, I made \$1000 in just one day!" But here's the catch: they don't actually have a bot. They just want to take your money.

Bibliography

- [1] Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno. chapter 23, pages 323–326. John Wiley & Sons, Ltd, 2015.
- [2] Daniel Genkin, Adi Shamir, and Eran Tromer. RSA key extraction via low-bandwidth acoustic cryptanalysis. Cryptology ePrint Archive, Paper 2013/857, 2013.
- [3] Theodor Schnitzler, Katharina Kohls, Evangelos Bitsikas, and Christina Pöpper. Hope of delivery: Extracting user locations from mobile instant messengers. In *Proceedings 2023 Network and Distributed System Security Symposium*, NDSS 2023. Internet Society, 2023.
- [4] Connie J Mulligan, Edward B Quinn, Dima Hamadmad, Christopher L Dutton, Lisa Nevell, Alexandra M Binder, Catherine Panter-Brick, and Rana Dajani. Epigenetic signatures of intergenerational exposure to violence in three generations of Syrian refugees. *Scientific Reports*, 15(1), 2 2025.
- [5] Martin Gardner. Mathematical games. *Scientific American*, 237(2):120–124, 8 1977.
- [6] Brian P. Hayes. The Magic Words are Squeamish Ossifrage. *American Scientist*, 82(4):312–316, 7 1994.
- [7] Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno. John Wiley & Sons, Ltd, 2015.
- [8] Alon Shakevsky, Eyal Ronen, and Avishai Wool. Trust dies in darkness: Shedding light on samsung’s TrustZone keymaster design. Cryptology ePrint Archive, Paper 2022/208, 2022.
- [9] Vincent Rijmen and Bart Preneel. Cryptanalysis of mcguffin. In *Fast Software Encryption: Second International Workshop. Leuven, Belgium, 14-16 December 1994, Proceedings*, volume 1008 of *Lecture Notes in Computer Science*, pages 353–358. Springer, 1994.
- [10] Louise Matsakis. How the West Got China’s Social Credit System Wrong. 7 2019.

Bibliography

- [11] Ron Wyden. DNI Clapper tells Wyden the NSA does not collect data on millions of Americans, 3 2013.
- [12] CNBC. Yahoo Security officer confronts NSA Director | CNBC, 2 2015.
- [13] Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno. *Hash Functions*, chapter 5, pages 77–88. John Wiley & Sons, Ltd, 2015.
- [14] Dan Boneh, Henry Corrigan-Gibbs, and Stuart Schechter. Balloon hashing: A memory-hard function providing provable protection against sequential attacks. Cryptology ePrint Archive, Paper 2016/027, 2016.
- [15] P h c. phc-winner-argon2/argon2-specs.pdf at master · P-H-C/phc-winner-argon2.
- [16] Ilya Mironov et al. Hash functions: Theory, attacks, and applications. *Microsoft Research, Silicon Valley Campus*, 10, 2005.
- [17] Bruce Schneier. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. John Wiley & Sons, 2 edition, 1996.
- [18] Alex Biryukov and Dmitry Khovratovich. Related-key cryptanalysis of the full AES-192 and AES-256. Cryptology ePrint Archive, Paper 2009/317, 2009.
- [19] Eli Biham and Adi Shamir. Differential fault analysis of secret key cryptosystems. In *Advances in Cryptology - CRYPTO '97, 17th Annual International Cryptology Conference, Santa Barbara, California, USA, August 17-21, 1997, Proceedings*, volume 1294 of *Lecture Notes in Computer Science*, pages 513–525. Springer, 1997.
- [20] John T. Kohl. The use of encryption in kerberos for network authentication. In Gilles Brassard, editor, *Advances in Cryptology — CRYPTO' 89 Proceedings*, pages 35–43, New York, NY, 1990. Springer New York.
- [21] Niels Ferguson, Bruce Schneier, and Tadayoshi Kohno. *Block Cipher Modes*, chapter 4, pages 63–76. John Wiley & Sons, Ltd, 2015.
- [22] Meltem Sonmez Turan. *ASCON-Based Lightweight Cryptography Standards for constrained devices*. January 2025.
- [23] D. Dolev and A. Yao. On the security of public key protocols. *IEEE Transactions on Information Theory*, 29(2):198–208, 1983.
- [24] Bruce Schneier. Security pitfalls in cryptography. 1998.
- [25] Python Software Foundation. secrets — generate secure random numbers for managing secrets. Accessed: 2024-12-06.
- [26] Python Software Foundation. Pep 506 – adding a secrets module to the standard library, 2015. Accessed: 2024-12-06.

Bibliography

- [27] Bruce Schneier. Cryptography after the aliens land. *IEEE Security and Privacy*, 16(5):86–88, 2018.
- [28] Daniel J. Bernstein. *Introduction to post-quantum cryptography*, pages 1–14. Springer Berlin Heidelberg, Berlin, Heidelberg, 2009.
- [29] Peter Gutmann and Stephan Neuhaus. Replication of quantum factorisation records with an 8-bit home computer, an abacus, and a dog. Cryptology ePrint Archive, Paper 2025/1237, 2025.
- [30] Ivan Damgård, Helene Haagh, Rebekah Mercer, Anca Nițulescu, Claudio Orlandi, and Sophia Yakubov. Stronger security and constructions of multi-designated verifier signatures. Cryptology ePrint Archive, Paper 2019/1153, 2019.
- [31] Elaine Barker. *Recommendation for key management*:. May 2020.
- [32] Adi Shamir. How to share a secret. *Communications of the ACM*, 22(11):612–613, 1979.
- [33] Santiago Cuéllar, Bill Harris, James Parker, Stuart Pernsteiner, and Eran Tromer. Cheesecloth: Zero-knowledge proofs of real-world vulnerabilities, 2023.
- [34] Peter Gutmann. Secure deletion of data from magnetic and solid-state memory. In *Proceedings of the sixth USENIX security symposium, san jose, CA*, volume 14, pages 77–89, 1996.
- [35] J. Alex Halderman, Seth D. Schoen, Nadia Heninger, William Clarkson, William Paul, Joseph A. Calandrino, Ariel J. Feldman, Jacob Appelbaum, and Edward W. Felten. Lest we remember: Cold boot attacks on encryption keys. In *17th USENIX Security Symposium (USENIX Security 08)*, San Jose, CA, July 2008. USENIX Association.
- [36] Yoongu Kim, Ross Daly, Jeremie Kim, Chris Fallin, Ji Hye Lee, Donghyuk Lee, Chris Wilkerson, Konrad Lai, and Onur Mutlu. Flipping bits in memory without accessing them: An experimental study of dram disturbance errors. In *2014 ACM/IEEE 41st International Symposium on Computer Architecture (ISCA)*, pages 361–372, 2014.
- [37] Andrew Kwong, Daniel Genkin, Daniel Gruss, and Yuval Yarom. Ram-bleed: Reading bits in memory without accessing them. In *2020 IEEE Symposium on Security and Privacy (SP)*, pages 695–711, 2020.
- [38] Eli Biham and Adi Shamir. Differential fault analysis of secret key cryptosystems. In Burton S. Kaliski, editor, *Advances in Cryptology — CRYPTO '97*, pages 513–525, Berlin, Heidelberg, 1997. Springer Berlin Heidelberg.

Bibliography

- [39] Paul Kocher, Jann Horn, Anders Fogh, , Daniel Genkin, Daniel Gruss, Werner Haas, Mike Hamburg, Moritz Lipp, Stefan Mangard, Thomas Prescher, Michael Schwarz, and Yuval Yarom. Spectre attacks: Exploiting speculative execution. In *40th IEEE Symposium on Security and Privacy (S&P'19)*, 2019.
- [40] Michael Schwarz, Martin Schwarzl, Moritz Lipp, Jon Masters, and Daniel Gruss. *NetSpectre: Read Arbitrary Memory over Network*, pages 279–299. 09 2019.
- [41] Sriram Sami, Yimin Dai, Sean Rui Xiang Tan, Nirupam Roy, and Jun Han. Spying with your robot vacuum cleaner: eavesdropping via lidar sensors. In *Proceedings of the 18th Conference on Embedded Networked Sensor Systems, SenSys '20*, page 354–367, New York, NY, USA, 2020. Association for Computing Machinery.
- [42] C. Metzler, D. Lindell, and G. Wetzstein. Keyhole Imaging: Non-Line-of-Sight Imaging and Tracking of Moving Objects Along a Single Optical Path. *IEEE Transactions on Computational Imaging*, 2021.
- [43] Rob J. van Emous. Towards systematic black-box testing for exploitable race conditions in web apps, June 2019.
- [44] Sarah Bird, Ilana Segall, and Martin Lopatka. Replication: Why we still can't browse in peace: On the uniqueness and reidentifiability of web browsing histories. In *Sixteenth Symposium on Usable Privacy and Security (SOUPS 2020)*, pages 489–503. USENIX Association, August 2020.
- [45] Yinzhi Cao, Song Li, and Erik Wijmans. (cross-)browser fingerprinting via os and hardware level features. In *Network and Distributed System Security Symposium*, 2017.
- [46] Artur Janc, Krzysztof Kotowicz, Lukas Weichselbaum, and Roberto Clapis. Information leaks via safari's intelligent tracking prevention, 2020.
- [47] Daniel Moore and Thomas Rid and. Cryptopolitik and the darknet. *Survival*, 58(1):7–38, 2016.
- [48] Farkhund Iqbal, Hamad Binsalleeh, Benjamin C.M. Fung, and Mourad Debbabi. Mining writeprints from anonymous e-mails for forensic investigation. *Digital Investigation*, 7(1):56–64, 2010.
- [49] Nick Bilton. *American kingpin: The epic hunt for the criminal mastermind behind the Silk Road*. Penguin, 2018.
- [50] B Evers, J Hols, E Kula, J Schouten, M Den Toom, RM Van der Laan, and JA Pouwelse. Thirteen years of tor attacks. *Computer Science, Delft University of Technology*, 2016.
- [51] Cormac Herley. Why do nigerian scammers say they are from nigeria? *WEIS*, June 2012.